

Figura 8.27 Una pila de enteros implementada en Java y C#.

```

class PilaDeEnteros
{private int[] EntradasPila = new int[20];
  private int PunteroPila = 0;

  public void insertar(int NuevaEntrada)
  {if (PunteroPila < 20)
    EntradasPila[PunteroPila++] = NuevaEntrada;
  }

  public int extraer ()
  {if (PunteroPila > 0) return EntradasPila[--PunteroPila];
   else return 0;
  }
}

```

PilaDeEnteros PilaUno = new PilaDeEnteros();

o en un programa C++ mediante la sentencia

PilaDeEnteros PilaUno();

Posteriormente en los programas, podemos insertar el valor 106 en PilaUno utilizando la sentencia

PilaUno.insertar(106);

o podemos extraer la entrada superior de PilaUno y colocarla en la variable AntigoValor utilizando la sentencia

AntigoValor = PilaUno.extraer();

Estas características son esencialmente las mismas que las asociadas con los tipos abstractos de datos. Sin embargo, existen diferencias entre los tipos abstractos de datos y las clases. Las clases son una extensión de los tipos abstractos de datos. Por ejemplo, como hemos visto en la Sección 6.5, los lenguajes orientados a objetos permiten que las clases hereden propiedades de otras clases

La librería STL

Las estructuras de datos de las que hemos hablado en este capítulo se han convertido en estructuras estándar de programación, tan estándar, de hecho, que muchos entornos de programación las tratan casi como si fueran primitivas. Podemos encontrar un ejemplo en el entorno de programación C++, que está ampliado con la librería STL (*Standard Template Library*, Librería estándar de plantillas). La STL contiene un conjunto de clases predefinidas que describen diversas estructuras de datos muy populares. En consecuencia, incorporando la STL en un programa C++, el programador queda liberado de describir estas estructuras en detalle. En lugar de ello, lo único que tiene que hacer es declarar los identificadores de los tipos apropiados, de la misma manera que declarábamos PilaUno como de tipo PilaDeEnteros en la Sección 8.6.

y también permiten que las clases contengan métodos especiales denominados constructores, que personalizan los objetos individuales en el momento de crearlos. Además, las clases se asocian normalmente con diversos grados de encapsulación (Sección 6.5), permitiendo proteger las propiedades internas de sus instancias frente a posibles atajos dañinos. Y, finalmente, una clase puede utilizarse como medio de agrupar procedimientos relacionados y puede, por tanto, estar constituida únicamente por definiciones de procedimientos. En este sentido, podríamos decir que una clase es un tipo abstracto más que un tipo abstracto de datos.

Podemos concluir que los conceptos de clases y de objetos constituyen otro paso en la evolución de las técnicas para representar abstracciones de datos en los programas. De hecho, es esta capacidad de definir y utilizar abstracciones de una forma conveniente lo que ha dotado de tanta popularidad al paradigma orientado a objetos.

Cuestiones y ejercicios

1. ¿En qué se parecen los tipos abstractos de datos y las clases? ¿En qué se diferencian?
2. ¿Cuál es la diferencia entre una clase y un objeto?
3. Describa una clase que podríamos utilizar como plantilla para construir objetos de tipo cola-de-enteros.

8.7 Punteros en el lenguaje máquina

En este capítulo hemos presentado los punteros y hemos mostrado cómo se utilizan a la hora de construir estructuras de datos. En esta sección vamos a considerar cómo se manipulan los punteros en lenguaje máquina.

Suponga que queremos escribir un programa en el lenguaje máquina descrito en el Apéndice C con el fin de extraer una entrada de una pila, como se describe en la Figura 8.12 y colocar dicha entrada en un registro de propósito general. En otras palabras, queremos cargar un registro con el contenido de la posición de memoria que contiene la entrada correspondiente a la cima de la pila. Nuestro lenguaje máquina proporciona dos instrucciones para cargar registros: una con el código de operación 2 y la otra con el código de operación 1. Recuerde que en el caso del código de operación 2, el campo de operando contiene el dato que hay que cargar, mientras que en el caso del código de operación 1, el campo de operando contiene la dirección del dato que hay que cargar.

No sabemos cuál va a ser el contenido, así que no podemos utilizar el código de operación 2 para conseguir nuestro objetivo. Además, tampoco podemos utilizar el código de operación 1, porque no sabemos cuál será la dirección. Después de todo, la dirección de la cima de la pila irá variando a medida que se ejecute el programa. Sin embargo, lo que sí conocemos es la dirección del puntero de pila. Es decir, conocemos la posición de la dirección del dato que queremos cargar. Lo que necesitamos, entonces, es un tercer código de operación

para cargar un registro, en el que el operando contenga la dirección de un puntero que apunte al dato que hay que cargar.

Para conseguir esto, ampliaremos el lenguaje del Apéndice C con el fin de incluir un código de operación D. Una instrucción con este código de operación tendría la forma DRXY, lo que significa que hay que cargar en el registro R el contenido de la posición de memoria cuya dirección se encuentra en la dirección XY (Figura 8.28). Así, si el puntero de pila se encuentra en la posición de memoria en la dirección AA, entonces la instrucción D5AA haría que el dato situado en la cima de la pila se cargue en el registro 5.

Sin embargo, esta instrucción no completa la operación de extracción del dato. También deberemos restar una unidad del puntero de pila para que ahora apunte a la nueva cima de la pila. Esto significa que, después de la instrucción de carga, nuestro programa en lenguaje máquina tendría que cargar el puntero de pila en un registro, restarle una unidad y volver a almacenar el resultado en memoria.

Utilizando uno de los registros como puntero de pila en lugar de usar una posición de memoria, podríamos eliminar la necesidad de mover el puntero de pila repetidamente entre los registros y la memoria. Pero esto implicaría que tendríamos que rediseñar la instrucción de carga para que esta espere encontrar el puntero en un registro en lugar de en la memoria principal. Por tanto, en lugar de la solución anterior, podríamos definir una instrucción con el código de operación D que tuviera la forma DR0S, lo que significaría que hay que cargar el registro R con el contenido de la celda de memoria a la que apunta el registro S (Figura 8.29). Entonces, podría efectuarse una operación completa de extracción ejecutando primero esta instrucción y luego otra instrucción (o instrucciones) para restar una unidad del valor almacenado en el registro S.

Observe que se necesita una instrucción similar para implementar una operación de inserción en la pila. Podemos por tanto ampliar aún más el lenguaje descrito en el Apéndice C, introduciendo el código de operación E, de

Figura 8.28 Nuestro primer intento de ampliar el lenguaje máquina del Apéndice C para poder utilizar punteros.

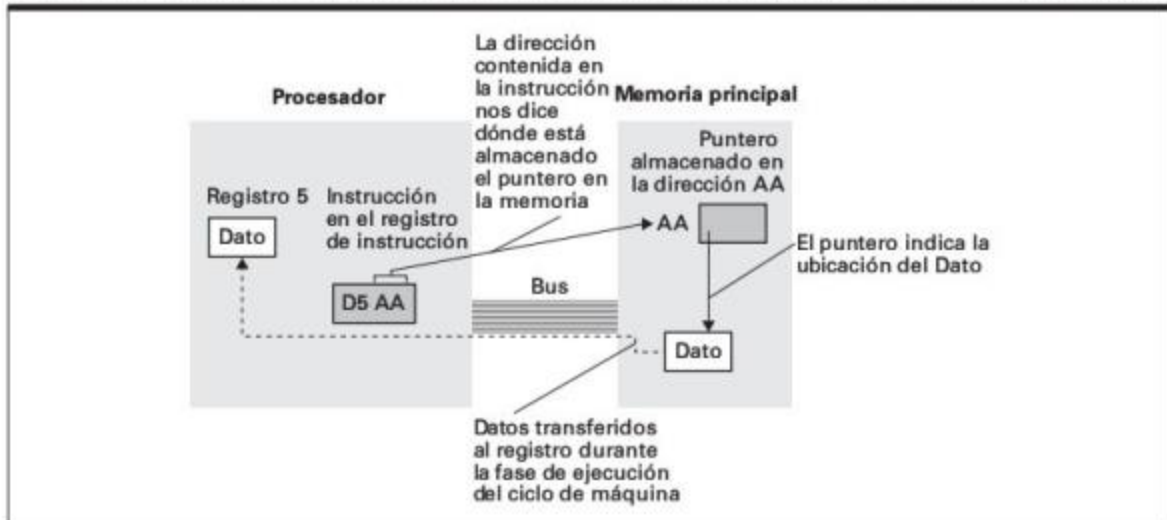
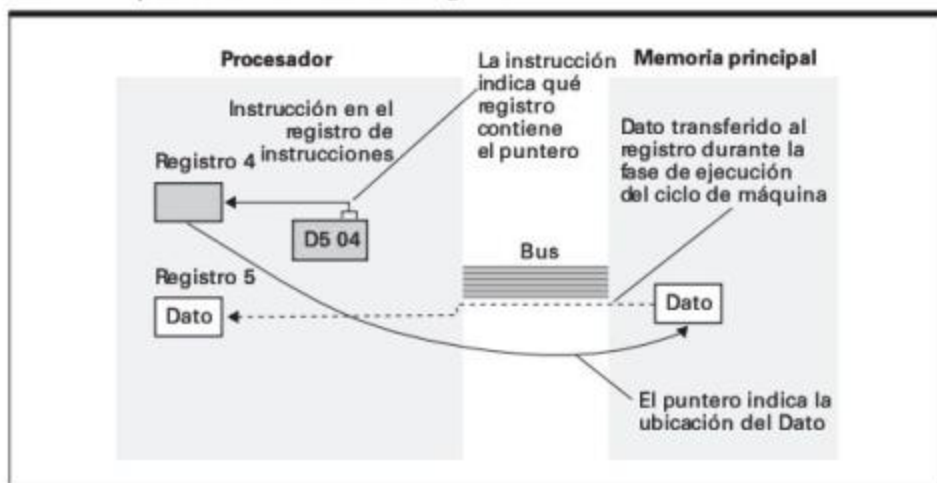


Figura 8.29 Carga de un registro a partir de una celda de memoria que se localiza por medio de un puntero almacenado en otro registro.



modo que una instrucción de la forma EROS signifique que hay que almacenar el contenido del registro R en la celda de memoria a la que apunta el registro S. De nuevo, para completar la operación de inserción, esta instrucción debería ir seguida por otra instrucción (o instrucciones) que sumara uno al valor contenido en el registro S.

Estos nuevos códigos de operación D y E que hemos propuesto no solo ilustran cómo se diseñan los lenguajes máquina para manipular punteros, sino que también ilustran una técnica de direccionamiento que no estaba presente en el lenguaje máquina original. Tal como se presenta en el Apéndice C, el lenguaje máquina utiliza dos medios de identificar los datos implicados en una instrucción. El primero de ellos queda ilustrado por una instrucción cuyo código de operación sea 2. En ella, el campo de operando contiene explícitamente el dato necesario. Esta técnica se denomina **direccionamiento inmediato**. El segundo medio para identificar datos queda ilustrado por las instrucciones con los códigos de operación 1 y 3. En ellas, los campos de operando contienen la dirección del dato necesario. Esta técnica se denomina **direccionamiento directo**. Sin embargo, nuestros nuevos códigos de operación D y E propuestos ilustran otra forma más de identificar los datos. Los campos de operando de estas instrucciones contienen la dirección de la dirección de los datos. Esta técnica se denomina **direccionamiento indirecto**. Las tres técnicas de direccionamiento son bastante comunes en los lenguajes máquina actuales.

Cuestiones y ejercicios

1. Suponga que ampliamos el lenguaje máquina descrito en el Apéndice C como se ha sugerido al final de esta sección. Suponga también que el registro 8 contiene el patrón DB, que la celda de memoria en la posición DB contiene el patrón CA y que la celda situada en la dirección CA con-

tiene el patrón A5. ¿Qué patrón de bits contendrá el registro 5 inmediatamente después de ejecutar cada una de las siguientes instrucciones?

- 25A5
 - 15CA
 - D508
- Utilizando las extensiones descritas al final de esta sección, escriba una rutina completa en lenguaje máquina para realizar una operación de extracción en una pila. Suponga que la pila está implementada como se muestra en la Figura 8.12, que el puntero de pila se encuentra en el registro F y que hay que extraer el valor situado en la cima de la pila y almacenarlo en el registro 5.
 - Utilizando las extensiones descritas al final de esta sección, escriba un programa para copiar el contenido de cinco celdas de memoria contiguas que comienzan en la dirección A0 a las cinco celdas que comienzan en la dirección B0. Suponga que el programa comienza en la dirección 00.
 - En este capítulo hemos presentado una instrucción de la forma DR0S. Suponga que ampliáramos esta instrucción a DRXS, con el siguiente significado "Cargar el registro R con el dato al que apunta el valor contenido en el registro S sumado con el valor X." Así, el puntero al dato se obtiene extrayendo el valor almacenado en el registro S y luego incrementando dicho valor en X. El valor del registro S no se ve alterado. (Si el registro F contuviera el valor 04, entonces la instrucción DE2F cargaría en el registro E el contenido de la celda de memoria situada en la dirección 06. El valor del registro F seguiría siendo 04.) ¿Qué ventajas tendría esta instrucción? ¿Qué pasaría con una instrucción de la forma DRTS, que significara "Cargar el registro R con el dato al que apunta el valor del registro S sumado al valor del registro T"?

Problemas de repaso

(Los problemas marcados con asterisco están asociados con las secciones opcionales.)

- Dibuje sendos diagramas que muestren cómo aparecería en la memoria de la máquina la matriz que se muestra a continuación, si se la almacenara con ordenación por filas y con ordenación por columnas.
- Suponga que almacenamos con ordenación por filas un array de seis filas y ocho columnas, comenzando en la dirección 20 (base diez). Si cada entrada de la matriz requiere una única celda de memoria, ¿cuál es la dirección correspondiente a la entrada de la cuarta columna de la tercera fila? ¿Y cuál sería la dirección si cada entrada requiriera dos celdas de memoria?

A	B	C	D
E	F	G	H
I	J	K	L

3. Repita el Problema 2 suponiendo que se utiliza ordenación por columnas en lugar de ordenación por filas.
4. ¿Qué complicaciones aparecen si se intenta implementar una lista dinámica utilizando una matriz unidimensional tradicional?
5. Describa un método para almacenar matrices tridimensionales. ¿Qué polinomio de dirección se utilizaría para localizar la entrada correspondiente al i -ésimo plano, j -ésima fila y k -ésima columna?
6. Suponga que almacenamos la lista de letras A, B, C, E, F y G en un bloque contiguo de posiciones de memoria. ¿Qué actividades se requieren para insertar la letra D en la lista, suponiendo que haya que preservar el orden alfabético de la lista?
7. La siguiente tabla representa el contenido de algunas posiciones de la memoria principal de una computadora, junto con la dirección de cada celda representada. Observe que algunas de las celdas contienen letras del alfabeto y que cada una de esas celdas va seguida por una celda vacía. Coloque las direcciones necesarias en esas celdas vacías de manera que cada celda que contenga una letra forme, junto con la siguiente posición, una entrada de una lista enlazada en la que las letras aparezcan en orden alfabético. (Use el cero como puntero NIL.) ¿Qué dirección debería contener el puntero inicial?

Dirección	Contenido
11	C
12	
13	G
14	
15	E
16	
17	B
18	
19	U
20	
21	F
22	

8. La siguiente tabla representa una parte de una lista enlazada en la memoria principal de una computadora. Cada entrada de la lista está compuesta por dos celdas: la primera contiene una letra del alfabeto y la segunda contiene un puntero a la siguiente entrada de la lista. Modifique los punteros para que la letra N deje de pertenecer a la lista. Luego sustituya la letra N por la letra G y modifique los punteros para que la nueva letra aparezca en la lista en su lugar apropiado, en orden alfabético.

Dirección	Contenido
30	J
31	38
32	B
33	30
34	X
35	46
36	N
37	40
38	K
39	36
40	P
41	34

9. La siguiente tabla representa una lista enlazada utilizando el mismo formato que en los problemas anteriores. Si el puntero inicial contiene el valor 44, ¿cuál es el nombre representado por la lista? Modifique los punteros para que la lista contenga el nombre Jean.

Dirección	Contenido
40	N
41	46
42	I
43	40
44	J
45	50
46	E
47	00
48	M
49	42
50	A
51	40

10. ¿Cuál de las siguientes rutinas insertará correctamente NuevaEntrada inmediata-

mente después de la entrada denominada *EntradaAnterior* en una lista enlazada? ¿Qué es lo que falla en la otra rutina?

Rutina 1:

1. Copiar el valor contenido en el campo de puntero de *EntradaAnterior* en el campo de puntero de *NuevaEntrada*.
2. Modificar el valor contenido en el campo de puntero de *EntradaAnterior* introduciendo la dirección de *NuevaEntrada*.

Rutina 2:

1. Modificar el valor contenido en el campo de puntero de *EntradaAnterior* introduciendo la dirección de *NuevaEntrada*.
2. Copiar el valor del campo de puntero de *EntradaAnterior* en el campo de puntero de *NuevaEntrada*.

segundo puntero después de cada letra nos encontremos las letras en orden alfabético. ¿Qué valor tendrá el puntero inicial de cada una de las dos listas representadas?

Dirección	Contenido
60	O
61	
62	
63	C
64	
65	
66	A
67	
68	
69	L
70	
71	
72	R
73	
74	

11. Diseñe un procedimiento para concatenar dos listas enlazadas (es decir, para colocar una delante de la otra, con el fin de formar una única lista).
12. Diseñe un procedimiento para combinar dos listas contiguas ordenadas en una única lista ordenada contigua. ¿Qué sucede si las listas están enlazadas?
13. Diseñe un procedimiento para invertir el orden de una lista enlazada.
14. a. Diseñe un algoritmo para imprimir una lista enlazada en orden inverso, utilizando una pila como estructura auxiliar de almacenamiento.
b. Diseñe un procedimiento recursivo para realizar esta misma tarea sin hacer un uso explícito de una pila. ¿En qué forma seguirá habiendo una pila implicada en esa solución recursiva?
15. En ocasiones, a una única lista enlazada se le asignan dos ordenaciones diferentes, asociando dos punteros con cada entrada en lugar de uno. Rellene la tabla que se muestra a continuación de modo que, al seguir el primer puntero después de cada letra aparezca el nombre Carol, pero al seguir el

16. La siguiente tabla representa una pila almacenada en un bloque contiguo de celdas de memoria, tal como se explica en el texto. Si la base de la pila está en la dirección 10 y el puntero de pila contiene el valor 12, ¿qué valor nos daría una instrucción de extracción? ¿Qué valor tendrá el puntero de pila después de la operación de extracción?

Dirección	Contenido
10	F
11	C
12	A
13	B
14	E

17. Dibuje una tabla que muestre el contenido final de las celdas de memoria si la instrucción del Problema 16 hubiera sido introducir la letra D en la pila, en lugar de extraer una letra. ¿Cuál será el valor del puntero de pila después de la instrucción de inserción?
18. Diseñe un procedimiento para extraer la entrada correspondiente al fondo de una pila, de modo que se conserve el resto de la pila. Los accesos a la pila deben realizarse únicamente mediante operaciones de inser-

ción y extracción. ¿Qué estructura auxiliar de almacenamiento debe utilizarse para resolver este problema?

19. Explique el término *backtracking* con un ejemplo adecuado.
20. Suponga que disponemos de dos pilas. Si solo se nos permite mover una entrada cada vez de una pila a otra, ¿qué reordenaciones de los datos originales serían posibles? ¿Qué ordenaciones serían posibles si dispusiéramos de tres pilas?
21. Suponga que disponemos de tres pilas y que solo se nos permite mover una entrada cada vez de una pila a otra. Diseñe un algoritmo para invertir dos entradas adyacentes en una de las pilas.
22. Suponga que queremos crear una pila de nombres, siendo la longitud de los mismos variable. ¿Por qué es conveniente almacenar los nombres en áreas separadas de la memoria y luego construir la pila con una serie de punteros a esos nombres, en lugar de permitir que la pila contenga los propios nombres?
23. ¿Cuál es la diferencia fundamental entre una lista enlazada y un árbol binario?
24. Suponga que deseamos implementar una "cola" en la que las nuevas entradas tengan prioridades asociadas, de modo que cada nueva entrada deberá ser colocada delante de aquellas entradas que tengan una prioridad menor. Describa un sistema de almacenamiento para implementar este tipo de "cola" y justifique las decisiones que tome.
25. Suponga que las entradas de una pila requieren una celda de memoria cada una y que el puntero de pila contiene el valor 101 en cualquier instante. ¿Cuál será el contenido del puntero de pila después de haber añadido tres entradas y eliminado dos?
26. a. Suponga que una cola implementada de forma circular se muestra en el estado indicado en el siguiente diagrama. Dibuje un diagrama que muestre la estructura después de efectuar las siguientes operaciones: insertar las letras G y R, extraer tres letras e insertar las letras D y P.



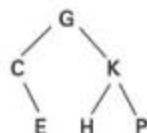
- b. ¿Qué error se produciría en el apartado (a) si se insertan las letras G, R, D y P antes de eliminar ninguna letra?
27. ¿Qué es una lista contigua? ¿Cuál es su limitación? ¿Cómo podría superarse esa limitación utilizando una lista enlazada?
28. Suponga que nos dan dos colas y que solo se puede desplazar cada vez una entrada desde el principio de una cola a la parte final de la otra. Diseñe un algoritmo para invertir dos entradas adyacentes en una de las colas.
29. La siguiente tabla representa un árbol almacenado en la memoria de una máquina. Cada nodo del árbol está compuesto por tres celdas. La primera de esas celdas contiene el dato (una letra), la segunda contiene un puntero al hijo izquierdo del nodo y la tercera un puntero al hijo derecho del nodo. El puntero NIL está representado por un valor 0. Si el valor del puntero raíz es 55, dibuje una imagen del árbol.

Dirección	Contenido
40	G
41	0
42	0
43	X
44	0
45	0
46	J
47	49
48	0
49	M
50	0
51	0
52	F
53	43
54	40
55	W
56	46
57	52

30. La siguiente tabla representa el contenido de un bloque de celdas de la memoria prin-

cial de una computadora. Observe que algunas de las celdas contienen letras del alfabeto y que cada una de esas celdas va seguida por dos celdas en blanco. Rellene las celdas en blanco de modo que el bloque de memoria represente el árbol que se muestra a continuación. Utilice la primera celda que sigue como puntero al hijo izquierdo de ese nodo y la siguiente celda como puntero al hijo derecho. Utilice el valor 0 para los punteros NIL. ¿Qué valor deberá contener el puntero raíz?

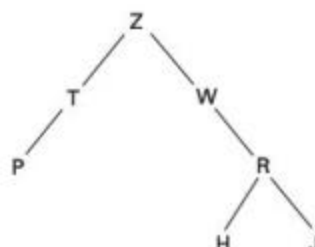
Dirección	Contenido
30	C
31	
32	
33	H
34	
35	
36	K
37	
38	
39	E
40	
41	
42	G
43	
44	
45	P
46	
47	



31. Explique las diferencias entre estructuras de datos estáticas y dinámicas.
32. Diseñe un algoritmo no recursivo para sustituir el algoritmo recursivo representado en la Figura 8.24. Utilice una pila para controlar cualquier paso atrás que pueda ser necesario.
33. Aplique el algoritmo recursivo de impresión de árboles de la Figura 8.24. Dibuje un diagrama que represente las activaciones

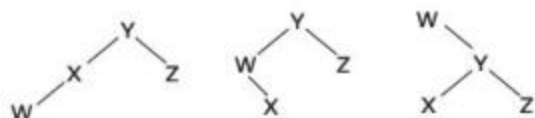
anidadas del algoritmo (y la posición actual en cada una) en el momento de imprimir el nodo X.

34. Manteniendo sin variaciones el nodo raíz y sin modificar la ubicación física de los elementos de datos, cambie los punteros del árbol del Problema 29 para que el algoritmo de impresión de árboles de la Figura 8.24 imprima los nodos alfabéticamente.
35. Dibuje un diagrama que muestre cómo aparecería en la memoria el árbol binario mostrado a continuación, si se almacenara sin punteros utilizando un bloque de celdas de memoria contiguas como el descrito en la Sección 8.3.



36. Suponga que las celdas contiguas que representan un árbol binario como se ha descrito en la Sección 8.3 contuvieran los valores R, B, A, D, y E, respectivamente. Dibuje un diagrama del árbol.
37. Proporcione un ejemplo en el que conveniría implementar una lista (la estructura conceptual) como un árbol (la estructura real subyacente). Proporcione un ejemplo en el que conveniría implementar un árbol (la estructura conceptual) en forma de lista (la estructura real subyacente).
38. Las estructuras de árbol enlazado que hemos analizado en el texto contenían punteros que permitían descender por el árbol, pasando de padre a hijo. Describa un sistema de punteros que permita ascender por el árbol, pasando de los hijos a los padres. ¿Y qué ocurre en el caso de que queramos movernos entre hermanos?
39. Describa una estructura de datos adecuada para representar una configuración durante una partida del juego de las tres en raya.

40. Indique cuáles de los árboles que se muestran a continuación se imprimirían por orden alfabético de los nodos si se aplica el algoritmo de la Figura 8.24.



41. Modifique el procedimiento de la Figura 8.24 para imprimir la "lista" en orden inverso.
42. Describa una estructura de árbol que pueda utilizarse para almacenar la historia genealógica de una familia. ¿Qué operaciones hay que realizar con el árbol? Si el árbol se implementa como una estructura enlazada, ¿qué punteros habría que asociar con cada nodo? Diseñe los procedimientos necesarios para realizar las operaciones que haya identificado, suponiendo que el árbol se implementa como una estructura enlazada con los punteros que haya decidido asociar con cada nodo. Utilizando su sistema de almacenamiento, explique cómo se podría localizar a todos los hermanos de una persona.
43. Suponga que los elementos 10, 20, 30, 40 y 50 se insertan en una cola. A continuación, se extraen tres elementos, ¿qué elementos quedarán en la cola?
44. En la implementación tradicional de un árbol, cada nodo se construye con un puntero separado para cada uno de los posibles hijos. El número de tales punteros es una decisión de diseño y representa el número máximo de hijos que cualquier nodo puede tener. Si un nodo tiene menos hijos que el número máximo de punteros admitidos, algunos de esos punteros tendrán el valor NIL. Pero ese tipo de nodo nunca puede tener más hijos que punteros. Describa cómo podría implementarse un árbol sin limitar el número de hijos que puede tener un nodo.
45. ¿Para cuáles de las siguientes operaciones debe emplearse una pila?
- Ordenación de elementos utilizando la técnica de ordenación por burbuja.
 - Evaluación de símbolos en una expresión algebraica.
 - Pulsaciones efectuadas por el usuario de una computadora al escribir una carta. Explique su respuesta.
46. Utilizando la instrucción de pseudocódigo `define type` presentada en la Sección 8.5, esboce una definición de un tipo abstracto de datos que represente una lista de nombres. En particular, ¿qué estructura contendrá la lista y qué procedimientos se proporcionarán para manipular la lista? (No hace falta incluir una descripción detallada de los procedimientos.)
47. Utilizando la instrucción de pseudocódigo `define type` presentada en la Sección 8.5, esboce una definición de un tipo abstracto de datos que represente una cola. A continuación, proporcione instrucciones de pseudocódigo que muestren cómo se podrían crear instancias de ese tipo y cómo podrían insertarse y borrarse entradas en dichas instancias.
48. a. ¿Cuál es la diferencia entre un tipo abstracto de datos y un tipo de datos primitivo?
b. ¿Cuál es la diferencia entre una estructura de datos de pila y una estructura de datos de cola?
49. Identifique las estructuras de datos y procedimientos que podrían aparecer en un tipo abstracto de datos que represente a un estudiante.
50. Identifique las estructuras de datos y procedimientos que podrían aparecer en un tipo abstracto de datos que represente una nave espacial simple en un videojuego.
- *51. Modifique la Figura 8.27 para que la clase defina una cola en lugar de una pila.
- *52. ¿Por qué decimos que una clase es más general que un tipo abstracto de datos tradicional?
- *53. Utilizando instrucciones de la forma `DROS` y `EROS` como se describe al final de la Sección 8.7, escriba una rutina completa en lenguaje máquina para insertar una entrada en una pila implementada como se muestra en la Figura 8.12. Suponga que el puntero de pila se encuentra en el registro

F y que la entrada que hay que insertar está en el registro 5.

- *54.** Suponga que cada entrada de una lista enlazada está formada por una celda de memoria de datos, seguida de un puntero a la siguiente entrada de la lista. Suponga también que queremos insertar entre las entradas situadas en las posiciones B5 y C4 otra nueva entrada, situada en la dirección de memoria A0. Utilizando el lenguaje descrito en el Apéndice C y los códigos de operación adicionales D y E como se des-

criben al final de la Sección 8.7, escriba una rutina en lenguaje máquina para llevar a cabo la inserción.

- *55.** ¿Qué ventajas tiene una instrucción de la forma DR0S como se describe en la Sección 8.7 con respecto a una instrucción de la forma DRXY? ¿Qué ventaja tiene una instrucción de la forma DRXS como se describe en la Cuestión/Ejercicio 4 de la Sección 8.7 con respecto a una instrucción de la forma DR0S?

Cuestiones sociales

Las siguientes cuestiones pretenden ser una guía para los problemas éticos/sociales/legales asociados con el campo de la computación. El objetivo no es responder simplemente a estas cuestiones. El lector debería considerar también por qué las ha contestado de la forma en que lo ha hecho y analizar si sus justificaciones son coherentes entre las distintas cuestiones.

1. Suponga que un analista software diseña una organización de datos que permite la manipulación eficiente de los datos en una aplicación concreta. ¿Cómo pueden protegerse los derechos sobre esa estructura de datos? ¿Es una estructura de datos la expresión de una idea (como un poema) y por tanto debería estar protegida por un copyright o, por el contrario, las estructuras de datos caen en el mismo agujero legal que los algoritmos? ¿Y qué pasa con las leyes de patentes?
2. ¿Hasta qué punto puede ser peor disponer de datos incorrectos que no disponer de ningún dato?
3. En muchos programas de aplicación, el tamaño hasta el que puede crecer una pila está determinado por la cantidad de memoria disponible. Si se consume el espacio disponible, el software está diseñado para generar un mensaje como por ejemplo "desbordamiento de pila" y terminar. En la mayoría de los casos este error nunca llega a producirse y el usuario no es consciente de él. ¿Quién sería responsable si se produce ese error y, como consecuencia, se pierde información importante? ¿Cómo podría el desarrollador del software minimizar su responsabilidad legal?
4. En una estructura de datos basada en un sistema de punteros, el borrado de un elemento suele consistir en modificar el puntero en lugar de borrar celdas de memoria. Por tanto, cuando se borra una entrada de una lista enlazada, la entrada borrada sigue estando en memoria hasta que su espacio de memoria sea ocupado por otros datos. ¿Qué problemas éticos y de seguridad surgen debido a esta persistencia de los datos borrados?
5. Es fácil transferir datos y programas de una computadora a otra. Así, es sencillo transferir a muchas máquinas el conocimiento que una máquina alberga. Por el contrario, a veces hace falta mucho tiempo para que un ser humano transfiera sus conocimientos a otros. Por ejemplo, se necesita

mucho tiempo para que una persona enseñe a otra un nuevo idioma. ¿Qué implicaciones podría tener esta diferencia en cuanto a la tasa de transferencia de los conocimientos, si las capacidades de las máquinas comienzan a desafiar las capacidades de los seres humanos?

6. El uso de punteros permite enlazar datos relacionados en la memoria de una computadora de una manera que recuerda a la forma en que algunos expertos creen que la información está asociada en la mente humana. ¿En qué sentido son similares a los vínculos cerebrales esos enlaces contenidos en la memoria de una computadora? ¿En qué sentido se diferencian? ¿Es ético intentar construir computadoras que se asemejen más a la mente humana?
7. ¿Cree que la popularización de la tecnología de computadoras ha planteado nuevos problemas éticos o que simplemente ha proporcionado un nuevo contexto en el que aplicar las teorías éticas previas?
8. Suponga que el autor de un libro de texto de introducción a las Ciencias de la computación quiere incluir ejemplos de programas para ilustrar los conceptos expuestos en el texto. Sin embargo, en aras de la claridad, muchos de los ejemplos deben ser versiones simplificadas de lo que en la práctica se usaría en un software de calidad profesional. El autor sabe que los ejemplos podrían ser utilizados por un lector inocente y terminar formando parte de una aplicación software importante, en la que lo apropiado sería emplear técnicas más robustas. ¿Qué debería hacer el autor: utilizar los ejemplos simplificados, insistir en poner ejemplos muy robustos aunque eso haga disminuir su valor pedagógico o no incluir ejemplos mientras no pueda obtener a la vez tanto la claridad como la robustez?

Lecturas adicionales

Carrano, F. M. *Data Abstraction and Problem Solving with C++: Walls and Mirrors*, 5ª ed. Boston, MA: Addison-Wesley, 2007.

Carrano, F. M. y J. Prichard. *Data Abstraction and Problem Solving with Java: Walls and Mirrors*, 2ª ed. Boston, MA: Addison-Wesley, 2006.

Gray, S. *Data Structures in Java: From Abstract Data Types to the Java Collections Framework*. Boston, MA: Addison-Wesley, 2007.

Main, M. *Data Structures and Other Objects Using Java*, 3ª ed. Boston, MA: Addison-Wesley, 2006.

Main, M. y W. Savitch. *Data Structures and Other Objects Using C++*, 4ª ed. Boston, MA: Addison-Wesley, 2010.

Shaffer, C. A. *Practical Introduction to Data Structures and Algorithm Analysis*, 2ª ed. Upper Saddle River, NJ: Prentice Hall, 2001.

Weiss, M. A. *Data Structures and Problem Solving Using Java*, 3ª ed. Boston, MA: Addison-Wesley, 2006.

Weiss, M. A. *Data Structures and Algorithm Analysis in C++*, 3ª ed. Boston, MA: Addison-Wesley, 2007.

Weiss, M. A. *Data Structures and Algorithm Analysis in Java*, 2ª ed. Boston, MA: Addison-Wesley, 2007.

Sistemas de bases de datos

Una base de datos es un sistema que convierte un conjunto de datos de gran tamaño en una herramienta abstracta, permitiendo al usuario buscar y extraer elementos pertinentes de información, de una forma cómoda para él. En este capítulo, vamos a explorar este tema, así como hacer incursiones en los campos de la minería de datos, que buscan técnicas para descubrir patrones ocultos en grandes conjuntos de datos, y de las estructuras de archivos tradicionales, que proporcionan muchas de las herramientas que se utilizan en los sistemas actuales de bases de datos y de minería de datos.

9.1 Fundamentos de las bases de datos

La importancia de los sistemas de bases de datos

El papel de los esquemas

Sistemas de gestión de bases de datos

Modelos de bases de datos

9.2 El modelo relacional

Problemas del diseño relacional

Operaciones relacionales

SQL

*9.3 Bases de datos orientadas a objetos

*9.4 Mantenimiento de la integridad de una base de datos

El protocolo de confirmación/anulación

Bloqueo

*9.5 Estructuras de archivo tradicionales

Archivos secuenciales

Archivos indexados

Archivos hash

9.6 Minería de datos

9.7 Impacto social de la tecnología de bases de datos

**Las secciones marcadas con asterisco se sugieren como secciones opcionales.*

La tecnología actual es capaz de almacenar cantidades enormemente grandes de datos, pero esas colecciones son inútiles a menos que seamos capaces de extraer aquellos elementos concretos de información que sean pertinentes para la tarea que tenemos entre manos. En este capítulo vamos a estudiar los sistemas de bases de datos y a aprender cómo esos sistemas aplican la abstracción con el fin de convertir grandes conglomerados de datos en fuentes de información útil. Como tema relacionado, investigaremos el campo de la minería de datos, que se está expandiendo rápidamente y cuyo objetivo consiste en desarrollar técnicas para identificar y explorar patrones dentro de conjuntos de datos. También examinaremos los principios en que se basan las estructuras de archivos tradicionales, que proporcionan el fundamento para los sistemas actuales de bases de datos y de minería de datos.

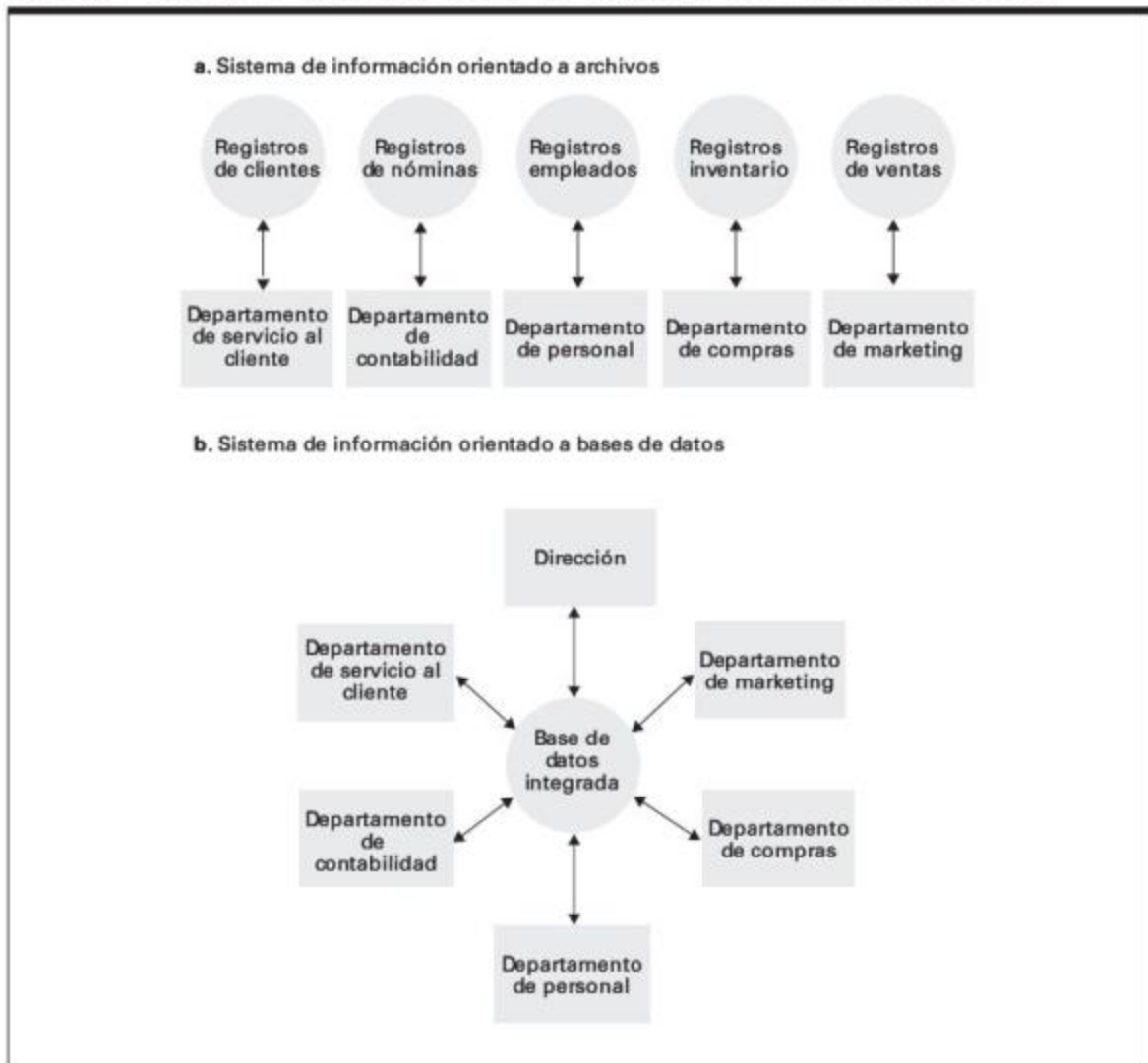
9.1 Fundamentos de las bases de datos

El término **base de datos** hace referencia a un conjunto de datos multidimensional en el sentido de que los enlaces internos existentes entre los distintos elementos hacen que se pueda acceder a la información con diversas perspectivas. Esto difiere de un sistema de archivos tradicional (Sección 9.5), que en ocasiones se denomina **archivo plano**, que no es más que un sistema de almacenamiento unidimensional, lo que quiere decir que presenta la información que contiene desde un único punto de vista. Mientras que un archivo plano que contenga información acerca de compositores de música y sus obras podría proporcionar una lista de obras musicales ordenadas por compositor, una base de datos podría presentar todas las obras de un mismo compositor, todos los compositores que hayan escrito algún tipo concreto de música y quizá también los compositores que hayan escrito variaciones de las obras de algún otro compositor.

La importancia de los sistemas de bases de datos

Históricamente, a medida que las computadoras fueron encontrando usos cada vez más amplios en el campo de la gestión de información, cada aplicación tendía a ser implementada como un sistema separado con su propio conjunto de datos. Las nóminas se procesaban utilizando el archivo de nóminas, el departamento de personal mantenía sus propios registros de empleados y el inventario se gestionaba mediante un archivo de inventario. Esto significaba que buena parte de la información requerida por una organización estaba duplicada por toda la empresa y que muchos elementos diferentes, pero relacionados, se almacenaban en sistemas separados. En este tipo de entorno, los sistemas de bases de datos surgieron como medio de integrar la información almacenada y mantenida por una organización concreta (Figura 9.1). Con este tipo de sistema, los mismos datos de ventas podían utilizarse para generar órdenes de compra de materias primas, generar informes sobre las tendencias del mercado, dirigir los anuncios de productos a los clientes que más probablemente pudieran responder de forma favorable a esa información y generar los bonos de los miembros del equipo comercial.

Estos conjuntos integrados de información proporcionaban un valioso recurso con el que tomar decisiones de gestión, suponiendo que pudiera acce-

Figura 9.1 Comparación entre una organización en archivos y la organización mediante bases de datos.

derse a la información de una manera significativa. A su vez, la investigación en el campo de las bases de datos se centró en el desarrollo de técnicas que permitieran aportar la información contenida en una base de datos al proceso de toma de decisiones. Se han hecho muchos progresos a este respecto. Hoy día, la tecnología de bases de datos, combinada con las técnicas de minería de datos constituye una importante herramienta de gestión, permitiendo a la dirección de una organización extraer información relevante a partir de enormes cantidades de datos que cubren todos los aspectos relativos a la organización y su entorno.

Además, los sistemas de bases de datos se han convertido en la tecnología subyacente que da soporte a muchos de los sitios más populares de la World Wide Web. El objetivo fundamental de sitios como Google, eBay y Amazon es

proporcionar una interfaz entre los clientes y las bases de datos. Para responder a la solicitud de un cliente, el servidor interroga a una base de datos, organiza los resultados en forma de página web y envía dicha página al cliente. Dichas interfaces web han popularizado un nuevo papel para la tecnología de bases de datos, en el que una base de datos ya no es un medio de almacenar los registros de una empresa, sino que se convierte en el propio producto de la empresa. De hecho, al combinar la tecnología de bases de datos con las interfaces web, Internet se ha convertido en una de las principales fuentes de información mundiales.

El papel de los esquemas

Entre las desventajas de la proliferación de tecnologías de bases de datos se encuentra la posibilidad de que personal no autorizado acceda a datos confidenciales. Alguien que está realizando un pedido en el sitio web de una empresa no debe tener acceso a los datos financieros de la empresa. De forma similar, un empleado del departamento de personal de la empresa puede necesitar acceder a los registros de empleados, pero no debería acceder a los registros de inventario o de ventas. Por tanto, la capacidad de controlar el acceso a la información contenida en la base de datos es tan importante como la capacidad de compartir esa información.

Para proporcionar a los diferentes usuarios acceso a diferentes tipos de informaciones dentro de una base de datos, los sistemas de bases de datos suelen utilizar lo que se denomina esquemas y subesquemas. Un **esquema** es una descripción de toda la estructura de la base de datos; el software de la base de datos utiliza esa descripción para mantener la base de datos. Un **subesquema** es una descripción de aquella parte de la base de datos que es relevante para las necesidades de un usuario concreto. Por ejemplo, un esquema para una base de datos de una universidad indicaría que cada registro de estudiante contiene elementos tales como la dirección y el teléfono actuales de dicho estudiante, además de su historial académico. Asimismo, indicaría que el registro de cada estudiante está enlazado con el registro de su tutor. A su vez, el registro de cada profesor de la universidad contendría la dirección de esa persona, el historial laboral, etc. Basándose en este esquema, se mantendría un sistema de enlaces que permitiera en último término conectar la información relativa a un estudiante con el historial laboral de un profesor.

Para impedir que el administrativo encargado de las matrículas de los estudiantes de la universidad utilice este sistema de enlaces con el fin de obtener información privilegiada acerca de los profesores, el acceso de ese administrativo a la base de datos debe restringirse a un subesquema cuya descripción de los registros de los profesores no incluya el historial laboral. Con este subesquema, el administrativo podría averiguar qué profesor es el tutor de un estudiante concreto, pero no podría obtener acceso a información adicional acerca del profesor. Por el contrario, el subesquema correspondiente al departamento de contabilidad proporcionaría el historial laboral de cada profesor, pero no incluiría la relación entre estudiantes y tutores. Por tanto, el departamento de contabilidad podría modificar el salario de un profesor pero no podría obtener los nombres de los estudiantes de los que es tutor.

Sistemas de gestión de bases de datos

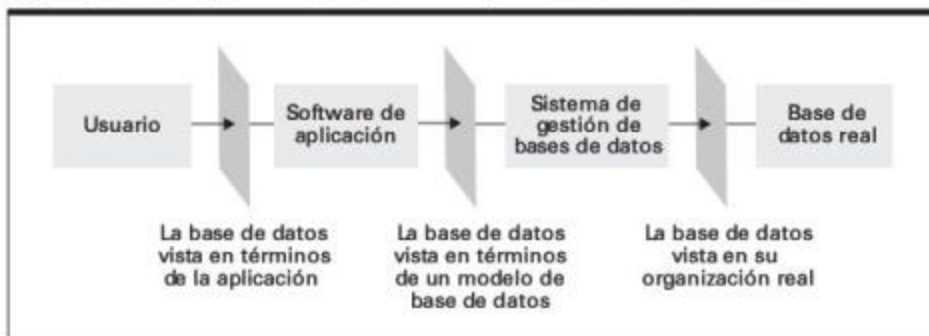
Una aplicación de bases de datos típica consta de varias capas de software, que agruparemos en dos capas principales: una capa de aplicación y una capa de gestión de base de datos (Figura 9.2). El software de aplicación gestiona la comunicación con el usuario de la base de datos y puede ser bastante complejo, como ilustran las aplicaciones en las que los usuarios acceden a una base de datos por medio de un sitio web. En tal caso, la capa completa de aplicación está compuesta por clientes dispersos por Internet y por un servidor que utiliza la base de datos con el fin de satisfacer las solicitudes de los clientes.

Observe que el software de aplicación no manipula directamente la base de datos. La manipulación real de la misma se lleva a cabo mediante el **sistema de gestión de base de datos (DBMS, Database Management System)**. Una vez que el software de aplicación ha determinado qué acción está solicitando el usuario, utiliza el DBMS como herramienta abstracta para obtener el resultado. Si la solicitud consiste en añadir o borrar datos, será el DBMS el que modifique en la práctica la base de datos. Si la solicitud consiste en extraer información, será el DBMS el que lleve a cabo las búsquedas requeridas.

Esta dicotomía entre el software de aplicación y el DBMS tiene varias ventajas. Una de ellas es que permite la construcción y uso de herramientas abstractas que, como ya hemos visto, constituyen uno de los principales conceptos simplificadores en el campo del diseño de software. Si se aíslan dentro del DBMS los detalles relativos a cómo está realmente almacenada la base de datos, el diseño del software de aplicación puede simplificarse enormemente. Por ejemplo, con un DBMS bien diseñado, el software de aplicación no tiene que preocuparse de si la base de datos está almacenada en una única máquina o está dispersa entre muchas máquinas conectadas en red, como **base de datos distribuida**. En lugar de ello, será el DBMS el que trate con estos problemas, permitiendo que el software de aplicación acceda a la base de datos sin preocuparse por dónde están realmente almacenados los datos.

Un segunda ventaja de separar el software de aplicación del DBMS es que este tipo de organización proporciona un medio de controlar el acceso a la base de datos. Obligando a que sea el DBMS el que realice todos los accesos a la base de datos, puede imponer las restricciones definidas por los distintos subesquemas. En particular, el DBMS puede utilizar el esquema completo de la base

Figura 9.2 Las capas conceptuales de una implementación de base de datos.



Bases de datos distribuidas

Con los avances relativos a las capacidades de red, los sistemas de bases de datos han crecido hasta abarcar una serie de bases de datos que se conocen con el nombre de bases de datos distribuidas y que consisten en datos que residen en diversas máquinas simultáneamente. Por ejemplo, una corporación internacional podría almacenar y mantener los registros de los empleados locales en los sitios locales, pero enlazar dichos registros por medio de una red, con el fin de crear una única base de datos distribuida.

Una base de datos distribuida puede contener datos fragmentados y/o replicados. El primer caso está ilustrado por el ejemplo anterior relativo a los registros de empleados, en el que diferentes fragmentos de la base de datos se almacenaban en diferentes aplicaciones. En el segundo caso, se almacenan en diferentes ubicaciones una serie de duplicados del mismo componente de base de datos. Esa replicación puede realizarse con el fin de reducir el tiempo necesario para extraer la información. Ambos casos plantean problemas que no están presentes en las bases de datos centralizadas más tradicionales: por ejemplo, cómo ocultar la naturaleza distribuida de la base de datos para que funcione como un sistema coherente o cómo garantizar que las partes replicadas de una base de datos continúen siendo copias exactas unas de otras a medida que se producen las actualizaciones. Además, el estudio de las bases de distribuidas es una de las áreas de investigación más activas en la actualidad.

de datos para sus necesidades internas, pero exigir que el software de aplicación empleado por cada usuario permanezca dentro de los límites descritos por el subesquema correspondiente a ese usuario.

Otra razón más para separar la interfaz de usuario y las tareas reales de manipulación de los datos en dos capas diferentes de software es la de conseguir la denominada **independencia de los datos**, la capacidad de modificar la organización de la propia base de datos sin tener que modificar el software de aplicación. Por ejemplo, el departamento de personal puede tener que agregar un campo al registro de cada empleado para indicar si ese empleado ha decidido participar en el nuevo programa de cobertura sanitaria de la empresa. Si el software de aplicación tratara directamente con la base de datos, ese cambio en el formato de los datos podría requerir que se hicieran modificaciones en todos los programas de aplicación que traten con la base de datos. Como resultado, la modificación impuesta por el departamento de personal podría provocar cambios tanto en el programa de nóminas como por ejemplo en el programa de impresión de etiquetas de correo para el boletín mensual de la empresa.

Esa separación entre el software de aplicación y un DBMS elimina la necesidad de modificar los programas. Para implementar un cambio en la base de datos requerido por un único usuario, lo único que hace falta es modificar el esquema global y los subesquemas de aquellos usuarios que se vean afectados por el cambio. Los subesquemas de los restantes usuarios seguirán siendo iguales, así que su software de aplicación, que está basado en esos subesquemas no modificados, no tendrá que ser alterado.

Modelos de bases de datos

Ya hemos visto repetidamente cómo se puede utilizar la abstracción para ocultar la complejidad interna de un componente. Los sistemas de gestión de bases de datos proporcionan un ejemplo más de este concepto. Estos sistemas ocultan la complejidad de la estructura interna de una base de datos, permitiendo al usuario de la misma imaginarse que la información almacenada en la base de datos está organizada con un formato más útil. En particular, un DBMS contiene rutinas que traducen los comandos expresados en términos de una vista conceptual de la base de datos a las acciones requeridas por el sistema real de almacenamiento de los datos. Esta vista conceptual de la base de datos se conoce como **modelo de base de datos**.

En las siguientes secciones consideraremos tanto el modelo de base datos relacional como el modelo de base de datos orientado a objetos. En el caso del modelo de base de datos relacional, la vista conceptual de la base de datos es la de una colección de tablas compuestas de filas y columnas. Por ejemplo, la información acerca de los empleados de una empresa puede visualizarse como una tabla que contiene una fila por empleado y una serie de columnas etiquetadas como nombre, dirección, número de identificación del empleado, etc. A su vez, el DBMS dispondrá de rutinas que permitirán al software de aplicación seleccionar ciertas entradas de una fila concreta de la tabla o quizá informar del rango de valores existente en la columna de salario (aún cuando la información no esté en realidad almacenada en filas y columnas).

Estas rutinas forman las herramientas abstractas utilizadas por el software de aplicación a la hora de acceder a la base de datos. Para ser más precisos, el software de aplicación suele escribirse en uno de los lenguajes de programación de propósito general existentes, como los presentados en el Capítulo 6. Estos lenguajes proporcionan los ingredientes básicos para las expresiones algorítmicas, pero carecen de instrucciones para manipular una base de datos. Sin embargo, un programa escrito en uno de esos lenguajes puede utilizar las rutinas proporcionadas por el DBMS en forma de subrutinas pre-escritas, lo que tiene el efecto en la práctica de ampliar las capacidades del lenguaje de forma tal que permite dar soporte a la imagen conceptual del modelo de base de datos.

La búsqueda de mejores modelos de bases de datos continúa siendo un campo de investigación activo. El objetivo es encontrar modelos que permitan conceptualizar fácilmente sistemas de datos complejos, que conduzcan a formas concisas de expresar las solicitudes de información y que produzcan sistemas de gestión de bases de datos eficientes.

Cuestiones y ejercicios

1. Identifique dos departamentos de una planta de fabricación que pudieran tener diferentes usos para la misma información de inventario o para una información similar. A continuación, describa cómo podría diferir el subesquema para los dos departamentos.
2. ¿Cuál es el propósito de un modelo de base de datos?
3. Resuma los papeles del software de aplicación y de un DBMS.

9.2 El modelo relacional

En esta sección vamos a examinar más de cerca del modelo de base de datos relacional. Este modelo representa los datos como si estuvieran almacenados en tablas rectangulares, denominadas **relaciones**, que son similares al formato en el que se visualiza la información en los programas de hoja de cálculo. Por ejemplo, el modelo relacional permite representar la información relativa a los empleados de una empresa mediante una relación como la mostrada en la Figura 9.3.

Una fila de una relación se denomina **tupla**. En la relación de la Figura 9.3, las tuplas están compuestas por la información acerca de un empleado concreto. Las columnas de una relación se denominan **atributos** porque cada entrada de una columna describe alguna característica, o atributo, de la entidad representada por la correspondiente tupla.

Problemas del diseño relacional

Un paso fundamental a la hora de diseñar una base de datos relacional es diseñar las relaciones que forman la base de datos. Aunque esto puede parecer una tarea simple, hay muchos aspectos sutiles esperando a atrapar a los diseñadores incautos.

Suponga que además de la información contenida en la relación de la Figura 9.3, queremos incluir información acerca de los puestos de trabajo que ocupan los empleados. Podríamos desear incluir un historial laboral asociado con cada empleado que constara de atributos tales como el puesto ocupado (secretaria, director de oficina, supervisor de planta), un código de identificación del puesto (que diferencia unos puestos de otros), el código de la categoría asociado a cada puesto, el departamento al que pertenece ese puesto y el periodo durante el que el empleado ha ocupado el puesto en términos de una fecha de inicio y una fecha de terminación. (Utilizaremos un asterisco como fecha de terminación si se trata del puesto actual del empleado.)

Una solución a este problema consistiría en ampliar la relación de la Figura 9.3 para incluir estos atributos como columnas adicionales de la tabla, como se muestra en la Figura 9.4. Sin embargo, un examen más atento del resultado permite detectar varios problemas. El primero es una falta de eficiencia debido a la redundancia. La relación ya no contiene solo una tupla por cada empleado, sino una tupla por cada asignación de un puesto de trabajo o de un empleado. Si un empleado ha ido progresando en la empresa a través de una secuencia de varios puestos de trabajo, varias tuplas de la nueva relación deberán contener la

Figura 9.3 Una relación que contiene información sobre los empleados.

Id Empl	Nombre	Direccion	NSS
25X15	Juan Barrio	San Juan 33	111223333
34Y70	Charo Garrido	Av Principal 563	999009999
23Y34	Gervasio Salas	Dr. Circo 1555	111005555
.	.	.	.
.	.	.	.
.	.	.	.

misma información acerca del empleado (nombre, dirección, número de identificación y número de la Seguridad Social). Por ejemplo, la información personal acerca de los empleados Barrio y Salas está repetida porque han ocupado más de un puesto. Además, cuando un puesto concreto ha sido ocupado por numerosos empleados, es preciso repetir en cada tupla que represente una asignación de ese puesto de trabajo tanto la información correspondiente al departamento asociado con ese puesto como el código de categoría apropiado. Por ejemplo, la descripción del puesto de director de planta está duplicada porque hay más de un empleado que ha ocupado este puesto.

Otro problema quizá más serio de esta relación ampliada es el que surge cuando consideramos el problema de borrar información de la base de datos. Por ejemplo, suponga que Juan Barrio es el único empleado que tiene el puesto identificado como D7. Si abandonara la empresa y lo borráramos de la base de datos representada en la Figura 9.4, perderíamos la información relativa al puesto de trabajo D7. Después de todo, la única tupla que contiene el hecho de que el puesto D7 requiere una categoría K2 es la tupla relacionada con Juan Barrio.

Podría argumentarse que la capacidad de borrar solamente una parte de una tupla resolvería el problema, pero esto introduciría otras complicaciones. Por ejemplo, ¿deberíamos también conservar la información relativa al puesto de trabajo F5 en una tupla parcial, o esta información ya se encuentra en algún otro lugar dentro de la relación? Además, la tentación de usar tuplas parciales es un sólido indicio de que puede mejorarse el diseño de la base de datos.

La causa de todos estos problemas es que hemos combinado más de un concepto en una misma relación. Tal como está propuesta, la relación ampliada de la Figura 9.4 contiene información que trata directamente con los empleados (nombre, número de identificación, dirección, número de la Seguridad Social), información acerca de los puestos disponibles en la empresa (identificación del puesto, título del puesto, departamento y código de categoría) e información relativa a la relación entre los empleados y los puestos de trabajo (fecha de inicio y fecha de terminación). Teniendo en cuenta esta

Figura 9.4 Una relación que contiene redundancia.

[illegible]

Sistemas de bases de datos para PC

Las computadoras personales se utilizan en una amplia variedad de aplicaciones, que van desde las más elementales a las muy sofisticadas. En las aplicaciones de "bases de datos" elementales, como por ejemplo almacenar la lista de felicitaciones de Navidad o mantener los registros de la liga de fútbol del colegio, se utilizan a menudo sistemas de hoja de cálculo en lugar de software de base de datos, ya que la aplicación requiere poco más que la capacidad de almacenar, imprimir y ordenar los datos. Sin embargo, existen verdaderos sistemas de bases de datos para el mercado del PC, siendo uno de ellos Microsoft Access. Es un sistema de base de datos relacional completo, tal como se describe en la Sección 9.2, junto con un software de generación de diagramas e informes. Access proporciona un ejemplo excelente de cómo los principios presentados en este texto forman la base de productos muy populares en el mercado actual.

observación podemos resolver nuestros problemas rediseñando el sistema con tres relaciones: una para cada una de las categorías anteriores. Podemos mantener la relación original de la Figura 9.3 (que ahora denominaremos relación EMPLEADO) e insertar la información adicional mediante dos nuevas relaciones denominadas PUESTO y ASI GNACI ON, lo que nos da la base de datos de la Figura 9.5.

Una base de datos compuesta por estas tres relaciones contiene la información pertinente acerca de los empleados en la relación EMPLEADO, la información sobre los puestos disponibles en la relación PUESTO y sobre el historial laboral en la relación ASI GNACI ON. La información adicional está disponible implícitamente combinando la información procedente de distintas relaciones. Por ejemplo, si conocemos el número de identificación de un empleado, podemos conocer los departamentos en que ese empleado ha trabajado, localizando primero todos los puestos que el empleado ha ocupado mediante la relación ASI GNACI ON y localizando después los departamentos asociados con esos puestos, por medio de la relación PUESTO (Figura 9.6). Mediante procesos como estos, cualquier información que pudiera obtenerse a partir de esa única relación de gran tamaño podrá ahora obtenerse a partir de las tres relaciones más pequeñas y sin los problemas citados.

Lamentablemente, dividir la información en varias relaciones no siempre es tan sencillo como en el ejemplo anterior. Por ejemplo, compare la relación original de la Figura 9.7 con los atributos *IdEmpl*, *Ti tPuesto* y *Dept*, con la descomposición propuesta en dos relaciones. A primera vista, el sistema de dos relaciones parece contener la misma información que el sistema de una sola relación, pero de hecho no es así. Considere por ejemplo el problema de localizar el departamento en el que trabaja un cierto empleado. Esto puede hacerse fácilmente en el sistema de una sola relación, consultando la tupla que contiene el número de identificación del empleado deseado y extrayendo el departamento correspondiente. Sin embargo, en el sistema de dos relaciones, la información deseada no está disponible necesariamente. Podemos encontrar el título del puesto del empleado deseado y un departamento que tenga dicho puesto, pero esto no implica necesariamente que el empleado buscado trabaje en ese departamento concreto, porque podría haber varios departamentos con puestos que tuvieran el mismo título.

Figura 9.5 Una base de datos de empleados compuesta por tres relaciones.

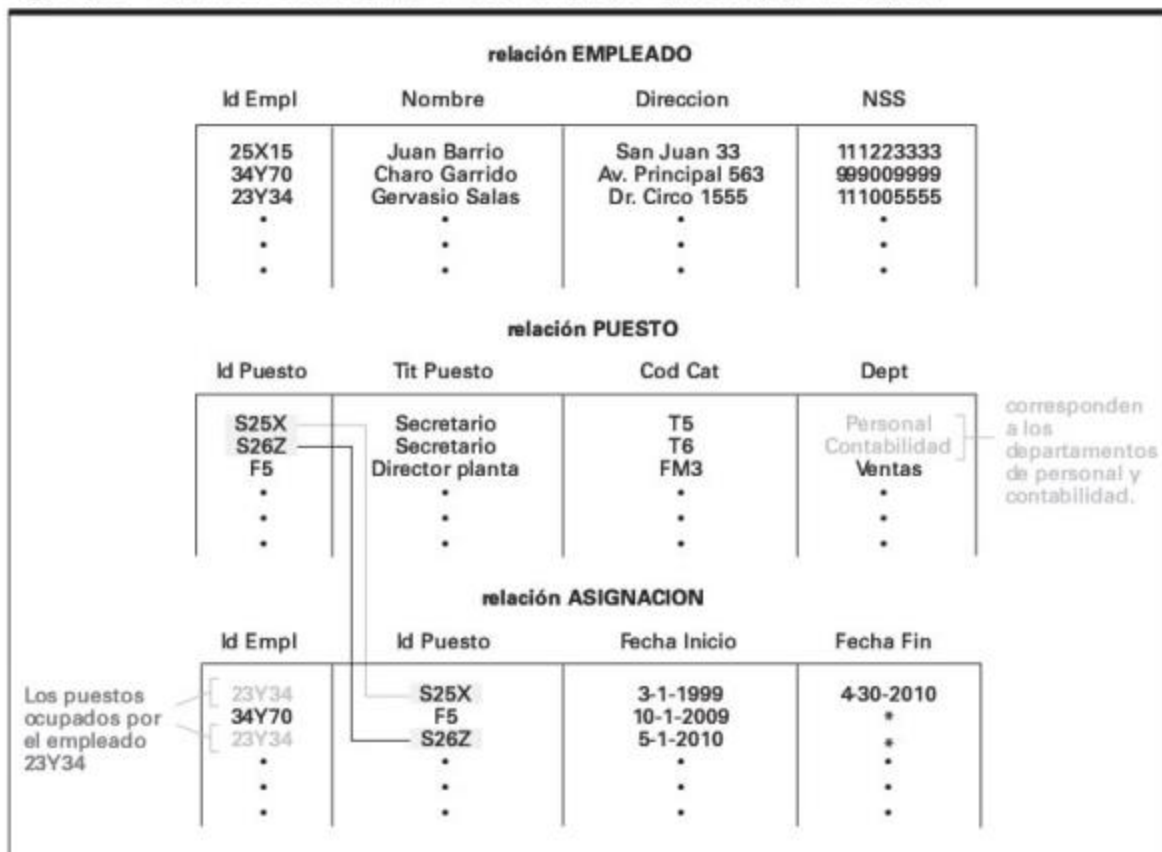
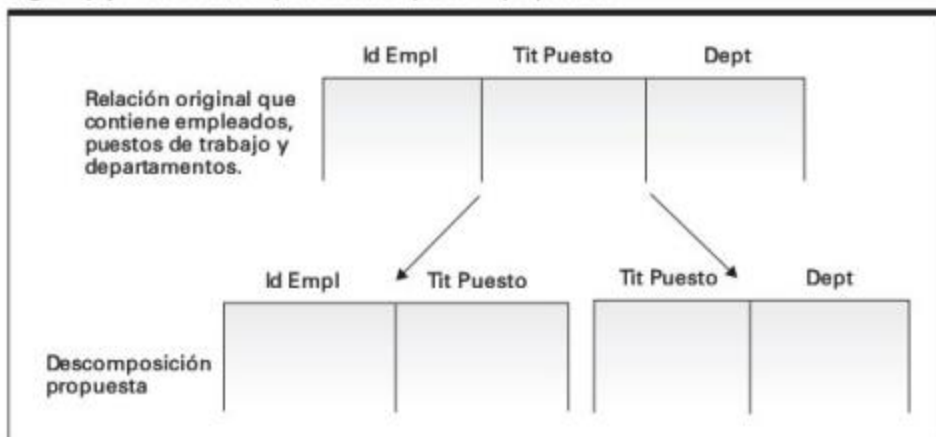
relación EMPLEADO			
Id Empl	Nombre	Dirección	NSS
25X15	Juan Barrio	San Juan 33	111223333
34Y70	Charo Garrido	Av. Principal 563	999009999
23Y34	Gervasio Salas	Dr. Circo 1555	111005555
relación PUESTO			
Id Puesto	Tit Puesto	Cod Cat	Dept
S25X	Secretario	T5	Personal
S26Z	Secretario	T6	Contabilidad
F5	Director planta	FM3	Ventas
.	.	.	.
.	.	.	.
.	.	.	.
relación ASIGNACION			
Id Empl	Id Puesto	Fecha Inicio	Fecha Fin
23Y34	S25X	1-3-1999	30-4-2010
34Y70	F5	1-10-2009	*
23Y34	S26Z	1-5-2010	*
.	.	.	.
.	.	.	.
.	.	.	.

Vemos, por tanto, que en ocasiones dividir una relación en relaciones más pequeñas provoca la pérdida de información, mientras que en otras ocasiones no es así (este último caso se denomina **descomposición sin pérdidas**). Estas características relacionales son muy importantes a la hora de acometer un diseño. El objetivo es identificar las características relacionales que pueden conducir a problemas en el diseño de bases de datos y encontrar formas de reorganizar dichas relaciones para eliminar esas características problemáticas.

Operaciones relacionales

Ahora que tenemos una comprensión básica de cómo pueden organizarse los datos en términos del modelo relacional, es el momento de ver cómo puede extraerse información de una base de datos compuesta por relaciones. Comenzaremos examinando algunas operaciones que podríamos querer llevar a cabo con relaciones.

En ocasiones, necesitamos seleccionar ciertas tuplas de una relación. Para extraer la información acerca de un empleado, tendremos que seleccionar la tupla de la relación EMPLEADO, que tenga el valor apropiado en el atributo correspondiente del número de identificación del empleado. O bien, para obtener una lista de los puestos de trabajo de un cierto departamento, tendremos

Figura 9.6 Localización de los departamentos en los que el empleado 23Y34 ha trabajado.**Figura 9.7** Una relación y una descomposición propuesta.

que seleccionar las tuplas de la relación PUESTO que tengan dicho departamento como atributo de departamento. El resultado de este proceso es otra relación compuesta por las tuplas seleccionadas en la relación padre. El resul-

tado de seleccionar información acerca de un empleado concreto nos dará una relación que contendrá una única tupla extraída de la relación EMPLEADO. El resultado de seleccionar las tuplas asociadas con un cierto departamento nos dará como resultado una relación que probablemente contendrá varias tuplas de la relación PUESTO.

En resumen, una operación que podemos desear llevar a cabo con una relación consiste en seleccionar tuplas que posean ciertas características y colocar dichas tuplas seleccionadas en una nueva relación. Para expresar esta operación, adoptaremos la sintaxis

NUEVA ← SELECT from EMPLEADO where IdEmpl = "34Y70"

La semántica de esta sentencia consiste en crear una nueva relación denominada NUEVA que contenga las tuplas (solo debería haber una en este caso) de la relación EMPLEADO cuyo atributo IdEmpl sea igual a 34Y70 (Figura 9.8).

A diferencia de la operación SELECT, que extrae filas de una relación, la operación PROJECT extrae columnas. Por ejemplo, suponga que al buscar los títulos de puestos de un cierto departamento, ya hemos seleccionado las tuplas de la relación PUESTO correspondientes al departamento objetivo y que hemos colocado esas tuplas en una relación denominada NUEVA1. La lista que estaremos buscando será la columna TitPuesto dentro de esta nueva relación. La operación PROJECT nos permite extraer esta columna (o varias columnas si hiciera falta) y colocar el resultado en una nueva relación. Podemos expresar ese tipo de operación mediante

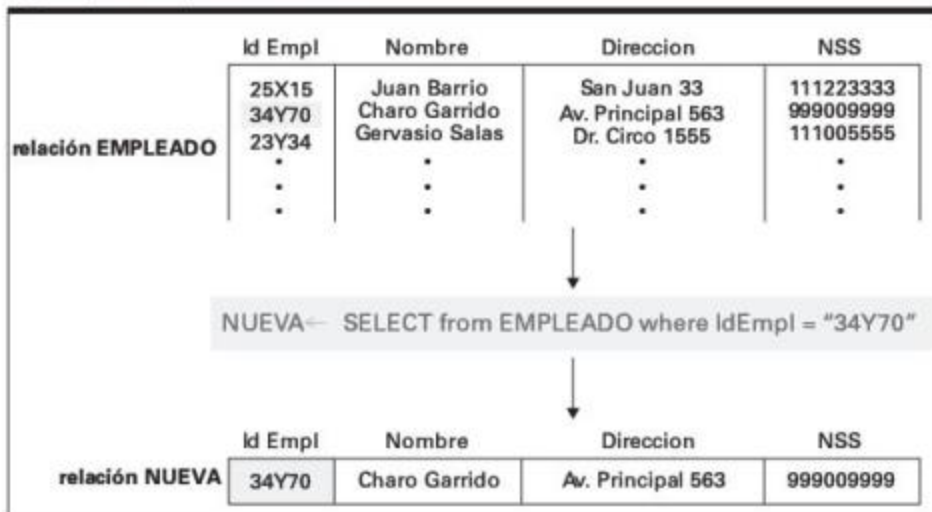
NUEVA2 ← PROJECT TitPuesto from NUEVA1

El resultado es la creación de una nueva relación (denominada NUEVA2) que contiene una única columna de valores, correspondientes a la columna TitPuesto de la relación NUEVA1.

Veamos otro ejemplo de la operación PROJECT. La sentencia

CORREO ← PROJECT Nombre, Direccion from EMPLEADO

Figura 9.8 La operación SELECT.



puede utilizarse para obtener un listado de los nombres y direcciones de todos los empleados. Esta lista se almacenará en la nueva relación (de dos columnas) denominada CORREO (Figura 9.9).

Otra operación que se utiliza en conjunción con las bases de datos relacionales es la operación JOIN. Se usa para combinar diferentes relaciones en una única relación. El resultado de combinar mediante JOIN dos relaciones produce una nueva relación cuyos atributos son los atributos de las relaciones originales (Figura 9.10). Los nombres de esos atributos serán los mismos que los de las relaciones originales, salvo porque cada uno tendrá como prefijo la relación original de la que procede. (Si la relación A que tiene los atributos V y W se combina mediante JOIN con la relación B, que contiene los atributos X, Y y Z, entonces el resultado tendrá cinco atributos denominados A.V, A.W, B.X, B.Y y B.Z.) Este convenio de denominación garantiza que los atributos de la nueva relación tenga nombres distintivos, incluso aunque las relaciones originales pudieran tener nombres de atributos en común.

Las tuplas (filas) de la nueva relación se generan concatenando las tuplas de las dos relaciones originales (véase de nuevo la Figura 9.10). Para determinar qué tuplas se combinan realmente, con el fin de formar tuplas de la nueva relación, se añade una condición para la aplicación de JOIN. Una de las posibles condiciones es que determinados atributos designados tengan el mismo valor. De hecho, este es el caso representado en la Figura 9.10, en el que se ilustra el resultado de ejecutar la sentencia

$C \leftarrow \text{JOIN } A \text{ and } B \text{ where } A.W = B.X$

En este ejemplo habrá que concatenar una tupla de la relación A con una tupla de la relación B únicamente en aquellos casos en los que los atributos W y X de las dos tuplas sean iguales. Por tanto, la concatenación de la tupla (r, 2) de la

Figura 9.9 La operación PROJECT.

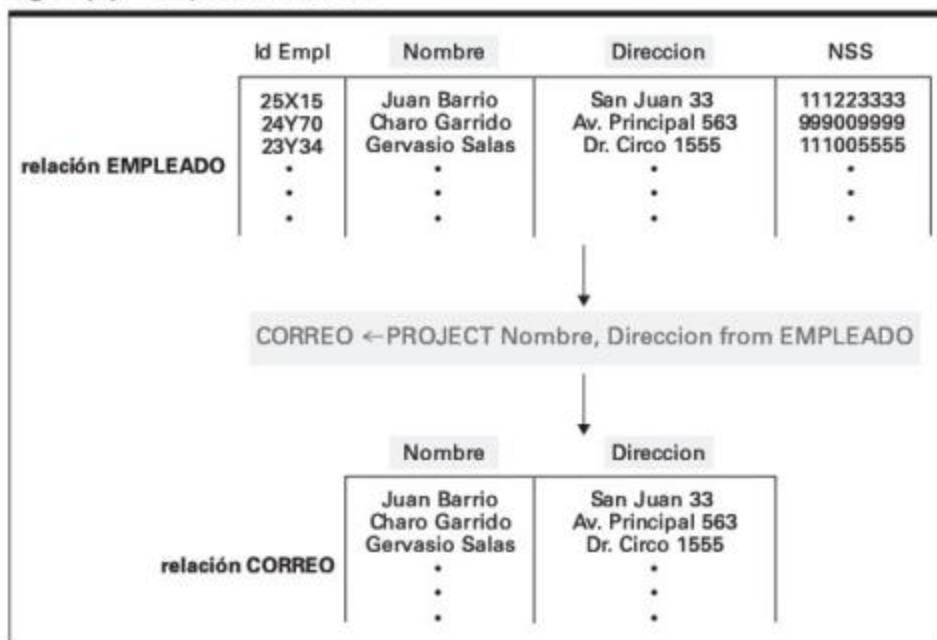
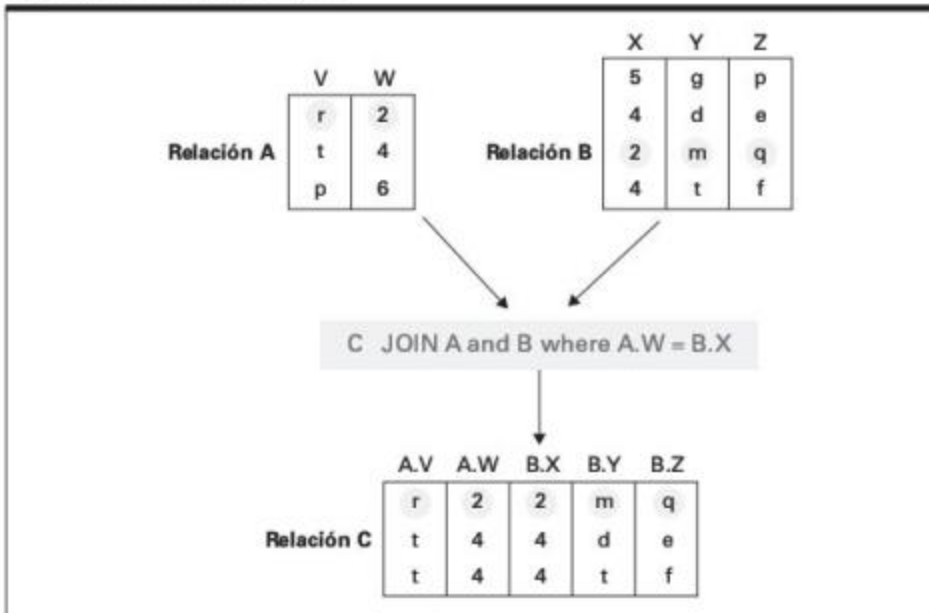


Figura 9.10 La operación JOIN.



relación A con la tupla (2, m, q) de la relación B aparece en el resultado, porque el valor del atributo W de la primera es igual al valor del atributo X en la segunda. Por el contrario, el resultado de concatenar la tupla (r, 2) de la relación A con la tupla (5, g, p) de la relación B no aparece en la relación final, porque estas tuplas no comparten valores comunes en los atributos W y X.

Veamos otro ejemplo: la Figura 9.11 representa el resultado de ejecutar la sentencia

$C \leftarrow \text{JOIN } A \text{ and } B \text{ where } A.W < B.X$

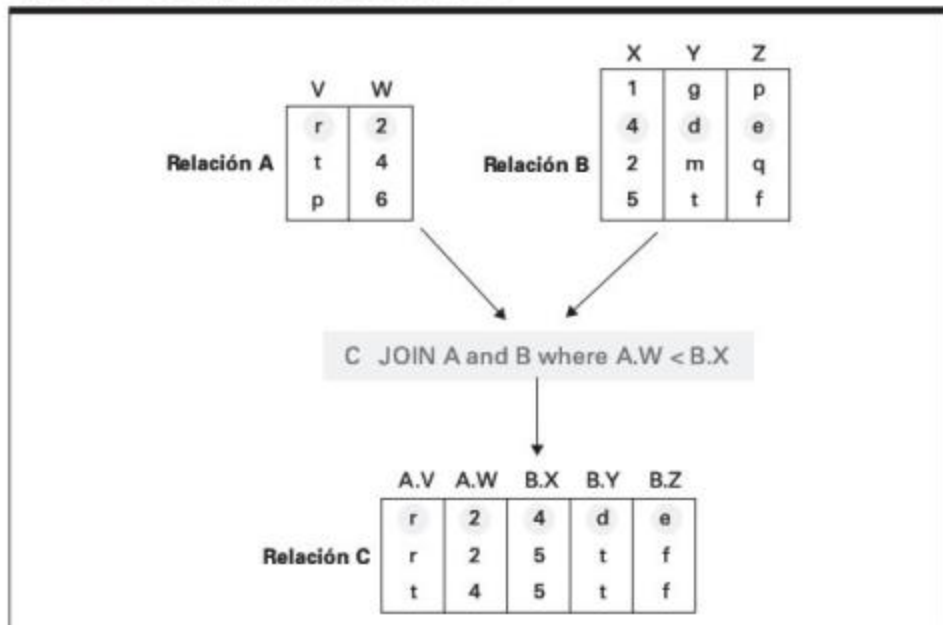
Observe que las tuplas del resultado son exactamente aquellas en las que el atributo W de la relación A es menor que el atributo X de la relación B.

Veamos ahora cómo puede utilizarse la operación JOIN con la base de datos de la Figura 9.5 para obtener un listado de todos los números de identificación de los empleados junto con el departamento en el que trabaja cada empleado. La primera observación que haremos es que la información requerida está distribuida entre más de una relación, por lo que el proceso de extraer la información requerirá algo más que selecciones y proyecciones. De hecho, la herramienta que necesitamos es la sentencia

$\text{NUEVA1} \leftarrow \text{JOIN ASIGNACION and PUESTO}$
 $\text{where ASIGNACION.IdPuesto} = \text{PUESTO.IdPuesto}$

que genera la relación NUEVA1, como se muestra en la Figura 9.12. A partir de esta relación, nuestro problema puede resolverse seleccionando primero aquellas tuplas en las que ASIGNACION.FechaFin sea igual a "" (lo que indica que es el puesto actual del empleado) y luego proyectando los atributos ASIGNACION.IdEmpl y PUESTO.Dept. En resumen, la información que necesitamos puede obtenerse a partir de la base de datos de la Figura 9.5 ejecutando la secuencia

Figura 9.11 Otro ejemplo de la operación JOIN.



```

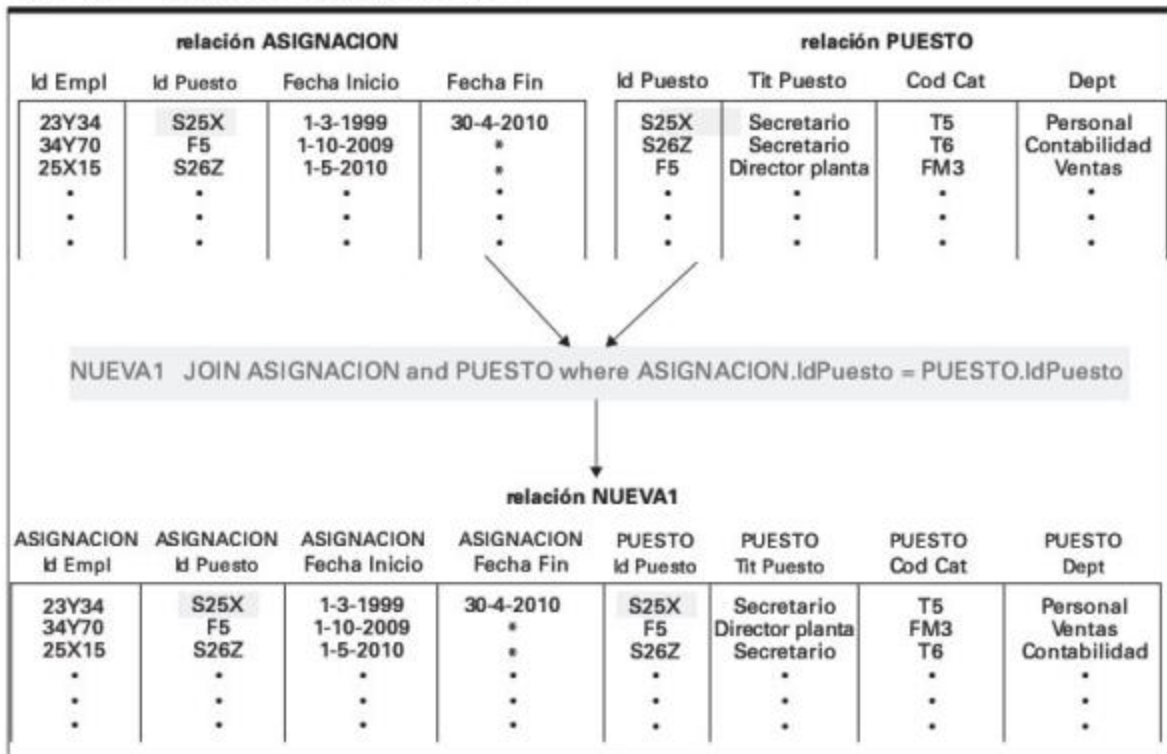
NUEVA1 ← JOIN ASIGNACION and PUESTO
           where ASIGNACION.IdPuesto = PUESTO.IdPuesto
NUEVA2 ← SELECT from NUEVA1 where ASIGNACION.FechaFin = "*"
LIST ← PROJECT ASIGNACION.IdEmpl, PUESTO.Dept from NUEVA2
  
```

SQL

Ahora que hemos presentado las operaciones relacionales básicas, reconsideremos la estructura global de un sistema de base de datos. Recuerde que una base de datos se almacena en realidad en un sistema de almacenamiento masivo. Para evitar que el programador de aplicaciones tenga que conocer los detalles de dichos sistemas, se proporciona un sistema de gestión de bases de datos que permite escribir el software de aplicación en términos de un modelo de base de datos, tal como el modelo relacional. El DBMS acepta comandos en términos del modelo elegido y los convierte en acciones relativas a la estructura de almacenamiento real. Esta conversión se gestiona mediante un conjunto de rutinas dentro del DBMS, que son utilizadas como herramientas abstractas por el software de aplicación. Así, un DBMS basado en el modelo relacional incluiría rutinas para realizar las operaciones SELECT, PROJECT y JOIN, que podrían ser invocadas desde el software de aplicación. De esta forma, el software de aplicación podría escribirse como si la base de datos estuviera realmente almacenada en la forma tabular simple del modelo relacional.

Los sistemas de gestión de bases de datos relacionales actuales no necesariamente proporcionan rutinas para realizar las operaciones SELECT, PROJECT y JOIN en su forma básica. En su lugar, proporcionan rutinas que pueden ser combinaciones de estos pasos básicos. Un ejemplo es el lenguaje SQL (*Structured Query Language*, Lenguaje de consulta estructurado), que forma la base

Figura 9.12 Una aplicación de la operación JOIN.



de la mayoría de los sistemas de consulta de bases de datos relacionales. Por ejemplo, SQL es el lenguaje subyacente en el sistema de bases de datos relacional MySQL utilizado por muchos servidores de bases de datos en Internet.

Una de las razones de la popularidad de SQL es que el Instituto ANSI lo ha estandarizado. Otra razón es que originalmente fue desarrollado y comercializado por IBM y se ha aprovechado así de muchos años de experiencia de uso. En esta sección explicamos cómo se expresan en SQL las consultas a una base de datos relacional.

Aunque vamos a ver que una consulta en SQL se expresa en una forma más bien imperativa, la realidad es que es esencialmente una sentencia declarativa. Trate de leer las sentencias SQL como descripciones de la información deseada más que como secuencias de las actividades que hay que realizar. La importancia de este aspecto es que SQL ahorra a los programadores de aplicaciones el engorro de desarrollar algoritmos para manipular relaciones, los programadores simplemente tienen que describir la información deseada.

Como primer ejemplo de sentencia SQL, reconsideremos nuestra última consulta en la que hemos desarrollado un proceso en tres pasos para obtener todos los números de identificación de los empleados junto con sus correspondientes departamentos. En SQL, esta consulta completa podría representarse mediante la única sentencia

```
select IdEmpl, Dept
from ASIGNACION, PUESTO
```

```

where ASIGNACION.IdPuesto = PUESTO.IdPuesto
and ASIGNACION.FechaFin = '*'

```

Como indica este ejemplo, cada sentencia de consulta SQL puede contener tres cláusulas: una cláusula `select`, una cláusula `from` y una cláusula `where`. En términos generales, lo que dicha sentencia está haciendo es solicitar el resultado de combinar mediante `JOIN` todas las relaciones enumeradas en la cláusula `from`, seleccionar mediante `SELECT` aquellas tuplas que satisfagan las condiciones indicadas en la cláusula `where` y luego proyectar mediante `PROJECT` aquellas tuplas enumeradas en la cláusula `select`. (Observe que la terminología está en cierto modo invertida, ya que la cláusula `select` de una sentencia SQL identifica los atributos utilizados en la operación `PROJECT`.) Veamos algunos ejemplos sencillos.

La sentencia

```

select Nombre, Direccion
from EMPLEADO

```

genera un listado de todos los nombres y direcciones de los empleados contenidos en la relación `EMPLEADO`. Observe que esto es simplemente una operación `PROJECT`.

La sentencia

```

select EmplId, Nombre, Direccion, NumSSN
from EMPLEADO
where Nombre = 'Charo Garrido'

```

genera toda la información de la tupla asociada con Charo Garrido en la relación `EMPLEADO`. Esto es esencialmente una operación `SELECT`.

La sentencia

```

select Nombre, Direccion
from EMPLEADO
where Nombre = 'Charo Garrido'

```

genera el nombre y la dirección de Charo Garrido contenidos en la relación `EMPLEADO`. Esto es una combinación de las operaciones `SELECT` y `PROJECT`.

La sentencia

```

select EMPLEADO.Nombre, ASIGNACION.FechaInicio
from EMPLEADO, ASIGNACION
where EMPLEADO.EmplId = ASIGNACION.IdEmpl

```

genera un listado de todos los nombres de empleado y las fechas en las que comenzaron a trabajar en ese puesto. Observe que esto es el resultado de combinar mediante `JOIN` las relaciones `EMPLEADO` y `ASIGNACION` y luego seleccionar mediante `SELECT` y `PROJECT` las tuplas y atributos apropiados, identificados en las cláusulas `where` y `select`.

Cerramos esta sección resaltando que SQL dispone de sentencias para definir la estructura de las relaciones, crear relaciones y modificar el contenido de las relaciones, además de para realizar consultas. A continuación proporcionamos algunos ejemplos de las sentencias `insert into`, `delete from` y `update`.

La sentencia

```
insert into EMPLEADO
values ('42Z12', 'Susana Burgos', 'San Pedro 33',
       '444661111')
```

añade una tupla a la relación EMPLEADO que contiene los valores indicados;

```
delete from EMPLEADO
where Name = 'Gervasio Salas'
```

borra de la relación EMPLEADO la tupla relativa a Gervasio Salas y

```
update EMPLEADO
set Direccion = 'Av. Secundaria 1812.'
where Nombre = 'Juan Barrio'
```

modifica la dirección de la tupla asociada con Juan Barrio en la relación EMPLEADO.

Cuestiones y ejercicios

1. Responda a las siguientes cuestiones basándose en la información parcial proporcionada en las relaciones EMPLEADO, PUESTO y ASIGNACION de la Figura 9.5:
 - a. ¿Quién es el secretario del departamento de contabilidad que tiene experiencia en el departamento de personal?
 - b. ¿Quién es el director de planta en el departamento de ventas?
 - c. ¿Qué puesto ocupa actualmente Gervasio Salas?
2. Basándose en las relaciones EMPLEADO, PUESTO y ASIGNACION presentadas en la Figura 9.5, escriba una secuencia de operaciones relacionales para obtener una lista de todos los títulos de los puestos de trabajo dentro del departamento de personal.
3. Basándose en las relaciones EMPLEADO, PUESTO y ASIGNACION presentadas en la Figura 9.5, escriba una secuencia de operaciones relacionales para obtener una lista de los nombres de todos los empleados junto con los departamentos correspondientes.
4. Convierta a SQL sus respuestas a las Cuestiones 2 y 3.
5. ¿En qué sentido proporciona independencia de los datos el modelo relacional?
6. ¿Cómo se enlazan entre sí las diferentes relaciones de una base de datos relacional?

9.3 Bases de datos orientadas a objetos

Otro modelo de base de datos es el basado en el paradigma de orientación a objetos. Esta solución conduce a obtener una **base de datos orientada a objetos**, compuesta por objetos que están enlazados entre sí con el fin de reflejar sus relaciones. Por ejemplo, una implementación orientada a objetos de la base de datos de empleados de la sección anterior podría estar compuesta por tres clases (tipos de objetos): EMPLEADO, PUESTO y ASIGNACION. Un objeto de la clase

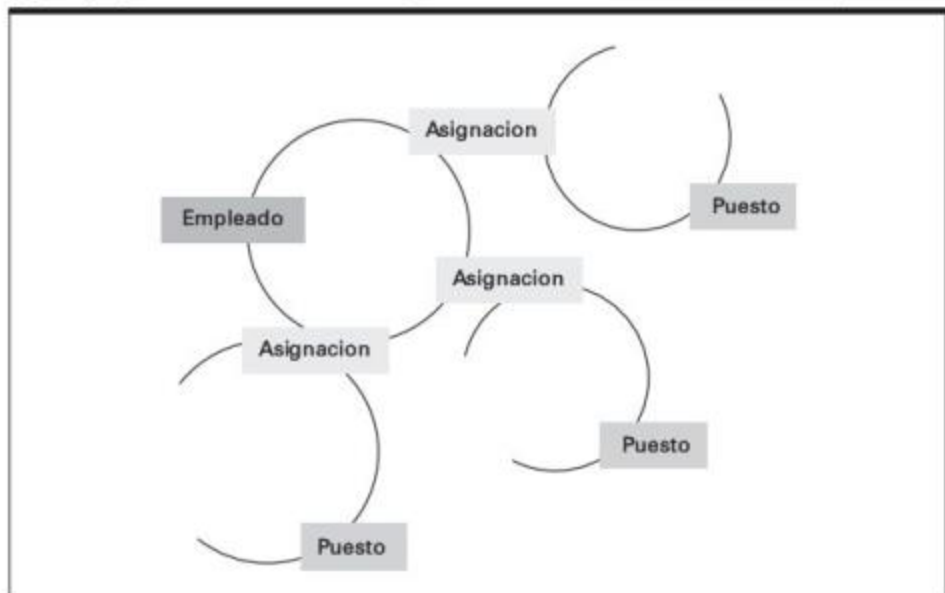
EMPLEADO contendría entradas tales como *IdEmpl*, *Nombre*, *Dirección* y *NSS*; un objeto de la clase PUESTO contendría entradas tales como *IdPuesto*, *Título*, *CodCat* y *Dept*; y cada objeto de la clase ASIGNACION podría contener entradas como *FechaInicio* y *FechaFin*.

En la Figura 9.13 se muestra una representación conceptual de ese tipo de base de datos, en la que los enlaces entre los distintos objetos están representados mediante líneas que conectan los objetos relacionados. Si nos centramos en un objeto de tipo EMPLEADO, lo vemos enlazado a un conjunto de objetos de tipo ASIGNACION, que representa las diversas asignaciones de puesto de trabajo que ese empleado en concreto ha tenido. A su vez, cada uno de estos objetos de tipo ASIGNACION está enlazado a un objeto de tipo PUESTO, que representa el puesto asociado con dicha asignación. Así, todas las asignaciones de un empleado pueden encontrarse siguiendo los enlaces que van desde el objeto que representa a ese empleado. De forma similar, pueden encontrarse todos los empleados que han ocupado un puesto concreto siguiendo los enlaces correspondientes al objeto que representa ese puesto de trabajo.

Los enlaces entre objetos en una base de datos orientada a objetos suelen ser mantenidos por el DBMS, por lo que el programador no tiene que preocuparse por los detalles de cómo se implementan estos enlaces a la hora de escribir el software de aplicación. En lugar de ello, cuando se añade un nuevo objeto a la base de datos, el software de aplicación simplemente especifica los otros objetos a los que debe estar enlazado. El DBMS crea entonces cualquier sistema de enlaces que pueda ser necesario con el fin de registrar esas asociaciones. En particular, un DBMS puede enlazar los objetos que representan las asignaciones de un determinado empleado de forma similar a una lista enlazada.

Otra tarea de un DBMS orientado a objetos es proporcionar almacenamiento permanente para los objetos que se le confían, un requisito que puede parecer obvio, pero que es inherentemente distinto de la forma en que normal-

Figura 9.13 Las asociaciones entre objetos en una base de datos orientada a objetos.



mente se tratan los objetos. Por lo general, cuando se ejecuta un programa orientado a objetos, los objetos creados durante la ejecución del programa se descartan una vez que este termina. En este sentido, los objetos se consideran transitorios, pero los objetos creados y añadidos a una base de datos deben guardarse después de que haya terminado el programa que los creó. Tales objetos se denominan **persistentes**. Por ello, la creación de objetos persistentes se aparta significativamente de la norma.

Los defensores de las bases de datos orientadas a objetos ofrecen numerosos argumentos para demostrar por qué la solución orientada a objetos es mejor en el diseño de bases de datos que el enfoque tradicional. Uno de esos argumentos es que la solución orientada a objetos permite que todo el sistema software (software de aplicación, DBMS y la propia base de datos) se diseñe bajo el mismo paradigma. Esto contrasta con la práctica tradicional de utilizar un lenguaje de programación imperativo para desarrollar el software de aplicación con el fin de interrogar a una base de datos relacional. Inherente a dicha tarea es el conflicto entre los paradigmas imperativo y relacional. Esta distinción es quizá demasiado sutil para el nivel con el que estamos abordando nuestro estudio, pero esa diferencia ha provocado muchos errores de software a lo largo de los años. Incluso a nuestro nivel, no es difícil apreciar que una base de datos orientada a objetos combinada con un programa de aplicación orientado a objetos, permite obtener una imagen homogénea de objetos que se comunican unos con otros a través del sistema. Por el contrario, una base de datos relacional combinada con un programa de aplicación imperativo sugiere la imagen de dos organizaciones inherentemente distintas tratando de encontrar una interfaz común.

Para entender otra ventaja que las bases de datos orientadas a objetos tienen con respecto a sus equivalentes relacionales, considere el problema de almacenar los nombres de los empleados en una base de datos relacional. Si almacenamos un nombre completo en forma de un único atributo en una relación, entonces las consultas relativas únicamente a los apellidos serán muy complicadas. Sin embargo, si almacenamos el nombre en forma de atributos individuales, tal como nombre de pila, primer apellido y segundo apellido, entonces el número de atributos se hace problemático, porque no todos los nombres se adaptan a una estructura específica, aún cuando la población esté restringida a una única cultura. En una base de datos orientada a objetos, estos problemas pueden ocultarse dentro del objeto que almacene el nombre del empleado. El nombre del empleado podrá guardarse como un objeto inteligente que será capaz de informar de cuál es el nombre del empleado relacionado en diversos formatos. De ese modo, desde fuera de esos objetos, sería igual de fácil tratar solo con los apellidos que con los nombres completos, los nombres de pila o los alias. Los detalles relacionados con cada una de las distintas perspectivas estarían encapsulados dentro de los objetos.

Esta capacidad de encapsular los aspectos técnicos de los diferentes formatos de datos resulta ventajosa también en otros casos. En una base de datos relacional, los atributos de una relación forman parte del diseño global de la base de datos, por lo que los tipos asociados con esos atributos influyen sobre toda la base de datos. (Las variables para el almacenamiento temporal deben declararse con el tipo apropiado y es preciso designar procedimientos para manipular datos de los diversos tipos.) Así, ampliar una base de datos relacional para incluir atributos de nuevos tipos (audio y vídeo) puede ser problemático. En

particular, puede que sea necesario ampliar diversos procedimientos dispersos por todo el diseño de la base de datos, con el fin de incorporar estos nuevos tipos de datos. Sin embargo, en un diseño orientado a objetos, los mismos procedimientos utilizados para extraer un objeto que represente el nombre de un empleado pueden utilizarse para extraer un objeto que represente una película, porque las distinciones de tipo pueden ocultarse dentro de los objetos implicados. Por tanto, la solución orientada a objetos es más compatible con la construcción de bases de datos multimedia, una característica que ya está demostrando sus grandes ventajas.

Otra ventaja más que el paradigma orientado a objetos ofrece para el diseño de bases de datos es el potencial para almacenar objetos inteligentes, en lugar de simplemente datos. Es decir, un objeto puede contener métodos que describan cómo debería responder el objeto a mensajes relativos a su contenido y a sus relaciones. Por ejemplo, cada objeto de la clase EMPLEADO de la Figura 9.13 podría contener métodos para leer y actualizar la información contenida en el objeto, así como un método para leer el historial laboral del empleado y quizá un método para modificar la asignación de puesto de trabajo de ese empleado. De la misma forma, cada objeto de la clase PUESTO podría tener un método para leer los datos específicos del puesto de trabajo y quizá otro método para informar de qué empleados han ocupado ese puesto de trabajo en concreto. Así, para extraer el historial laboral de un empleado, no necesitaríamos construir un procedimiento elaborado. En lugar de ello, podríamos simplemente pedir al objeto de empleado apropiado que informara acerca de su historial laboral. Esta capacidad de construir bases de datos cuyos componentes respondan de forma inteligente a las consultas ofrece múltiples y excitantes posibilidades que van mucho más allá de las disponibles con las bases de datos relacionales tradicionales.

Cuestiones y ejercicios

1. ¿Qué métodos contendría una instancia de un objeto de la clase ASIGNACION en la base de datos de empleados que hemos descrito en esta sección?
2. ¿Qué es un objeto persistente?
3. Identifique algunas clases, junto con algunas de sus características internas, que puedan emplearse en una base de datos orientada a objetos utilizada para gestionar el inventario de un almacén.
4. Identifique una ventaja que una base de datos orientada a objetos pueda tener si la comparamos con una base de datos relacional.

9.4 Mantenimiento de la integridad de una base de datos

Los sistemas baratos de gestión de bases de datos para uso personal son sistemas relativamente simples. Tienden a tener un único objetivo (ocultar al usuario los detalles técnicos de la implementación de la base de datos). Las bases de

datos mantenidas por esos sistemas son relativamente pequeñas y por lo general contienen información cuya pérdida o corrupción podría ser incómoda, pero raras veces desastrosa. Cuando surge de verdad un problema, el usuario puede normalmente corregir los errores directamente o recargar la base de datos a partir de una copia de seguridad para luego efectuar manualmente las correcciones requeridas con el fin de actualizar dicha copia. Este proceso puede ser incómodo, pero el coste de evitar esa incomodidad tiende a ser mayor que la incomodidad en sí. En cualquier caso, esa incomodidad está restringida a solo unas pocas personas y las pérdidas financieras asociadas suelen ser limitadas.

Sin embargo, en el caso de los sistemas de bases de datos comerciales, multiusuario y de gran tamaño, los riesgos son mucho mayores. El coste de tener datos incorrectos o de que se pierdan los datos puede ser enorme y tener consecuencias devastadoras. En estos entornos, uno de los principales papeles del DBMS es mantener la integridad de la base de datos protegiéndola frente a problemas tales como operaciones que por alguna razón solo lleguen a completarse parcialmente o como la posibilidad de que diversas operaciones interfieran de manera inadvertida, haciendo que al final la base de datos contenga información incorrecta. Este es el papel del DBMS que vamos a analizar en esta sección.

El protocolo de confirmación/anulación (*commit/rollback*)

Una única transacción, como por ejemplo una transferencia de fondos de una cuenta bancaria a otra, la cancelación de una reserva en una línea aérea o la matrícula de un estudiante en un curso universitario, puede implicar múltiples pasos en el nivel de la base de datos. Por ejemplo, una transferencia de fondos entre cuentas bancarias requiere que se reduzca el saldo de una de las cuentas y se incremente correspondientemente el saldo de la otra. En el tiempo que media entre esos pasos, la información de la base de datos puede ser incoherente. De hecho, el dinero habrá desaparecido durante el breve periodo que media entre el momento de disminuir el saldo de la primera cuenta y el momento de aumentar el saldo de la segunda. De la misma forma, cuando se reasigna el asiento de un pasajero en un vuelo, puede haber un instante en el que el pasajero no tenga asiento asignado o un instante en el que la lista de pasajeros parezca tener un pasajero más de lo que realmente tiene.

En el caso de bases de datos grandes sujetas a una gran carga de transacciones es bastante probable que cualquier instantánea tomada en un momento aleatorio encuentre a la base de datos en mitad de la realización de alguna transacción. Por tanto, cualquier solicitud para que se ejecute una transacción o cualquier error de funcionamiento de un equipo es probable que se produzca en algún instante en el que la base de datos se encuentra en un estado incoherente.

Consideremos primero el problema de un error de funcionamiento. El objetivo del DBMS es garantizar que ese tipo de problemas no dejen a la base de datos en un estado incoherente permanente. Esto suele llevarse a cabo manteniendo un registro en un sistema de almacenamiento no volátil, como por ejemplo un disco magnético, en el que se van anotando todas las actividades de las transacciones. Antes de permitir a una transacción modificar la base de

datos, se anota en el registro la modificación que se va a realizar. De ese modo, el registro contendrá un archivo permanente de todas las acciones de cada transacción.

El punto en el que todos los pasos que componen una transacción se han anotado en el registro se denomina **punto de confirmación**. Es en ese momento cuando el DBMS dispone de la información que necesita para reconstruir la transacción por su propia cuenta en caso de que llegara a ser necesario. En ese punto, el DBMS "se compromete" con la transacción en el sentido de que acepta la responsabilidad de garantizar que las actividades de la transacción se reflejarán en la base de datos. En el caso de que se produzca un fallo en un equipo, el DBMS puede utilizar la información contenida en su registro para reconstruir las transacciones que se han completado (confirmado) desde que se llevó a cabo la última copia de seguridad.

Si surgen problemas antes de que una transacción haya alcanzado su punto de confirmación, el DBMS puede encontrarse con una transacción parcialmente ejecutada que no puede completarse. En este caso, el registro puede utilizarse para **anular** (deshacer) las actividades realizadas por la transacción. En el caso de un error de funcionamiento, el DBMS podría recuperarse anulando aquellas transacciones que estuvieran incompletas (no confirmadas) en el momento en que se produjo el error.

La anulación de transacciones no está restringida, sin embargo, al proceso de recuperarse de errores de funcionamiento de los equipos. De hecho, suelen ser parte de la operación normal de un DBMS. Por ejemplo, una transacción podría terminarse antes de haber completado sus pasos debido a un intento de acceder a información privilegiada. O puede que se vea implicada en un interbloqueo, en el que una serie de transacciones competidoras se encuentran esperando por los datos que están usando las otras. En estos casos, el DBMS puede utilizar el registro para anular una transacción y evitar así errores de base de datos debido a la existencia de transacciones incompletas.

Para enfatizar la delicada naturaleza del diseño de sistemas DBMS, es necesario recalcar que existen sutiles problemas inherentes al proceso de anulación. La anulación de una transacción puede afectar a las entradas de la base de datos que hayan sido utilizadas por otras transacciones. Por ejemplo, la transacción que se está anulando podría haber actualizado un saldo de una cuenta bancaria y otra transacción podría haber ya basado sus actividades en ese valor actualizado. Esto podría significar que esas transacciones adicionales tengan que ser también anuladas, lo que puede a su vez afectar a otras transacciones. El resultado es el problema conocido como de la **anulación en cascada**.

Bloqueo (*locking*)

Consideremos ahora el problema de la ejecución de una transacción mientras que la base de datos se encuentra en un estado intermedio de otra transacción, una situación que puede llevar a interacciones inadvertidas entre las transacciones y producir resultados erróneos. Por ejemplo, puede surgir el problema conocido con el nombre de **problema de totalización incorrecta** si una transacción se encuentra en mitad del proceso de transferir dinero de una cuenta bancaria a otra al mismo tiempo que otra transacción trata de calcular el total

de los depósitos existentes en el banco. Esto podría resultar en un total que sea demasiado grande o demasiado pequeño, dependiendo del orden en que se efectúen los pasos de la transferencia. Otra posibilidad es la que se conoce con el nombre del **problema de la actualización perdida**, que podemos ejemplificar mediante dos transacciones, cada una de las cuales efectúa un pago contra una misma cuenta. Si una de las transacciones lee el saldo actual de la cuenta justo en el instante en el que la otra acaba de leer el saldo pero no ha calculado todavía el nuevo, entonces ambas transacciones basarán su cálculo del nuevo saldo en el mismo valor de saldo inicial. Por ello, el efecto de uno de los pagos no se verá reflejado en la base de datos.

Para resolver estos problemas, un DBMS podría obligar a las transacciones a que se ejecuten en su totalidad una a una, manteniendo a todas las nuevas transacciones en una cola hasta que las que la preceden hayan completado su tarea. Pero una transacción invierte a menudo una gran cantidad de tiempo esperando a que se realicen operaciones con el almacenamiento masivo. Entremezclando la ejecución de las transacciones, el tiempo durante el que una transacción está esperando puede ser utilizado por otra transacción para procesar datos que haya extraído. Por ello, la mayoría de los grandes sistemas de gestión de bases de datos disponen de un planificador para coordinar la compartición de tiempo entre transacciones, de forma bastante similar a como un sistema operativo de multiprogramación coordina la intercalación de procesos (véase la Sección 3.3).

Para protegerse frente anomalías tales como el problema de la totalización incorrecta y el problema de la actualización perdida, esos planificadores incorporan un **protocolo de bloqueo** mediante el que los elementos de una base de datos que estén siendo actualmente utilizados por alguna transacción se marcan como en uso. Estas marcas se denominan bloqueos y decimos que los elementos marcados están bloqueados. Existen dos tipos comunes de bloqueos: **bloqueos compartidos** y **bloqueos exclusivos**. Estos bloqueos se corresponden con los dos tipos de acceso a los datos que puede necesitar una transacción (acceso compartido y acceso exclusivo). Si una transacción no va a modificar un elemento de datos, entonces necesitará un acceso compartido, lo que quiere decir que otras transacciones también estarán autorizadas a ver los datos. Sin embargo, si la transacción va a modificar el elemento, deberá tener acceso exclusivo, lo que quiere decir que debe ser la única transacción con acceso a esos datos.

En un protocolo de bloqueo, cada vez que una transacción solicita acceso a un elemento de datos, también debe informar al DBMS del tipo de acceso que requiere. Si una transacción solicita acceso compartido a un elemento que no está bloqueado o que está bloqueado con un acceso compartido, entonces el DBMS concede el acceso y el elemento se marca con un bloqueo compartido. Sin embargo, si el elemento solicitado ya está marcado con un acceso exclusivo, el acceso adicional se deniega. Si una transacción solicita acceso exclusivo a un elemento, esa solicitud solo se concede si el elemento no tiene ningún bloqueo asociado. De esta manera, una transacción que vaya a alterar los datos protegerá a esos datos frente a otras transacciones obteniendo acceso exclusivo, mientras que múltiples transacciones pueden compartir el acceso a un elemento si ninguna de ellas va a modificarlo. Por supuesto, una vez que una

transacción ha terminado con un elemento, se lo notifica al DBMS y este elimina el bloqueo asociado.

Para gestionar el caso en el que se rechaza la solicitud de acceso efectuada por una transacción hay disponibles varios métodos. Uno de ellos es que la transacción se ve forzada simplemente a esperar hasta que el elemento solicitado pasa a estar disponible. Sin embargo, esta solución puede conducir a un interbloqueo, ya que dos transacciones que requieran acceso exclusivo a los mismos dos elementos de datos podrían bloquearse entre sí, si cada una de ellas obtiene acceso exclusivo a uno de los elementos y luego insiste en esperar a obtener acceso al otro. Para evitar esos interbloques, algunos sistemas de gestión de bases de datos dan prioridad a las transacciones más antiguas. Es decir, si una transacción más antigua solicita acceso a un elemento que está bloqueado por una transacción más reciente, se obliga a esta última a liberar todos sus elementos de datos y sus actividades se deshacen, basándose en el registro. Después, a la transacción antigua se le proporciona acceso al elemento que solicitó y se fuerza a la transacción más reciente a comenzar de nuevo. Si a causa de este mecanismo, una transacción reciente es forzada a anularse repetidamente, se irá haciendo más antigua durante el proceso y terminará siendo una de las transacciones con mayor prioridad. Este protocolo, conocido con el nombre de **protocolo de desalojo y espera** (*wound-wait*, las transacciones más antiguas desalojan a las más recientes y las más recientes esperan a que las más antiguas terminen), garantiza que todas las transacciones tengan la oportunidad de terminar completando su tarea.

Cuestiones y ejercicios

1. ¿Cuál es la diferencia entre una transacción que ha alcanzado su punto de confirmación y otra que no lo ha hecho?
2. ¿Cómo podría un DBMS protegerse frente a un proceso de anulación en cascada masivo?
3. Muestre cómo el entremezclado incontrolado de dos transacciones, una que tiene que deducir 100 euros de una cuenta y la otra que tiene que deducir 200 euros de la misma cuenta, podría producir saldos finales de 100, 200 y 300 euros, suponiendo que el saldo inicial sea de 400 euros.
4.
 - a. Resuma los posibles resultados de una transacción que solicite acceso compartido a un elemento de una base de datos.
 - b. Resuma los posibles resultados de una transacción que solicite acceso exclusivo a un elemento de una base de datos.
5. Describa la secuencia de sucesos que conduciría a un interbloqueo entre transacciones que estén realizando operaciones en un sistema de base de datos.
6. Describa cómo podría romperse el interbloqueo de su respuesta a la Cuestión 5. ¿Requeriría su solución utilizar el registro del sistema de gestión de la base de datos? Razone su respuesta.

9.5 Estructuras de archivo tradicionales

En esta sección vamos a dejar momentáneamente de lado el estudio de los sistemas de base de datos multidimensionales, con el fin de considerar las estructuras de archivo tradicionales. Estas estructuras representan el inicio histórico de los sistemas de almacenamiento y recuperación de datos a partir de los cuales ha evolucionado la tecnología actual de base de datos. Muchas de las técnicas desarrolladas para estas estructuras (como las de indexación y *hashing*) son herramientas importantes en la construcción de las bases de datos masivas y complejas, que se utilizan en la actualidad.

Archivos secuenciales

Un **archivo secuencial** es un archivo al que se accede de forma serie, desde su principio hasta su final, como si la información del archivo estuviera dispuesta en una única fila de gran longitud. Entre los ejemplos podemos citar los archivos de audio, los archivos de video, archivos que contienen programas y archivos que contienen documentos de texto. De hecho, la mayoría de los archivos creados por un usuario típico de una computadora personal son archivos secuenciales. Por ejemplo, cuando se guarda una hoja de cálculo, su información se codifica y almacena como un archivo secuencial, a partir del cual el software de manejo de hojas de cálculo puede reconstruir la hoja de cálculo guardada.

Los archivos de texto, que son archivos secuenciales en los que cada registro lógico es un único símbolo codificado mediante ASCII o Unicode, suelen servir como herramienta básica para construir archivos secuenciales más elaborados, como los archivos de registros de empleados. Lo único que hace falta es establecer un formato uniforme para la representación de la información relativa a cada empleado en forma de cadena de texto; codificar esa información con dicho formato y luego grabar los registros de empleados resultantes uno detrás de otro, como una única cadena de texto. Por ejemplo, podríamos construir un archivo de empleados simple decidiendo introducir cada registro de empleado como una cadena de 31 caracteres, formada por un campo de 25 caracteres que contendrá el nombre del empleado (relleno con los caracteres de espacio suficientes como para completar el campo de 25 caracteres), seguido por un campo de 6 caracteres que representará el número de identificación de ese empleado. El archivo final sería una larga cadena de caracteres codificados en la que cada bloque de 31 caracteres representaría la información relativa a un único empleado (Figura 9.14). La información se extraería del archivo en términos de los registros lógicos, compuestos por bloques de 31 caracteres. Dentro de cada uno de estos bloques, los campos individuales se identificarían de acuerdo con el formato uniforme empleado para construir los bloques.

Los datos de un archivo secuencial deben guardarse en el almacenamiento masivo de forma tal que la naturaleza secuencial del archivo se conserve. Si el sistema de almacenamiento masivo es secuencial (como sucede, por ejemplo, con una cinta magnética o un CD), esta tarea será muy sencilla. Simplemente tendremos que guardar el archivo en el medio de almacenamiento de acuerdo con las propiedades secuenciales de dicho medio. Después, para procesar el archivo, simplemente habrá que leer y procesar los datos contenidos en el

A medida que se extrae cada registro de empleado, se calcula la nómina de ese empleado y se genera el cheque correspondiente. La actividad de procesar ese tipo de archivo secuencial se podría ejemplificar mediante la sentencia

```
while (EOF no se ha alcanzado) do
    (extraer el siguiente registro del archivo y procesarlo)
```

Cuando los registros lógicos dentro de un archivo secuencial se identifican mediante valores clave de los campos, el archivo suele disponerse de modo que los registros aparezcan en el orden determinado por esos valores clave (que quizá sea alfabético o numérico). Ese tipo de disposición simplifica la tarea de procesamiento de la información del archivo. Por ejemplo, suponga que procesar las nóminas requiere que se actualice el registro de cada empleado para reflejar la información contenida en el archivo de asignaciones horarias de ese empleado. Si tanto el archivo que contiene los datos horarios como el que contiene los registros de empleados están en el mismo orden de acuerdo con las mismas claves, entonces este proceso de actualización puede gestionarse accediendo a ambos archivos secuencialmente, utilizando el registro horario extraído de un archivo para actualizar el registro correspondiente del otro archivo. Esta es una mejora significativa con respecto al proceso repetido de búsqueda que se necesitaría si los archivos no estuvieran en el mismo orden. Debido a ello, la actualización de archivos secuenciales clásicos suele realizarse en múltiples pasos. En primer lugar, se registra la nueva información (como por ejemplo el conjunto de asignaciones horarias) en un archivo secuencial que se conoce como archivo de transacción y este archivo de transacción se ordena para hacerlo corresponder con el orden del archivo que hay que actualizar, que se denomina archivo maestro. Después, se actualizan los registros del archivo maestro extrayendo secuencialmente los registros de ambos archivos.

Una ligera variante de este proceso de actualización es el proceso de combinar dos archivos secuenciales para formar un nuevo archivo que contenga los registros de los dos archivos originales. Suponemos que los registros de los archivos de entrada están ordenados de manera ascendente, de acuerdo con un campo clave común y también asumimos que hay que combinar los archivos de forma que se genere un archivo de salida cuyas claves también estén en

Figura 9.15 Un procedimiento para combinar dos archivos secuenciales.

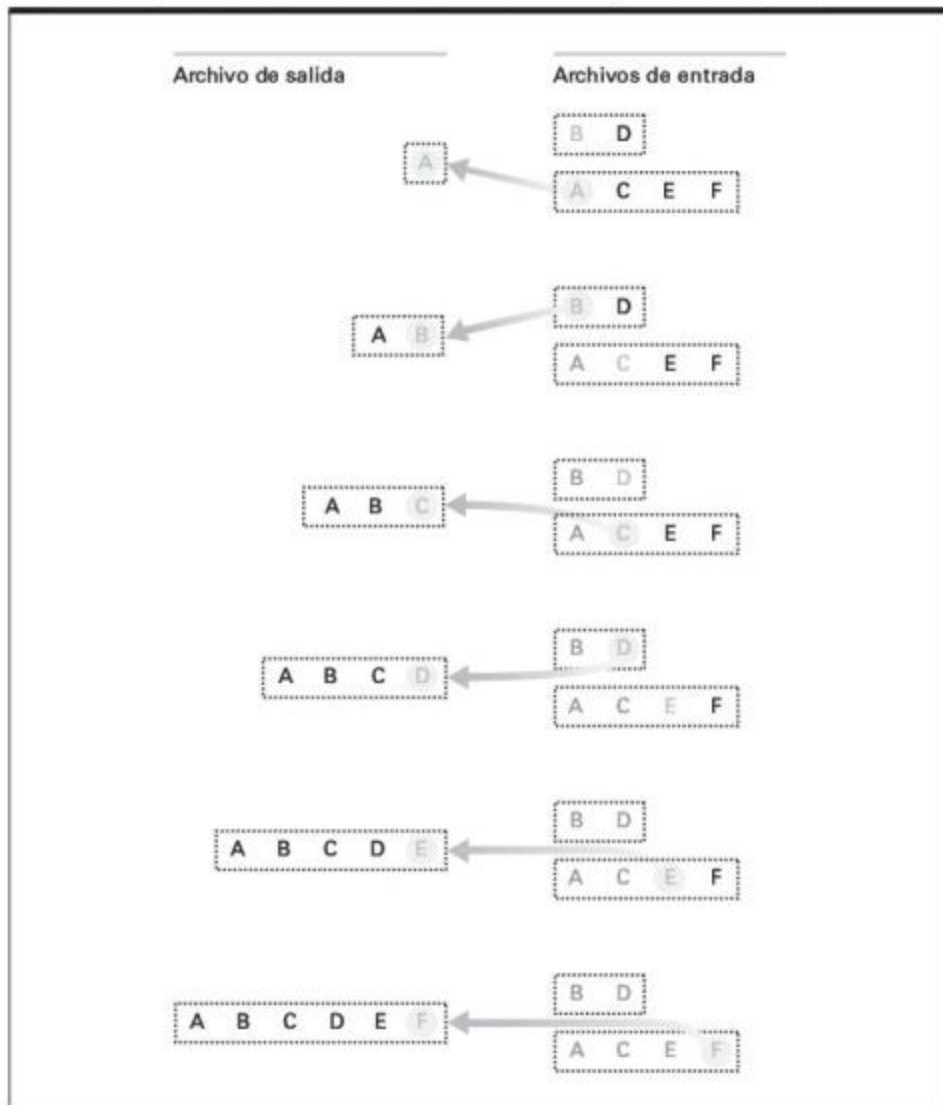
```
procedure CombinarArchivos (ArchivoEntradaA, ArchivoEntradaB, ArchivoSalida)
if (ambos archivos de entrada en EOF) then (Parar, con ArchivoSalida vacía)
if (ArchivoEntradaA no está en EOF) then (Declarar su primer registro como registro actual del archivo)
if (ArchivoEntradaB no está en EOF) then (Declarar su primer registro como registro actual del archivo)
while (ninguno de los archivos de entrada está en EOF) do
    (Poner en ArchivoSalida aquel de los dos registros actuales que tenga el "menor" valor en el campo clave;
    if (ese registro actual es el último de su correspondiente archivo de entrada)
        then (Declarar que ese archivo de entrada está en EOF)
        else (Declarar el siguiente registro de ese archivo de entrada como registro actual del archivo)
    )
Comenzando con el registro actual del archivo de entrada que no esté en EOF,
copiar los registros restantes en ArchivoSalida.
```

orden ascendente. En la Figura 9.15 se resume el algoritmo clásico de combinación. El aspecto que hay que resaltar es que el archivo de salida se construye a medida que se van explorando secuencialmente los dos archivos de entrada (Figura 9.16).

Archivos indexados

Los archivos secuenciales resultan ideales para almacenar datos que van a procesarse en el orden en el que están almacenadas las entradas del archivo. Sin embargo, dichos archivos son poco eficientes cuando hay que extraer los registros del archivo en un orden impredecible. En estas situaciones, lo que hace falta es

Figura 9.16 Aplicación del algoritmo de combinación. (Se utilizan letras para representar registros completos. La letra concreta indica el valor del campo clave del registro.)



una forma de identificar rápidamente la ubicación del registro lógico deseado. Una solución común consiste en utilizar un índice para el archivo, de forma bastante similar a como se emplea un índice en un libro para localizar los temas que el libro contiene. Este tipo de sistema de archivos se denomina **archivo indexado**.

Un índice de un archivo contiene una lista de las claves almacenadas en el archivo, junto con entradas que indican dónde está almacenado el registro que contiene cada clave. Así, para encontrar un registro concreto, lo que hay que hacer es localizar la clave de identificación en el índice y luego extraer el bloque de información almacenado en la ubicación almacenada con dicha clave.

El índice de un archivo suele guardarse como un archivo independiente en el mismo dispositivo de almacenamiento masivo que el archivo indexado. Normalmente, el índice se transfiere a la memoria principal antes de que comience el procesamiento del archivo, para que se pueda acceder a él fácilmente cuando se necesite acceder a los registros del archivo (Figura 9.17).

Un ejemplo clásico de archivo indexado podría ser el que se utiliza en el contexto de mantenimiento de los registros de empleados. En ese caso, podemos utilizar un índice para evitar las largas búsquedas a la hora de extraer un registro individual. En particular, si indexamos el archivo que contiene los registros de empleados por los números de identificación de empleado, entonces podremos extraer rápidamente el registro de un empleado si conocemos su número de identificación. Otro ejemplo lo tenemos en los CD de audio en los que se utiliza un índice para acceder de forma relativamente rápida a cada canción o fragmento individual.

A lo largo de los años, se han utilizado numerosas variantes del concepto básico de índice. Una de esas variantes construye un índice de forma jerárquica de modo que el índice adopta una estructura en capas o de árbol. Un ejemplo prominente sería el sistema de directorios jerárquico utilizado por la mayoría de los sistemas operativos para organizar el almacenamiento de archivos. En ese caso, los directorios, o carpetas, desempeñan el papel de índices, cada uno de los cuales contiene enlaces a sus subíndices. Desde esta perspectiva, el sistema de archivos completo podría considerarse como un único gran archivo indexado.

Figura 9.17 Apertura de un archivo indexado.



Archivos hash

Aunque la indexación proporciona un acceso relativamente rápido a las entradas contenidas en una estructura de almacenamiento de datos, lo hace a expensas de tener que realizar una serie de tareas de mantenimiento del índice. El **hashing** es una técnica que proporciona un acceso similar sin tal sobrecarga. Al igual que en el caso de un sistema indexado, el hashing permite localizar un registro por medio de un valor clave, pero en lugar de buscar la clave en un índice, el hashing identifica la ubicación del registro directamente a partir de la clave.

Un sistema hash puede resumirse de la forma siguiente: se divide el espacio de almacenamiento de datos en varias secciones, denominadas **fragmentos** (*buckets*), cada uno de los cuales es capaz de albergar varios registros. Los registros se dispersan entre los fragmentos de acuerdo con un algoritmo que convierte los valores clave en números de fragmento (esta conversión entre valores clave y números de fragmento se denomina **función hash**). Cada registro se almacena en el fragmento identificado mediante este proceso. De este modo, un registro que haya sido almacenado en la estructura de almacenamiento podrá extraerse aplicando primero la función hash a la clave de identificación del registro, con el fin de determinar el fragmento apropiado y extrayendo después el contenido de dicho fragmento, para finalmente buscar el registro deseado en ese fragmento extraído.

El hashing no se utiliza solo como medio de extraer datos de un sistema de almacenamiento masivo, sino también como medio de extraer grandes bloques de datos almacenados en la memoria principal. Cuando se aplica el hashing a una estructura guardada en un sistema de almacenamiento masivo, el resultado se denomina **archivo hash**. Cuando se aplica a una estructura guardada en la memoria principal, el resultado suele denominarse **tabla hash**.

Autenticación mediante hashing

El hashing es mucho más que un medio de construir sistemas eficientes de almacenamiento de datos. Por ejemplo, el hashing se puede utilizar como medio de autenticar los mensajes transferidos a través de Internet. La técnica consiste en obtener un valor hash del mensaje de una manera secreta. Este valor se transfiere entonces junto con el propio mensaje. Para autenticar el mensaje, el receptor calcula el valor hash del mensaje recibido (de la misma forma secreta) y confirma que el valor generado concuerda con el original. (Asumimos que la probabilidad de que un mensaje alterado tenga el mismo valor hash es muy pequeña.) Si el valor obtenido no concuerda con el original, quedará claro que el mensaje ha sido corrompido. Los lectores interesados en este tema pueden buscar información en Internet acerca de MD5, que es una función hash utilizada ampliamente en aplicaciones de autenticación.

Resulta bastante esclarecedor considerar las técnicas de detección de errores como una aplicación de la técnica de hash a la autenticación. Por ejemplo, el uso de bits de paridad es esencialmente un sistema en el que se aplica una función hash a un patrón de bits para generar un 0 o un 1. Este valor se transfiere posteriormente junto con el patrón original. Si el patrón recibido por el destinatario no genera el mismo valor al aplicarle la función hash se considera que el patrón se ha corrompido.

Apliquemos la técnica de hashing al archivo clásico de empleados en el que cada registro contiene información acerca de un único empleado de la empresa. En primer lugar, establecemos varias áreas disponibles de almacenamiento masivo, que desempeñarán el papel de los fragmentos. El número de fragmentos y el tamaño de cada uno de ellos son decisiones de diseño que luego analizaremos. Por el momento, vamos a suponer que hemos creado 41 fragmentos, a los que denominaremos fragmento número 0, fragmento número 1, hasta el fragmento número 40 (enseguida explicaremos la razón por la que hemos seleccionado 41 fragmentos, en lugar de un número par como 40).

Vamos a suponer que el número de identificación se utiliza como clave para localizar el registro del empleado. Nuestra siguiente tarea entonces consistirá en desarrollar una función hash para convertir estas claves en números de fragmentos. Aunque los "números" de identificación de los empleados puedan tener la forma 25X3Z o J2X35 y sean, por tanto, no numéricos, el hecho es que están almacenados como patrones de bits y podemos interpretar esos patrones de bits como números. Utilizando esta interpretación numérica podemos dividir cualquier clave entre el número de fragmentos disponibles y quedarnos con el resto, que en nuestro caso será un entero comprendido en el rango entre 0 y 40. Por tanto, podemos emplear el resto de este proceso de división para identificar uno de los 41 fragmentos (Figura 9.18).

Utilizando esto como función hash, podemos construir el archivo considerando cada registro individualmente, aplicando nuestra función hash consistente en dividir su clave entre 41 para obtener un número de fragmento y almacenando luego el registro en dicho fragmento (Figura 9.19). Posteriormente, si necesitamos extraer un registro, solo tenemos que aplicar la función hash a la clave con el fin de identificar el fragmento apropiado, para después buscar en ese fragmento el registro en cuestión.

Llegados a este punto vamos a analizar nuestra decisión de dividir el área de almacenamiento en 41 fragmentos. En primer lugar, observe que para obtener un sistema de hash eficiente, los registros que queremos almacenar deben estar distribuidos de manera uniforme entre los fragmentos. Si un número des-

Figura 9.18 Aplicación de la función hash al valor 25X3Z del campo clave, para asignarle uno de los 41 fragmentos.

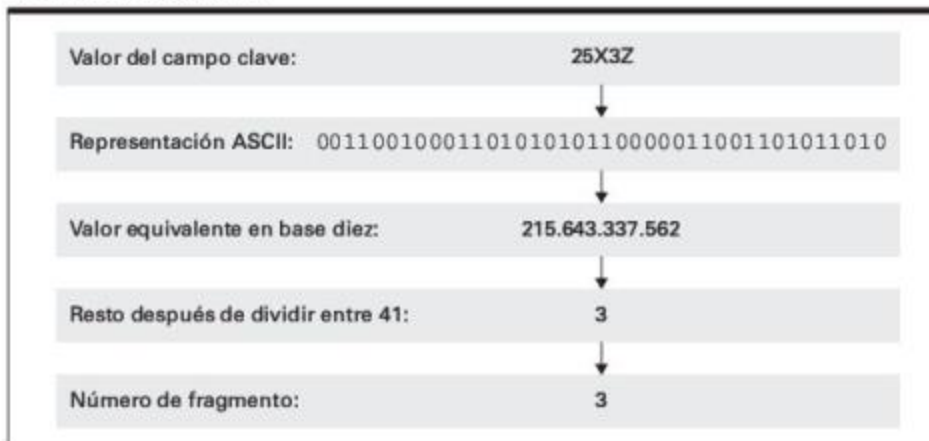
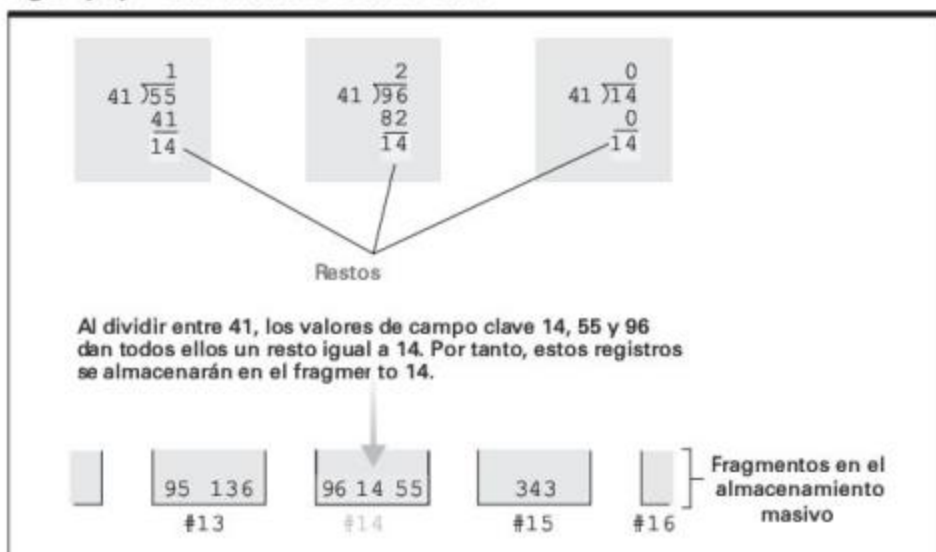


Figura 9.19 Rudimentos de un sistema hash.



proporcionado de claves produjeran, mediante la función hash, el mismo número de fragmento (un fenómeno denominado **agrupación**), entonces habría un número desproporcionado de registros almacenados en un mismo fragmento. A su vez, el extraer un registro de ese fragmento requeriría una búsqueda muy larga, perdiéndose así todas las ventajas que hubiéramos podido ganar mediante la técnica de hash.

Observe ahora que si hubiéramos decidido dividir el área de almacenamiento en 40 fragmentos en lugar de en 41, nuestra función hash hubiera implicado dividir las claves entre el valor 40 en lugar de 41. Pero si un dividendo y un divisor tienen un factor común, dicho factor estará presente también en el resto. En particular, si las claves de las entradas almacenadas en nuestro archivo hash resultaran ser múltiplos de 5 (que es también un divisor de 40), entonces el factor 5 aparecería en los restos al dividir entre 40, y las entradas tenderían a agruparse en aquellos fragmentos que estuvieran asociados con los restos 0, 5, 10, 15, 20, 25, 30 y 35. Una situación similar se produciría en el caso de claves que fueran múltiplos de 2, 4, 8, 10 y 20, porque todas ellas son también factores de 40. En consecuencia, hemos decidido dividir el área de almacenamiento en 41 fragmentos, porque la elección de 41, al ser un número primo, elimina la posibilidad de que existan factores comunes y reduce por tanto la probabilidad de que se produzcan agrupamientos.

Lamentablemente, nunca puede eliminarse del todo la posibilidad de agrupamiento. Incluso con una función hash bien diseñada es extremadamente probable que dos claves tengan el mismo valor hash (un fenómeno denominado **colisión**) en las etapas tempranas de construcción del archivo. Para entender por qué, considere el siguiente escenario.

Suponga que hemos encontrado una función hash que distribuye arbitrariamente los registros entre 41 fragmentos. Suponga también que nuestro sistema de almacenamiento está vacío y que vamos a insertar nuevos registros de

uno en uno. Al insertar el primer registro se colocará en un segmento vacío. Sin embargo, al insertar el siguiente solo seguirán vacíos 40 de los 41 fragmentos, por lo que la probabilidad de que el segundo registro se almacene en un fragmento vacío es solo de 40/41. Suponiendo que el segundo registro se almacene en un fragmento vacío, el tercer registro solo encontrará 39 fragmentos vacíos, por lo que la probabilidad de que se almacene en uno de ellos será de 39/41. Continuando con este proceso, vemos que si los primeros siete registros se almacenan en fragmentos vacíos, el octavo fragmento tendrá solo una probabilidad de 34/41 de ser guardado en uno de los fragmentos vacíos restantes.

Este análisis nos permite calcular la probabilidad de que los primeros ocho registros queden todos ellos guardados en fragmentos vacíos: esa probabilidad será el producto de las probabilidades de que cada registro se almacene en un fragmento vacío, asumiendo que también las entradas precedentes fueran almacenadas en un fragmento vacío. Esta probabilidad es igual a

$$(41/41) (40/41) (39/41) (38/41) \dots (34/41) = 0,482$$

Como vemos, el resultado es inferior a un medio; es decir, al distribuir registros entre 41 fragmentos, la probabilidad de que se haya producido ya una colisión en el momento de almacenar el octavo registro es mayor que la probabilidad de que no se haya producido.

La alta probabilidad de las colisiones indica que, independientemente de lo bien que se elija una función hash, es preciso diseñar los sistemas hash teniendo presente el fenómeno del agrupamiento. En particular, es posible que un fragmento termine por llenarse y se desborde. Una solución a este problema sería permitir que los fragmentos se expandan en tamaño. Otra solución es permitir que los fragmentos desborden sus registros sobrantes en un área de desbordamiento que se haya reservado para ese propósito. En cualquiera de los casos, los fenómenos de agrupamiento y de desbordamiento de fragmentos pueden degradar significativamente el rendimiento de un archivo hash.

Las investigaciones demuestran que, como regla general, los archivos hash se comportan bien siempre que el cociente entre el número de registros y la capacidad total de registros del archivo (un cociente que se conoce como **factor de carga**) permanezca por debajo del 50 por ciento. Sin embargo, en cuanto el factor de carga comienza a superar el 75 por ciento, el rendimiento del sistema suele degradarse (aparece el fenómeno del agrupamiento, haciendo que algunos fragmentos se llenen y posiblemente se desborden). Por esta razón, los sistemas de almacenamiento hash suelen reconstruirse con una capacidad mayor en cuanto su factor de carga se aproxima al 75 por ciento. Como conclusión, podemos decir que la eficiencia en la extracción de registros que se obtiene al implementar un sistema hash tiene sus costes de implementación asociados.

Cuestiones y ejercicios

1. Siga el algoritmo de combinación presentado en la Figura 9.15, asumiendo que uno de los archivos de entrada contiene registros cuyos valores de campo clave son B y E, mientras que el otro contiene A, C, D y F.

2. El algoritmo de combinación es la base de un conocido algoritmo de ordenación que se denomina ordenación por combinación. Describa cuál podría ser este algoritmo. (*Sugerencia:* cualquier archivo no vacío puede considerarse como un conjunto de archivos de una única entrada.)
3. ¿El carácter de secuencial es una propiedad física o conceptual de un archivo?
4. ¿Cuáles son los pasos requeridos a la hora de extraer un registro de un archivo indexado?
5. Explique por qué una función hash mal elegida puede provocar que un sistema de almacenamiento hash sea poco más que un archivo secuencial.
6. Suponga que construimos un sistema de almacenamiento hash utilizando la función de división hash tal como la hemos presentado en el texto, pero con seis fragmentos de almacenamiento. Para cada uno de los siguientes valores clave, identifique el fragmento en el que se almacenaría el registro que tenga dicha clave. ¿Cuál es el problema existente y por qué?

a. 24	b. 30	c. 3	d. 18	e. 15
f. 21	g. 9	h. 39	i. 27	j. 0
7. ¿Cuántas personas habrá que juntar antes de que la probabilidad de que dos miembros del grupo cumplan años el mismo día sea mayor del 50 por ciento? ¿Cómo se relaciona este problema con los conceptos presentados en esta sección?

9.6 Minería de datos

Un campo en rápida expansión que está estrechamente relacionado con la tecnología de bases de datos es el de la minería de datos, que son técnicas que permiten descubrir patrones dentro de conjuntos de datos. La minería de datos se ha convertido en una herramienta de gran importancia en numerosas áreas, incluyendo el marketing, la gestión de inventarios, el control de calidad, la gestión de riesgos crediticios, la detección de fraudes y el análisis de inversiones. Las técnicas de minería de datos tienen incluso aplicación en entornos tan improbables como la identificación de las funciones de determinados genes codificados en las moléculas de ADN y la caracterización de propiedades de los organismos.

Las actividades de la minería de datos difieren de las consultas tradicionales a una base de datos en que la minería de datos trata de identificar patrones previamente desconocidos por oposición a las consultas tradicionales, que simplemente solicitan que se extraigan hechos ya almacenados. Además, la minería de datos se practica con colecciones de datos estáticas, denominadas **almacenes de datos**, en lugar de con bases de datos operacionales "en línea", que están sujetas a actualizaciones frecuentes. Estos almacenes de datos suelen ser "instantáneas" de bases de datos o de conjuntos de bases de datos. Se utilizan en lugar de las bases de datos operacionales reales porque la localiza-

ción de patrones en un sistema estático es mucho más sencilla que en un sistema dinámico.

También debemos observar que el tema de la minería de datos no está restringido al dominio de la computación, sino que sus tentáculos se extienden bastante más allá para entrar en el campo de la estadística. De hecho, muchos argumentarían que puesto que la minería de datos tiene su origen en los intentos de realizar análisis estadísticos sobre grandes conjuntos de datos diversos es una aplicación de la estadística, más que un campo de las Ciencias de la computación.

Dos formas muy comunes de la minería de datos son la **descripción de clases** y la **discriminación de clases**. La descripción de clases trata de identificar propiedades que caractericen a un grupo dado de elementos de datos, mientras que la discriminación de clases trata de identificar propiedades que separen a dos grupos. Por ejemplo, las técnicas de descripción de clases se utilizarían para identificar las características de las personas que han comprado vehículos pequeños y baratos, mientras que las técnicas de discriminación de clases se emplearían para encontrar las propiedades que distinguen a los clientes que compran vehículos usados de aquellos que compran vehículos de primera mano.

Otra forma de minería de datos es el **análisis de agrupamiento**, que trata de descubrir clases. Observe que esta técnica difiere de la de descripción de clases, que intenta descubrir propiedades de los miembros pertenecientes a clases que ya han sido identificadas. Para ser más precisos, el análisis de agrupamiento trata de encontrar propiedades de los elementos de datos que conduzcan al descubrimiento de grupos. Por ejemplo, al analizar la información acerca de las edades de las personas que han visto una película concreta, el análisis de agrupamiento podría descubrir que la base de espectadores se descompone en dos grupos de edad: un grupo de edad de 4 a 10 años y otro grupo de 25 a 40 años (¿quizá esa película atrajo a los niños y a sus padres?).

Otra forma más de minería de datos es el **análisis de asociaciones**, que implica la búsqueda de enlaces entre grupos de datos. Es el análisis de asociaciones el que podría revelar, por ejemplo, que los clientes que compran patatas fritas también compran cerveza y refrescos, o que las personas que hacen sus compras durante las horas de trabajo normales también cobran una pensión de algún organismo público.

El **análisis de excepciones** es otra forma de minería de datos. Esta técnica trata de identificar aquellos elementos de datos que no se ajustan a la norma. El análisis de excepciones puede utilizarse para localizar errores en los conjuntos de datos, para detectar el robo de tarjetas de crédito observando las desviaciones súbitas con respecto a los patrones normales de compra de un cliente y quizá para identificar terroristas potenciales detectando comportamientos inusuales.

Por último, la forma de minería de datos denominada **análisis de patrones secuenciales** trata de identificar patrones de comportamiento a lo largo del tiempo. Por ejemplo, el análisis de patrones secuenciales podría revelar tendencias implícitas en los sistemas económicos, como por ejemplo los mercados de valores, o en sistemas medioambientales, como por ejemplo las condiciones climáticas.

Bioinformática

Los avances en la tecnología de bases de datos y en las técnicas de minería de datos están ampliando el repertorio de herramientas que los biólogos tienen a su disposición en las áreas de investigación que implican la identificación de patrones y la clasificación de compuestos orgánicos. El resultado es un nuevo campo de la Biología denominado Bioinformática. Teniendo su origen en los esfuerzos para decodificar el ADN, la Bioinformática abarca ahora tareas tales como la catalogación de proteínas y los intentos de comprender las secuencias de interacción proteínica (denominadas rutas bioquímicas). Aunque normalmente se considera parte de la Biología, la Bioinformática es un ejemplo de cómo las Ciencias de la computación están influyendo en otros campos e incluso integrándose en ellos.

Como indica este último ejemplo, los resultados de la minería de datos pueden utilizarse para predecir comportamientos futuros. Si una entidad posee las propiedades que caracterizan a una clase, entonces esa entidad se comportará probablemente como los miembros de esa clase. Sin embargo, muchos proyectos de minería de datos tratan simplemente de comprender mejor datos ya existentes, como lo atestigua el uso de la minería de datos a la hora de desentrañar los misterios del ADN. En cualquier caso, el ámbito de posibles aplicaciones de la minería de datos es enorme, así que este campo promete constituir un área activa de investigación durante los años venideros.

Observe que la tecnología de bases de datos y la de minería de datos están emparentadas, por lo que las investigaciones en uno de los dos campos tendrán repercusiones en el otro. Las técnicas de bases de datos se utilizan ampliamente para proporcionar a los almacenes de datos la capacidad de presentar la información en forma de **cubos de datos** (datos contemplados desde múltiples perspectivas, el término *cubo* se utiliza para evocar la imagen de múltiples dimensiones) y es esa capacidad de presentación de la información lo que hace posible la minería de datos. A su vez, a medida que los investigadores en la minería de datos mejoran las técnicas para implementar cubos de datos, estos resultados irán sirviendo para mejorar el campo del diseño de bases de datos.

Para terminar, es preciso reconocer que una adecuada aplicación de la minería de datos abarca mucho más que la identificación de patrones dentro de un conjunto de datos. Es preciso aplicar un juicio inteligente para determinar si esos patrones son significativos o simples coincidencias. El hecho de que una administración concreta de lotería haya vendido un alto número de boletos premiados probablemente no debería considerarse como significativo para alguien que está planeando comprar un billete de lotería, pero el descubrimiento de que los clientes que compran bolsitas de aperitivos también tienden a adquirir alimentos congelados sí que podría constituir una información significativa para los gerentes de supermercados. De la misma forma, la minería de datos abarca un amplio rango de cuestiones éticas, relativas a los derechos de los individuos representados en el almacén de datos, a la precisión y el uso de las conclusiones que se extraigan e incluso si la propia minería de datos es aceptable.

Cuestiones y ejercicios

1. ¿Por qué la minería de datos no se lleva a cabo con bases de datos “en línea”?
2. Proporcione un ejemplo adicional de patrón que podría encontrarse para cada uno de los tipos de minería de datos identificados en el texto.
3. Identifique algunas de las diferentes perspectivas que podrían utilizarse con un cubo de datos a la hora de aplicar técnicas de minería de datos a la información relativa a las ventas de una empresa.
4. ¿En qué difiere la minería de datos de las consultas tradicionales de bases de datos?

9.7 Impacto social de la tecnología de bases de datos

Con el desarrollo de la tecnología de las bases de datos, esa información que antaño estaba enterrada en misteriosos archivos ha pasado a ser accesible. En muchos casos, los sistemas automatizados de gestión de bibliotecas hacen que resulte fácil conocer los hábitos de lectura de los usuarios; los vendedores minoristas mantienen registros de las compras efectuadas por sus clientes y los motores de búsqueda en Internet también conservan memoria de las solicitudes que los clientes realizan. A su vez, esta información está potencialmente disponible para las empresas de marketing, las fuerzas de seguridad, los partidos políticos, los empresarios y las propias personas individuales.

Esto es bastante representativo de los problemas potenciales que presenta todo el espectro existente de aplicaciones de bases de datos. La tecnología ha hecho que sea fácil recopilar cantidades enormes de datos y combinar o comparar diferentes conjuntos de datos con el fin de detectar relaciones que de otro modo permanecerían ocultas bajo la avalancha de información. Las ramificaciones de esto, tanto positivas como negativas, son enormes. Y no son un simple objeto de debate académico, sino que son una auténtica realidad.

En algunos casos, el proceso de recopilación de datos se hace de manera abierta; en otros, se trata de una actividad mucho más sutil. Como ejemplos del primer caso podemos citar aquellos en los que se nos pide explícitamente que proporcionemos información. Esa información puede ser suministrada de manera voluntaria, como sucede en las encuestas o en los formularios de registro para un concurso, o puede ser aportada de manera involuntaria, como cuando nos es impuesta por las leyes existentes. En ocasiones, el que sea voluntario o no depende de nuestro punto de vista. ¿Proporcionar información personal a la hora de solicitar un crédito es un hecho voluntario o involuntario? La diferencia está en si recibir el crédito es una necesidad o se trata, por el contrario, de algo prescindible. Para utilizar una tarjeta de crédito ahora se requiere en muchos sitios que permitamos que nuestra firma se archive en formato digitalizado. De nuevo, el proporcionar esa información puede considerarse algo voluntario o involuntario, dependiendo de la situación de cada cual.

Los otros casos más sutiles de recopilación de datos evitan la comunicación directa con la persona. Por ejemplo, las empresas de tarjetas de crédito, que

registran los hábitos de compra de los poseedores de sus tarjetas, los sitios web que registran las identidades de aquellos que visitan el sitio o los activistas que anotan los números de matrícula de los vehículos que se encuentran en el aparcamiento de algunas de las instituciones que son objeto de su ira. En estos casos, la persona que está siendo sometida a ese proceso de recopilación de datos puede no ser consciente de que esa información se está recopilando, y es probable que sea aún menos consciente de la existencia de esas bases de datos que se construyen a partir de dicha información.

En ocasiones, las actividades subyacentes de recopilación de datos son evidentes por sí mismas, solo con que uno se pare a pensar durante unos momentos. Por ejemplo, una tienda puede ofrecer descuentos a sus clientes habituales, exigiéndoles que proporcionen de antemano a la tienda sus datos personales. El proceso de registro puede implicar el entregar al cliente algún tipo de tarjeta de identificación, que luego habrá que presentar en el momento de la compra, para poder obtener el descuento. El resultado es que esa tienda será capaz de compilar un registro de los hábitos de compra de los clientes. Un registro cuyo valor es mucho mayor que el de los descuentos concedidos.

Por supuesto, la fuerza que impulsa este florecimiento de los procesos de recopilación de datos es, precisamente, el valor de esos datos, que se ve amplificado por los avances en la tecnología de bases de datos, los cuales permiten enlazar esos datos de distintas maneras con el fin de descubrir información que de otro modo permanecería oculta. Por ejemplo, los hábitos de compra de los poseedores de tarjetas de crédito pueden clasificarse y compararse, con el fin de obtener perfiles de cliente que tienen un valor inmenso en las tareas de marketing. Los formularios de suscripción para revistas de culturismo pueden enviarse a aquellos que hayan adquirido recientemente aparatos de gimnasia, mientras que los formularios de suscripción a revistas sobre entrenamiento de perros pueden dirigirse a aquellos que hayan comprado en las últimas semanas comida para perros. Las distintas formas de combinar la información son, en ocasiones, muy imaginativas. Por ejemplo, en Estados Unidos se comparan los registros de asistencia social con los registros policiales, con el fin de localizar y detener a aquellos que han violado la libertad condicional y, en 1984, el gobierno americano llegó a utilizar antiguas listas de registro con fechas de cumpleaños, de un popular fabricante de helados con el fin de identificar a aquellos ciudadanos que no se habían registrado para hacer el servicio militar.

Hay distintos enfoques para proteger a la sociedad del uso abusivo de las bases de datos. Uno es aplicar remedios legales. Lamentablemente, el que se apruebe una ley contra un determinado tipo de actuación, no impide que esa actuación se produzca, sino solo la hace ilegal. Un ejemplo sobresaliente en Estados Unidos es la Ley de Intimididad de 1974, cuyo objetivo era proteger a los ciudadanos del uso abusivo de las bases de datos gubernamentales. Una de las medidas de esta ley requería que los organismos gubernamentales publicitaran la existencia de sus bases de datos mediante un registro federal que permitiera a los ciudadanos acceder a su información personal y corregirla. Sin embargo, los organismos gubernamentales tardaron mucho en ajustarse a esta norma. Esto no implica necesariamente que hubiera una intención maliciosa: en muchos casos, el problema era la burocracia. Pero el simple hecho de que un sistema burocrático pueda estar construyendo bases de datos con información personal que luego es incapaz de publicitar no es precisamente tranquilizador.

Otra técnica, quizá más potente, para controlar el abuso de las bases de datos es la opinión pública. Nadie abusaría de las bases de datos si el castigo fuera superior a los beneficios que se obtienen. Y el castigo que las empresas temen más es precisamente una opinión pública adversa, porque puede repercutir directamente en los resultados de la empresa. A principios de la década de 1990, fue la opinión pública la que terminó por hacer que las empresas de tarjetas de crédito desistieran de vender listas de correo para propósitos de marketing. Más recientemente, America Online (uno de los principales proveedores de servicios Internet) renunció por presiones de la opinión pública a su política de vender información relativa a sus clientes a empresas de telemarketing. Ha habido incluso organismos gubernamentales que han tenido que plegarse ante las críticas de la opinión pública. En 1997, la Administración de la Seguridad Social de Estados Unidos modificó sus planes de hacer disponibles a través de Internet los registros de los afiliados a la Seguridad Social, cuando la opinión pública cuestionó la seguridad de esa información. En esos casos, se obtuvieron resultados en cuestión de días, lo que contrasta enormemente con los dilatados periodos de tiempo que suelen asociarse con los procesos legales.

Por supuesto, en muchos casos las aplicaciones de bases de datos son beneficiosas tanto para el que posee los datos como para el que los proporciona, aunque siempre existe una pérdida de intimidad que no hay que tomarse a la ligera. Tales problemas de intimidad son graves cuando la precisión de la información es adecuada, pero se hacen realmente enormes cuando, encima, la información es errónea. Imagine la sensación de desesperanza cuando uno se da cuenta de que le han rebajado el límite de sus tarjetas de crédito debido a algún tipo de información errónea. E imagine después cómo se multiplicarían nuestros problemas en un entorno en el que esa información errónea fuera compartida fácilmente con otras instituciones.

Los problemas de intimidad son uno de los mayores efectos colaterales de los avances de la tecnología en general y de las técnicas de bases de datos en particular. Y continuarán representando un grave problema en el próximo futuro. Las soluciones a estos problemas exigen que los ciudadanos se informen, estén permanentemente alerta y sean activos.

Cuestiones y ejercicios

1. ¿Debería concederse a las fuerzas de seguridad acceso a bases de datos con el propósito de identificar a personas con tendencias criminales, aún cuando esas personas pueden no haber cometido ningún crimen?
2. ¿Debería concederse a las empresas de seguros acceso a bases de datos para identificar a personas que tengan problemas médicos potenciales, aún cuando esas personas no hayan mostrado ningún síntoma?
3. Suponga que se encuentra en una situación financiera desahogada. ¿Qué ventajas podría obtener si esta información fuera compartida por diversas instituciones? ¿Qué desventajas podría traerle la distribución de esa misma información? ¿Y en el caso de que su situación financiera no fuera desahogada?

4. ¿Qué papel tiene la prensa libre a la hora de controlar el abuso de bases de datos? (Por ejemplo, ¿en qué grado contribuye la prensa a la formación de la opinión pública o a poner de manifiesto los abusos?)

Problemas de repaso

(Los problemas marcados con asterisco están asociados con las secciones opcionales.)

- Resuma las diferencias entre un archivo plano y una base de datos.
- ¿Qué queremos decir al hablar de independencia de los datos?
- ¿Cuál es el papel de un sistema DBMS en la arquitectura de niveles de una implementación de base de datos?
- ¿Cuál es la diferencia entre un esquema y un subesquema?
- Identifique dos ventajas de separar el software de aplicación del DBMS.
- Describa la similitudes entre un tipo abstracto de datos (Capítulo 8) y un modelo de base de datos.
- Identifique el nivel dentro de un sistema de base de datos (usuario, programador del software de aplicación, diseñador del software DBMS) en el que tiene lugar cada una de estas actividades o en el que se presenta cada uno de estos problemas:
 - ¿Cómo habría que almacenar los datos en un disco para maximizar la eficiencia?
 - ¿Hay algún asiento libre en el vuelo 243?
 - ¿Cómo debe organizarse una relación en el almacenamiento masivo?
 - ¿Cuántas veces hay que permitir que un usuario escriba mal su contraseña antes de dar por terminada la conversación?
 - ¿Cómo puede implementarse la operación PROJECT?
- ¿Cuáles de las siguientes tareas son gestionadas por un DBMS?
 - Garantizar que el acceso de un usuario a la base de datos está restringido al subesquema apropiado.
 - Traducir los comandos expresados en términos del modelo de base de datos a acciones compatibles con el sistema real de almacenamiento de datos.
 - Ocultar el hecho de que los datos de la base de datos en la práctica están dispersos entre múltiples computadoras conectadas en red.
- Describa cómo se representaría en una base de datos relacional la siguiente información acerca de líneas aéreas, vuelos (para un día concreto) y pasajeros:
 Líneas aéreas: Clear Sky, Long Hop y Tree Top
 Vuelos de Clear Sky: CS205, CS37 y CS102
 Vuelos de Hop: LH67 y LH89
 Vuelos de Tree Top: TT331 y TT809
 Suarez tiene reservas en los vuelos CS205 (asiento 12B), CS37 (asiento 18C) y LH 89 (asiento 14A).
 Barrios tiene reservas en los vuelos CS37 (asiento 18B) y LH89 (asiento 14B).
 Salas tiene reservas en los vuelos LH67 (asiento 5A) y TT331 (asiento 4B).
- ¿Hasta qué punto es significativo el orden en que se aplican las operaciones SELECT y PROJECT a una relación? Es decir, ¿en qué condiciones el seleccionar y luego proyectar producirá los mismos resultados que si primero proyectamos y luego seleccionamos?
- Proporcione un argumento que demuestre que la cláusula "where" de la operación JOIN, tal como se describe en la Sección 9.2, no es necesaria. (Es decir, demuestre que

cualquier consulta que utilice una cláusula "where" puede expresarse también utilizando una operación JOIN que concatene cada tupla de una relación con cada tupla de la otra.)

12. En función de las relaciones que se muestran a continuación, ¿cuál sería el aspecto de la relación RESULT después de ejecutar cada una de estas sentencias:
- RESULT $\leftarrow$ PROJECT U and W from X1
 - RESULT $\leftarrow$ SELECT from X1 where W = 50
 - RESULT $\leftarrow$ PROJECT S from X2
 - RESULT $\leftarrow$ JOIN X1 and X2 where X1.W $\geq$ X2.R

relación X1			relación X2	
U	V	W	R	S
AA	ZZ	50	30	JJ
BB	DD	30	40	KK
CC	QQ	50		

13. Utilizando los comandos SELECT, PROJECT y JOIN, escriba una secuencia de instrucciones para responder a cada una de las siguientes cuestiones acerca de los componentes y de sus fabricantes en términos de la siguiente base de datos:

relación COMPONENTE

NombreComp	Peso
Tornillo 2X	1
Tornillo 2Z	1.5
Tuerca V5	0.5

relación FABRICANTE

NombreEmpresa	NombreComp	Coste
Empresa X	Tornillo 2Z	.03
Empresa X	Tuerca V5	.01
Empresa Y	Tornillo 2X	.02
Empresa Y	Tuerca V5	.01
Empresa Y	Tornillo 2Z	.04
Empresa Z	Tuerca V5	.01

- ¿Qué empresas fabrican el tornillo 2Z?
- Obtenga una lista de los componentes fabricados por la Empresa X junto con el coste de cada componente.
- ¿Qué empresas fabrican algún componente con un peso igual a 1?

14. Responda al Problema 13 utilizando SQL.

15. Utilizando los comandos SELECT, PROJECT y JOIN, escriba secuencias que permitan responder a las siguientes cuestiones acerca de las relaciones EMPLEADO, PUESTO y ASIGNACION de la Figura 9.5:

- Obtenga una lista de los nombres y direcciones de los empleados de la empresa.
- Obtenga una lista de los nombres y direcciones de aquellos que han trabajado o están trabajando en el departamento de personal.
- Obtenga una lista de los nombres y direcciones de aquellos que están trabajando en el departamento de personal.

16. Responda al problema anterior utilizando SQL.

17. Diseñe una base de datos relacional que contenga información acerca de compositores de música, sus vidas y sus obras. (Evite redundancias similares a las de la Figura 9.4.)

18. Diseñe una base de datos relacional que contenga información acerca de autores, de sus obras y de sus editores. (Evite redundancias similares a las de la Figura 9.4.)

19. Diseñe una base de datos relacional que contenga información acerca de una editorial y de los títulos que ha publicado. (Evite redundancias similares a las de la Figura 9.4.)

20. Diseñe una base de datos relacional que contenga información acerca de empresas, productos y clientes. (Evite redundancias similares a las de la Figura 9.4.)

21. Diseñe una base de datos relacional que contenga información acerca de los empleados de una empresa, de sus proyectos y de la distribución de los empleados para los proyectos. En este sistema, cada empleado puede trabajar en más de un proyecto y un proyecto puede tener más de un empleado trabajando en él. (Evite redundancias similares a las de la Figura 9.4.)

22. Escriba una secuencia de instrucciones (utilizando las operaciones SELECT, PROJECT y

JOIN) para extraer los campos *IdPuesto*, *FechaInicio* y *FechaFin* para cada puesto del departamento de contabilidad a partir de la base de datos relacional descrita en la Figura 9.5.

23. Responda al problema anterior utilizando SQL.
24. Escriba una secuencia de instrucciones (utilizando las operaciones SELECT, PROJECT y JOIN) para extraer los campos *Nombre*, *NSS*, *TitPuesto* y *CodCat* de cada uno de los empleados actuales a partir de la base de datos relacional descrita en la Figura 9.5.
25. Responda al problema anterior utilizando SQL.
26. Escriba una secuencia de instrucciones (utilizando las operaciones SELECT, PROJECT y JOIN) para extraer los campos *Nombre* y *NSS* del empleado cuyo *IdPuesto* es S25X a partir de la base de datos relacional descrita en la Figura 9.5.
27. Responda al problema anterior utilizando SQL.
28. ¿Cuál es la diferencia entre la información suministrada por la única relación

Nombre	Departamento	NumTelefono
Juarez	Ventas	555-2222
Salas	Ventas	555-3333
Barrio	Personal	555-4444

y las dos relaciones

Nombre	Departamento
Juarez	Ventas
Salas	Ventas
Barrio	Personal

Departamento	NumTelefono
Ventas	555-2222
Ventas	555-3333
Personal	555-4444

29. Diseñe una base de datos relacional que contenga información acerca de componen-

tes de automóviles y sus subcomponentes. Asegúrese de permitir que un componente pueda contener componentes más pequeños y, al mismo tiempo, estar contenido en otros componentes más grandes.

30. Elija un sitio web popular, como por ejemplo www.rediffmail.com, www.yahoo.com o www.amazon.com y diseñe una base de datos relacional que pudiera servir como base de datos de soporte para el sitio.

31. En referencia a la base de datos representada en la Figura 9.5, indique a qué cuestión se respondería mediante el siguiente segmento de programa:

```
TEMP ← SELECT from ASIGNACION
      where FechaFin = "*"
RESULT ← PROJECT IdPuesto,
               FechaInicio from TEMP
```

32. Traduzca la consulta del problema anterior a SQL.

33. En referencia a la base de datos representada en la Figura 9.5, indique a qué cuestión se respondería mediante el siguiente segmento de programa:

```
TEMP1 ← JOIN EMPLEADO and ASIGNACION
      where EMPLEADO.IdEmpl =
            ASIGNACION.IdEmpl
TEMP2 ← SELECT from TEMP1 where
      FechaFin = "*"
RESULT ← PROJECT Nombre, FechaInicio
      from TEMP2
```

34. Traduzca la consulta del problema anterior a SQL.

35. En referencia a la base de datos representada en la Figura 9.5, indique a qué cuestión se respondería mediante el siguiente segmento de programa:

```
TEMP1 ← JOIN EMPLEADO and ASIGNACION
      where EMPLEADO.IdEmpl =
            ASIGNACION.IdEmpl
TEMP2 ← SELECT from TEMP1 where
      IdPuesto = "F5"
RESULT ← PROJECT Nombre from TEMP2
```

36. Traduzca la consulta del problema anterior a SQL.

37. Traduzca la siguiente sentencia SQL

```
select PUESTO, TitPuesto
from EMPLEADO, PUESTO
where EMPLEADO.IdPuesto =
      PUESTO.IdPuesto
and EMPLEADO.IdEmpl = "12Y34"
```

a una secuencia de operaciones SELECT, PROJECT y JOIN.

38. Traduzca la siguiente sentencia SQL

```
select PROYECTO.FechaInicio
from PROYECTO, ESTUDIANTE
where PROYECTO.IdEstudiante =
      ESTUDIANTE.IdEstudiante
and ESTUDIANTE.Nombre = "David Rayo"
```

a una secuencia de operaciones SELECT, PROJECT y JOIN.

39. Describa el efecto que la siguiente sentencia SQL tendría sobre la base de datos del Problema 13.

```
insert into FABRICANTE
values (' Empresa Z', 'Tornillo 2X',
      .03)
```

40. Describa el efecto que la siguiente sentencia SQL tendría sobre la base de datos del Problema 13.

```
update FABRICANTE
set Coste = .03
where NombreEmpresa = 'Empresa Y'
and NombreComp = 'Tornillo 2X'
```

- *41. Identifique algunos de los objetos que podríamos esperar encontrar en una base de datos orientada a objetos que se utilizara para mantener el inventario de una frutería. ¿Qué métodos esperaríamos encontrar en cada uno de esos objetos?

- *42. Identifique algunos de los objetos que podríamos esperar encontrar en una base de datos orientada a objetos utilizada para mantener los registros de los libros de una biblioteca. ¿Qué métodos esperaríamos encontrar en cada uno de esos objetos?

- *43. ¿Qué información incorrecta sería generada por la siguiente secuencia de transacciones T1 y T2?

T1 está diseñada para calcular la suma de las cuentas bancarias A y B; T2 está diseñada para transferir 100 euros de la cuenta A a la B. T1 comienza extrayendo el saldo de la cuenta A; a continuación, T2 realiza su transferencia y, finalmente, T1 extrae el saldo de la cuenta B e informa de la suma de los valores que ha extraído.

- *44. Explique cómo podría resolverse el error existente en el Problema 43 utilizando el protocolo de bloqueo descrito en el texto.

- *45. ¿Qué efecto tendría el protocolo de desalojo y espera sobre la secuencia de sucesos del Problema 43, si T1 fuera la transacción más reciente? ¿Y si la transacción más reciente fuera T2?

- *46. Suponga que una transacción trata de sumar 100 euros a una cuenta bancaria cuyo saldo es de 200 euros, al mismo tiempo que otra transacción trata de retirar 100 euros de la misma cuenta. Describa un entremezclado de estas transacciones que conduciría a un saldo final de 100 euros. Describa un entremezclado de estas transacciones que conduciría a un saldo final de 300 euros.

- *47. ¿Cuál es la diferencia entre una transacción con acceso exclusivo y otra con acceso compartido a un elemento de la base de datos y por qué esa diferencia es importante?

- *48. Los problemas expuestos en la Sección 9.4 en relación con las transacciones concurrentes no están limitados a los entornos de bases de datos. ¿Qué problemas similares surgirían a la hora de acceder a un documento de un procesador de textos? (Si dispone de un PC con un procesador de textos, intente acceder al mismo documento con dos activaciones de ese programa procesador de textos y vea lo que sucede.)

- *49. Suponga que un archivo secuencial contiene 50.000 registros y que hacen falta 5 milisegundos para consultar cada entrada. ¿Cuánto estima que tendríamos que esperar a la hora de extraer un registro situado en mitad del archivo?

- *50.** Enumere los pasos que se ejecutan en el algoritmo de combinación de la Figura 9.15 si uno de los archivos de entrada está vacío al comienzo.
- *51.** Modifique el algoritmo de la Figura 9.15 para manejar el caso en el que ambos archivos de entrada contengan un registro con el mismo valor en el campo clave. Suponga que estos registros son idénticos y que solo uno de ellos debe aparecer en el registro de salida.
- *52.** Diseñe un sistema por el que pueda procesarse como archivo secuencial un archivo almacenado en un disco, con una de dos posibles ordenaciones.
- *53.** Describa cómo podría construirse un archivo secuencial que contenga información acerca de los subscriptores de una revista, utilizando un archivo de texto como estructura subyacente.
- *54.** Diseñe una técnica que permita implementar como archivo de texto un archivo secuencial cuyos registros lógicos no tengan todos el mismo tamaño. Por ejemplo, suponga que deseamos construir un archivo secuencial en el que cada registro lógico contenga información acerca de un novelista, así como una lista de las obras de dicho autor.
- *55.** ¿Qué ventajas tiene un archivo indexado con respecto a un archivo hash? ¿Qué ventajas tiene un archivo hash con respecto a un archivo indexado?
- *56.** En el capítulo se ha trazado un paralelismo entre el índice de un archivo tradicional y el sistema de directorios mantenido por un sistema operativo. ¿En qué se diferencian un índice tradicional y el directorio de archivos de un sistema operativo?
- *57.** Si partimos un archivo hash en 10 fragmentos, ¿cuál es la probabilidad de que al menos dos de tres registros seleccionados arbitrariamente correspondan al mismo fragmento? (Suponga que la función hash no da a ningún fragmento prioridad con respecto a los otros.) ¿Cuántos registros habrá que almacenar en el archivo hasta para que sea más probable que se produzcan colisiones que no se produzcan?
- *58.** Resuelva el problema anterior, suponiendo que el archivo se parte en 100 fragmentos en lugar de en solo 10.
- *59.** Si estamos utilizando la técnica de división explicada en el capítulo para obtener una función hash y dividimos el área de almacenamiento del archivo en 23 fragmentos, ¿en qué fragmento deberemos buscar para encontrar el registro cuya clave, al ser interpretada como un valor binario, tenga el entero 124?
- *60.** Compare la implementación de un archivo hash con la de una matriz homogénea bidimensional. ¿En qué sentido son similares los papeles de la función hash y del polinomio de dirección?
- *61.** Indique una ventaja que tenga
- a. un archivo secuencial con respecto a un archivo indexado.
 - b. un archivo secuencial con respecto a un archivo hash.
 - c. un archivo indexado con respecto a un archivo secuencial.
 - d. un archivo indexado con respecto a un archivo hash.
 - e. un archivo hash con respecto a un archivo secuencial.
 - f. un archivo hash con respecto a un archivo indexado.
- *62.** ¿En qué se parece un archivo secuencial a una lista enlazada?

Cuestiones sociales

Las siguientes cuestiones pretenden ser una guía para los problemas éticos/sociales/legales asociados con el campo de la computación. El objetivo no es responder simplemente a estas cuestiones. El lector debería considerar también por qué las ha contestado de la forma en que lo ha hecho y analizar si sus justificaciones son coherentes entre las distintas cuestiones.

1. En Estados Unidos, los registros de ADN de todos los presos federales están ahora almacenados en una base de datos para utilizarlos en investigaciones criminales. ¿Sería ético permitir que se usara esta información para otros propósitos, por ejemplo, para investigaciones médicas? En caso afirmativo, ¿para qué propósitos sí que sería ético? En caso negativo, ¿por qué no? ¿Cuáles son los pros y los contras en cada caso?
2. ¿Hasta qué punto debería una universidad estar autorizada a hacer pública la información acerca de sus estudiantes? ¿Qué pasa con sus nombres y direcciones? ¿Y qué pasa con la distribución de datos sin identificar a los estudiantes? ¿Es su respuesta coherente con la que ha proporcionado a la Cuestión 1?
3. ¿Qué restricciones son apropiadas en relación con la construcción de bases de datos con información personal? ¿Qué información tiene derecho el gobierno a poseer acerca de sus ciudadanos? ¿Qué información tiene derecho a tener una empresa de seguros sobre sus clientes? ¿Qué información tiene una empresa derecho a poseer en relación con sus empleados? ¿Deberían implementarse controles en todos estos casos? En caso afirmativo, ¿cuáles?
4. ¿Es apropiado que una empresa de tarjetas de crédito venda a las empresas de marketing información acerca de los hábitos de compra de sus clientes? ¿Es aceptable que una empresa de comercialización de coches deportivos venda su lista de correo a una revista de coches deportivos? ¿Es aceptable que el organismo encargado de gestionar los impuestos en Estados Unidos venda a los intermediarios de bolsa los nombres y direcciones de aquellos contribuyentes que han obtenido ganancias de capital significativas? Si no es capaz de responder a estas preguntas con un simple sí o no y sin matizaciones, ¿cuál cree que sería una política aceptable?
5. ¿Hasta qué grado es responsable el diseñador de una base de datos del modo en que se utilice la información contenida en dicha base de datos?
6. Suponga que una base de datos permite por error un acceso no autorizado a la información que contiene. Si alguien obtiene y utiliza esa información de forma maliciosa, ¿hasta qué punto comparten los diseñadores de la base de datos la responsabilidad por el mal uso de esa información? ¿Depende su respuesta del esfuerzo que el criminal haya necesitado para descubrir el fallo en el diseño de la base de datos y obtener la información no autorizada?
7. El uso cada vez mayor de técnicas de minería de datos plantea numerosas cuestiones relativas a la ética y a la intimidad. ¿Cree que nuestro derecho a la intimidad habrá sido vulnerado si la minería de datos revela ciertas

características acerca de la población global de nuestra ciudad? ¿Cree que el uso de las técnicas de minería de datos promueve prácticas empresariales adecuadas o prácticas fraudulentas? ¿Hasta qué grado es correcto obligar a los ciudadanos a participar en la elaboración de un censo, sabiendo que va a extraerse de esos datos más información que la que se está solicitando explícitamente en los cuestionarios individuales? ¿Proporciona la minería de datos a las empresas de marketing una ventaja injusta con respecto a las audiencias a las que se dirigen sus mensajes publicitarios, que no sospechan nada acerca de las técnicas que se utilizan? ¿Hasta qué punto es buena o mala la práctica de construir perfiles de clientes?

8. ¿Hasta qué punto debería permitirse a una persona o a una empresa recopilar y almacenar información acerca de las personas? ¿Qué pasa si esa información que se recopila ya está disponible públicamente, aunque esté dispersa entre distintas fuentes? ¿Hasta qué grado deben las personas o empresas que recopilen esa información proteger la información obtenida?
9. Muchas bibliotecas ofrecen un servicio de referencias, para que los usuarios puedan solicitar la ayuda de un bibliotecario a la hora de buscar información. ¿Cree que la existencia de Internet y de las tecnologías de bases de datos ha hecho que este servicio quede obsoleto? En caso afirmativo, ¿cree que es un paso hacia adelante o un paso hacia atrás? En caso negativo, ¿por qué no? ¿Cómo afecta a la existencia de las bibliotecas la generalización de Internet y de la tecnología de bases de datos?
10. ¿Hasta qué grado cree que está usted mismo expuesto a la posibilidad de robo de su identidad? ¿Qué pasos puede dar para minimizar ese riesgo? ¿Qué daños cree que podría sufrir si fuera víctima de un robo de identidad? ¿Sería usted responsable en caso de que se produjera ese robo de identidad?

Lecturas adicionales

Beg, C. E. y T. Connolly. *Database Systems: A Practical Approach to Design, Implementation and Management*, 4ª ed. Boston, MA: Addison-Wesley, 2005.

Berstein, A., M. Kifer y P. M. Lewis. *Database Systems*, 2ª ed. Boston, MA: Addison-Wesley, 2006.

Date, C. J. *An Introduction to Database Systems*, 8ª ed. Boston, MA: Addison-Wesley, 2004.

Date, C. J. *Databases, Types and the Relational Model*, 3ª ed. Boston, MA: Addison-Wesley, 2007.

Elmasri, R. y S. Navathe. *Fundamentals of Database Systems*, 6ª ed. Boston, MA: Addison-Wesley, 2011.

Patrick, J. J. *SQL Fundamentals*, 3ª ed. Upper Saddle River, NJ: Prentice-Hall, 2009.

Silberschatz, A., H. Korth y S. Sudarshan. *Database Systems Concepts*, 8ª ed. New York: McGraw-Hill, 2009.

Ullman, J. D. y J. D. Widom. *A First Course in Database Systems*, 3ª ed. Upper Saddle River, NJ: Prentice-Hall, 2008.

Gráficos por computadora

En este capítulo vamos a explorar el campo de los gráficos por computadora, un campo que está teniendo un impacto muy significativo en la producción de películas y videojuegos interactivos. De hecho, los avances en este campo están liberando a las empresas de producción de contenido audiovisual de las restricciones de la realidad y hay mucha gente que sostiene que las animaciones generadas por computadora pueden llegar pronto a eliminar la necesidad de utilizar actores tradicionales, estudios y medios fotográficos en los sectores de la televisión y del cine.

10.1 El ámbito de los gráficos por computadora

10.2 Panorámica de los gráficos 3D

10.3 Modelado

Modelado de objetos individuales
Modelado de escenas completas

10.4 Generación (rendering)

Interacción entre la luz y las superficies
Recorte, conversión de barrido y eliminación de caras ocultas

Sombreado (*shading*)

Hardware para la cadena de generación

*10.5 Iluminación global de las escenas

Trazado de rayos
Radiosidad

10.6 Animación

Fundamentos de animación
Cinemática y Dinámica
El proceso de animación

**Las secciones marcadas con asterisco se sugieren como secciones opcionales.*

Los gráficos por computadora son la rama de las Ciencias de la computación que aplica tecnología informática a la producción y manipulación de representaciones visuales. Se asocian con una amplia variedad de temas, incluyendo la presentación de textos, la generación de gráficos y diagramas, el desarrollo de interfaces gráficas de usuario, la manipulación de fotografías, la producción de videojuegos y la creación de películas animadas. Sin embargo, el término *gráficos por computadora* se utiliza cada vez más para hacer referencia a un campo específico denominado gráficos 3D y la mayor parte de este capítulo se concentrará en este tema. Comenzaremos definiendo lo que son los gráficos 3D y clarificando su papel dentro de esa interpretación más amplia del término gráficos por computadora.

10.1 El ámbito de los gráficos por computadora

Con la aparición de las cámaras digitales, se ha incrementado rápidamente la popularidad del software para la manipulación de imágenes codificadas digitalmente. Este software permite "retocar" fotografías, eliminando manchas y esos malditos "ojos rojos" que aparecen en los retratos, así como cortar y pegar partes de diferentes fotografías con el fin de crear imágenes que no reflejen necesariamente la realidad.

A menudo se aplican técnicas similares para crear efectos especiales en los sectores de la televisión y del cine. De hecho, dichas aplicaciones fueron uno de los motivos principales para que estos sectores pasaran de los sistemas analógicos, como por ejemplo las películas físicas, a las imágenes codificadas digitalmente. Entre las aplicaciones podemos citar el eliminar la aparición de los cables de soporte en las tomas, la superposición de múltiples imágenes o la producción de cortas secuencias de imágenes nuevas que se emplean para modificar la acción originalmente capturada por una cámara.

Además de software para manipular fotografías digitales e imágenes de películas, existe ahora una amplia variedad de paquetes software de utilidad/aplicación que sirven de ayuda para la producción de imágenes bidimensionales; estos paquetes software van desde los simples dibujos basados en líneas hasta sofisticados paquetes de creación artística (un ejemplo elemental muy conocido es esa aplicación de Microsoft denominada Paint). Como mínimo, estos programas permiten al usuario dibujar puntos y líneas, insertar formas geométricas simples como óvalos y rectángulos, rellenar de color regiones de la imagen y cortar y pegar partes específicas de un dibujo.

Observe que todas las aplicaciones anteriores tratan con la manipulación de formas e imágenes planas bidimensionales. Son, por tanto, ejemplos de dos campos relacionados de investigación: uno de ellos son los **gráficos 2D** y el otro es el **procesamiento de imágenes**. La distinción es que los gráficos 2D se centran en la tarea de convertir formas bidimensionales (círculos, rectángulos, letras, etc.) en patrones de píxeles con el fin de producir una imagen, mientras que el campo del procesamiento de imágenes, con el que nos toparemos de nuevo más adelante cuando abordemos el estudio de la inteligencia artificial, se centra en analizar los píxeles de una imagen con el fin de identificar patrones que puedan utilizarse para mejorar, o quizá, para "entender" la imagen. En resumen, los gráficos 2D tratan con la producción de imágenes mientras que el procesamiento de imágenes se ocupa del análisis de esas imágenes.

A diferencia del proceso de convertir formas bidimensionales en imágenes, como sucede en los gráficos 2D, el campo de los **gráficos 3D** trata con la conversión de formas tridimensionales en imágenes. El proceso consiste en construir versiones de escenas tridimensionales codificadas digitalmente y luego simular el proceso fotográfico, con el fin de generar imágenes de esas escenas. El objetivo es análogo al de la fotografía tradicional, salvo porque la escena que está siendo "fotografiada" mediante técnicas de gráficos 3D no existe como realidad física, sino que tan solo "existe" como un conjunto de datos y algoritmos. En otras palabras, los gráficos 3D implican "fotografiar" mundos virtuales (Figura 10.1), mientras que la fotografía tradicional pretende fotografiar el mundo real.

Es importante observar que la creación de una imagen mediante gráficos 3D implica dos pasos diferentes. Uno de ellos es la creación, codificación, almacenamiento y manipulación de la escena que se quiere fotografiar. El otro es el propio proceso de producción de la imagen. El primero es un proceso creativo, artístico; el segundo es un proceso computacionalmente muy costoso. Estos son temas que iremos viendo en las cuatro secciones siguientes.

El hecho de que los gráficos 3D generen "fotografías" de escenas virtuales los convierte en ideales para su uso en la producción de videojuegos interactivos y de películas, en donde las restricciones de la realidad limitarían la acción si no se emplearan estas técnicas avanzadas. Un videojuego interactivo está compuesto por un entorno virtual tridimensional codificado, con el que el jugador interactúa. Las imágenes que ve el jugador se generan mediante tecnología de gráficos 3D. Las películas animadas se crean de forma similar, salvo porque es el animador humano el que interactúa con el entorno virtual en lugar de que el que interactúe sea el espectador. El producto que al final se distribuye al

Figura 10.1 Una "fotografía" de un mundo virtual generada mediante gráficos 3D (de *Toy Story* por Walt Disney Pictures/Pixar Animation Studios) © Disney/Pixar.



público es una secuencia de imágenes bidimensionales, que estará determinada por el director/productor de la película.

En la Sección 10.6 investigaremos más en detalle el uso de los gráficos 3D en las animaciones. Por el momento, cerramos esta sección imaginando a dónde nos pueden conducir estas aplicaciones a medida que avance la tecnología de gráficos 3D. Hoy día, las películas se distribuyen como secuencias de imágenes bidimensionales. Aunque los proyectores que permiten visualizar esta información han progresado con respecto a los dispositivos analógicos que utilizaban cintas de películas, para terminar utilizando tecnología digital que emplea reproductores de DVD y pantallas planas, esos proyectores siguen teniendo que tratar únicamente con representaciones bidimensionales.

Imagine, sin embargo, cómo podría cambiar esto a medida que mejore nuestra capacidad de crear y manipular mundos virtuales tridimensionales realistas. En lugar de "fotografiar" estos mundos virtuales y distribuir una película en forma de secuencia de imágenes bidimensionales podríamos distribuir los mundos virtuales. Un espectador potencial recibiría acceso al "estudio" de la película, en lugar de limitarse a verla. Este estudio tridimensional sería entonces visualizado por medio de un "proyector de gráficos 3D", de forma bastante similar a como se visualizan los videojuegos utilizando "consolas de juegos" especializadas. El espectador podría primero contemplar una "trama sugerida", viendo así la película tal como la hubiera imaginado el director/productor. Pero el espectador podría también interactuar con el estudio virtual de una forma parecida a la de los videojuegos, con el fin de generar otros escenarios. Las posibilidades son inmensas, especialmente cuando consideramos también el potencial de las interfaces persona-máquina tridimensionales que se están desarrollando.

Cuestiones y ejercicios

1. Resuma la diferencia entre el procesamiento de imágenes, los gráficos 2D y los gráficos 3D.
2. ¿En qué sentido difieren los gráficos 3D de la fotografía tradicional?
3. ¿Cuáles son los dos pasos principales a la hora de producir una "fotografía" utilizando gráficos 3D?

10.2 Panorámica de los gráficos 3D

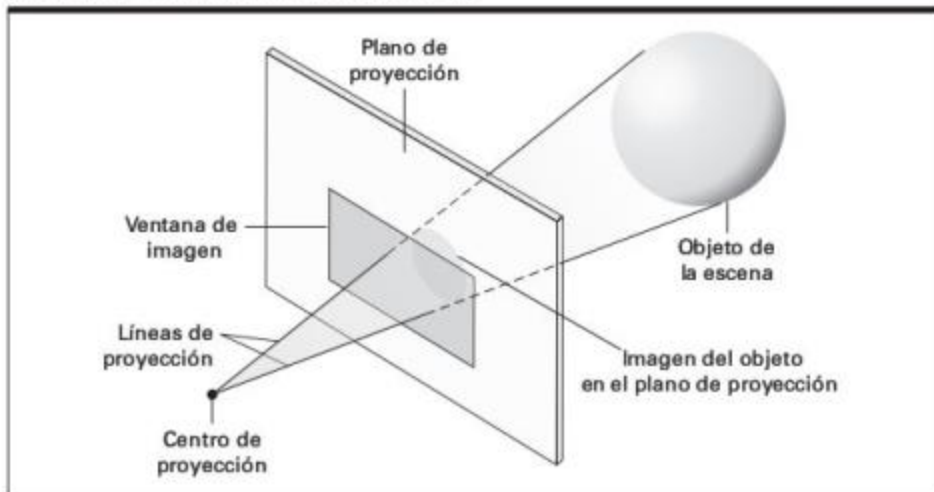
Iniciamos nuestro estudio sobre los gráficos 3D considerando el proceso completo de creación y visualización de imágenes, un proceso que consta de tres pasos: modelado, generación y visualización. La fase de modelado (que exploraremos en detalle en la Sección 10.3) es análoga al diseño y construcción de un estudio en la industria del cine tradicional, salvo porque la escena de gráficos 3D se "construye" a partir de datos y de algoritmos codificados digitalmente. Es decir, la escena generada en un contexto de utilización de gráficos por computadora puede que nunca llegue a existir en realidad.

El siguiente paso es generar una imagen bidimensional de la escena, calculando cómo aparecerían los objetos de la escena en una fotografía hecha con una cámara que estuviera situada en una posición específica. Este paso se denomina **generación** (*rendering*), que será el tema de las Secciones 10.4 y 10.5. La generación o reproducción implica aplicar las matemáticas de la geometría analítica para calcular la proyección de los objetos de la escena sobre una superficie plana, que se conoce con el nombre de **plano de proyección**; este proceso de proyección es en cierto modo análogo a la manera en la que una cámara proyecta una escena sobre una película (Figura 10.2). El tipo de proyección aplicada es una **proyección en perspectiva**, lo que quiere decir que todos los objetos se proyectan a lo largo de líneas rectas, denominadas **proyectores**, que salen de un punto común, denominado **centro de proyección** o **punto de vista**. (Este tipo de proyección difiere de la **proyección paralela** en la que las líneas de proyección son paralelas. Una proyección en perspectiva produce una imagen similar a la que normalmente percibe el ojo humano, mientras que una proyección paralela produce un "verdadero" perfil de un objeto, que a veces resulta útil en el contexto de los dibujos utilizados en el campo de la ingeniería.)

La parte restringida del plano de proyección que define las fronteras de la imagen final se conoce como **ventana de imagen**. Corresponde con el rectángulo que se muestra en la pantalla de la mayoría de las cámaras digitales para indicar los límites de la fotografía que se va a tomar. De hecho, la pantalla de la mayor parte de las cámaras permite ver una parte del plano de proyección de la cámara más grande que la propia ventana de imagen. (Puede que se vea en la pantalla la parte superior de la cabeza de la "tía Marta", pero a menos que toda la cabeza caiga dentro de la ventana de imagen, la pobre tía Marta aparecerá recortada en la fotografía final.)

Una vez identificada la parte de la escena que se proyecta sobre la ventana de imagen, habrá que calcular la apariencia de cada píxel de la imagen final. Este proceso píxel a píxel puede ser computacionalmente muy complejo, por-

Figura 10.2 El paradigma de los gráficos 3D.



que requiere determinar cómo interactúan los objetos de la escena con la luz: una superficie dura y brillante bajo una luz intensa tiene que mostrarse de forma diferente que una superficie suave y transparente bajo una luz indirecta. A su vez, el proceso de generación utiliza numerosas técnicas de distintos campos, incluyendo la ciencia de los materiales y la Física. Además, determinar la apariencia de un objeto requiere a menudo disponer de información de los otros objetos que componen la escena. El objeto podría estar dentro de la sombra proyectada por otro objeto, o bien el objeto puede ser un espejo, cuya apariencia será esencialmente la de algún otro objeto.

A medida que se determina la apariencia de cada píxel, los resultados se almacenan colectivamente en forma de representación de mapa de bits de la imagen en un área de almacenamiento denominada **memoria de fotograma**. Este buffer puede ser un área de la memoria principal o, en el caso del hardware diseñado específicamente para aplicaciones gráficas, puede ser un bloque de circuitos de memoria de propósito especial.

Finalmente, la imagen almacenada en la memoria de fotograma se visualiza directamente o se transfiere a un medio de almacenamiento más permanente para su visualización posterior. Si la imagen está siendo producida para su uso en una película, se la puede almacenar, e incluso también modificar, antes de su presentación final. Sin embargo, en un videojuego interactivo o en un simulador de vuelo las imágenes deben ser visualizadas a medida que son producidas en tiempo real, lo que constituye un requisito que a menudo limita la calidad de las imágenes creadas. Esta es la razón por la que la calidad gráfica de las producciones animadas distribuidas por las productoras de cine es superior a la de los videojuegos interactivos actuales.

Vamos a terminar nuestra introducción a los gráficos 3D analizando un sistema típico de videojuegos. El propio juego es, esencialmente, un mundo virtual codificado, junto con un software que permite que el jugador manipule dicho mundo. A medida que el jugador manipula el mundo virtual, el sistema genera repetidamente la escena y almacena las imágenes en la memoria de fotograma. Para satisfacer las restricciones de tiempo real, buena parte de este proceso de generación se lleva a cabo mediante hardware de propósito especial. De hecho, la presencia de este hardware es una de las características que diferencian a una consola de juegos de una computadora personal genérica. Por último, el dispositivo de visualización del sistema de juegos muestra el contenido de la memoria de fotograma, creando en el jugador la ilusión de que la escena va cambiando.

Cuestiones y ejercicios

1. Resuma los tres pasos implicados en la producción de una imagen utilizando gráficos 3D.
2. ¿Cuál es la diferencia entre el plano de proyección y la ventana de imagen?
3. ¿Qué es una memoria de fotograma?

10.3 Modelado

Un proyecto de gráficos 3D comienza de forma bastante similar a una función teatral: es preciso diseñar un "estudio" y hace falta recopilar o construir el utillaje adecuados. En la terminología de los gráficos por computadora, el estudio se denomina **escena** y el utillaje son los **objetos**. Recuerde que una escena de gráficos 3D es virtual porque está compuesta por objetos que se "construyen" como modelos codificados digitalmente, en lugar de ser estructuras físicas tangibles.

En esta sección vamos a explorar los temas relacionados con la "construcción" de objetos y escenas. Comenzaremos con los problemas del modelado de objetos individuales y concluiremos considerando la tarea de recopilar dichos objetos para formar una escena.

Modelado de objetos individuales

En la producción teatral, el grado con el que el escenario se adapta a la realidad depende de cómo se vaya a utilizar en la escena. Puede que no necesitemos poner en el escenario un automóvil completo, es posible que el teléfono no necesite funcionar y la escena de fondo puede estar pintada sobre una tela. De la misma forma, en el caso de los gráficos por computadora, el grado de precisión con el que el modelo software de un objeto tenga que reflejar las propiedades reales del objeto dependerá de los requisitos de cada situación concreta. Será necesario un mayor grado de detalle para modelar los objetos que vayan a estar en primer plano que los que vayan a ser objetos de fondo. Además, también puede generarse más detalle en aquellos casos que no estén sometidos a tremendas restricciones de tiempo real.

Algunos modelos de objetos pueden ser realmente simples, mientras que otros pueden tener una enorme complejidad. Como regla general, los modelos más precisos permiten obtener imágenes de mayor calidad, pero exigen tiempos más largos de generación. A su vez, buena parte de la investigación actual en el campo de los gráficos por computadora trata de desarrollar técnicas para la construcción de modelos de objetos altamente detallados, pero que sigan siendo eficientes. Parte de esta investigación se ocupa del desarrollo de modelos que puedan proporcionar diferentes niveles de detalle, dependiendo del papel último del objeto dentro de la escena; el resultado de ese desarrollo sería un único modelo de objeto que podría utilizarse dentro de un entorno cambiante.

La información requerida para describir un objeto incluye la forma del objeto, además de otras propiedades adicionales, como las características superficiales que determinan cómo interactuará el objeto con la luz. Por ahora, vamos a considerar la tarea de modelado de la forma.

Forma La forma de un objeto en el campo de los gráficos 3D suele describirse como un conjunto de pequeñas superficies planas denominadas **parches planos**, cada una de las cuales tiene forma de polígono. El conjunto de todos estos polígonos forma una **mallla poligonal** que aproxima la forma del objeto que se está describiendo (Figura 10.3). Utilizando parches planos pequeños, la aproximación puede hacerse tan precisa como se desee.

Figura 10.3 Una malla poligonal para una esfera.

Los parches planos de una malla poligonal suelen elegirse de forma triangular, porque cada triángulo puede representarse mediante sus tres vértices, que es el número mínimo de puntos requerido para identificar una superficie plana en el espacio tridimensional. En cualquier caso, las mallas poligonales se representan mediante el conjunto de los vértices de sus parches planos componentes.

La representación mediante malla poligonal de un objeto puede obtenerse de varias formas. Una de ellas consiste en comenzar con una descripción geométrica precisa del objeto deseado y luego utilizar dicha descripción para construir una malla poligonal. Por ejemplo, la geometría analítica nos dice que una esfera (centrada en el origen) de radio r queda descrita por la ecuación

$$r^2 = x^2 + y^2 + z^2$$

Basándose en esta fórmula, podemos calcular las ecuaciones para las líneas de latitud y longitud de la esfera, identificar los puntos en los que esas líneas intersectan y luego usar esos puntos como vértices de una malla poligonal. Pueden aplicarse otras técnicas similares a otras formas geométricas tradicionales y esta es la razón por la que los personajes en las animaciones por computadora más sencillas a menudo parecen estar compuestos a partir de estructuras tales como esferas, cilindros y conos.

Las formas más generales pueden describirse por medios analíticos más sofisticados. Uno de ellos está basado en el uso de las **curvas de Bezier** (denominadas así por Pierre Bezier, que desarrolló el concepto a principios de la década de 1970 cuando trabajaba como ingeniero para la empresa automovilística Renault), que permiten definir un segmento de línea curva en el espacio tridimensional utilizando únicamente unos cuantos puntos, denominados puntos de control (dos de los cuales representan los extremos del segmento de curva, mientras que los otros indican cómo está distorsionada esa curva). Por ejemplo, la Figura 10.4 muestra una curva definida mediante cuatro puntos de

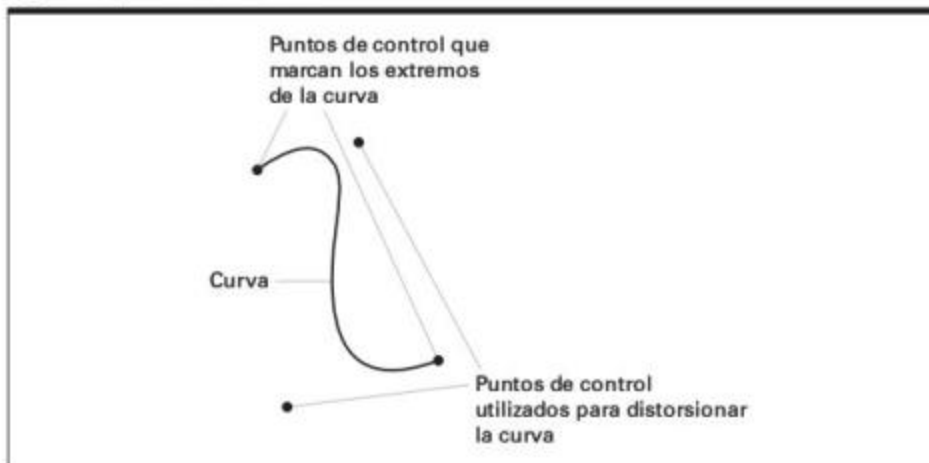
control. Observe cómo la curva parece haber sido estirada hacia los dos puntos de control que no identifican los extremos del segmento. Moviendo esos puntos de control, la curva puede doblarse para componer diferentes formas. (Es posible que el lector ya haya experimentado con este tipo de técnicas a la hora de construir líneas curvas con paquetes de dibujo como Paint de Microsoft.) Aunque no analizaremos el tema con más detalle aquí, las técnicas de Bezier para la descripción de curvas pueden ampliarse para describir superficies tridimensionales conocidas con el nombre de **superficies de Bezier**. A su vez, las superficies de Bezier han demostrado una gran eficiencia como primer paso en el proceso de obtención de mallas poligonales para superficies complejas.

Puede que se esté preguntando por qué es necesario convertir una descripción precisa de una forma geométrica, como por ejemplo la fórmula concisa de una esfera o las fórmulas que describen una superficie de Bezier, en una aproximación de esa forma geométrica utilizando una malla poligonal. La respuesta es que la representación de la forma de todos los objetos mediante mallas poligonales permite aplicar una solución uniforme al proceso de generación, un convenio que permite generar escenas completas de manera más eficiente. Así, aunque las fórmulas geométricas proporcionan descripciones precisas de las formas, se utilizan simplemente como herramientas para la construcción de mallas poligonales.

Otra forma de obtener una malla poligonal es construir la malla mediante la fuerza bruta. Esta técnica es popular en aquellos casos en los que una forma desafía todos los intentos de representarla mediante técnicas matemáticas elegantes. El procedimiento consiste en construir un modelo físico del objeto y luego anotar la ubicación de una serie de puntos de la superficie del modelo tocando la superficie con un dispositivo de tipo lápiz que registra su posición en el espacio tridimensional, un proceso que se conoce como **digitalización**. El conjunto de puntos obtenido se puede entonces utilizar como conjunto de vértices para la obtención de una malla poligonal que describa la forma del objeto.

Lamentablemente, algunas formas son tan complejas que obtener modelos realistas mediante un modelado geométrico o una digitalización manual no resulta factible. Entre los ejemplos podríamos citar las estructuras intrincadas

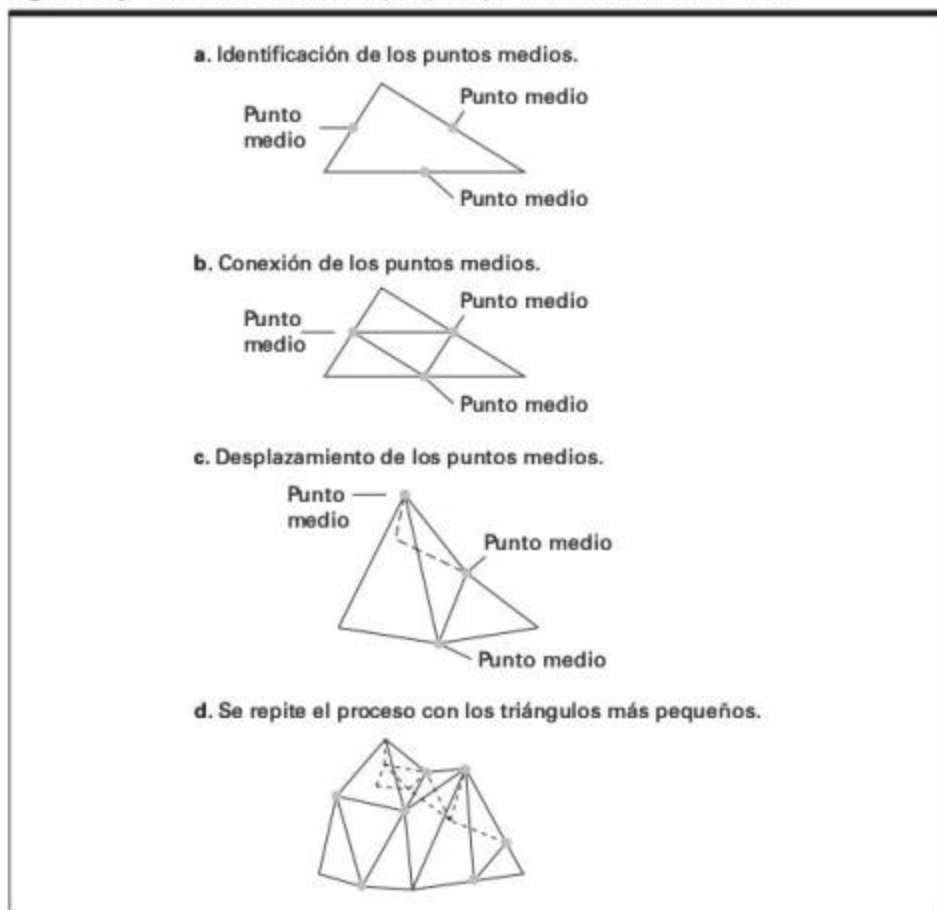
Figura 10.4 Curva de Bezier.



de algunas plantas, como los árboles; las secciones complejas de terrenos, como las cordilleras montañosas y sustancias gaseosas tales como nubes, el humo o las llamas de una hoguera. En estos casos, las mallas poligonales se obtienen escribiendo programas que generen automáticamente la forma geométrica deseada. Estos programas se conocen conjuntamente con el nombre de **modelos procedimentales**. En otras palabras, un modelo procedimental es un programa que aplica un algoritmo con el fin de generar una cierta estructura.

Por ejemplo, se han utilizado modelos procedimentales para generar cordilleras montañosas ejecutando los pasos siguientes: comenzando con un único triángulo, se identifican los puntos medios de las aristas del triángulo (Figura 10.5a). A continuación, se conectan estos puntos medios con el fin de formar un total de cuatro triángulos más pequeños (Figura 10.5b). Ahora, manteniendo fijos los vértices originales del triángulo, se desplazan los puntos intermedios en el espacio tridimensional (permitiendo que las aristas del triángulo se alarguen o contraigan) distorsionando así las formas triangulares (Figura 10.5c). Este proceso se repite con cada uno de los triángulos más pequeños (Figura 10.5d) y se sigue aplicando hasta obtener el grado de detalle deseado.

Figura 10.5 Creación de una malla poligonal para una cordillera montañosa.



Los modelos procedimentales proporcionan un medio eficiente de generar múltiples objetos complejos que sean similares entre sí, pero todos ellos distintos. Por ejemplo, un modelo procedimental se puede utilizar para construir una diversidad de objetos realistas que representen árboles, cada uno de ellos con su propia estructura de ramas, todas ellas similares. Una técnica para la obtención de esos modelos de árboles consiste en aplicar reglas de ramificación para "hacer crecer" esos objetos árbol de forma bastante similar a como un analizador sintáctico (Sección 6.4) construye un árbol de análisis sintáctico a partir de las reglas gramaticales. De hecho, el conjunto de reglas de ramificación utilizado en estos casos también se denomina gramática. Una gramática se puede diseñar para "hacer crecer" abetos, y emplear otra gramática distinta para "hacer crecer" olmos.

Otro método para construir modelos procedimentales consiste en simular la estructura subyacente de un objeto mediante un gran conjunto de partículas. Dichos sistemas se denominan **sistemas de partículas**. Usualmente, los sistemas de partículas aplican ciertas reglas predefinidas para mover las partículas

Fractales

La construcción de una cordillera montañosa por medio de un modelo procedimental, tal como se describe en el texto (véase la Figura 10.5) es un ejemplo del papel que desempeñan los fractales en los gráficos 3D. Técnicamente hablando, un **fractal** es un objeto geométrico cuya "dimensión de Hausdorff es mayor que su dimensión topológica". Lo que esto significa intuitivamente es que el objeto se forma "empaquetando" copias de un objeto de una dimensión menor. (Piense en la ilusión de anchura creada "empaquetando" múltiples segmentos de línea paralelos.) Un fractal normalmente se forma por medio de un proceso recursivo, en el que cada activación de la recursión "empaqueta" copias adicionales (aunque más pequeñas) del patrón que se está utilizando para construir el fractal. El fractal resultante es autosimilar, en el sentido de que cada porción, al ser ampliada, aparece como una copia del fractal completo.

Un ejemplo tradicional de fractal es el copo de nieve de von Koch, que se forma sustituyendo repetidamente los segmentos de línea recta de la estructura



por versiones más pequeñas de la misma estructura. Esto conduce a una secuencia de refinamientos que continúa de la forma siguiente:



Los fractales suelen ser la columna vertebral de los modelos procedimentales en el campo de los gráficos 3D. De hecho, han sido utilizados para generar imágenes realistas de cordilleras montañosas, de vegetación, de nubes y de humo.

que componen el sistema, quizá de una manera que recuerda en parte a las interacciones moleculares, con el fin de generar la forma geométrica deseada. Por ejemplo, se han utilizado sistemas de partículas para generar una animación de salpicaduras de agua, como veremos posteriormente al hablar de la animación. (Imagine un cubo de agua modelado como un cubo de canicas. A medida que el cubo se mueve, las canicas contenidas en el cubo también se mueven, simulando el movimiento del agua.) Otros ejemplos de aplicaciones de los sistemas de partículas serían escenas de llamas titilantes de un fuego, de nubes y de multitudes.

La salida de un modelo procedimental normalmente es una malla poligonal que se aproxima a la forma del objeto deseado. En algunos casos, como al generar una cordillera montañosa a partir de triángulos, la malla es una consecuencia natural del propio proceso de generación. En otros casos, como cuando se hace crecer un árbol mediante reglas de ramificación, la malla puede construirse como paso final adicional. Por ejemplo, en el caso de los sistemas de partículas, las partículas situadas en el borde exterior del sistema son candidatos naturales para actuar como vértices de la malla poligonal final.

El grado de precisión de la malla generada por un modelo procedimental puede depender de cada situación concreta. Un modelo procedimental para un árbol que vaya a aparecer al fondo de una escena podría generar una malla bastante burda que refleje solo la forma básica del árbol, mientras que un modelo procedimental para un árbol que vaya a aparecer en primer plano podría generar una malla en la que se distinguieran las ramas y las hojas individuales.

Características superficiales Un modelo compuesto simplemente de una malla poligonal captura únicamente la forma de un objeto. La mayoría de los sistemas de generación son capaces de enriquecer esos modelos durante el propio proceso de generación, con el fin de simular diversas características superficiales, a solicitud del usuario. Por ejemplo, aplicando diferentes técnicas de sombreado (que presentaremos en la Sección 10.4), un usuario puede especificar que la malla poligonal de una bola sea generada como una bola suave y de color rojo o como una bola rugosa de color verde. En algunos casos, esa flexibilidad es deseable, pero en aquellas situaciones en las que hace falta una generación fiel de un objeto original, es preciso incluir en el modelo información más específica acerca del objeto, para que el sistema de generación sepa qué es lo que tiene que hacer.

Existen diversas técnicas de codificar información acerca de un objeto más allá de su forma geométrica. Por ejemplo, junto con cada vértice de una malla poligonal, podría codificarse el color que el objeto original tiene en ese punto. Esta información podría entonces emplearse durante la generación para recrear la apariencia del objeto original.

En otros casos, pueden asociarse patrones de color con la superficie de un objeto por medio de un proceso conocido con el nombre de **aplicación de texturas**. La aplicación de texturas es similar al proceso de aplicar papel pintado a una pared, en el sentido de que asocia una imagen predefinida con la superficie del objeto. Esta imagen puede ser una fotografía digital, el dibujo realizado por un artista, o quizá una imagen generada por computadora. Imágenes tradicionalmente utilizadas como texturas son las de paredes de ladrillo, las superficies de madera vetada y las superficies de mármol.

Suponga, por ejemplo, que deseáramos modelar una pared de ladrillos. Podríamos representar la forma de la pared con una simple malla poligonal compuesta por un rectángulo uniforme de gran longitud. Entonces podríamos suministrar junto con esta malla una imagen bidimensional de una pared de ladrillos. Después, durante el proceso de generación, esta imagen podría aplicarse al objeto rectangular para generar la apariencia de una pared de ladrillos. Para ser más precisos, cada vez que el proceso de generación necesita determinar la apariencia de un punto de la pared, simplemente utilizaría la apariencia del correspondiente punto en la imagen de la pared de ladrillos.

Cuando mejor funciona la aplicación de texturas es cuando se aplica a superficies relativamente planas. El resultado puede parecer artificial si hace falta distorsionar la imagen de la textura significativamente para cubrir una superficie curva (imagine los problemas que surgen al tratar de cubrir con papel pintado una pelota) o si la imagen de la textura envuelve completamente a un objeto, provocando que aparezca una junta allí donde el patrón de textura utilizado no encaje bien consigo mismo. En cualquier caso, la aplicación de texturas ha demostrado ser un medio eficiente de simular la textura de los objetos y se utiliza ampliamente en situaciones sensibles al tiempo real, como en el caso de los videojuegos interactivos.

Búsqueda de realismo La construcción de modelos de objetos que permitan obtener imágenes realistas es uno de los temas activos de investigación hoy día. De especial interés son los materiales asociados con personajes vivos, como por ejemplo la piel, el pelo y las plumas. Buena parte de estas investigaciones son específicas de cada sustancia concreta y abarcan técnicas tanto de modelado como de generación. Por ejemplo, para obtener modelos realistas de la piel humana, algunos investigadores han incorporado en los modelos el grado con el que la luz penetra en las capas de piel de la dermis y la epidermis, y tratan de modelar también el modo en el que los contenidos de dichas capas afectan a la apariencia de la piel.

Otro ejemplo es el relativo al modelado del cabello humano. Si el cabello se observa desde una cierta distancia, entonces puede que resulten suficientes las técnicas de modelado más tradicionales. Pero en los primeros planos puede resultar difícil conseguir un pelo de apariencia realista. Entre los problemas existentes están por ejemplo las propiedades translúcidas del cabello, la profundidad textural, la forma en que el cabello cae o se ensortija y la manera en que responde a fuerzas externas como el viento. Para resolver estos problemas, algunas aplicaciones recurren al procedimiento de modelar mechones individuales de cabello, que es una tarea ingente, porque el número de cabellos existente en la cabeza de una persona puede ser del orden de 100.000. Sin embargo, todavía más sorprendente es que algunos investigadores han construido modelos de cabello que tienen en cuenta la textura, la variación de color y las características mecánicas de cada pelo.

Otro ejemplo en el que se ha procurado conseguir un considerable grado de detalle es a la hora de modelar la ropa. En este caso, se han utilizado los detalles de los patrones de entretejido para conseguir diferencias texturales apropiadas entre los distintos tipos de tejido, como por ejemplo tejidos satinados; asimismo, se han combinado las propiedades detalladas de los distintos tipos de hilo con datos de patrones de tejido, con el fin de crear modelos de tejidos que

permiten obtener imágenes de primer plano extremadamente realistas. Además, los conocimientos de física y de ingeniería mecánica se han aplicado a las hebras individuales de materiales tejidos, con el fin de calcular imágenes que reflejan aspectos tales como el estiramiento de las hebras y las roturas del tejido.

La producción de imágenes realistas es un área activa de investigación que, como ya hemos dicho, incorpora técnicas tanto de los procesos de modelado como de los de generación. Normalmente, a medida que se van haciendo progresos, las nuevas técnicas se incorporan primero en aplicaciones que no están sujetas a restricciones de tiempo real, como por ejemplo el software gráfico de los estudios de producción de películas de cine, en las que existe un retardo significativo entre los procesos de modelado/generación y la presentación final de las imágenes. [Este tipo de avances puede observarse comparando atentamente los personajes de *Toy Story 2* de Disney (1999) con los de la película original *Toy Story* (1995). Por ejemplo, se aplicaron técnicas recientemente desarrolladas para mejorar el aspecto de las juntas entre las mallas poligonales que representan las características faciales, como por ejemplo los límites entre la nariz y el resto de la cara.] A medida que estas nuevas técnicas se refinan y se simplifican, terminan utilizándose en las aplicaciones de tiempo real, con lo que la calidad de los gráficos en estos entornos también mejora. Es muy posible que una verdadera interacción realista en tiempo real con distintos tipos de mundos virtuales no esté demasiado lejos en el futuro.

Modelado de escenas completas

Una vez que los objetos de una escena se han descrito adecuadamente y se han codificado digitalmente, a cada uno se le asigna una ubicación, un tamaño y una orientación dentro de la escena. Este conjunto de información se enlaza entonces con el fin de obtener una estructura de datos denominada **grafo de escena**. Además, el grafo de escena contiene enlaces a objetos especiales que representan a las fuentes de luz, así como a un objeto concreto que representa a la cámara. Es aquí donde se almacenan la ubicación, la orientación y las propiedades focales de la cámara.

Por tanto, un grafo de escena es análogo a la disposición de un estudio en la fotografía tradicional. Contiene la cámara, las luces, el utillaje y las imágenes de fondo (todo lo que vaya a contribuir a la apariencia final de la fotografía cuando se apriete el disparador), todo ello colocado en los lugares apropiados. La diferencia es que una composición fotográfica tradicional contiene objetos físicos, mientras que un grafo de escena contiene representaciones de objetos codificadas digitalmente. En resumen, el grafo de escena describe un mundo virtual.

Para recalcar el ámbito que abarca un grafo de escena, considere de nuevo la imagen de la Figura 10.1 e imagine el grafo de escena utilizado para producirla. Los personajes, la pared, la colcha de la cama, el cabecero, el paquete situado detrás de Buzz Lightyear (el soldado espacial), las molduras de la ventana, los árboles situados detrás de la ventana y las fuentes de luz fueron todos ellos modelados con el grado de detalle apropiado y representados en el grafo de escena. De hecho, algunos objetos que podríamos inicialmente considerar como una única estructura, como por ejemplo Woody (el muñeco cowboy),

estaban en realidad representados dentro del grafo de escena como conglomerados de numerosos objetos individuales.

El posicionamiento de la cámara dentro de una escena tiene muchas repercusiones. Como hemos dicho anteriormente, el detalle con el que se modelen los objetos dependerá de la ubicación de cada objeto dentro de la escena. Los objetos que aparecen en primer plano requieren un mayor grado de detalle que los objetos de fondo, y la diferencia entre lo que es el primer plano y lo que constituye el fondo depende de la posición de la cámara. Si la escena se va a utilizar en un contexto análogo al de un escenario teatral, entonces los papeles del primer plano y el fondo están bien establecidos y pueden construirse los modelos de los objetos correspondientemente. Sin embargo, si el contexto requiere que la posición de la cámara se altere para obtener diferentes imágenes, puede que sea necesario ajustar entre una "fotografía" y otra el grado de detalle proporcionado por cada modelo de objeto. Esta es una de las áreas de investigación actuales. Podríamos concebir una escena compuesta por modelos "inteligentes" que fueran refinando sus mallas poligonales y otras características a medida que la cámara se desplazara dentro de la escena.

Un ejemplo interesante de escenario con cámara móvil es el que se genera en los sistemas de realidad virtual en los que se permite a los seres humanos experimentar la sensación de moverse por un mundo imaginario tridimensional. El mundo imaginario está representado por un grafo de escena y las personas ven este entorno por medio de una cámara que se desplaza por la escena de acuerdo con los movimientos que la propia persona haga. En realidad, para proporcionar la percepción de profundidad necesaria para la visión en tres dimensiones se utilizan dos cámaras: una que representa el ojo derecho de la persona y otra que representa su ojo izquierdo. Mostrando la imagen obtenida por cada cámara delante del ojo apropiado, la persona tiene la ilusión de estar dentro de la escena tridimensional, y cuando se añaden sensaciones sonoras y táctiles a la experiencia, la ilusión puede ser bastante realista.

Televisión 3D

Existen varias tecnologías para producir imágenes 3D para televisión, pero todas dependen del mismo efecto visual estereoscópico, dos imágenes ligeramente distintas que lleguen a los ojos izquierdo y derecho serán interpretadas por el cerebro asignando a las imágenes una sensación de profundidad. Los mecanismos más baratos para conseguir este efecto requieren la utilización de gafas especiales con lentes de filtro. Las lentes coloreadas más antiguas (usadas en el cine en la década de 1950) o las más modernas lentes polarizadas filtran diferentes partes de una misma imagen proyectada en la pantalla, lo que da como resultado que a cada ojo llegue una imagen distinta. Otra tecnología más cara requiere el uso de gafas "activas" que obturan alternativamente las lentes izquierda y derecha de forma sincronizada con una televisión 3D que va conmutando rápidamente entre la imagen izquierda y la derecha. Finalmente, se están desarrollando televisiones 3D que no requieren gafas especiales ni emplear ningún tipo de casco. Utilizan complicadas matrices de filtros o lentes de ampliación en la superficie de la pantalla con el fin de proyectar las imágenes izquierda y derecha hacia la cabeza del espectador con ángulos ligeramente distintos, lo que hace que los ojos izquierdo y derecho contemplen imágenes diferentes.

Para terminar, debemos decir que la construcción de un grafo de escena es un paso fundamental dentro del proceso de generación de gráficos 3D. Puesto que el grafo contiene toda la información requerida para producir la imagen final, su terminación marca el final del proceso del modelado artístico y el comienzo del proceso computacionalmente muy costoso de generación de la imagen. De hecho, una vez construido el grafo de escena, la tarea gráfica pasa a ser la de calcular las proyecciones, determinar los detalles de las superficies en puntos específicos y simular los efectos de la luz, un conjunto de tareas que es en buena medida independiente de la aplicación concreta.

Cuestiones y ejercicios

1. A continuación se indican cuatro puntos (codificados utilizando el sistema de coordenadas rectangular tradicional) que representan los vértices de un parche plano. Describa la forma del parche. (Para aquellos que no hayan estudiado geometría analítica, cada tripla especifica cómo alcanzar el punto en cuestión partiendo de la esquina de una habitación. El primer número nos dice cuánto hay que caminar a lo largo de la pared situada a nuestra derecha. El segundo número indica cuánto hay que andar en una dirección paralela a la pared situada a nuestra izquierda. El tercer número nos dice cuánto tenemos que elevarnos respecto del suelo. Si un número es negativo, tendrá que imaginarse que es un fantasma y que puede caminar hacia atrás a través de las paredes y el suelo.)
(0, 0, 0) (0, 1, 1) (0, 2, 1) (0, 1, 0)
2. ¿Qué es un modelo procedimental?
3. Enumere algunos de los objetos que podrían estar presentes en un grafo de escena utilizado para generar la imagen de un parque.
4. ¿Por qué las formas se representan mediante mallas poligonales aún cuando podrían representarse de manera más precisa mediante ecuaciones geométricas?
5. ¿Qué es la aplicación de texturas?

10.4 Generación (*rendering*)

Ahora es el momento de analizar el proceso de generación, que implica determinar cómo aparecerán los objetos de un grafo de escena cuando se proyecten sobre el plano de proyección. Hay diversas formas de llevar a cabo la tarea de generación. Esta sección se centra en la técnica tradicional utilizada en la mayoría de los sistemas gráficos más populares (videojuegos, computadoras personales, etc.) del "mercado de consumo" actual. En las siguientes secciones investigaremos dos alternativas a esta técnica.

Comencemos con algo de información básica acerca de la interacción entre la luz y los objetos. Después de todo, la apariencia de un objeto está determinada por la luz emitida por ese objeto, así que determinar la apariencia de un

objeto equivale en último término a la tarea de simular el comportamiento de los rayos luminosos.

Interacción entre la luz y las superficies

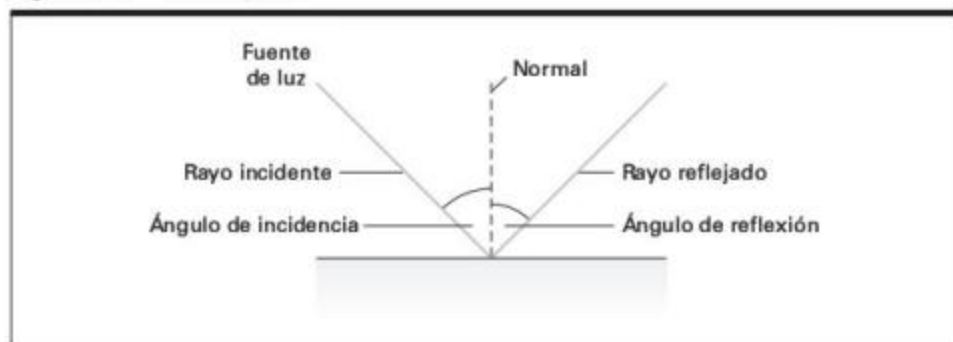
Dependiendo de las propiedades del material de un objeto, la luz que incide sobre su superficie puede ser absorbida, rebotar en la superficie en forma de luz reflejada o atravesar la superficie (y curvarse) como luz refractada.

Reflexión Consideremos un rayo de luz reflejado en una superficie opaca y plana. El rayo llega viajando en línea recta e incide sobre la superficie formando un ángulo denominado **ángulo de incidencia**. El ángulo con el que el rayo se refleja es idéntico al de incidencia. Como se muestra en la Figura 10.6, esos ángulos se miden respecto a una línea perpendicular (o **normal**) a la superficie (una línea normal a la superficie suele denominarse simplemente "normal" como por ejemplo en la frase "El ángulo de incidencia se mide respecto a la normal"). El rayo incidente, el rayo reflejado y la normal se encuentran en un mismo plano.

Si la superficie es suave, los rayos de luz paralelos (como por ejemplo los que lleguen de la misma fuente luminosa) que incidan sobre la superficie en una misma área se verán reflejados esencialmente en la misma dirección y saldrán del objeto en forma de rayos paralelos. Esa luz reflejada se denomina **luz especular**. Fíjese en que la luz especular solo puede observarse cuando la orientación de la superficie y de la fuente luminosa hace que la luz se refleje en dirección al espectador. Así, normalmente suele aparecer en forma de áreas brillantes sobre una superficie. Además, como la luz especular tiene un contacto mínimo con la superficie, tiende a ser del color de la fuente luminosa original.

Sin embargo, las superficies raramente son perfectamente suaves, por lo que muchos rayos luminosos pueden incidir sobre la superficie en determinados puntos cuya orientación difiere de la prevalente en esa superficie. Además, los rayos luminosos a menudo penetran en las capas superficiales y rebotan entre las partículas que componen la superficie, antes de salir finalmente en forma de luz reflejada. El resultado es que muchos rayos serán dispersados en distintas direcciones. Esta luz dispersada se denomina **luz difusa**. A diferencia de la luz especular, la luz difusa es visible en un amplio rango de direcciones.

Figura 10.6 Luz reflejada.

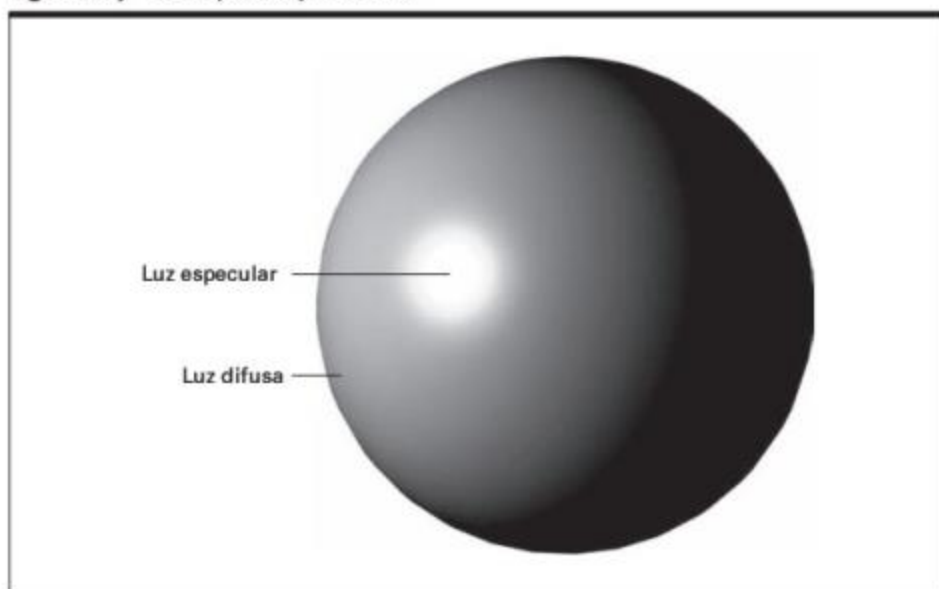


Puesto que tiende a tener un contacto prolongado con la superficie, la luz difusa es más susceptible a las propiedades de absorción del material y por tanto, tiende a adoptar el color del objeto.

La Figura 10.7 muestra una bola iluminada por una única fuente de luz. El área brillante en la superficie de la bola es producida por la luz especular. El resto del hemisferio orientado hacia la fuente de luz se percibe gracias a la luz difusa. Observe que el hemisferio orientado en dirección contraria a la fuente de luz principal no es visible gracias a la luz que se refleja directamente de esa fuente. La capacidad de ver esta otra parte de la bola se debe a la **luz ambiente**, que es luz dispersada que no está asociada con ninguna fuente luminosa ni ninguna dirección concreta. Las partes de las superficies iluminadas por la luz ambiente suelen aparecer con un color oscuro uniforme.

La mayor parte de las superficies reflejan tanto la luz especular como la difusa. Las características de la superficie determinan la proporción de cada tipo de luz. Las superficies suaves parecen brillantes porque reflejan más luz especular que difusa. Las superficies rugosas parecen más opacas porque reflejan más luz difusa que especular. Además, debido a las propiedades de detalle de algunas superficies, la relación entre luz especular y luz difusa varía dependiendo de la luz incidente. La luz que incida sobre ese tipo de superficies desde una dirección podría verse reflejada principalmente como luz especular, mientras que la luz que incida desde otra dirección podría verse reflejada principalmente como luz difusa. De ese modo, la apariencia de la superficie cambiaría de brillante a opaca a medida que fuera girando. Dichas superficies se denominan **superficies anisótropas**, por oposición a las **superficies isotrópicas**, cuyos patrones de reflexión son simétricos. Podemos encontrar ejemplos de superficies anisótropas en tejidos como el satén, en el que las características de la tela alteran la apariencia del material dependiendo de su orientación. En la superficie de hierba de un campo de atletismo, la disposición de la hierba (normal-

Figura 10.7 Luz especular y luz difusa.



mente determinada por la forma en la que se corta) produce efectos visuales anisótropos, como por ejemplo patrones de bandas claras y oscuras.

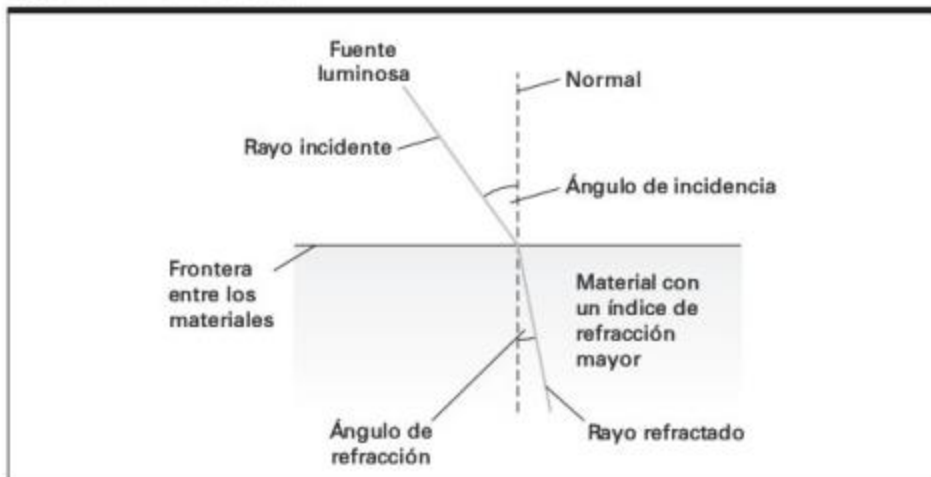
Refracción Consideremos ahora la luz que incide sobre un objeto que sea transparente en lugar de opaco. En este caso, los rayos luminosos atraviesan el objeto en lugar de rebotar en su superficie. A medida que los rayos penetran en la superficie su dirección se ve modificada, un fenómeno que se conoce como **refracción** (Figura 10.8). El grado de refracción está determinado por el índice de refracción de los materiales implicados. El índice de refracción está relacionado con la densidad del material. Los materiales densos tienden a tener un índice de refracción mayor que los materiales menos densos. Cuando un rayo de luz pasa a un material con un índice de refracción mayor (como por ejemplo al pasar del aire al agua), se curva hacia la normal en el punto de entrada. Si el rayo de luz pasa a un material con un índice de refracción menor, se curva alejándose de la normal.

Para generar objetos transparentes de forma apropiada, el software de generación debe conocer los índices de refracción de los materiales implicados. Pero la cosa no termina ahí. El software de generación también debe saber qué lado de la superficie de un objeto representa el interior del mismo, por oposición al exterior. ¿Está la luz entrando en el objeto o saliendo de él? Las técnicas para obtener este tipo de información son a veces bastante sutiles. Por ejemplo, si acordamos listar siempre los vértices de cada polígono de una malla poligonal en el sentido contrario a las agujas del reloj, visto el polígono desde fuera del objeto, entonces la lista codificada de manera inteligente indicará qué lado del parche representa la parte exterior del objeto.

Recorte, conversión de barrido y eliminación de caras ocultas

Ahora vamos a centrarnos en el proceso de producir una imagen a partir de un grafo de escena. Nuestro enfoque consistirá, por ahora, en seguir las técnicas utilizadas en la mayoría de los sistemas de videojuegos interactivos. Tomadas en conjunto, estas técnicas forman un paradigma muy bien consolidado que se

Figura 10.8 Luz refractada.

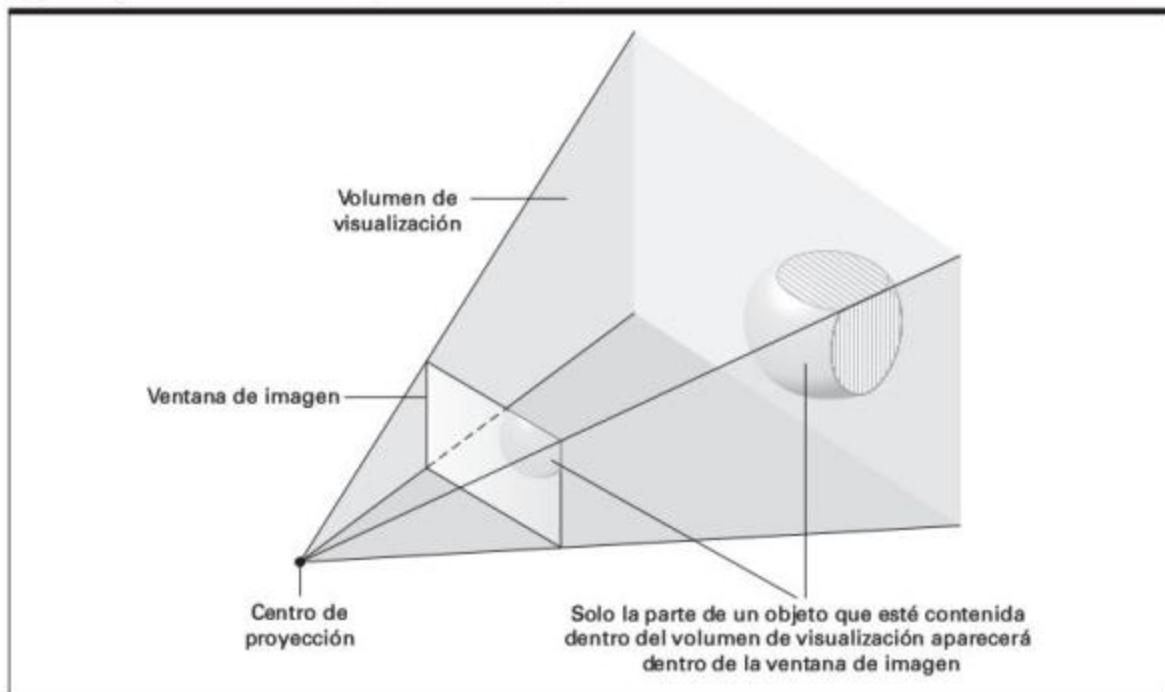


conoce con el nombre de **cadena de generación** (*rendering pipeline*). Consideraremos algunos de los pros y los contras de este enfoque al final de esta sección y en la siguiente sección exploraremos dos alternativas. Es útil observar que la cadena de generación maneja objetos opacos, por lo que la refracción no es un problema. Además, ignora las interacciones entre unos objetos y otros, por lo que por el momento no vamos a preocuparnos por elementos como los espejos y las sombras.

La cadena de generación comienza identificando la región dentro de una escena tridimensional que contiene los objetos (o partes de objetos) que pueden ser "vistos" por la cámara. Esta región, denominada **volumen de visualización**, es el espacio contenido dentro de la pirámide definida por las cuatro líneas rectas que salen del centro de proyección y pasan por las esquinas de la ventana de imagen (Figura 10.9).

Una vez identificado el volumen de visualización, la tarea consiste en descartar todos aquellos objetos o partes de objetos que no estén contenidos dentro del volumen de visualización. Después de todo, la proyección de esa parte de la escena caerá fuera de la ventana de imagen y, por tanto, no aparecerá en la imagen final. El primer paso consiste en descartar todos aquellos objetos que estén completamente fuera del volumen de visualización. Para simplificar este proceso, un grafo de escena puede estar organizado como una estructura de árbol en la que los objetos situados en distintas regiones de la escena estén almacenados en diferentes ramas. De esta manera pueden descartarse grandes secciones del grafo de escena simplemente ignorando determinadas ramas completas del árbol.

Figura 10.9 Identificación de la región de la escena que cae dentro del volumen de visualización.



Aliasing

¿Ha notado alguna vez la apariencia “parpadeante” que las corbatas y camisas de rayas tienen en las pantallas de televisión? Este es el resultado de un fenómeno conocido como **aliasing**, que se produce cuando un patrón de la imagen deseada se combina de manera inapropiada con la densidad de los píxeles que forman la imagen. Por ejemplo, suponga que una parte de la imagen deseada está compuesta por franjas alternativas blancas y negras, pero que el centro de todos los píxeles resulta caer únicamente en la franjas negras. En este caso, el objeto se representaría como completamente negro. Pero si el objeto se mueve ligeramente, el centro de todos los píxeles podría caer en la franjas blancas, lo que significa que el objeto cambiaría súbitamente pasando a ser de color blanco. Existen distintas formas de compensar este efecto tan molesto. Una de ellas consiste en generar cada píxel promediando una área pequeña de la imagen, en lugar de utilizar únicamente la apariencia de un único punto muy preciso.

Después de identificar y descartar los objetos que caen completamente fuera del volumen de visualización, los objetos restantes se recortan mediante un proceso denominado precisamente **recorte** (*clipping*), que esencialmente corta las partes de los objetos que caen fuera del volumen de visualización. Para ser más precisos, el recorte es el proceso de comparar cada parche plano individual con los límites del volumen de visualización y recortar aquellas partes del parche que quedan fuera. El resultado es una serie de mallas poligonales (de parches planos posiblemente recortados) que caen completamente dentro del volumen de visualización.

El siguiente paso de la cadena de generación consiste en identificar los puntos de los parches planos restantes que hay que asociar con cada posición de píxel en la imagen final. Es importante darse cuenta de que solo esos puntos contribuirán a la imagen final. Si un determinado detalle de un objeto cae entre dos posiciones de píxel no quedará representado por ningún píxel y no será visible por tanto en la imagen final. Esta es la razón por la que se indica claramente el número de píxeles que tiene cada aparato en el mercado de las cámaras digitales. Cuantos más píxeles haya, más probable es que podamos capturar en nuestras fotografías los pequeños detalles.

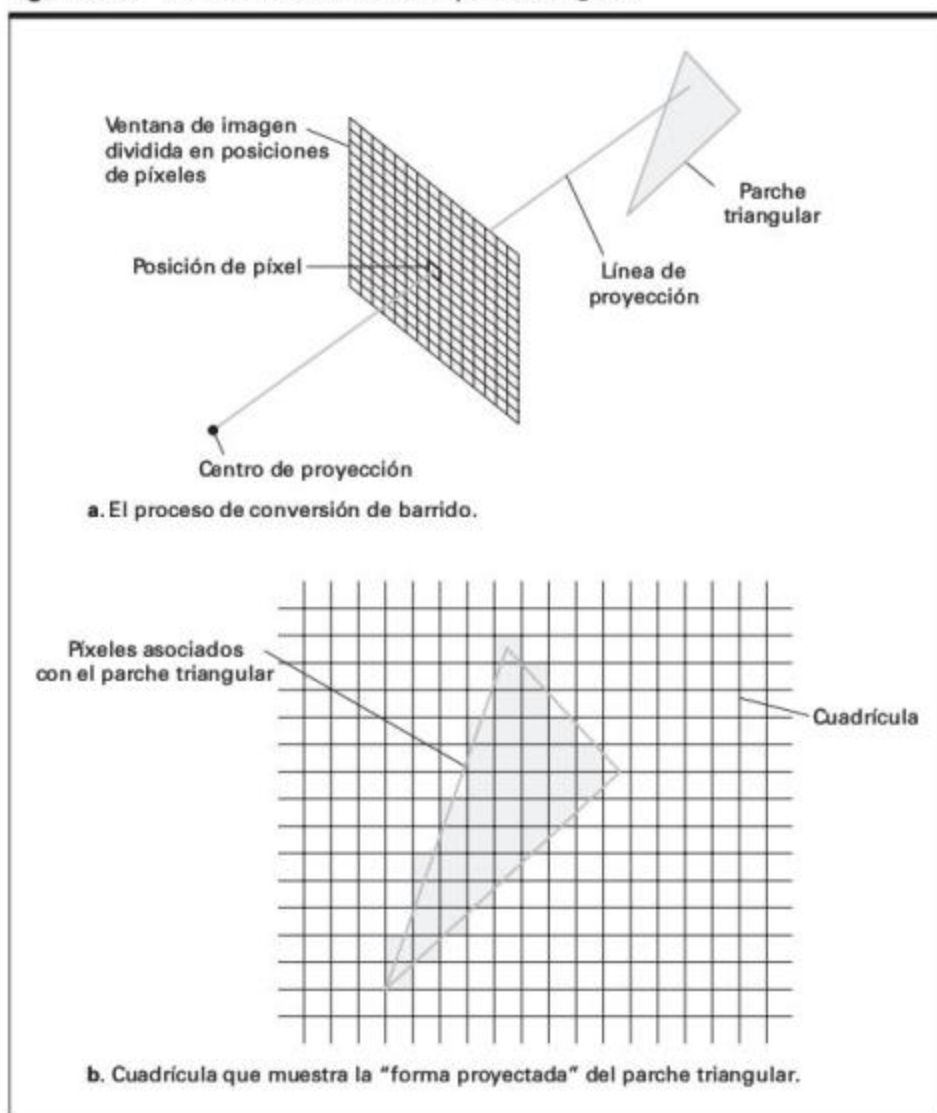
El proceso de asociar las posiciones de los píxeles con puntos de la escena se denomina **conversión de barrido** (porque implica convertir los parches en filas horizontales de píxeles, denominadas líneas de barrido) o **tramado** (*rasterization*, porque una matriz de píxeles es como una trama o “*raster*”). La conversión de barrido se lleva a cabo tirando líneas rectas (proyectores) desde el centro de proyección a través de cada posición de píxel de la ventana de imagen y luego identificando los puntos en los que estos proyectores intersectan con los parches planos. Estos serán entonces los puntos de los parches en los que deberemos determinar la apariencia de un objeto. De hecho, estos serán los puntos que se representarán mediante los píxeles en la imagen final.

La Figura 10.10 muestra la conversión de barrido de un único parche triangular. La parte (a) de la figura muestra cómo se utiliza un proyector para asociar una posición de píxel con un punto del parche. La parte (b) muestra la imagen en píxeles del parche, tal como ha sido determinada por la conversión

de barrido. Toda la matriz de píxeles está representada mediante una cuadrícula y en ella hemos sombreado los píxeles asociados con el triángulo. Observe que la figura también ilustra la distorsión que puede aparecer a la hora de realizar la conversión de barrido de una forma geométrica cuyas características sean pequeñas en relación con el tamaño de los píxeles. Esos bordes dentados les resultarán familiares a la mayoría de los usuarios de pantallas de computadoras personales.

Lamentablemente, el realizar la conversión de barrido de una escena completa (o incluso de un único objeto) no es tan sencillo como realizar la conversión de barrido de un único parche. Esto se debe a que, cuando hay múltiples parches implicados, uno de ellos puede bloquear la visión de otro. Así, aún cuando una línea de proyección interseccione con un parche plano, dicho punto

Figura 10.10 Conversión de barrido de un parche triangular.



del parche puede no ser visible en la imagen final. Identificar y descartar los puntos de una escena cuya visión está bloqueada es un proceso que se denomina **eliminación de caras ocultas**.

Una versión específica del proceso de eliminación de caras ocultas es lo que se denomina **eliminación de caras traseras**, que implica descartar todos los parches de una malla poligonal que representen la "parte trasera" de un objeto. Observe que la eliminación de caras traseras es relativamente sencilla, porque los parches del lado posterior de un objeto pueden identificarse fácilmente: son aquellos cuya orientación se aleja de la cámara.

Sin embargo, resolver el problema completo de eliminación de caras ocultas requiere mucho más que la simple eliminación de las caras traseras. Imagine por ejemplo una escena en la que hubiera un automóvil delante de un edificio. Habrá parches planos tanto del automóvil como del edificio que se proyectarán sobre la misma área de la ventana de imagen. Allí donde se produzca un solapamiento, los datos de los píxeles que terminen por almacenarse en la memoria de fotograma deberían indicar la apariencia del objeto situado en el primer plano (el automóvil) y no del objeto situado al fondo (el edificio). En resumen, si una línea de proyección intersecta con más de un parche plano, será el punto correspondiente al parche situado más cerca de la ventana de imagen el que habrá que generar.

Una solución simple para resolver este problema relativo a los objetos situados en el primer plano y el fondo es la que se conoce con el nombre de **algoritmo del pintor**, que consiste en disponer los objetos de la escena de acuerdo con su distancia a la cámara y luego realizar la conversión de barrido primero para los objetos más distantes, haciendo así que los resultados de la conversión de barrido de los objetos más próximos sustituyan a cualquier resultado que se haya obtenido previamente. Lamentablemente, el algoritmo del pintor falla en aquellos casos en los que hay objetos físicamente entremezclados. Por ejemplo, parte de un árbol podría encontrarse detrás de un objeto, mientras que otra parte del mismo árbol podría estar delante de ese objeto.

Otras soluciones más generales al problema de representar objetos de primer plano y de fondo se centran en los píxeles individuales en lugar de en objetos completos. Una técnica muy popular utiliza un área de almacenamiento adicional, denominada **buffer z** (y también buffer de profundidad), que contiene una entrada por cada píxel de la imagen (o, lo que es lo mismo, por cada entrada de píxel de la memoria de fotograma). Cada posición del buffer z se emplea para almacenar la distancia entre la cámara y el objeto actualmente representado por la correspondiente entrada de la memoria de fotograma; esa distancia se calcula a lo largo de la línea de proyección. Así, con la ayuda de un buffer z puede resolverse el problema de qué objetos están en primer plano y cuáles en el fondo, calculando y almacenando la apariencia de un píxel solo si los datos correspondientes a ese píxel no han sido almacenados todavía en la memoria de fotograma, o si el punto del objeto que estamos considerando actualmente está más próximo que el del objeto previamente generado; para ver si está más próximo se emplea la información de distancia almacenada en el buffer z.

Para ser más precisos, cuando se utiliza un buffer z, el proceso de generación puede llevarse a cabo de la forma siguiente: en todas las entradas del buffer z se almacena un valor que representa la distancia máxima con respecto a la cámara que puede tener un objeto, para que lo tengamos en consideración

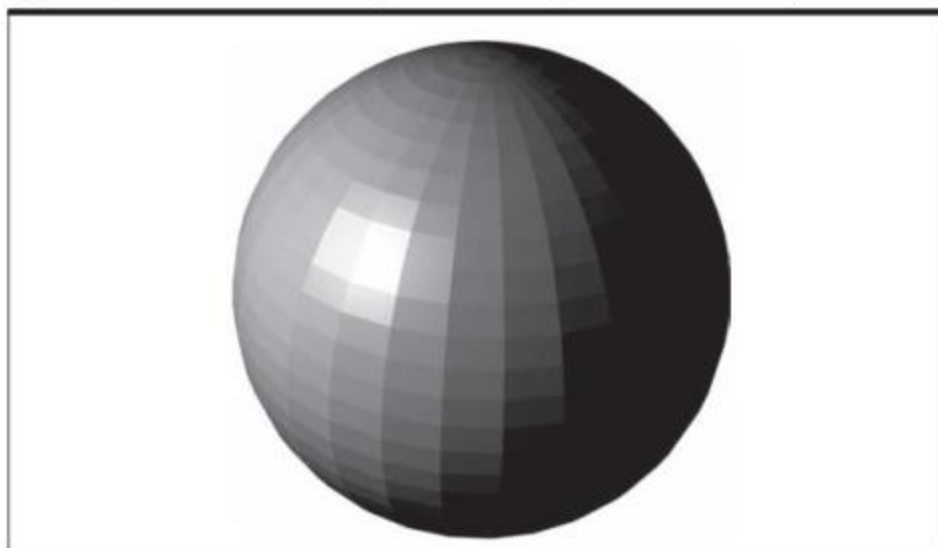
durante la generación. Después, cada vez que se tiene en cuenta un nuevo punto de un parche plano para generarlo, primero comparamos su distancia a la cámara con el valor del buffer z asociado con la posición del píxel actual. Si dicha distancia es inferior al valor almacenado en el buffer z, se calcula la apariencia del punto, se almacena el resultado en la memoria de fotograma y se sustituye la antigua entrada en el buffer z por la distancia al punto que se acaba de generar (observe que si la distancia al punto es mayor que el valor almacenado en el buffer z, no es necesario llevar a cabo ninguna acción, porque el punto de ese parche está demasiado alejado como para tenerlo en cuenta o su visión está bloqueada por otro punto más próximo que ya ha sido generado).

Sombreado (*shading*)

Una vez que el proceso de conversión de barrido ha identificado un punto de un parche plano que debe aparecer en la imagen final, la tarea de generación se convierte en la de determinar la apariencia del parche en dicho punto. Este proceso se denomina **sombreado**. Observe que el sombreado implica calcular las características de la luz proyectada hacia la cámara desde el punto en cuestión, características que dependerán a su vez de la orientación que la superficie tenga en dicho punto. Después de todo, es la orientación de la superficie en dicho punto la que determina el grado de luz especular, difusa y ambiente que será percibida por la cámara.

Una solución sencilla al proceso de sombreado, denominada **sombreado plano**, consiste en utilizar la orientación de un parche plano como la orientación de cada punto del parche; es decir, asumir que la superficie en cada parche es plana. Sin embargo, el resultado es que la imagen final aparecerá con facetas, como se muestra en la Figura 10.11 en lugar de redondeada como en la Figura 10.7. En un cierto sentido, el sombreado plano produce una imagen de la propia malla poligonal, en lugar de producir una imagen del objeto que está siendo modelado mediante dicha malla.

Figura 10.11 Una esfera tal como aparecería si se la genera con sombreado plano.

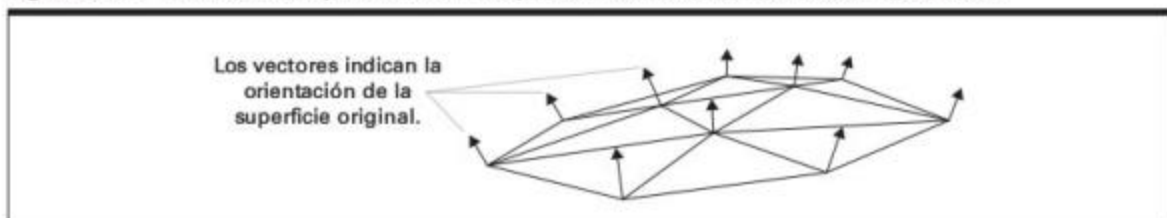


Para obtener una imagen más realista, el proceso de generación debe combinar la apariencia de los parches planos individuales para generar una superficie de aspecto suavemente curvo. Esto se consigue estimando la verdadera orientación de la superficie original en cada punto individual que se esté generando.

Dichos esquemas de estimación realizan normalmente su trabajo a partir de datos que indican la orientación de la superficie en los vértices de la malla poligonal. Hay diversas formas de obtener estos datos. Una de ellas es codificar la orientación de la superficie original en cada vértice y asociar estos datos a la malla poligonal como parte del proceso de modelado. Esto produce una malla poligonal con flechas, que se denominan **vectores normales**, asociadas a cada vértice. Cada vector normal apunta hacia fuera en la dirección perpendicular a la superficie original. El resultado es una malla poligonal que puede visualizarse como se muestra en la Figura 10.12 (otra técnica consiste en calcular la orientación de cada uno de los parches adyacentes a un vértice y luego utilizar una "media" de dichas orientaciones como estimación de la orientación de la superficie en el vértice).

Independientemente de cómo se obtenga la orientación de la superficie original en los vértices de una malla poligonal, tenemos a nuestra disposición diversas estrategias para el sombreado de un parche plano a partir de esos datos. Entre esas estrategias se incluyen el **sombreado de Gouraud** y el **sombreado de Phong**, siendo la distinción entre estos dos tipos bastante sutil. Ambos comienzan utilizando la información acerca de la orientación de la superficie en los vértices de un parche, con el fin de aproximar la orientación de la superficie a lo largo de las aristas del parche. Después, el sombreado de Gouraud aplica dicha información para determinar la apariencia de la superficie a lo largo de las aristas del parche y, finalmente, interpola esa apariencia de los bordes con el fin de estimar la apariencia de la superficie en los puntos interiores del parche. Por el contrario, el sombreado de Phong interpola la orientación de la superficie a lo largo de las aristas del parche con el fin de estimar la orientación de la superficie en los puntos situados en su interior y solo entonces pasa a considerar las cuestiones acerca de la apariencia de los puntos. En resumen, el sombreado de Gouraud convierte la información de orientación en información de color y luego interpola esa información de color. El sombreado de Phong interpola la información de orientación hasta estimar la orientación del punto en cuestión y luego convierte dicha información de orientación en información de color. El resultado es que el sombreado de Phong es más proclive a mostrar la luz especular en el interior de un parche porque es más sensible a los cambios en la orientación de la superficie. (Véase la Cuestión 3 al final de esta sección.)

Figura 10.12 Una vista conceptual de una malla poligonal con vectores normales en los vértices.



Finalmente, hay que decir que las técnicas básicas de sombreado pueden ampliarse con el fin de dar a una superficie una apariencia de textura. Un ejemplo denominado **aplicación de protuberancias** (*bump mapping*) es básicamente una forma de generar pequeñas variaciones en la orientación aparente de una superficie para que parezca ser rugosa. Es decir, la aplicación de protuberancias añade un cierto grado de aleatoriedad al proceso de interpolación aplicado por los algoritmos de sombreado tradicionales, de modo que la superficie global parece tener una textura como la ilustrada en la Figura 10.13.

Hardware para la cadena de generación

Como ya hemos dicho, los procesos de recorte, conversión de barrido, eliminación de caras ocultas y sombreado forman una secuencia que se conoce con el nombre de cadena de generación. Además, los algoritmos para realizar estas tareas con eficiencia son bien conocidos y han sido implementados directamente en circuitos electrónicos, que se han miniaturizado mediante la tecnología VLSI con el fin de producir chips que realicen automáticamente toda la cadena de generación. Hoy día, incluso los dispositivos más baratos son capaces de generar millones de parches planos por segundo.

La mayoría de los sistemas de computadoras diseñados para aplicaciones de gráficos, incluyendo las consolas para videojuegos, incorporan estos dispositivos en su diseño. En el caso de sistemas de computadoras de propósito más general, esta tecnología puede añadirse en forma de **tarjeta gráfica** o **adaptador gráfico**, que se conecta al bus de la computadora como una controladora especializada (véase el Capítulo 2). Ese hardware reduce significativamente el tiempo requerido para ejecutar el proceso de generación.

El hardware de la cadena de generación también reduce la complejidad del software de aplicación gráfica. Básicamente, lo único que el software tiene que hacer es proporcionar el grafo de la escena al hardware de gráficos. El hard-

Figura 10.13 Una esfera tal como aparecería al ser generada utilizando una aplicación de protuberancias.



ware realiza entonces los pasos de la cadena y coloca los resultados en la memoria de fotograma. Así, desde la perspectiva del software, toda la cadena de generación se reduce a un único paso que utiliza al hardware como herramienta abstracta.

Por ejemplo, consideremos de nuevo el caso de un videojuego interactivo. Para inicializar el juego, el software transfiere el grafo de escena al hardware gráfico. El hardware entonces genera la escena y coloca la imagen en la memoria de fotograma, desde donde automáticamente se muestra en la pantalla del monitor. A medida que va progresando el juego, el software simplemente actualiza el grafo de escena con el que trabaja el hardware gráfico con el fin de reflejar los cambios en la situación del juego. Y el hardware genera repetidamente la escena, colocando cada vez la imagen actualizada en la memoria de fotograma.

Debemos observar, sin embargo, que la funcionalidad y las capacidades de comunicación de los distintos tipos de hardware gráfico existentes varían sustancialmente. Así, si se desarrolla una aplicación, como por ejemplo un videojuego para una plataforma específica, será necesario modificar esa aplicación si se transfiere a otro entorno. Para reducir esta dependencia con respecto a los detalles específicos de los sistemas gráficos, se han desarrollado interfaces software estándar que desempeñan un papel de intermediario entre el hardware gráfico y el software de aplicación. Estas interfaces están compuestas por rutinas software que convierten una serie de comandos estandarizados en las instrucciones específicas requeridas por un determinado sistema de hardware gráfico. Como ejemplos podemos citar **OpenGL** (*Open Graphics Library*, Biblioteca abierta de gráficos), que es un sistema no propietario desarrollado por Silicon Graphics y ampliamente utilizado en el sector de los videojuegos, así como **Direct3D**, que fue desarrollado por Microsoft para su uso en entornos Microsoft Windows.

Para terminar, debemos observar que, a pesar de todas las ventajas asociadas con la cadena de generación, también existen algunas desventajas, la más significativa es el hecho de que la cadena implementa únicamente un **modelo de iluminación local**, lo que quiere decir que la cadena genera cada objeto independientemente de los objetos restantes. Es decir, con un modelo de iluminación local, cada objeto se genera con respecto a las fuentes luminosas como si fuera el único objeto de la escena. El resultado es que las interacciones luminosas entre los objetos, como por ejemplo las sombras y las reflexiones, no se verán capturadas en la escena. Por el contrario, en un **modelo de iluminación global** sí que se tienen en cuenta las interacciones entre objetos. Hablaremos en la siguiente sección de dos técnicas para la implementación de un modelo de iluminación global. Por ahora, limitémonos a decir que estas técnicas exceden las capacidades del procesamiento en tiempo real de las tecnologías actuales.

Sin embargo, esto no significa que los sistemas que utilizan hardware de cadena de generación no sean capaces de producir algunos efectos de iluminación global. De hecho, se han desarrollado algunas técnicas muy inteligentes para sobreponerse a algunas de las restricciones impuestas por un modelo de iluminación global. En particular, puede simularse la apariencia de las **sombras proyectadas**, que son sombras que se proyectan sobre el suelo, dentro del contexto de un modelo de iluminación local; el procedimiento consiste

en hacer una copia de la malla poligonal del objeto que está proyectando la sombra, aplastar esa malla duplicada hasta hacerla plana, colocarla sobre el suelo y asignarle un color oscuro. En otras palabras, la sombra se modela como si fuera otro objeto, que luego puede generarse mediante el hardware tradicional de cadena de generación para generar la ilusión de una sombra. Dichas técnicas son populares tanto en aplicaciones de "consumo" como los videojuegos interactivos, como en aplicaciones de "nivel profesional" como por ejemplo los simuladores de vuelo.

Cuestiones y ejercicios

1. Indique la diferencia entre la luz especular, la luz difusa y la luz ambiente.
2. Defina los términos *recorte* y *conversión de barrido*.
3. El sombreado de Gouraud y el sombreado de Phong pueden resumirse de la forma siguiente: el sombreado de Gouraud utiliza la orientación de la superficie de un objeto a lo largo de las aristas de un parche con el fin de determinar la apariencia de la superficie en las aristas y luego interpola esas apariencias en el interior del parche para determinar la apariencia de cada punto concreto del interior. El sombreado de Phong interpola las orientaciones de las aristas con el fin de calcular las orientaciones de los puntos en el interior del parche y luego utiliza esas orientaciones para determinar la apariencia de cada punto concreto. ¿Cómo diferirá la apariencia de un objeto si utilizamos un tipo de sombreado u otro?
4. ¿Cuál es la importancia de la cadena de generación?
5. Describa cómo podrían simularse las reflexiones en un espejo utilizando un modelo de iluminación local.

10.5 Iluminación global de las escenas

Los investigadores están trabajando actualmente en dos alternativas a la cadena de generación. Ambas implementan un modelo de iluminación global y proporcionan así la posibilidad de acabar con las restricciones del modelo de iluminación local que son inherentes a la cadena de generación tradicionalmente utilizada. Una de estas alternativas es la técnica de trazado de rayos y la otra es la de radiosity. Ambas son procesos muy meticulosos y que requieren una enorme cantidad de tiempo de computación, como vamos a ver a continuación.

Trazado de rayos

El **trazado de rayos** es esencialmente el proceso de seguir un rayo luminoso hacia atrás, con el fin de localizar su origen. El proceso comienza seleccionando uno de los píxeles que hay que generar, identificando la línea recta que pasa por ese píxel y por el centro de proyección y luego trazando el rayo luminoso que incide sobre la ventana de imagen según esa línea. Este proceso de

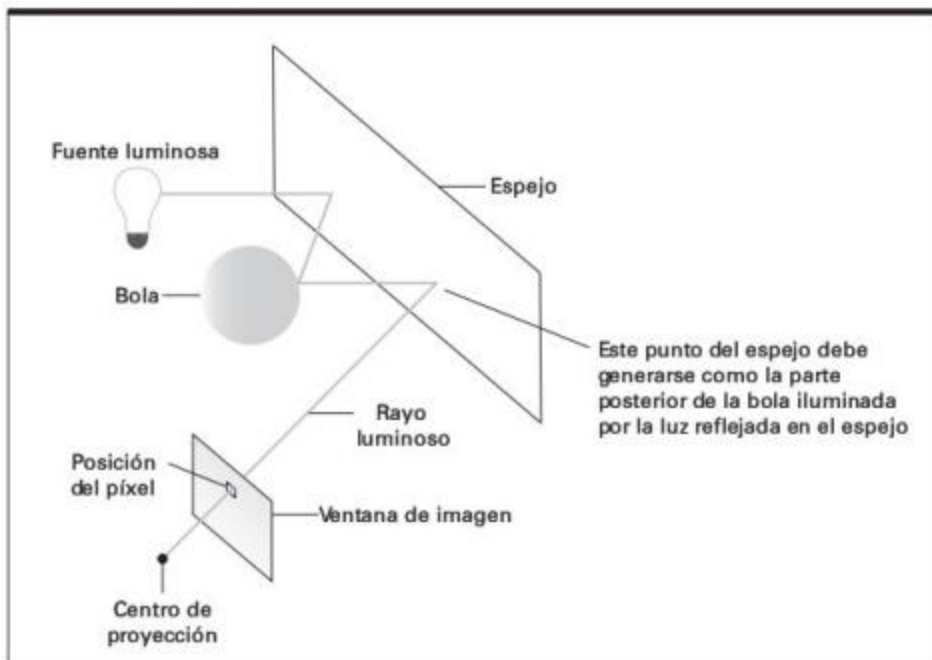
trazado implica seguir la línea hacia dentro de la escena hasta que entre en contacto con un objeto. Si el objeto es una fuente luminosa, el proceso de trazado de rayos termina y el píxel se genera como un punto de la fuente luminosa. En caso contrario, se evalúan las propiedades de la superficie del objeto para determinar la dirección del rayo luminoso incidente que fue reflejado, con el fin de producir el rayo que estamos siguiendo hacia atrás. El proceso sigue entonces hacia atrás a ese rayo incidente para localizar su origen, en cuyo punto puede que se identifique y que se tenga que trazar otro rayo.

En la Figura 10.14 se muestra un ejemplo de trazado de rayos en el que podemos ver cómo se traza hacia atrás un rayo a través de la ventana de imagen, hasta alcanzar la superficie de un espejo. A partir de ahí, el rayo se traza hasta una bola brillante, luego de nuevo hacia el espejo y de allí a la fuente luminosa. Basándose en la información obtenida durante este proceso de trazado, el píxel de la imagen debe aparecer como un punto de la bola, que está iluminado por la fuente luminosa reflejada en el espejo.

Una desventaja del trazado de rayos es que solo traza las reflexiones especulares. Por tanto, todos los objetos generados con este método tenderán a tener una apariencia brillante. Para contrarrestar este efecto, puede aplicarse una variante del trazado de rayos que se denomina **trazado de rayos distribuido**. La diferencia es que en lugar de trazar un único rayo hacia atrás a partir de un punto de reflexión, el trazado de rayos distribuido traza múltiples rayos a partir de dicho punto, cada uno de ellos apuntando en una dirección ligeramente distinta.

Otra variante del trazado de rayos básico es aplicable cuando la escena contiene objetos transparentes. En este caso, será preciso considerar dos efectos cada vez que se trace un rayo hacia atrás hasta alcanzar una superficie. Uno de

Figura 10.14 Trazado de rayos.



ellos es la reflexión y el otro es la refracción. Por ejemplo, observe la apariencia transparente del casco de Buzz (el guerrero espacial) en la Figura 10.1, así como los resaltes especulares próximos a la superficie superior del casco. En este caso, la tarea de trazar el rayo original se subdivide en dos tareas: trazar hacia atrás la reflexión y trazar también hacia atrás la refracción.

Normalmente, el trazado de rayos se implementa de forma recursiva, estando encargada cada activación de trazar un rayo hasta su fuente. La primera activación puede trazar su rayo hasta una superficie opaca brillante. En ese punto, reconocería que su rayo es la reflexión de un rayo incidente, calcularía la dirección de dicho rayo incidente e invocaría otra activación del proceso, con el fin de trazar ese rayo incidente. Esta segunda activación realizaría una tarea similar para encontrar el origen de su rayo, un proceso que podría resultar en que se invocaran otras activaciones adicionales.

Pueden utilizarse diversas condiciones para terminar con el trazado de rayos recursivo. El rayo puede trazarse hasta alcanzar una fuente luminosa, o bien el rayo puede salir de la escena sin incidir sobre ningún objeto o el número de activaciones puede alcanzar un límite predeterminado. Otra condición de terminación más puede estar basada en las propiedades de absorción de las superficies encontradas. Si una superficie es altamente absorbente, como por ejemplo una superficie oscura mate, entonces cualquier rayo incidente tendrá poco efecto sobre la apariencia de la superficie y el trazado de rayos puede darse por terminado. La absorción acumulativa también puede tener un efecto similar; es decir, el trazado de rayos puede terminar después de visitar varias superficies moderadamente absorbentes.

Estando basado en un modelo de iluminación global, el trazado de rayos está libre de muchas de las restricciones inherentes a la cadena de generación tradicional. Por ejemplo, los problemas de la eliminación de superficies ocultas y de detección de sombras están resueltas de forma natural en el proceso de trazado de rayos. Lamentablemente, el trazado de rayos tiene como principal desventaja el hecho de que requiere muchísimo tiempo de procesamiento. A medida que se traza cada reflexión hasta su fuente, el número requerido de cálculos crece enormemente, un problema que se complica todavía más cuando se permiten las refracciones o se aplica el trazado de datos distribuido. Por ello, el trazado de rayos no se implementa en los sistemas de tiempo real de "consumo", como los videojuegos interactivos, sino que donde podemos encontrarlo es en aplicaciones de tipo "profesional" que no son sensibles al tiempo real, como por ejemplo el software gráfico utilizado por los estudios de cine.

Radiosidad

Otra alternativa a la cadena de generación tradicional es la **radiosidad**. Mientras que el trazado de rayos va analizando punto a punto para trazar rayos luminosos individuales, la radiosidad adopta un enfoque más "regional" considerando la energía luminosa total radiada entre pares de parches planos. Esta energía luminosa radiada es esencialmente luz difusa. La energía luminosa radiada de un objeto es generada por ese objeto (como en el caso de una fuente luminosa) o reflejada en el objeto. La apariencia de cada objeto se determina entonces considerando la energía luminosa que recibe de otros objetos.

El grado con el que la luz radiada por un objeto afecta a la apariencia de otro está determinado por una serie de parámetros denominados **factores de forma**. Cada pareja de parches de la escena que hay que generar tiene asociado un factor de forma propio. Estos factores de forma tienen en cuenta las relaciones geométricas entre los parches indicados, como son distancia de separación y las orientaciones relativas. Para determinar la apariencia de un parche en la escena, se calcula la cantidad de energía luminosa recibida de todos los demás parches de la escena, utilizando el factor de forma apropiado en cada cálculo. Los resultados se combinan para generar unos valores únicos de color e intensidad para cada parche. Estos valores se interpolan después entre parches adyacentes, empleando técnicas similares al sombreado de Gouraud con el fin de obtener una superficie con contornos suaves, en lugar de una superficie con facetas.

Dado que son numerosos los parches que hay que considerar, la técnica de radiosidad es computacionalmente muy intensa. Por ello, como el trazado de rayos, su aplicación cae fuera de las capacidades de los sistemas gráficos de tiempo real actualmente disponibles en el mercado de consumo. Otro problema con la técnica de radiosidad es que, como trata con unidades compuestas por parches completos en lugar de con puntos individuales, no consigue capturar los detalles relativos a la luz especular, lo que significa que todas las superficies generadas mediante la técnica de radiosidad tienden a presentar una apariencia apagada.

Sin embargo, la radiosidad tiene sus ventajas. Una de ellas es que determinar la apariencia de los objetos utilizando la radiosidad es independiente de la cámara. Por tanto, una vez que se han realizado los cálculos de radiosidad para una escena, la generación de la escena puede completarse rápidamente para diversas posiciones de la cámara. Otra es que la radiosidad captura muchas de las características más sutiles de la luz, como el **sangrado de color**, que es el efecto por el que el color de un objeto afecta al tono de otros objetos situados cerca de él. Por ello, la radiosidad tiene sus propios nichos de mercado. Uno es el del software gráfico utilizado en entornos de diseño arquitectónico. De hecho, la luz existente dentro de un edificio que se esté diseñando está compuesta fundamentalmente por luz difusa y luz ambiente, de modo que los efectos especulares no son significativos, y el hecho de que se puedan gestionar de manera eficiente los cambios en la posición de la cámara implica que un arquitecto podrá ver rápidamente las distintas habitaciones desde distintas perspectivas.

Cuestiones y ejercicios

1. ¿Por qué el trazado de rayos sigue los rayos hacia atrás desde la ventana de imagen a la fuente luminosa, en lugar de ir hacia adelante desde la fuente luminosa a la ventana de imagen?
2. ¿Cuál es la diferencia entre el trazado de rayos simple y el trazado de rayos distribuido?
3. Cite dos desventajas de la técnica de radiosidad.
4. ¿En qué se parecen el trazado de rayos y la radiosidad? ¿En qué se diferencian?

10.6 Animación

Volvamos ahora nuestra atención hacia el tema de la animación por computadora, que es el uso de tecnología de computadoras para generar y visualizar imágenes en movimiento.

Fundamentos de la animación

Comenzamos presentando algunos conceptos básicos de animación.

Fotogramas La animación se consigue mostrando una secuencia de imágenes, denominadas **fotogramas**, en rápida sucesión. Estos fotogramas capturan la apariencia de una escena cambiante a intervalos de tiempo regulares, de modo que su presentación secuencial crea la ilusión de estar observando la escena continuamente a lo largo del tiempo. La velocidad estándar de visualización en la industria del cine es de 24 imágenes por segundo. El estándar en vídeo para televisión es de 60 o de 50 fotogramas por segundo (aunque como uno de cada dos fotogramas está diseñado para entremezclarse con el fotograma precedente, con el fin de generar una imagen detallada completa, el vídeo también puede clasificarse como un sistema de 30 o de 25 fotogramas por segundo).

Los fotogramas pueden producirse mediante técnicas fotográficas tradicionales o pueden generarse artificialmente por medio de sistemas de gráficos por computadora. Es más, las dos técnicas pueden combinarse. Por ejemplo, a menudo se utiliza software de gráficos 2D para modificar imágenes fotográficas, por ejemplo para eliminar los cables en una escena, superponer imágenes o crear la ilusión de **morfismo**, que es el proceso mediante el que un objeto parece transformarse en otro.

Un examen más detallado del proceso de morfismo proporciona algunas interesantes cuestiones acerca del proceso de animación. La construcción de un efecto de morfismo comienza identificando un par de fotogramas clave que serán los que enmarquen la secuencia de morfismo. Uno de ellos será la última imagen antes de que el morfismo tenga lugar. El otro será la primera imagen después de que ya haya ocurrido. (En la producción de películas tradicionales

Kineografías

Una kineografía es un libro de fotogramas de animación que simula el movimiento cuando se hacen pasar las páginas rápidamente. Cualquiera puede hacer una kineografía con este libro de texto (suponiendo que no haya llenado ya los márgenes de garabatos). Simplemente, dibuje un punto en el margen de la primera página y luego basándose en la posición de lo que hay impreso en la tercera página, coloque otro punto en esa tercera página en una posición ligeramente distinta al de la primera. Repita este proceso en cada página impar consecutiva hasta llegar al final del libro. Ahora, haga pasar rápidamente las páginas con el pulgar y mire cómo se mueve el punto de un lado a otro. ¡Ya está! Acaba de crear su propia kineografía y quizá haya dado así el primer paso en su carrera dentro del sector de la animación. Para experimentar con la Cinemática, trate de dibujar una figura con una serie de líneas en lugar de un simple punto, y haga que la figura parezca caminar. Ahora experimente con la Dinámica generando la imagen de una gota de agua al chocar con el suelo.

esto requiere “filmarse” dos secuencias de la acción: una que conduce hasta el momento que comienza el morfismo y la otra que continúa después de que el morfismo se haya producido.) Determinadas características geométricas, como por ejemplo puntos y líneas, denominadas **puntos de control**, en el fotograma que precede al morfismo se asocian con otras características similares del fotograma posterior al morfismo y luego se construye el morfismo aplicando técnicas matemáticas que van distorsionando incrementalmente una imagen y aproximándola a la otra, usando como guías los puntos de control. Grabando las imágenes generadas durante este proceso de distorsión, se obtiene una corta de secuencia de imágenes producidas artificialmente que rellena el hueco existente entre los fotogramas clave originales y crea la ilusión del morfismo.

El guión Un proyecto típico de animación comienza con la creación de un **guión** (*storyboard*), que es una secuencia de imágenes bidimensionales que narran la historia completa en forma de bocetos de escenas correspondientes a puntos clave de la presentación. El papel que desempeñe el guión dependerá de si el proyecto de animación se implementa utilizando técnicas 2D o 3D. En un proyecto que emplee gráficos 2D, el guión suele evolucionar hasta convertirse en el conjunto final de fotogramas, de forma bastante similar a como lo hacía en los estudios Disney en la década de 1920. En aquellos días, una serie de artistas, llamados maestros animadores, iba refinando el guión para generar fotogramas detallados, denominados **fotogramas clave**, que establecían la apariencia de los personajes y de la escena a intervalos regulares de la animación. Después, los ayudantes de animación dibujaban fotogramas adicionales que permitían rellenar los huecos entre los fotogramas clave, de modo que la animación pudiera visualizarse de forma continua y suave. Este proceso de rellenado de huecos se denominaba **interpolación**.

La principal diferencia entre este proceso y el utilizado hoy día es que los animadores ahora utilizan software de gráficos 2D y de procesamiento de imágenes para dibujar los fotogramas clave, mientras que buena parte del proceso de interpolación se ha automatizado, con lo que el papel de los ayudantes de animación ha desaparecido en la práctica.

Animación 3D La mayor parte de las animaciones para videojuegos y producciones cinematográficas se crean ahora utilizando gráficos 3D. En estos casos, el proyecto sigue comenzando por la creación de un guión compuesto por imágenes bidimensionales. Sin embargo, en lugar de evolucionar hasta convertirse en el producto final, como en los proyectos de gráficos 2D, el guión se utiliza como guía en la construcción de un mundo virtual tridimensional. Este mundo virtual se “fotografía” después repetidamente, a medida que los objetos contenidos en él se mueven, de acuerdo con el guión o con la progresión del videojuego.

Quizá debiéramos hacer aquí una pausa para aclarar lo que queremos decir al afirmar que un objeto se mueve dentro de una escena generada por computadora. Recuerde que el “objeto” es en realidad un conjunto de datos almacenado en el grafo de la escena. Entre los datos de este conjunto, hay una serie de valores que indican la posición y la orientación del objeto. Por tanto, “mover” un objeto consiste simplemente en ir cambiando estos valores. Una vez realizados estos cambios, se utilizarán los nuevos valores durante el proceso de generación. En consecuencia, el objeto parecerá haberse movido en la imagen bidimensional final.

Desenfoque

En el campo de la fotografía tradicional, se han hecho muchos esfuerzos para tratar de generar imágenes nítidas de objetos que se mueven rápidamente. En el campo de la animación, el problema es justamente el opuesto. Si cada fotograma de una secuencia que muestra un objeto en movimiento presenta ese objeto como una imagen perfectamente definida, entonces el movimiento puede parecer "entrecortado". Sin embargo, las imágenes perfectamente nítidas son la consecuencia natural de la creación de fotogramas como imágenes individuales de objetos estáticos en un grafo de escena. Por ello, los animadores suelen distorsionar artificialmente las imágenes de los objetos en movimiento en un fotograma generado por computadora. Una de las técnicas utilizadas, denominada **supermuestreo**, consiste en producir múltiples imágenes en las que el objeto en movimiento está ligeramente desplazado de una a otra, y luego superponer dichas imágenes para obtener un único fotograma. Otra técnica consiste en alterar la forma del objeto en movimiento de manera que parezca alargarse en la dirección del movimiento.

Cinemática y Dinámica

El grado con el que el movimiento dentro de una escena de gráficos 3D está automatizado o controlado por un animador humano varía de una aplicación a otra. El objetivo, por supuesto, es automatizar el proceso completo. Con este fin, una gran parte de las investigaciones se han dirigido a tratar de encontrar formas de identificar y simular el movimiento de los fenómenos que tienen lugar de manera natural. A este respecto, hay dos áreas de la Mecánica que han resultado ser particularmente útiles.

Una de ellas es la **Dinámica**, que trata de describir el movimiento de un objeto aplicando las leyes de la Física con el fin de determinar el efecto de las fuerzas que actúan sobre el objeto. Por ejemplo, además de una posición, a un objeto de la escena se le puede asignar una dirección de movimiento, una velocidad y una masa. Estos elementos pueden entonces utilizarse para determinar el efecto que la gravedad o las colisiones con otros objetos tendrían sobre el objeto en cuestión, lo que permitiría al software calcular la posición correcta del objeto en el siguiente fotograma.

Por ejemplo, considere la tarea de construir una secuencia animada que muestre un chorro de agua cayendo en un contenedor. Podríamos utilizar un sistema de partículas en el grafo de escena para representar el agua, en el que cada partícula representaría una pequeña unidad de agua (piense en el agua como si estuviera compuesta de grandes "moléculas" del tamaño de canicas). Después, podríamos aplicar las leyes de la Física para calcular los efectos de la gravedad sobre las partículas, así como la interacción entre las propias partículas, por ejemplo si empezamos a mover el contenedor de lado a lado. Esto nos permitirá calcular la posición de cada partícula a intervalos de tiempo regulares y, utilizando la posición de las partículas exteriores como vértices de una malla poligonal, podríamos obtener una malla que describiera la superficie del agua. Nuestra animación podría entonces obtenerse "fotografiando" repetidamente esta malla, a medida que va progresando la simulación.

La otra rama de la Mecánica utilizada para simular el movimiento es la **Cinemática**, que trata de describir el movimiento de un objeto en términos de

cómo se mueven las partes del objeto unas con respecto a otras. Las aplicaciones de la Cinemática son muy importantes a la hora de animar figuras articuladas, en las que es necesario mover extremidades tales como brazos y piernas. Estos movimientos se pueden modelar más fácilmente simulando patrones de movimiento de las articulaciones, que calculando los efectos de las fuerzas individuales ejercidas por los músculos y la gravedad. Así, mientras que la Dinámica puede ser la técnica preferida a la hora de determinar la trayectoria de una pelota que rebota en el suelo, el movimiento del brazo de un personaje animado se determinaría aplicando la Cinemática para calcular las rotaciones apropiadas de las articulaciones del hombro, el codo y la muñeca. Por ello, muchas investigaciones en el campo de la animación de personajes vivientes se centran en los problemas de anatomía y en cómo la estructura de las articulaciones y las extremidades influyen en el movimiento.

Un método típico de aplicar la Cinemática consiste en comenzar representando un personaje mediante una figura compuesta por líneas rectas, que simula la estructura del esqueleto del personaje que se quiere animar. Cada sección de la figura se cubre luego con una malla poligonal que representa la superficie del personaje en las proximidades de dicha sección y se establecen una serie de reglas para determinar cómo hay que conectar entre sí las mallas adyacentes. Después, la figura puede manipularse (mediante software o un animador humano) reposicionando las articulaciones de la estructura del esqueleto, de la misma manera que se manipularía una marioneta. Los puntos en los que las "cuerdas" de la marioneta se conectan al modelo se denominan **avars**, que es la abreviatura de "*articulation variables*", variables de articulación (o más recientemente, "*animation variables*", variables de animación).

En la práctica, los **avars** se utilizan para controlar algo más que las simples posiciones de las articulaciones del esqueleto. Por ejemplo, el cowboy llamado Woody de *Toy Story* (Figura 10.1) tenía aproximadamente 100 avars asociados solo con la cara, lo que permitía a los animadores ajustar sus rasgos faciales con el fin de expresar emociones y mover la boca para que se ajustará a las palabras pronunciadas por el personaje.

Gran parte de la investigación en la aplicación de la Cinemática ha estado dirigida hacia el desarrollo de algoritmos para calcular automáticamente secuencias de posiciones de las extremidades que imiten el movimiento natural. Debido a ello, ahora hay disponibles algoritmos que generan secuencias realistas de movimientos tales como el caminar.

Sin embargo, gran parte de la animación basada en la Cinemática sigue generándose por el procedimiento de dirigir a un personaje a través de una secuencia predefinida de posiciones de las articulaciones y las extremidades. Estas posiciones pueden ser definidas gracias a la creatividad de un animador o pueden obtenerse mediante la técnica de **captura de movimiento**, que implica registrar las posiciones de un modelo vivo a medida que el modelo realiza la acción deseada. Para ser más precisos, después de aplicar cinta reflectante en determinados puntos estratégicos del cuerpo de una persona, puede fotografiarse a esa persona desde múltiples ángulos mientras que lanza una pelota. Después, observando la ubicación de la cinta reflectante en las diversas fotografías, se pueden identificar las orientaciones precisas de los brazos y las piernas de esa persona a medida que progresa la acción de lanzar la bola y esas

orientaciones pueden luego transferirse a un personaje como parte de una secuencia de animación.

El proceso de animación

El objetivo último de las investigaciones en el campo de la animación consiste en automatizar el proceso completo de animación. Podríamos imaginarnos un software que, a partir de los parámetros apropiados, generara automáticamente la secuencia de animación deseada. El progreso hecho en esta dirección queda ilustrado por el hecho de que la industria del cine ahora genera imágenes de multitudes, escena de batallas y estampidas de animales por medio de “robots” virtuales individuales que se mueven automáticamente dentro de un grafo de escena, realizando cada uno de ellos la tarea que tiene asignada.

Un caso interesante es el que tuvo lugar al filmar los ejércitos fantásticos de orcos y de humanos para la trilogía de *El señor de los anillos*. Cada guerrero que aparecía en pantalla estaba modelado como un objeto “inteligente” diferente, con sus propias características físicas y con una personalidad asignada aleatoriamente, que le proporcionaba las tendencias de atacar o de huir. En las simulaciones de prueba para la batalla del Abismo de Helm en la segunda parte de la trilogía, los orcos tenían configurada su tendencia a huir con un valor demasiado alto, por lo que simplemente salían corriendo al enfrentarse con los guerreros humanos (este es quizá el primer caso de extras virtuales que consideran que un trabajo es demasiado peligroso).

Por supuesto, una gran cantidad de animaciones siguen siendo creadas hoy día por animadores humanos. Sin embargo, en lugar de dibujar fotogramas bidimensionales a mano como en la década de 1920, estos animadores utilizan software para manipular objetos virtuales tridimensionales dentro de un grafo de escena, de una manera que recuerda la forma de controlar las marionetas como hemos explicado anteriormente al hablar de la Cinemática. De esta forma, un animador es capaz de crear una serie de escenas virtuales que se “fotografian” para generar la animación. En algunos casos, esta técnica se emplea para generar tan solo las escenas correspondientes a los fotogramas clave y luego se emplea software para generar las escenas intermedias, generando automáticamente la escena mientras el software aplica la Dinámica y la Cinemática para desplazar los objetos del grafo de la escena entre la posición correspondiente a un fotograma clave y la siguiente.

A medida que progresan las investigaciones en el campo de los gráficos por computadora y a medida que la tecnología continúa mejorando, el proceso de animación podrá irse automatizando cada vez más. Está por ver si el papel de los animadores humanos, de los actores humanos y de los estudios físicos llegará a quedar obsoleto, pero hay mucha gente que piensa que ese día no está tan lejos. De hecho, los gráficos 3D pueden llegar a afectar a la industria del cine mucho más de lo que esta se vió afectada durante la transición del cine mudo al cine sonoro.

Cuestiones y ejercicios

1. Las imágenes que vemos los seres humanos tienden a permanecer en nuestros sistemas perceptivos durante aproximadamente 200 milise-

gundos. Basándose en esta aproximación, ¿cuántas imágenes por segundo habrá que presentar a una persona para crear una animación? Compare esta aproximación con el número de fotogramas por segundo utilizado en la industria de cine.

2. ¿Qué es un guión (*storyboard*)?
3. ¿Qué es la interpolación entre fotogramas?
4. Defina los términos *Cinemática* y *Dinámica*.

Problemas de repaso

1. ¿Cuáles de las siguientes serían aplicaciones de los gráficos 2D y cuáles de los gráficos 3D?
 - a. Diseño de la maqueta de las páginas de una revista.
 - b. Dibujo de una imagen utilizando Microsoft Paint
 - c. Generación de imágenes a partir de un mundo virtual para un videojuego.
2. En el contexto de los gráficos 3D, ¿qué se correspondería con cada uno de los siguientes elementos de la fotografía tradicional? Explique sus respuestas.
 - a. Película.
 - b. Rectángulo del visor de una cámara.
 - c. La escena que se está fotografiando.
3. Al usar una proyección de perspectiva, ¿en qué condiciones una esfera de la escena no generará un círculo en el plano de proyección?
4. Al usar una proyección de perspectiva, ¿puede una imagen de un segmento de línea recta transformarse alguna vez en un segmento de línea curva en el plano de proyección? Justifique su respuesta.
5. Suponga que un extremo de una estaca recta de ocho metros de altura está situado a cuatro metros del centro de proyección. Suponga además que una línea recta trazada desde el centro de proyección a uno de los extremos de la estaca intersecta con el plano de proyección en un punto que se encuentra a un metro del centro de proyección. Si la estaca es paralela al plano de proyección, ¿qué longitud tendrá la imagen de la estaca en el plano de proyección?
6. Explique la diferencia entre una proyección paralela y una proyección en perspectiva.
7. Explique la relación entre la ventana de imagen y la memoria de fotograma.
8. Indique una diferencia importante entre la aplicación de gráficos 3D para producir una película y para generar la animación para un juego interactivo. Explique su respuesta.
9. Enumere algunas propiedades de un objeto que puedan incorporarse a un modelo de ese objeto para su uso en una escena de gráficos 3D. Identifique algunas propiedades que probablemente no se representarían en el modelo. Explique su respuesta.
10. Identifique algunas propiedades físicas de un objeto que no queden capturadas por un modelo que solo contenga una malla poligonal. (Es decir, una malla poligonal no constituye por sí sola un modelo del objeto.) Explique cómo podría añadirse una de esas propiedades al modelo del objeto.
11. ¿Podrían cuatro puntos cualesquiera en el espacio tridimensional ser los vértices de un parche que forme una malla poligonal? Explique su respuesta.
12. Cada uno de los siguientes conjuntos representa los vértices (expresados en el sistema de coordenadas rectangulares tradicional) de un parche de una malla poligonal. Describa la forma de la malla.

- Parche 1: (0, 0, 0) (0, 2, 0)
 (2, 2, 0) (2, 0, 0)
 Parche 2: (0, 0, 0) (1, 1, 1)
 (2, 0, 0)
 Parche 3: (2, 0, 0) (1, 1, 1)
 (2, 2, 0)
 Parche 4: (2, 2, 0) (1, 1, 1)
 (0, 2, 0)
 Parche 5: (0, 2, 0) (1, 1, 1)
 (0, 0, 0)

13. Cada uno de los siguientes conjuntos representa los vértices (expresados en el sistema de coordenadas rectangulares tradicional) de un parche de una malla poligonal. Describa la forma de la malla.

- Parche 1: (0, 0, 0) (0, 4, 0)
 (2, 4, 0) (2, 0, 0)
 Parche 2: (0, 0, 0) (0, 4, 0)
 (1, 4, 1) (1, 0, 1)
 Parche 3: (2, 0, 0) (1, 0, 1)
 (1, 4, 1) (2, 4, 0)
 Parche 4: (0, 0, 0) (1, 0, 1)
 (2, 0, 0)
 Parche 5: (2, 4, 0) (1, 4, 1)
 (0, 4, 0)

14. Diseñe una malla poligonal que represente un sólido rectangular. Utilice el sistema de coordenadas rectangulares tradicional para codificar los vértices y dibuje un boceto que represente la solución.
15. Utilizando no más de ocho parches triangulares, diseñe una malla poligonal para aproximar la forma de una esfera de radio igual a la unidad. (Con solo ocho parches, la malla será una aproximación muy burda de la esfera, pero el objetivo es ver si ha comprendido el concepto de malla poligonal, más que generar una representación precisa de una esfera.) Represente los vértices de los parches utilizando el sistema de coordenadas rectangulares tradicional y dibuje un boceto de la malla.
16. ¿Por qué los siguientes cuatro puntos no pueden ser los vértices de un parche plano?
 (0, 0, 0) (1, 0, 0)
 (0, 1, 0) (0, 0, 1)

17. Suponga que los puntos (1, 0, 0), (1, 1, 1) y (1, 0, 2) son los vértices de un parche plano. ¿Cuál o cuáles de los siguientes segmentos de línea son normales a la superficie del parche?
- El segmento de línea que va de (1, 0, 0) a (1, 1, 0)
 - El segmento de línea que va de (1, 1, 1) a (2, 1, 1)
 - El segmento de línea que va de (1, 0, 2) a (0, 0, 2)
 - El segmento de línea que va de (1, 0, 0) a (1, 1, 1)
18. Identifique dos "tipos" de modelos procedimentales.
19. Compare los dos métodos más importantes de eliminación de superficies ocultas, el buffer z y el algoritmo del pintor, en lo respecta al procedimiento y a las ventajas y desventajas de cada uno de ellos.
20. ¿Cuál de las siguientes técnicas se emplea para cortar una parte de una imagen? Explique su respuesta.
- Anti-aliasing.
 - Recorte.
 - Conversión de barrido.
 - Trazado de rayos.
21. ¿Qué es un supermuestreo? ¿Para qué se utiliza?
22. ¿Qué complicaciones introduce el hecho de que la cámara de un grafo de escena pueda cambiar su posición y su orientación?
23. Suponga que la superficie del parche plano con vértices (0, 0, 0), (0, 2, 0), (2, 2, 0) y (2, 0, 0) es suave y brillante. Si un rayo luminoso tiene su origen en el punto (0, 0, 1) e incide en la superficie en el punto (1, 1, 0), ¿a través de cuáles de los siguientes puntos pasará el rayo reflejado?
- (0, 0, 1)
 - (1, 1, 1)
 - (2, 2, 1)
 - (3, 3, 1)
24. Suponga que una boya sirve de soporte a una fuente luminosa situada diez metros por encima de la superficie del agua en calma. ¿En qué punto de la superficie del agua verá un observador el reflejo de la luz,

- si ese observador está a quince metros de la boya y a cinco metros por encima de la superficie del agua?
25. Si un pez está nadando por debajo de la superficie del agua, que está en calma, y un observador está contemplando al pez desde fuera del agua, ¿dónde parecerá estar el pez desde la perspectiva del observador?
- Por encima y hacia atrás con respecto a su posición real.
 - En su posición real.
 - Por debajo y hacia delante de su posición real.
26. Suponga que los puntos $(1, 0, 0)$, $(1, 1, 1)$ y $(1, 0, 2)$ son los vértices de un parche plano y que esos vértices están enumerados en sentido contrario a las agujas del reloj, vistos desde fuera del objeto. En cada uno de los siguientes casos, indique si un rayo luminoso que tenga su origen en el punto indicado incidiría sobre la superficie del parche desde el exterior o desde el interior del objeto.
- $(0, 0, 0)$
 - $(2, 0, 0)$
 - $(2, 1, 1)$
 - $(3, 2, 1)$
27. Proporcione un ejemplo en el que un objeto situado fuera del volumen de visualización pudiera continuar apareciendo en la imagen final. Explique su respuesta.
28. Describa el contenido y el propósito de un buffer z.
29. ¿Cómo se generan los siguientes tipos de luz?
- Luz ambiente.
 - Luz especular.
 - Luz difusa.
- Proporcione una explicación breve.
30. Suponga que la superficie de un objeto está cubierta por franjas verticales alternativas de color naranja y azul, cada una de las cuales tiene un centímetro de anchura. Si el objeto se sitúa en una escena de modo que las posiciones de los píxeles queden asociadas con puntos del objeto a intervalos de dos centímetros, ¿cuáles serían las posibles apariencias del objeto en la imagen final? Explique su respuesta.
31. Aunque la aplicación de texturas y la aplicación de protuberancias son formas de asociar una "textura" con una superficie, son técnicas considerablemente distintas. Escriba un breve párrafo comparando las dos.
32. ¿Cómo ayuda un fotograma clave a construir un guión?
33. ¿Qué es el sangrado de color?
34. Indique en qué se diferencia el hardware de una computadora diseñada para videojuegos interactivos de un PC de propósito general?
35. ¿Qué es la captura de movimiento? ¿Dónde se utiliza?
36. ¿Es el procesamiento de imágenes un campo de los gráficos? Explique su respuesta.
37. ¿Qué es un modelo procedimental? ¿Por qué se utiliza?
38. ¿Cuáles son las ventajas de OpenGL? ¿Qué desventaja tiene?
39. ¿Qué es la eliminación de caras traseras? ¿Cuáles son las ventajas de este método?
40. Si comparamos una imagen de una escena generada mediante el trazado de rayos tradicional con una imagen similar de la misma escena producida mediante la técnica de radiosity, ¿qué diferencias presentarán ambas imágenes?
41. ¿Cuántos fotogramas harían falta para generar una producción animada de 90 minutos para el cine?
42. Proporcione ejemplos de elementos de hardware que reduzcan la complejidad del software de aplicación de gráficos?
43. Explique cómo ayudaría el uso de un buffer z a la hora de crear una secuencia de animación que muestre un único objeto moviéndose en una escena.
44. ¿Para qué se utilizan las curvas de Bezier?

Cuestiones sociales

Las siguientes cuestiones pretenden ser una guía para los problemas éticos/sociales/legales asociados con el campo de la computación. El objetivo no es responder simplemente a estas cuestiones. El lector debería considerar también por qué las ha contestado de la forma en que lo ha hecho y analizar si sus justificaciones son coherentes entre las distintas cuestiones.

1. Suponga que la animación por computadora alcanza el punto en el que ya no se necesitan actores reales en el cine ni en la televisión. ¿Cuáles serían las consecuencias? ¿Qué efectos en cascada tendría el que ya no hubiera "estrellas de cine"?
2. Con el desarrollo de las cámaras digitales y del software relacionado, la capacidad de modificar o fabricar fotografías está al alcance del público en general. ¿Qué cambios traerá esto a la sociedad? ¿Qué problemas éticos y legales podrían surgir?
3. ¿Hasta qué punto las fotografías pueden ser propiedad de alguien? Suponga que una persona carga su fotografía en un sitio web y alguna otra persona descarga esa fotografía, la modifica para que el sujeto aparezca en una situación comprometida y distribuye la versión modificada. ¿Qué podría hacer el sujeto de esa fotografía?
4. ¿Hasta qué grado es responsable el programador que ha ayudado a desarrollar un videojuego violento de las consecuencias que este pueda tener? ¿Debería restringirse el acceso de los niños a los videojuegos? En caso afirmativo, ¿cómo podría restringirse el acceso y quién debería restringirlo? ¿Y qué sucede con otros grupos sociales, como los criminales convictos?

Lecturas adicionales

Angel, E. *Interactive Computer Graphics, A Top-Down Approach Using OpenGL*, 5ª ed. Boston, MA: Addison-Wesley, 2009.

Bowman, D. A., E. Kruijff, J. J. LaViola, Jr. y I. Poupyrev. *3D User Interfaces Theory and Practice*. Boston, MA: Addison-Wesley, 2005.

Hill, Jr., F. L. y S. Kelley. *Computer Graphics Using OpenGL*. 3ª ed. Upper Saddle River, NJ: Prentice-Hall, 2007.

McConnell, J. J. *Computer Graphics, Theory into Practice*. Sudbury, MA: Jones and Bartlett, 2006.

Parent, R. *Computer Animation, Algorithms and Techniques*, 2ª ed. San Francisco, CA: Morgan Kaufmann, 2008.

Inteligencia artificial

En este capítulo vamos a explorar la rama de las Ciencias de la computación conocida como inteligencia artificial. Aunque este campo es relativamente joven ha producido algunos resultados sorprendentes, como por ejemplo jugadores de ajedrez expertos, computadoras que parecen aprender y razonar, y máquinas que coordinan sus actividades de cara a conseguir un objetivo común, como por ejemplo ganar un partido de fútbol. En el campo de la inteligencia artificial, la ciencia ficción de hoy día podría muy bien llegar a ser la realidad de mañana.

11.1 Inteligencia y máquinas

- Agentes inteligentes
- Metodologías de investigación
- El test de Turing

11.2 Percepción

- Comprensión de las imágenes
- Procesamiento del lenguaje

11.3 Razonamiento

- Sistemas de producción
- Árboles de búsqueda
- Heurística

11.4 Áreas adicionales de investigación

- Representación y manipulación del conocimiento
- Aprendizaje
- Algoritmos genéticos

11.5 Redes neuronales artificiales

- Propiedades básicas
- Entrenamiento con las redes neuronales artificiales
- Memoria asociativa

11.6 Robótica

11.7 Consideración de las consecuencias

La inteligencia artificial es el campo de las Ciencias de la computación que trata de construir máquinas autónomas, máquinas que sean capaces de llevar a cabo tareas complejas sin intervención humana. Este objetivo requiere que las máquinas sean capaces de percibir y razonar. Dichas capacidades caen dentro de la categoría de actividades de sentido común que, aunque son naturales para la mente humana, están resultando difíciles para la máquinas. El resultado es que el trabajo dentro de este campo continúa planteando grandes desafíos. En este capítulo vamos a abordar algunos de los temas de esta extensa área de investigación.

11.1 Inteligencia y máquinas

El campo de la inteligencia artificial es muy amplio y se mezcla con otros campos como la psicología, la neurología, las matemáticas, la lingüística y la ingeniería eléctrica y mecánica. Por tanto, para centrar nuestro análisis, comenzaremos considerando el concepto de agente y los tipos de comportamiento inteligente que un agente puede exhibir. De hecho, buena parte de la investigación en el campo de la inteligencia artificial puede clasificarse en función del comportamiento de un agente.

Agentes inteligentes

Un **agente** es un "dispositivo" que responde a estímulos procedentes de su entorno. Es natural pensar en un agente como si fuera una máquina individual, como por ejemplo un robot, aunque un agente puede adoptar otras formas; por ejemplo, la de un aeroplano autónomo, la de un personaje en un videojuego interactivo o la de un proceso que se comunica con otros procesos a través de Internet (quizá como cliente, como servidor o como igual, *peer*). La mayor parte de los agentes disponen de sensores que les permiten recibir datos de su entorno; además, tienen actuadores con los que pueden ejercer una influencia sobre ese entorno. Como ejemplos de sensores podríamos citar los micrófonos, las cámaras, los sensores de proximidad y los dispositivos de muestreo del aire o del suelo. Como ejemplos de actuadores tendríamos las piernas, las alas, las pinzas y los sintetizadores de voz.

Gran parte de la investigación en inteligencia artificial puede caracterizarse en el contexto de la construcción de agentes que se comportan de manera inteligente, lo que quiere decir que las acciones de los actuadores del agente deben constituir respuestas racionales a los datos recibidos a través de sus sensores. A su vez, podemos clasificar todas esas investigaciones considerando los diferentes niveles de esas respuestas.

La respuesta más simple es una acción refleja, que es sencillamente una respuesta predeterminada a los datos de entrada. Para obtener un comportamiento más "inteligente" hacen falta niveles superiores de respuesta. Por ejemplo, podemos dotar a un agente con un conocimiento de su entorno y exigirle que ajuste sus acciones correspondientemente. El proceso de lanzar una pelota de béisbol es, en buena medida, una acción refleja pero el determinar cómo y hacia dónde lanzarla requiere un conocimiento del entorno actual (por ejemplo, saber qué jugadores están en las distintas bases del campo). En el campo de la inteligencia artificial, el problema de determinar cómo almacenar, actuali-

zar, extraer y aplicar al proceso de toma de decisiones esos conocimientos acerca del mundo real sigue planteando grandes desafíos.

Si queremos que el agente persiga un objetivo, como por ejemplo ganar una partida de ajedrez o maniobrar a través de un pasillo atestado de personas, hace falta otro nivel de respuesta aún más sofisticado. Ese comportamiento dirigido por objetivos requiere que la respuesta del agente (o su secuencia de respuestas) sea el resultado del proceso de formar deliberadamente un plan de acción o de seleccionar la mejor acción posible de entre todas las opciones existentes.

En algunos casos, las respuestas de un agente mejoran a lo largo del tiempo, a medida que el agente aprende. Esto podría adoptar la forma de un proceso de desarrollo de un **conocimiento procedimental** (aprender "cómo") o de un proceso de almacenamiento de **conocimiento declarativo** (aprender "qué"). El aprendizaje de conocimiento procedimental suele implicar un proceso de prueba y error, mediante el que el agente aprende las acciones apropiadas a base de ser castigado por las acciones incorrectas y premiado por las que son acertadas. De acuerdo con este enfoque, se han desarrollado agentes que mejoran a lo largo del tiempo sus habilidades en juegos de competición, como las damas y el ajedrez. El aprendizaje de conocimiento declarativo suele consistir en ampliar o modificar los "hechos" presentes en el almacén de conocimiento de un agente. Por ejemplo, un jugador de béisbol debe ajustar repetidamente su base de datos de conocimiento (por ejemplo, para reflejar la posición cambiante de los jugadores en las distintas bases del campo) a partir de la cual se determinan las respuestas racionales a los sucesos futuros.

Para producir respuestas racionales a los estímulos, un agente debe "comprender" los estímulos recibidos a través de sus sensores. Es decir, el agente tiene que ser capaz de extraer información de los datos que sus sensores generan o, en otras palabras, el agente debe ser capaz de percibir. En algunos casos, se trata de un proceso sencillo. Las señales obtenidas de un giróscopo deben codificarse en formas que sean compatibles con los cálculos necesarios para determinar las respuestas. Pero en otros casos, la extracción de información de los datos de entrada resulta difícil. Como ejemplos podemos citar los intentos de comprender el lenguaje y las imágenes. Del mismo modo, los agentes tienen que ser capaces de formular sus respuestas en términos compatibles con los actuadores de los que disponen. Puede que esto sea un proceso sencillo, o ese proceso puede exigir al agente que formule las respuestas como frases completas habladas, lo que significa que el agente debe ser capaz de expresarse de forma oral. Por ello, temas tales como el procesamiento y análisis de imágenes, la comprensión del lenguaje natural y la generación del habla son áreas de investigación de gran importancia.

Los atributos de los agentes que hemos identificado representan áreas de investigación tanto actuales como pasadas. Por supuesto, no son completamente independientes entre sí. Nos gustaría poder desarrollar agentes que poseyeran todos esos atributos, obteniendo así agentes que comprendieran los datos recibidos de su entorno y desarrollaran nuevos patrones de respuesta a través de un proceso de aprendizaje, cuyo objetivo sería maximizar las habilidades del agente. Sin embargo, aislando los diversos tipos de comportamiento racional y tratando de obtenerlos de manera independiente, los investigadores obtienen avances que pueden combinarse posteriormente con los progresos realizados en otras áreas, con el fin de producir agentes más inteligentes.

Terminamos esta subsección presentando un agente que proporcionará el contexto para las explicaciones contenidas en las Secciones 11.2 y 11.3. El agente está diseñado para resolver el puzzle de los ocho cuadrados, el cual está compuesto por ocho piezas cuadradas etiquetadas del 1 al 8 y montadas en un marco capaz de albergar un número total de nueve de esas piezas, dispuestas en tres filas y tres columnas (Figura 11.1). Entre las piezas situadas en el marco hay un hueco al que puede moverse cualquiera de las piezas adyacentes, lo que permite ir mezclando las piezas del marco. El problema consiste en mover las piezas otra vez a sus posiciones iniciales (Figura 11.1) partiendo de una disposición desordenada de las mismas.

Nuestro agente adoptará la forma de una caja equipada con una pinza, una cámara de vídeo y un dedo con un extremo recubierto de goma, para que no se deslice al empujar algún objeto (Figura 11.2). Cuando se enciende el agente por primera vez, su pinza se abre y se cierra, como si estuviera pidiendo el puzzle. Cuando colocamos un puzzle con las ocho piezas desordenadas en la pinza, esta se cierra para sujetarlo. Después de un breve periodo de tiempo, el dedo de la máquina desciende y comienza a empujar las piezas de un lado a otro dentro del marco, hasta colocarlas en su posición original. Llegados a este punto, la máquina suelta el puzzle y se apaga automáticamente.

Esta máquina solucionadora de puzzles exhibe dos de los atributos de los agentes que hemos identificado anteriormente. En primer lugar, debe ser capaz de percibir, en el sentido de que debe extraer el estado actual del puzzle de la imagen recibida a través de la cámara. Consideraremos la cuestión de la comprensión de imágenes en la Sección 11.2. En segundo lugar, la máquina debe desarrollar e implementar un plan para conseguir el objetivo. Consideraremos estas cuestiones en la Sección 11.3.

Metodologías de investigación

Para orientarse en el campo de la inteligencia artificial, es útil entender que este campo está avanzando en dos direcciones distintas. Una de las líneas de avance es la ingenieril, en la que los investigadores están tratando de desarrollar sistemas que exhiban un comportamiento inteligente. La otra línea de avance es la teórica, en la que los investigadores tratan de desarrollar una comprensión computacional de la inteligencia animal, y especialmente de la inteligencia humana. Podemos clarificar esta dicotomía considerando la manera en la que se persiguen esos dos objetivos. La técnica ingenieril conduce a una metodología orientada al rendimiento, porque el objetivo último es construir un producto que satisfaga ciertos objetivos de rendimiento. El enfoque teórico

Figura 11.1 El puzzle de ocho piezas una vez resuelto.

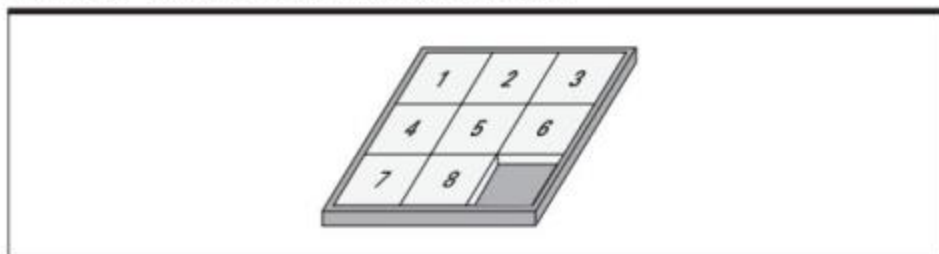
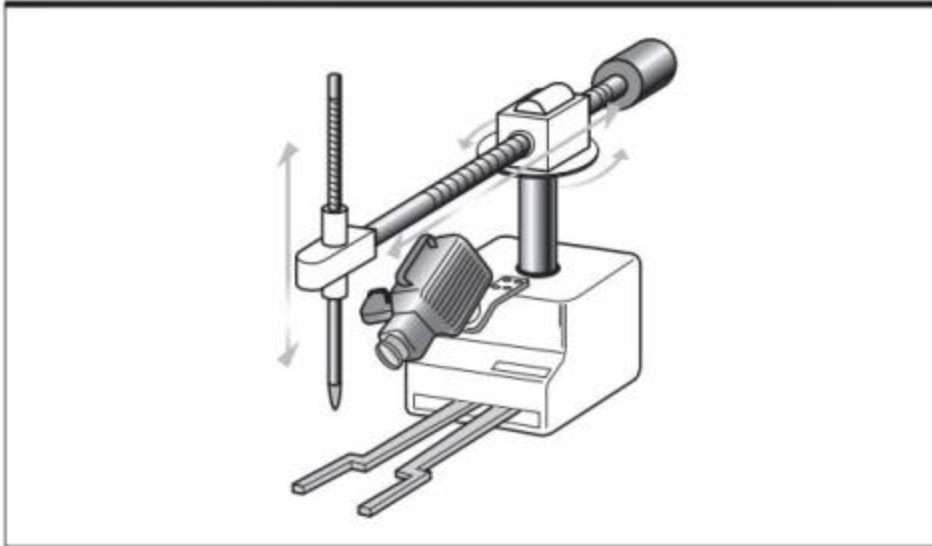


Figura 11.2 Nuestra máquina solucionadora de puzzles.



conduce a una metodología orientada a la simulación, porque el objetivo último es ampliar nuestra comprensión de la inteligencia, por lo que el énfasis se pone en el proceso subyacente, más que en el comportamiento exterior.

Por ejemplo, considere los campos del procesamiento del lenguaje natural y de la lingüística. Estos dos campos están íntimamente relacionados y ambos se benefician de las investigaciones realizadas en el otro, a pesar de lo cual los objetivos últimos son diferentes. Los lingüistas están interesados en aprender cómo procesan el lenguaje los seres humanos, por lo que tratan de alcanzar objetivos más teóricos. Los investigadores en el campo del procesamiento del lenguaje natural están interesados en desarrollar máquinas que puedan manipular el lenguaje natural, por lo que sus objetivos tienden a adoptar una orientación ingenieril. Por ello, los lingüistas operan en un modo orientado a la simulación, construyendo sistemas cuyo objetivo es el de probar teorías. Por el contrario, los investigadores en el campo del procesamiento del lenguaje natural operan en un modo orientado al rendimiento, construyendo sistemas con el fin de realizar tareas. Los sistemas construidos de este último modo (como por ejemplo los traductores de documentos y los sistemas que permiten a las máquinas responder a las órdenes verbales) dependen en gran medida del conocimiento obtenido por los lingüistas, pero a menudo aplican "atajos" que resulta que funcionan adecuadamente en el entorno restringido en el que opera cada sistema concreto.

Como ejemplo elemental, considere la tarea de desarrollar un intérprete de órdenes (*shell*) para un sistema operativo que reciba instrucciones del mundo exterior a través de comandos verbales en español. En este caso, la *shell* (un agente) no necesita preocuparse de todo el idioma español. Para ser más precisos, la *shell* no necesita distinguir entre los distintos significados de la palabra *copiar* (¿Es un nombre o un verbo? ¿Debe incluir la connotación de plagio?). En lugar de ello, la *shell* necesita simplemente distinguir la palabra *copiar* de otros comandos como *renombrar* y *borrar*. Por tanto, esa *shell* podría realizar su

tarea simplemente viendo si sus entradas se corresponden con patrones de audio predeterminados. El rendimiento de un sistema de este tipo podría ser satisfactorio para un ingeniero, pero la forma en que se obtiene ese comportamiento no resultaría estéticamente placentera para un teórico.

El test de Turing

En el pasado, el **test de Turing** (propuesto por Alan Turing en 1950) ha servido como manera de medir el progreso en el campo de la inteligencia artificial. Hoy día, la importancia que se concedía al test de Turing se ha desvanecido, aunque continúa formando parte del folklore propio del campo de la inteligencia artificial. La propuesta de Turing consistía en permitir que una persona, a la que llamaremos interrogador, se comunicara con un sujeto de prueba por medio de un teclado, sin que el interrogador supiera si el sujeto de prueba era un ser humano o una máquina. En este entorno, diríamos que una máquina se comporta de manera inteligente si el interrogador no es capaz de distinguirla de un ser humano. Turing predijo que para el año 2000 las máquinas tendrían un 30 por ciento de posibilidades de pasar un test de Turing de cinco minutos de duración, una conjetura que ha resultado ser sorprendentemente precisa.

Una de las razones por la que ya no se considera el test de Turing como una medida significativa de la inteligencia es que se puede **producir** con facilidad una inquietante apariencia de inteligencia. Un ejemplo muy conocido surgió como resultado del programa DOCTOR (una versión del sistema más general denominado ELIZA) desarrollado por Joseph Weizenbaum a mediados de la década de 1960. Este programa interactivo estaba diseñado para proyectar la imagen de un analista rogeriano realizando una sesión de psicoterapia. La computadora desempeñaba el papel del analista, mientras que el usuario hacía de paciente. Internamente, lo único que DOCTOR hacía era reestructurar las frases pronunciadas por el paciente de acuerdo con algunas reglas bien definidas y volver a plantearlas al paciente. Por ejemplo, en respuesta a la frase "Hoy estoy cansado", DOCTOR podría haber contestado "¿Por qué cree que está cansado hoy?". Si DOCTOR era incapaz de reconocer la estructura de la frase, simplemente respondía con algo como "Continúe" o "Eso es muy interesante".

Los orígenes de la inteligencia artificial

Los intentos de construir máquinas que imiten el comportamiento humano son muy antiguos, pero muchos expertos coincidirán en que el campo moderno de la inteligencia artificial tiene su origen en 1950. Ese es el año en el que Alan Turing publicó el artículo "*Computing Machinery and Intelligence*" (máquinas de computación e inteligencia) en el que proponía que podrían programarse máquinas que exhibieran comportamiento inteligente. El nombre del campo, *inteligencia artificial*, fue acuñado unos cuantos años después en la ahora legendaria propuesta escrita por John McCarthy, en la que sugería que "se llevara a cabo un estudio sobre inteligencia artificial durante el verano de 1956 en el Dartmouth College" para explorar "la conjetura de que cualquier aspecto del aprendizaje o cualquier otra característica de la inteligencia puede ser, en principio, descrita de forma tan precisa que es posible construir una máquina para simularla."

El propósito de Weizenbaum al desarrollar DOCTOR era el estudio de la comunicación en lenguaje natural. El tema de la psicoterapia simplemente proporcionaba un entorno en el que el programa podía "comunicarse". Sin embargo, para consternación de Weizenbaum, varios psicólogos propusieron utilizar su programa en psicoterapias reales. (La tesis rogeriana es que es el paciente, y no el psicoanalista, el que debe conducir la conversación durante la terapia, así que esos psicólogos argumentaban que una computadora podría posiblemente participar en la conversación tan bien como podría hacerlo un terapeuta.) Además, DOCTOR proyectaba una imagen de comprensión tan fuerte que muchos de los que se "comunicaban" con el programa llegaban a aficionarse a esos diálogos de preguntas y respuestas que mantenían con la máquina. En cierto sentido, DOCTOR pasó el test de Turing. El resultado fue que se plantearon cuestiones tanto éticas como técnicas y Weizenbaum terminó por convertirse en un activo defensor de la tesis de que hay que mantener la dignidad humana en este mundo de avances tecnológicos.

Otros ejemplos más recientes de "éxitos" a la hora de pasar el test de Turing serían los virus de Internet que llevan a cabo diálogos "inteligentes" con una víctima humana, para convencerla de que desactive sus protecciones contra el *malware*. Además, fenómenos similares a los test de Turing son los que se producen en el contexto de determinados juegos de computadora, como por ejemplo los programas para jugar al ajedrez. Aunque estos programas seleccionan sus movimientos aplicando simplemente técnicas de fuerza bruta (similares a las que explicaremos en la Sección 11.3), las personas que compiten contra la computadora experimentan a menudo la sensación de que las máquinas poseen creatividad e incluso una personalidad. Sensaciones similares se experimentan en el campo de la robótica, en el que se han construido máquinas con atributos físicos que permiten proyectar características inteligentes. Como ejemplos podemos citar los perros robot de juguete, que proyectan personalidades adorables simplemente moviendo la cabeza o levantando las orejas en respuesta a un sonido.

Cuestiones y ejercicios

1. Identifique diversos tipos de acciones "inteligentes" que un agente podría llevar a cabo.
2. Una planta colocada en una habitación oscura en la que solo exista una fuente luminosa tenderá a crecer hacia la luz. ¿Se trata de una respuesta inteligente? ¿Posee inteligencia la planta? ¿Cómo definiría entonces la inteligencia?
3. Suponga que diseñamos una máquina expendedora para suministrar varios productos dependiendo del botón que se pulse. ¿Diría que esa máquina es "consciente" de qué botón se ha pulsado? ¿Cómo definiría entonces el hecho de "ser consciente"?
4. Si una máquina pasa el test de Turing, ¿estaría de acuerdo en afirmar que es inteligente? En caso negativo, ¿estaría de acuerdo en afirmar que parece inteligente?

5. Suponga que utiliza un programa de mensajería para hablar con alguien a través de Internet y que mantiene con ese interlocutor una conversación coherente y con sentido durante diez minutos. Si posteriormente averiguara que ha estado conversando con una máquina, ¿concluiría que esa máquina es inteligente? Explique su respuesta.

11.2 Percepción

Para responder de manera inteligente a las entradas procedentes de sus sensores, un agente debe ser capaz de comprender esas entradas. Es decir, el agente tiene que ser capaz de percibir. En esta sección vamos a explorar dos áreas de investigación en el campo de la percepción que han demostrado ser especialmente complicadas: la comprensión de las imágenes y del lenguaje.

Comprensión de las imágenes

Consideremos los problemas planteados por la máquina solucionadora de puzzles que hemos presentado en la sección anterior. La apertura y cierre de la pinza de la máquina no supone obstáculo de importancia y la capacidad de detectar la presencia del puzzle en la pinza durante este proceso es bastante sencilla de implementar, porque nuestra pinza requiere muy poca precisión. Incluso el problema de centrar la cámara sobre el puzzle puede solventarse diseñando simplemente la pinza de modo que coloque el puzzle en una posición predeterminada que resulte adecuada para su visualización. En consecuencia, el primer comportamiento inteligente que la máquina requiere es la extracción de información a través de un medio visual.

Es importante comprender que el problema al que se enfrenta nuestra máquina a la hora de examinar el puzzle no es el de simplemente producir y almacenar una imagen. La tecnología es capaz de hacer esto desde hace muchos años, como por ejemplo en los sistemas tradicionales de fotografía y televisión. Más bien, el problema consiste en comprender la imagen con el fin de extraer de ella el estado actual del puzzle (y quizá, posteriormente, monitorizar el movimiento de las piezas).

En el caso de nuestra máquina solucionadora de puzzles, las posibles interpretaciones de la imagen de un puzzle son relativamente limitadas. Podemos suponer que lo que aparece es siempre una imagen que contiene los dígitos 1 a 8 en un patrón bien organizado. El problema consiste simplemente en extraer la disposición de estos dígitos. Para ello, vamos a imaginar que la imagen del puzzle se ha codificado en forma de bits en la memoria de la computadora, representando cada bit el nivel de brillo de un píxel concreto. Asumiendo que el tamaño de la imagen es uniforme (la máquina sostiene el puzzle en una posición predeterminada delante de la cámara), nuestra máquina puede detectar qué pieza se encuentra en cada posición por el procedimiento de comparar las diferentes secciones de la imagen con una serie de plantillas pregrabadas, que estarán compuestas por los patrones de bits generados por los dígitos individuales que se usan en el puzzle. A medida que vayan detectándose correspondencias, se irá revelando el estado actual del puzzle.

Esta técnica de reconocimiento de imágenes es un método utilizado en los lectores ópticos de caracteres. Sin embargo, tiene la desventaja de exigir un cierto grado de uniformidad en cuanto al estilo, el tamaño y la orientación de los símbolos que se están leyendo. En particular, el patrón de bits generado por un carácter físicamente grande no se ajusta a la plantilla correspondiente a una versión más pequeña del mismo símbolo, incluso aunque las formas sean iguales. Además, es fácil imaginar hasta qué punto se incrementa la dificultad del problema a la hora de intentar procesar material manuscrito.

Otra solución al problema del reconocimiento de caracteres se basa en intentar encontrar correspondencias entre las características geométricas en lugar de buscar correspondencias relativas a la apariencia exacta de los símbolos. En este caso, el dígito 1 podría caracterizarse como una única línea vertical, el 2 podría considerarse una línea curva abierta unida a una línea recta horizontal por su parte inferior, y así sucesivamente. Este método de reconocimiento de símbolos consta de dos etapas: la primera consiste en extraer las características de la imagen que se está procesando y la segunda en comparar esas características con la de los símbolos conocidos. Como sucede con la técnica basada en la detección de correspondencias con plantillas pregrabadas, esta técnica de reconocimiento de caracteres tampoco es perfecta. Por ejemplo, una serie de errores poco importantes en la imagen pueden generar un conjunto de características geométricas completamente distintas, como sucede al tratar de distinguir entre una O y una C o, si hablamos del puzzle de ocho piezas, al tratar de distinguir el 3 del 8.

Nosotros tenemos suerte en nuestra aplicación del puzzle porque no necesitamos comprender imágenes de escenas tridimensionales genéricas. Considere, por ejemplo, la ventaja que tenemos al poder estar seguros de que las formas que hay que reconocer (los dígitos de 1 a 8) están aisladas en diferentes partes de la imagen, en lugar de aparecer como imágenes solapadas, que es lo que sucede comúnmente en la mayoría de los escenarios de carácter general. Por ejemplo, en una fotografía genérica, no solo tenemos el problema de reconocer un objeto desde diferentes ángulos, sino que también tenemos que enfrentarnos con el hecho de que algunas partes del objeto pueden estar ocultas a la vista.

La tarea de comprender imágenes genéricas suele abordarse mediante un proceso en dos pasos: (1) **procesamiento de imágenes**, que hace referencia a la tarea de identificar las características de la imagen y (2) **análisis de imágenes**, que hace referencia a la tarea de comprender lo que esas características significan. Ya hemos observado esta dicotomía en el contexto de reconocimiento de símbolos mediante sus características geométricas. En esa situación, el procesamiento de imágenes está representado por el proceso de identificar las características geométricas presentes en la imagen, mientras que el análisis de imágenes está representado por el proceso de identificar el significado de esas características.

El procesamiento de imágenes abarca numerosos temas distintos. Uno de ellos es el del realce de contornos, que es el proceso de aplicar técnicas matemáticas para clarificar las fronteras entre distintas regiones de una imagen. En cierto sentido, el realce de contorno es un intento de convertir una fotografía en un dibujo compuesto por líneas. Otra de las actividades dentro del procesamiento de imágenes se conoce con el nombre de localización de regiones. Este

Inteligencia artificial fuerte e inteligencia artificial débil

La conjetura de que las máquinas pueden programarse para exhibir comportamiento inteligente se conoce como **inteligencia artificial débil** y es aceptada, en distinto grado, por un gran número de expertos en la actualidad. Sin embargo, la conjetura de que las máquinas pueden programarse para poseer inteligencia y, de hecho, consciencia, conjetura que se conoce con el nombre de **inteligencia artificial fuerte**, es objeto de grandes debates. Los oponentes de la inteligencia artificial fuerte argumentan que una máquina es inherentemente distinta de un ser humano y que por tanto jamás puede sentir amor, diferenciar el bien del mal y pensar acerca de sí misma de la misma manera que un ser humano lo hace. Sin embargo, los defensores de la inteligencia artificial fuerte argumentan que la mente humana está construida a partir de pequeños componentes que, considerados individualmente, no son humanos y tampoco conscientes, pero que al combinarse, sí lo son. ¿Por qué no podrá darse el mismo fenómeno en las máquinas, argumentan?

El problema a la hora de resolver el debate de la inteligencia artificial fuerte es que atributos tales como la inteligencia y la consciencia son características internas que no pueden identificarse directamente. Como Alan Turing señaló, otorgamos inteligencia a otros seres humanos porque se comportan de manera inteligente, aún cuando no seamos capaces de observar sus estados mentales internos. ¿Estamos entonces preparados para actuar igual con las máquinas, si estas exhiben las características externas de la consciencia? ¿Por qué?

es el proceso de identificar aquellas áreas de la imagen que tienen propiedades comunes, como por ejemplo el mismo brillo, color o textura. Esas regiones probablemente representen secciones de la imagen que pertenecen a un mismo objeto. (Es la capacidad de reconocer regiones lo que permite a las computadoras añadir color a las películas antiguas en blanco y negro.) Otra actividad más dentro del ámbito del procesamiento de imágenes es el suavizado, que es el proceso de eliminar fallos en la imagen. El suavizado evita que los errores de la imagen confundan a los otros pasos del procesamiento de imágenes, aunque una cantidad excesiva de suavizado puede hacer que se pierda también información importante.

El suavizado, el realce de contornos y la localización de regiones son pasos del proceso de identificación de los distintos componentes de una imagen. El análisis de imágenes es el proceso de determinar qué es lo que esos componentes representan y, en último término, qué es lo que la imagen significa. Aquí nos encontramos con problemas tales como reconocer desde diferentes perspectivas objetos cuya visión está parcialmente obstruida. Una técnica de análisis de imágenes consiste en comenzar con una suposición acerca de lo que puede ser la imagen y luego tratar de asociar los componentes de la imagen con los objetos cuya presencia se conjetura. Esta parece ser una solución que los seres humanos aplicamos. Por ejemplo, en ocasiones nos resulta difícil reconocer un objeto inesperado en un entorno en el que nuestra visión esté desenfocada, pero en cuanto tenemos una pista acerca de lo que el objeto puede ser, podemos identificarlo fácilmente.

Los problemas asociados con el análisis de imágenes genéricas son enormes y todavía quedan por hacer muchas investigaciones en este área. De hecho, el análisis de imágenes es uno de esos campos que ilustra cómo muchas

tareas que la mente humana realiza de forma rápida y aparentemente fácil continúan desafiando las capacidades de las máquinas.

Procesamiento del lenguaje

Otro problema de percepción que ha demostrado ser difícil de resolver es el de la comprensión del lenguaje. El éxito obtenido al traducir lenguajes de programación de alto nivel a lenguaje máquina (Sección 6.4) hizo pensar a los primeros investigadores que la capacidad de programar a las computadoras para que comprendieran el lenguaje natural se podría conseguir en solo unos pocos años. De hecho, la capacidad de traducir programas proporciona la ilusión de que la máquina comprende realmente el lenguaje que se está traduciendo. (Recuerde de la Sección 6.1 la historia de Grace Hopper acerca de esos directivos que creían que estaba enseñando a las computadoras a entender el alemán.)

Lo que esos investigadores no comprendían eran las enormes diferencias entre los lenguajes de programación formales y los lenguajes naturales como el español, el inglés, el alemán o el latín. Los lenguajes de programación se construyen a partir de primitivas bien diseñadas, de modo que cada sentencia tiene una única estructura gramatical y un único significado. Por el contrario, una sentencia en un lenguaje natural puede tener múltiples significados, dependiendo del contexto o incluso de la manera en que sea comunicada. Por ello, para comprender el lenguaje natural, los humanos dependen fuertemente de la posesión de conocimientos adicionales.

Por ejemplo, las frases

Velázquez pintaba personas.

y

Juan es un burro.

tienen varios significados, que no pueden distinguirse traduciendo o analizando sintácticamente cada palabra de forma independiente. En lugar de ello, hace falta la capacidad de comprender el contexto en el que se dice la frase. En otros casos, el verdadero significado de una frase no coincide con su traducción literal. Por ejemplo,

¿Sabes qué hora es?

significa a menudo "Dime qué hora es, por favor", o si quien la dice ha estado esperando durante un buen rato, puede querer decir "Llegas muy tarde".

Para descubrir el significado de una frase en el lenguaje natural, se requieren por tanto varios niveles de análisis. El primero de ellos es el **análisis sintáctico**. Es en este nivel cuando se reconoce a *María* como sujeto de la frase

María le dió a Juan una tarjeta de cumpleaños.

mientras que en la frase

Juan recibió una tarjeta de cumpleaños de María.

se reconoce que el sujeto es *Juan*.

Otro nivel de análisis es el **análisis semántico**. A diferencia del proceso de análisis sintáctico, que simplemente identifica el papel gramatical de cada palabra, el análisis semántico tiene la tarea de identificar el papel semántico de

cada palabra de la frase. El análisis semántico trata de identificar cosas tales como la acción descrita, el agente de esa acción (que puede ser o no el sujeto de la frase) y el objeto de la acción. Es el análisis semántico lo que permite saber que las frases "María le dió a Juan una tarjeta de cumpleaños" y "Juan recibió una tarjeta de cumpleaños de María" dicen lo mismo.

Un tercer nivel de análisis es el **análisis contextual**. Es en este nivel cuando se incluye en el proceso de comprensión el contexto de la frase. Por ejemplo, es fácil identificar el papel gramatical de cada palabra en la frase

El actor rechazó el papel.

Podemos incluso realizar un análisis semántico identificando la acción implicada como *rechazó*, el agente como *actor*, y así sucesivamente. Pero hasta que no consideramos el contexto de la frase, no queda claro el significado de la misma. En concreto, tendrá un significado distinto si la palabra *papel* hace referencia a un personaje de una obra teatral o a una hoja de papel. Precisamente, es en el nivel contextual cuando podrá revelarse finalmente el verdadero significado de la pregunta "¿Sabes qué hora es?".

Hay que recalcar que los diversos niveles de análisis (sintáctico, semántico y contextual) no son necesariamente independientes entre sí. Por ejemplo, imagine que el pie de una fotografía dice

Detenidos comiendo bocadillos.

En este caso, las personas que aparecen en la fotografía pueden ser el sujeto de la frase (los *detenidos*) si lo que la fotografía muestra es a unas personas que habían sido anteriormente detenidas y que ahora están comiéndose unos bocadillos. Sin embargo, las personas que aparecen en la fotografía serán el objeto si lo que la fotografía muestra son unas personas que están siendo detenidas mientras se comen un bocadillo. Por tanto, la frase tiene más de una estructura gramatical posible y cuál de las dos sea correcta dependerá del contexto, es decir, de lo que la fotografía muestre.

Otra área de investigación en el procesamiento del lenguaje natural es la que se refiere a documentos completos en lugar de a frases individuales. En este caso, los problemas caen dentro de dos categorías distintas: **recuperación de información** y **extracción de información**. La recuperación de información hace referencia a la tarea de identificar documentos relacionados con el tema que estemos tratando. Un ejemplo sería el problema al que se enfrentan los usuarios de la World Wide Web al tratar de buscar sitios relacionados con un tema concreto. La solución más popular consiste en buscar sitios de acuerdo con una serie de palabras clave, pero esto genera a menudo una avalancha de sitios carentes de interés, al mismo tiempo que puede no proporcionar ningún enlace que sí tiene importancia simplemente por el hecho de que en ese sitio se utiliza "automóvil" en lugar de "coche" o "vehículo". Lo que hace falta es un mecanismo de búsqueda que comprenda el contenido de los sitios que se analizan. La dificultad de obtener ese tipo de comprensión es la razón por la que muchos investigadores están adoptando técnicas como XML con el fin de obtener una Web semántica, como ya hemos indicado en la Sección 4.3.

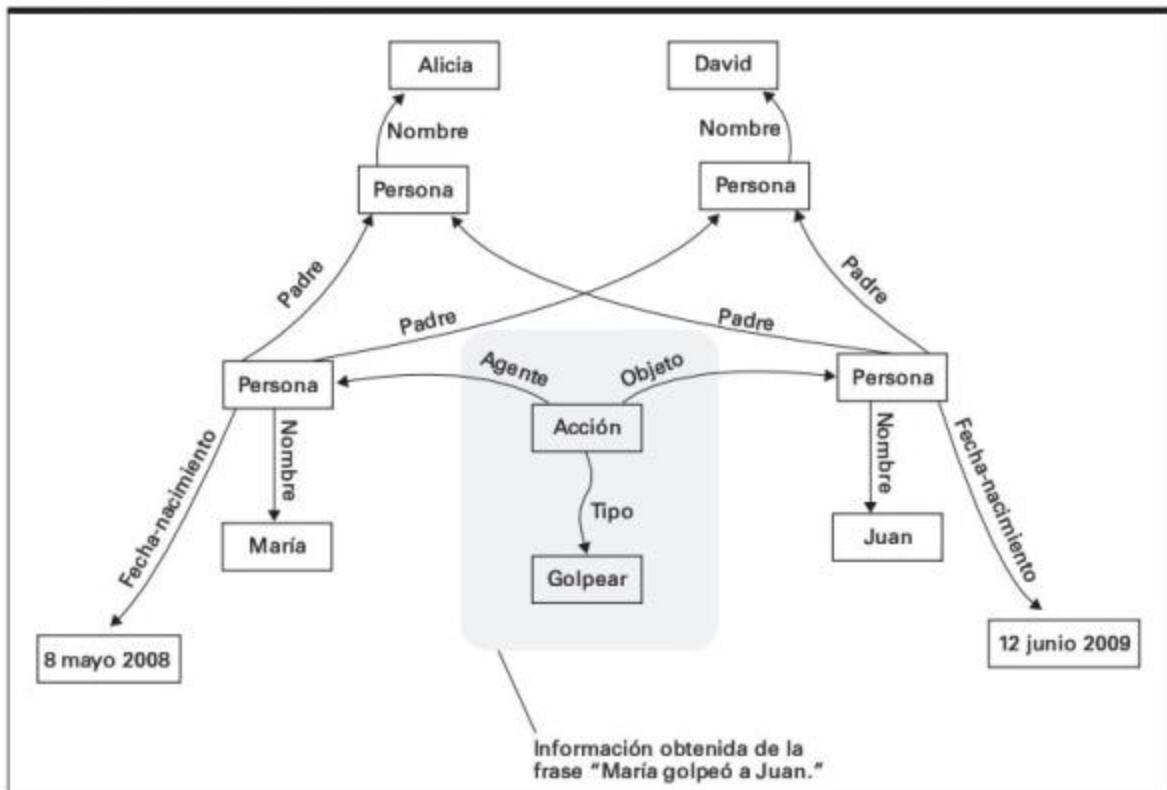
La extracción de información hace referencia a la tarea de extraer información de los documentos, con el fin de darle una forma que resulte útil para otras aplicaciones. Esto puede implicar identificar la respuesta a una cuestión

específica o guardar la información de manera tal que puedan responderse cuestiones en algún momento posterior. Una de esas maneras en las que puede almacenarse la información se conoce con el nombre de **marco**, que es básicamente una plantilla en la que se guardan detalles específicos. Por ejemplo, considere un sistema para leer un periódico. El sistema podría utilizar diversos marcos, uno por cada tipo de artículo que pueda aparecer en un periódico. Si el sistema identifica que un artículo trata sobre un robo, intentará rellenar las casillas necesarias dentro del marco correspondiente a los robos. Este marco solicitaría, probablemente, elementos de información tales como la dirección del robo, la fecha y la hora en que se produjo, los objetos que fueron sustraídos, etc. Por el contrario, si el sistema identifica que el artículo informa sobre un desastre natural, rellenará el marco correspondiente a los desastres naturales, que indicará al sistema que debe identificar el tipo de desastre, los daños producidos, etc.

Otra manera en la que los extractores de información guardan la información extraída se conoce con el nombre de **red semántica**. Se trata, esencialmente, de una gran estructura de datos enlazados, en la que se emplean punteros para indicar las asociaciones entre los distintos elementos de datos. La Figura 11.3 muestra parte de una red semántica en la que se ha resaltado la información obtenida a partir de la frase

María golpeó a Juan.

Figura 11.3 Una red semántica.

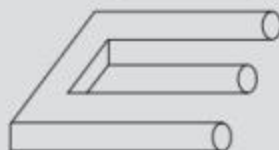


Inteligencia artificial en la palma de la mano

Las técnicas de inteligencia artificial están incorporándose cada vez más en las aplicaciones para teléfonos inteligentes. Por ejemplo, Google ha desarrollado Google Goggles, una aplicación para teléfono inteligente que proporciona un motor de búsqueda visual. Basta con hacer una fotografía de un libro, de un paisaje o de un anuncio utilizando la cámara del teléfono y Goggles realiza el procesamiento de imágenes, el análisis de imágenes y el reconocimiento de textos, después de lo cual inicia una búsqueda en la Web para identificar el objeto. Por ejemplo, un turista inglés que esté de visita en Francia puede hacer una fotografía de una señal de tráfico, de un menú o de cualquier otro texto y hacer que la aplicación la traduzca al inglés. Además de en Goggles, Google está trabajando activamente en el campo de la traducción de idiomas voz a voz. Muy pronto seremos capaces de hablar en español a través de nuestro teléfono y hacer que esas palabras sean traducidas automáticamente al inglés, al chino o a cualquier otro idioma. Los teléfonos inteligentes irán teniendo cada vez más capacidades a medida que se vaya utilizando la tecnología de la inteligencia artificial en formas cada vez más innovadoras.

Cuestiones y ejercicios

1. ¿Cómo difieren los requisitos de un sistema de vídeo de un robot si el robot lo utiliza para controlar sus actividades, frente al caso en que esa información recopilada por el sistema de vídeo se transmita a una persona que sea quien controle remotamente el robot?
2. ¿Qué es lo que nos dice que el siguiente dibujo no tiene sentido? ¿Cómo podría programarse en una máquina ese tipo de razonamiento?



3. ¿Cuántos bloques hay en la pila de bloques que se muestra a continuación? ¿Cómo podría programarse una máquina para responder a este tipo de preguntas de forma correcta?



4. ¿Cómo sabemos que las dos frases "Nada es mejor que una felicidad completa" y "Un tazón de sopa es mejor que nada" no implican que "un tazón de sopa es mejor que una felicidad completa"? ¿Cómo podríamos transferir a una máquina nuestra capacidad de realizar esta diferenciación?

5. Identifique las ambigüedades que aparecen al traducir la frase "Han puesto un banco nuevo en la plaza".
6. Compare los resultados de analizar sintácticamente las dos frases siguientes. A continuación, explique cómo difieren estas frases semánticamente.
El granjero construyó la cerca en el campo.
El granjero construyó la cerca en el verano.
7. Basándose en la red semántica de la Figura 11.3, ¿cuál es la relación familiar entre María y Juan?

11.3 Razonamiento

Utilicemos ahora la máquina solucionadora de puzzles presentada en la Sección 11.1 para explorar las técnicas para el desarrollo de agentes dotados de capacidad de razonamiento elemental.

Sistemas de producción

Una vez que nuestra máquina solucionadora de puzzles ha descifrado las posiciones de las piezas a partir de la imagen visual, tiene que tratar de averiguar qué movimientos hay que hacer para resolver el puzzle. Una solución a este problema que se nos podría ocurrir es preprogramar la máquina con soluciones para todas las posibles disposiciones de las piezas. Entonces la tarea de la máquina consistiría simplemente en seleccionar y ejecutar el programa apropiado. Sin embargo, el puzzle de ocho piezas tiene más de 100.000 configuraciones posibles, por lo que la idea de proporcionar una solución explícita para cada una no es muy tentadora. Por tanto, nuestro objetivo será programar la máquina de manera que pueda construir soluciones para el puzzle por su cuenta. Es decir, la máquina debe programarse para realizar actividades de razonamiento básicas.

El desarrollo de las capacidades de razonamiento en una máquina ha sido objeto de investigación durante muchos años. Uno de los resultados de estas investigaciones es el reconocimiento de que existe una gran clase de problemas de razonamiento que presentan características comunes. Dichas características se pueden definir mediante una entidad abstracta conocida con el nombre de **sistema de producción**, que está formada por tres componentes principales:

1. *Un conjunto de estados.* Cada **estado** es una situación que puede presentarse dentro del entorno de aplicación concreto en el que nos encontremos. El estado del que se parte se denomina **estado inicial**; el estado (o estados) deseado se denomina **estado objetivo**. En nuestro caso, un estado es una de las posibles configuraciones del puzzle de ocho piezas; el estado inicial es la configuración del puzzle en el momento de entregárselo a la máquina y el estado objetivo es la configuración del puzzle resuelto, tal como se muestra en la Figura 11.1.
2. *Un conjunto de producciones (reglas o movimientos).* Una **producción** es una operación que puede realizarse en el entorno de aplicación con el

fin de pasar de un estado a otro. Cada producción puede estar asociada con una serie de precondiciones; es decir, pueden existir condiciones que deban estar presentes obligatoriamente en el entorno para que una determinada producción pueda aplicarse. (En nuestro caso, las producciones son los movimientos de las piezas. Cada movimiento de una pieza tiene la precondición de que el hueco debe estar situado al lado de la pieza en cuestión.)

3. *Un sistema de control.* El **sistema de control** está compuesto de la lógica que resuelve el problema de pasar del estado inicial al estado objetivo. En cada paso del proceso, el sistema de control debe decidir qué producción aplicar a continuación de entre todas las producciones cuyas precondiciones estén satisfechas. (Dado un estado concreto en nuestro puzzle de ocho piezas, habrá varias piezas colocadas junto al hueco y por tanto, varias producciones aplicables. El sistema de control debe decidir qué pieza mover.)

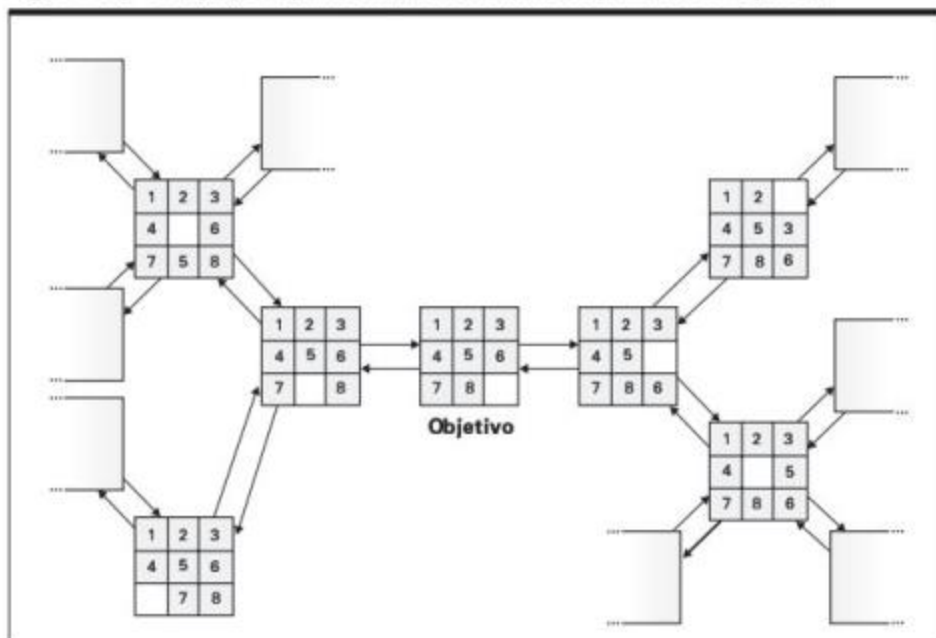
Observe que la tarea asignada a nuestra máquina solucionadora de puzzles puede formularse en el contexto de un sistema de producción. En este tipo de entorno, el sistema de control adopta la forma de un programa. Este programa inspecciona el estado actual del puzzle, identifica una secuencia de producciones que lleve hasta el estado objetivo y ejecuta esa secuencia. Es tarea nuestra, por tanto, diseñar un sistema de control para resolver el puzzle de ocho piezas.

Un concepto importante en el desarrollo de un sistema de control es el de **espacio del problema**, que es el conjunto de todos los estados, producciones y precondiciones existentes en un sistema de producción. Un espacio del problema suele conceptualizarse a menudo en la forma de un **grafo de estados**. Aquí, el término *grafo* hace referencia a una estructura que los matemáticos denominarían **grafo dirigido** y que no es otra cosa que un conjunto de ubicaciones denominadas **nodos** conectados mediante flechas. Un grafo de estados consta de un conjunto de nodos que representan los estados del sistema conectados mediante flechas que representan las producciones que hacen pasar al sistema de un estado a otro. Dos nodos del grafo de estados estarán conectados mediante una flecha si y solo si existe una producción que transforma el sistema desde el estado situado en el origen de la flecha al estado situado en el extremo final de la flecha.

Debemos hacer hincapié en que al igual que el número de estados posibles nos impedía proporcionar de manera explícita soluciones preprogramadas para el puzzle de ocho piezas, ese mismo problema de magnitud nos impide representar explícitamente el grafo de estados completo. Un grafo de estados es por tanto simplemente una forma de conceptualizar el problema al que nos enfrentamos, pero no algo que podamos dibujar en su totalidad. De todos modos, es útil analizar (y posiblemente ampliar) parte del estado de grafos para el puzzle de ocho piezas que se muestra en la Figura 11.4.

Cuando se contempla en términos del grafo de estados, el problema al que se enfrenta el sistema de control se convierte en el de intentar encontrar una secuencia de flechas que conduzca desde el estado inicial al estado objetivo. Después de todo, esta secuencia de flechas representa una secuencia de producciones que resuelve el problema original. Por tanto, independientemente de la aplicación, la tarea del sistema de control puede considerarse como la de

Figura 11.4 Una pequeña parte del grafo de estados del puzzle de ocho piezas.



encontrar una ruta a través del grafo de estados. Esta visión universal de los sistemas de control es el premio que obtenemos al analizar los problemas que requieren razonamiento en términos de los sistemas de producción. Si un problema puede caracterizarse en términos de un sistema de producción, entonces su solución podrá formularse en términos de una búsqueda de una ruta.

Para hacer hincapié en este punto, consideremos cómo podríamos formular otras tareas en términos de sistemas de producción y cómo podríamos, por tanto, llevarlas a cabo en el contexto de sistemas de control que intenten encontrar rutas a través de grafos de estados. Uno de los problemas clásicos en el campo de la inteligencia artificial es el de practicar juegos tales como el ajedrez. Estos juegos tienen una complejidad moderada en un contexto bien definido, por lo que proporcionan un entorno ideal para la comprobación de teorías. En el ajedrez, los estados del sistema de producción subyacente son las posibles configuraciones del tablero; las producciones son los movimientos de las piezas y el sistema de control está integrado en los jugadores (humanos o no). El nodo inicial del grafo de estados representa el tablero con las piezas en sus posiciones iniciales. Ramificándose a partir de este nodo hay una serie de flechas que conducen a aquellas configuraciones del tablero que pueden alcanzarse después del primer movimiento de una partida; ramificándose a partir de esas configuraciones habrá otras flechas a aquellas configuraciones a las que se pueda llegar mediante el siguiente movimiento y así sucesivamente. Con esta formulación, podemos imaginar una partida de ajedrez compuesta por dos jugadores, cada uno de los cuales trata de encontrar una ruta a través de un enorme grafo de estados que les lleve hasta el nodo objetivo que hayan elegido.

Quizá un ejemplo menos obvio de un sistema de producción sea el problema de obtener conclusiones lógicas a partir de una serie de hechos previa-

mente establecidos. Las producciones en este contexto serán las reglas de la lógica, denominadas **reglas de inferencia**, que permiten formar nuevos enunciados a partir de los antiguos. Por ejemplo, los enunciados "Todos los superhéroes son nobles" y "Superman es un superhéroe" pueden combinarse para producir el enunciado "Superman es noble". Los estados en este tipo de sistemas constan de conjuntos de enunciados que se sabe que son ciertos en determinados momentos concretos del proceso de deducción: el estado inicial será el conjunto de enunciados básicos (a menudo denominados axiomas) a partir de los cuales hay que extraer conclusiones y un estado objetivo será cualquier conjunto de enunciados que contenga la conclusión propuesta.

Por ejemplo, la Figura 11.5 muestra la parte de un grafo de estados que podría recorrerse para sacar la conclusión de que "Sócrates es mortal" a partir del conjunto de enunciados "Sócrates es un hombre", "Todos los hombres son humanos" y "Todos los humanos son mortales". Podemos ver en la figura cómo el cuerpo de conocimientos va pasando de un estado a otro, a medida que el proceso de razonamiento aplica las producciones apropiadas con el fin de generar enunciados adicionales.

Hoy día, esos sistemas de razonamiento que a menudo se implementan en lenguajes de programación lógica (Sección 6.7), constituyen la columna vertebral de la mayoría de los **sistemas expertos**, que son paquetes software diseñados para simular el razonamiento de tipo causa-efecto que los expertos humanos seguirían al verse enfrentados a las mismas situaciones. Por ejemplo, los sistemas expertos médicos se utilizan como ayuda para el diagnóstico de enfermedades o el desarrollo de tratamientos.

Figura 11.5 Razonamiento deductivo en el contexto de un sistema de producción.



Árboles de búsqueda

Hemos visto que en el contexto de un sistema de producción, el trabajo del sistema de control consiste en explorar el grafo de estados para encontrar una ruta que nos lleve desde el nodo inicial al nodo objetivo. Un método simple de realizar esta búsqueda consiste en recorrer cada una de las flechas que salen del estado inicial y anotar, en cada caso, el estado de destino. A continuación, se recorren las flechas que salen de esos nuevos estados y se anotan de nuevo esos resultados, y así sucesivamente. La búsqueda de un determinado objetivo se difundirá así a partir del estado inicial, como una gota de tinta en el agua. Este proceso continúa hasta que uno de los nuevos estados sea un estado objetivo, en cuyo punto sabremos que hemos encontrado una solución, por lo que el sistema de control tan solo necesitará encontrar las producciones que llevan desde el estado inicial hasta el estado objetivo a lo largo de la ruta descubierta.

El efecto de esta estrategia es el de construir un árbol, denominado **árbol de búsqueda**, que está compuesto por aquella parte del grafo de estados que ha sido explorada por el sistema de control. El nodo raíz del árbol de búsqueda será el estado inicial y los hijos de cada nodo son aquellos estados que resultan alcanzables a partir del padre aplicando una producción. Cada arco entre distintos nodos de un árbol de búsqueda representará la aplicación de una única producción y cada ruta que va desde la raíz a una hoja representa una ruta entre los correspondientes estados del grafo de estados.

En la Figura 11.7 se ilustra el árbol de búsqueda que se generaría al resolver el puzzle de ocho piezas a partir de la configuración mostrada en la Figura 11.6. La rama situada más a la izquierda de este árbol representa un intento de resolver el problema moviendo primero la pieza número 6 hacia arriba; la rama central representa el intento de solucionar el puzzle moviendo la pieza número 2 hacia la derecha y la rama de la derecha del árbol representa lo que se obtendría al mover la pieza número 5 hacia abajo. Como vemos, el árbol de búsqueda muestra que si comenzamos moviendo la pieza 6 hacia arriba, entonces la única producción permitida a continuación será mover la pieza 8 hacia la derecha. (De hecho, en ese punto también podríamos mover la pieza número 6 hacia abajo, pero eso sería invertir simplemente la producción anterior, volviendo a un estado por el que ya hemos pasado, lo que no es un movimiento muy inteligente.)

El estado objetivo aparece en el último nivel del árbol de búsqueda de la Figura 11.7. Puesto que esto indica que se ha encontrado una solución, el sistema de control puede terminar su procedimiento de búsqueda y comenzar a construir la secuencia de instrucciones que se utilizará para resolver el puzzle dentro del entorno externo. Para ello, basta con seguir el simple proceso de ascender por el árbol de búsqueda desde la posición del nodo objetivo al mismo

Figura 11.6 Un puzzle de ocho piezas sin resolver.

1	3	5
4	2	
7	8	6

tiempo que se insertan las producciones representadas por los arcos del árbol en una pila, a medida que nos las vamos encontrando. Aplicando esta técnica al árbol de búsqueda de la Figura 11.7 se obtiene la pila de producciones mostrada en la Figura 11.8. El sistema de control podrá ahora resolver el puzzle del mundo exterior ejecutando las instrucciones según las va sacando de esta pila.

Hay otra observación que conviene realizar. Recuerde que los árboles de los que hemos hablado en el Capítulo 8 utilizan un sistema de punteros que apunta hacia *abajo* del árbol, permitiéndonos así movernos desde un nodo

Figura 11.7 Un ejemplo de árbol de búsqueda.

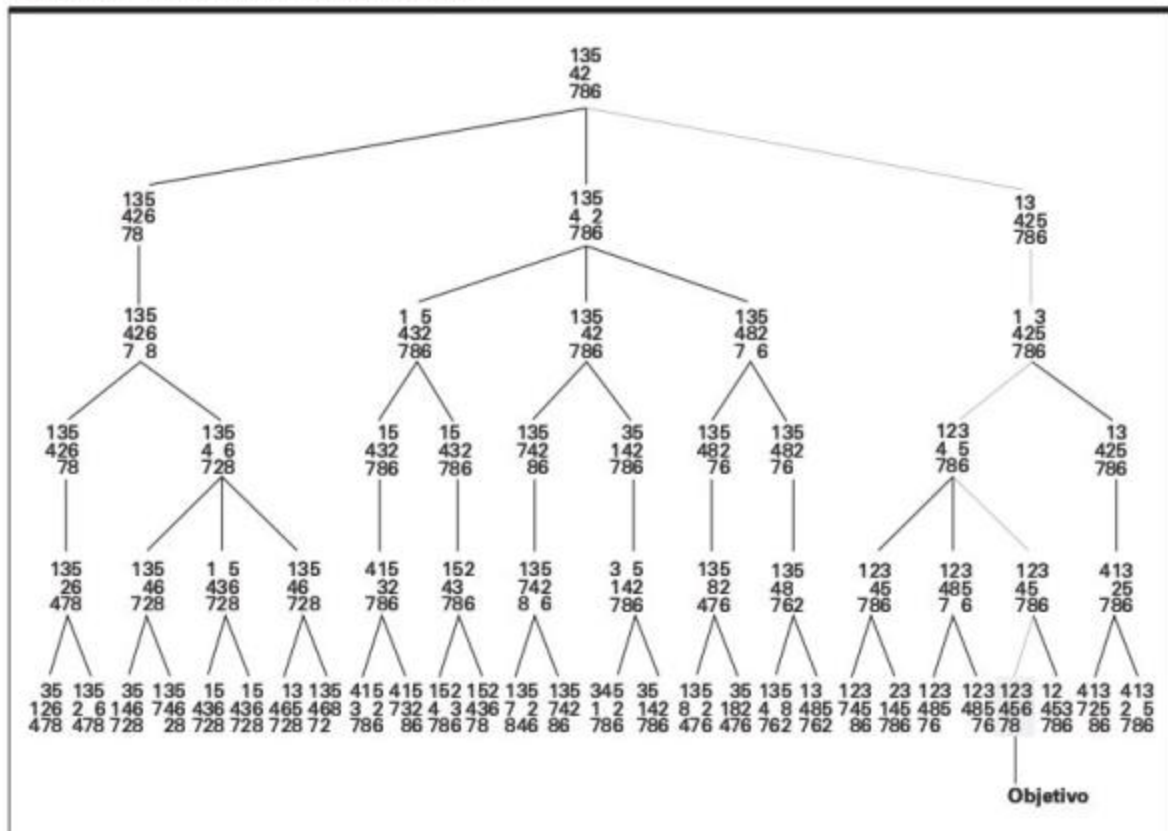
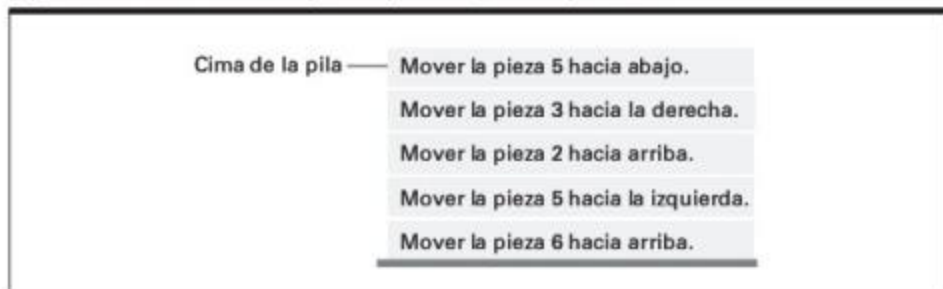


Figura 11.8 Producciones apiladas para su ejecución posterior.



padre a sus hijos. Sin embargo, en el caso de un árbol de búsqueda, el sistema de control debe ser capaz de moverse desde un hijo hacia su padre, a medida que se desplaza hacia *arriba* del árbol, desde el estado objetivo hasta el estado inicial. Esos árboles se construyen con su sistema de punteros apuntando hacia arriba en lugar de hacia abajo. Es decir, cada nodo hijo contiene un puntero a su padre, en lugar de ser los nodos padre los que tienen punteros a los nodos hijo. (En algunas aplicaciones, se utilizan ambos conjuntos de punteros, con el fin de permitir el movimiento dentro del árbol en ambas direcciones.)

Heurística

Para nuestro ejemplo de la Figura 11.7, hemos elegido una configuración inicial que produce un árbol de búsqueda manejable. Pero en realidad, el árbol de búsqueda generado en un intento de resolver un problema más complejo podría crecer hasta alcanzar un tamaño muchísimo mayor. En una partida de ajedrez, hay veinte posibles movimientos para la primera jugada, así que el nodo raíz del árbol de búsqueda tendrá veinte hijos, en lugar de los tres que había en el caso de nuestro ejemplo. Además, una partida de ajedrez puede consistir fácilmente en 30 o 35 parejas de movimientos. Incluso en el caso del puzzle de ocho piezas, el árbol de búsqueda puede llegar a ser muy grande si no se alcanza el objetivo rápidamente. Como resultado, desarrollar un árbol de búsqueda completo puede ser tan imposible como representar el grafo de estados en su totalidad.

Una estrategia para resolver este problema consiste en cambiar el orden en el que se construye el árbol de búsqueda. En lugar de construirlo **en anchura** (lo que quiere decir que se construye nivel a nivel), podemos explorar las rutas más prometedoras con mayor profundidad y considerar las otras opciones solo si la selección original no conduce a ninguna parte. Esto da como resultado una construcción **en profundidad** del árbol de búsqueda, lo que quiere decir que el árbol se genera construyendo rutas verticales en lugar de niveles horizontales. Para ser más precisos, esta solución se denomina a menudo construcción **con prioridad para los mejores**, para reflejar el hecho de que la ruta vertical que se decide explorar es aquella que parece ofrecer el mayor potencial.

La solución de prioridad para los mejores es similar a la estrategia que utilizaríamos los humanos al enfrentarnos al puzzle de ocho piezas. Raramente analizaríamos varias opciones al mismo tiempo, que es lo que se hace en la solución de búsqueda en anchura. En lugar de ello, lo que probablemente haríamos sería seleccionar la opción que nos parezca más prometedora y seguirla en primer lugar. Observe que decimos la solución que *parezca* más prometedora. Rara vez sabremos con seguridad qué opción es la mejor en un punto concreto. Simplemente seguimos nuestra intuición, que por supuesto puede no llevarnos a ninguna parte. Pero aunque en ocasiones no nos lleve a ninguna parte, el uso de esa información intuitiva parece proporcionar a los seres humanos una ventaja con respecto a los métodos de fuerza bruta consistentes en dedicar la misma atención a todas las opciones posibles. Por tanto, parece prudente aplicar esos métodos intuitivos a los sistemas de control automatizados.

Para ello, necesitamos una forma de identificar cuáles son los estados más prometedores de entre un conjunto de estados posibles. Nuestra solución consiste en utilizar un **heurístico**, que en nuestro caso es un valor cuantitativo

asociado con cada estado; con ese valor se intenta medir la "distancia" que hay entre ese estado y el objetivo más próximo. En cierto sentido, nuestro heurístico es una medida del coste previsto. Dada la posibilidad de elegir entre dos estados, aquel que tenga el valor heurístico más bajo será el estado a partir del cual se podrá, aparentemente, alcanzar un objetivo con un menor coste. Este estado representará, por tanto, la dirección que debemos explorar.

Un heurístico debe tener dos características. En primer lugar, debe ser una estimación razonable de la cantidad de trabajo restante en la solución si se llegase a alcanzar el estado asociado. Esto quiere decir que puede proporcionar información significativa a la hora de elegir entre varias opciones: cuanto mejor sea la estimación proporcionada por el heurístico, mejores serán las decisiones que tomemos basándonos en esa información. En segundo lugar, el heurístico debe ser fácil de calcular. Esto quiere decir que su uso debe darnos la posibilidad de beneficiar al proceso de búsqueda, en lugar de entorpecerlo. Si el cálculo del heurístico es extremadamente complicado, entonces a lo mejor tiene más sentido que invirtamos nuestro tiempo realizando una búsqueda en anchura.

Un heurístico simple en el caso del puzzle de ocho piezas consistiría en estimar la "distancia" hasta el objetivo contando el número de piezas que están descolocadas, esta solución se basaría en la conjetura de que un estado en el que haya cuatro piezas descolocadas está más lejos del objetivo (y es por tanto menos prometedor) que un estado en el que solo estén descolocadas dos piezas. Sin embargo, este heurístico no tiene en cuenta lo lejos que las piezas mal colocadas están de su posición correcta. Si las dos piezas del segundo caso están

Inteligencia basada en el comportamiento

Los primeros trabajos en el campo de la inteligencia artificial enfocaban el problema en el contexto de tratar de escribir explícitamente programas que simularan la inteligencia. Sin embargo, muchos expertos argumentan hoy día que la inteligencia del ser humano no está basada en la ejecución de programas complejos, sino en una serie de funciones simples de tipo estímulo-respuesta, que han ido evolucionando a lo largo de las generaciones. Esta teoría de la "inteligencia" se conoce con el nombre de inteligencia basada en el comportamiento, porque las funciones "inteligentes" de tipo estímulo-respuesta parecen ser el resultado de comportamientos que han hecho que ciertos individuos sobrevivan y se reproduzcan, mientras que otros no pudieron hacerlo.

La inteligencia basada en el comportamiento parece responder a diversas cuestiones que se plantean en el campo de la inteligencia artificial, como por ejemplo por qué las máquinas basadas en la arquitectura de von Neumann tienen unas habilidades computacionales muy superiores a las de los humanos pero sin embargo luchan infructuosamente por exhibir sentido común. Por ello, la inteligencia basada en el comportamiento promete ser una de las influencias principales en las investigaciones realizadas dentro del campo de la inteligencia artificial. Como se describe en el texto, las técnicas basadas en el comportamiento se han aplicado en el campo de las redes neuronales artificiales con el fin de enseñar a las neuronas a comportarse de determinadas maneras deseadas; se ha usado también en el campo de los algoritmos genéticos para proporcionar una alternativa al proceso de programación más tradicional y en robótica para mejorar el comportamiento de las máquinas mediante estrategias reactivas.

muy alejadas de sus posiciones correctas, podrían necesitarse muchas producciones para moverlas a través del puzzle.

Por tanto, un heurístico ligeramente mejor sería medir la distancia a la que está cada pieza de su destino, y sumar esos valores para obtener un único valor global. Una pieza que esté inmediatamente adyacente a su destino final se asociaría con una distancia de uno, mientras que una pieza cuya esquina toque el cuadrado de su destino final se asociaría con una distancia de dos (porque debe moverse al menos una posición en sentido vertical y una posición en sentido horizontal). Este heurístico es fácil de calcular y produce una estimación burda del número de movimientos necesario para transformar el puzzle desde su estado actual hasta el objetivo. Por ejemplo, el valor heurístico asociado con la configuración de la Figura 11.9 es siete (porque las piezas 2, 5 y 8 están a una distancia de uno de su destino final, mientras que las piezas 3 y 6 están a una distancia de dos de su posición correcta). De hecho, en realidad hacen falta siete movimientos para transformar esta configuración del puzzle en la configuración correcta.

Ahora que disponemos de un heurístico para el puzzle de ocho piezas, el siguiente paso consiste en incorporarlo a nuestro proceso de toma de decisiones. Recuerde que un ser humano enfrentado a una decisión tiende a seleccionar la opción que parece más próxima al objetivo. Por tanto, nuestro procedimiento de búsqueda debería considerar el heurístico de cada nodo hoja del árbol de búsqueda y continuar la búsqueda de un nodo hoja que tenga asociado el valor más pequeño. Esta estrategia es la adoptada en la Figura 11.10, que presenta un algoritmo para desarrollar un árbol de búsqueda y ejecutar la solución obtenida.

Apliquemos este algoritmo al puzzle de ocho piezas partiendo de la configuración inicial mostrada en la Figura 11.6. En primer lugar, establecemos este estado inicial como el nodo raíz y anotamos su valor heurístico, que es cinco. Después, la primera pasada a través del cuerpo de la sentencia `while` nos dice que añadamos los tres nodos que pueden alcanzarse a partir del estado inicial, como puede verse en la Figura 11.11. Observe que hemos anotado el valor heurístico de cada nodo hoja entre paréntesis debajo del nodo.

El nodo objetivo no ha sido alcanzado por lo que volvemos a pasar por el cuerpo de la sentencia `while`, ampliando esta vez nuestra búsqueda a partir del nodo situado más a la izquierda ("el nodo hoja situado más a la izquierda que tenga el valor heurístico más bajo"). Después de esto, el árbol de búsqueda tendrá la forma mostrada en la Figura 11.12.

Figura 11.9 Un puzzle de ocho piezas sin resolver.

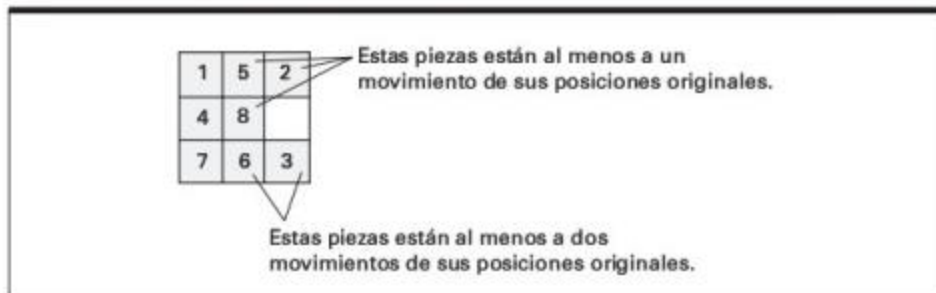


Figura 11.10 Un algoritmo para un sistema de control que utiliza heurísticos.

```

Establecer el nodo inicial del grafo de estados como el nodo raíz
del árbol de búsqueda y anotar su valor heurístico.
while (el nodo objetivo no se haya alcanzado) do
  [Seleccionar el nodo hoja de más a la izquierda que tenga el
   valor heurístico más bajo de todos los nodos hoja.
   Asociar a este nodo seleccionado como hijos aquellos nodos
   que puedan alcanzarse mediante una única producción.
   Anotar el heurístico de cada uno de estos nuevos nodos dentro
   del árbol de búsqueda a continuación del nodo
  ]
  Recorrer el árbol de búsqueda desde el nodo objetivo hasta la raíz,
  introduciendo en una pila la producción asociada con cada
  arco recorrido.
  Resolver el problema original ejecutando las producciones según
  se van sacando de la pila.

```

El valor heurístico del nodo hoja más a la izquierda ahora es cinco, lo que indica que quizá esta rama no sea después de todo una buena opción para continuar el análisis. El algoritmo tiene esto en cuenta y en la siguiente pasada a través de la sentencia **while** decide que hay que ampliar el árbol a partir del nodo situado más a la derecha (que ahora es el "el nodo hoja situado más a la izquierda que tenga el valor heurístico más bajo"). Habiéndolo ampliado de esta forma, el árbol de búsqueda tendrá la apariencia que se muestra en la Figura 11.13.

En este punto, el algoritmo parece estar bien encaminado. Puesto que el valor heurístico de este último nodo es solo tres, la sentencia **while** dice que hay que continuar explorando esta ruta y la búsqueda prosigue hacia el objetivo, generando el árbol de búsqueda que se ilustra en la Figura 11.14. Comparando esto con el árbol de la Figura 11.7 vemos que, incluso con el movimiento en falso que realizó al principio el nuevo algoritmo, el uso de la información heurística ha reducido considerablemente el tamaño del árbol de búsqueda y ha generado un proceso mucho más eficiente.

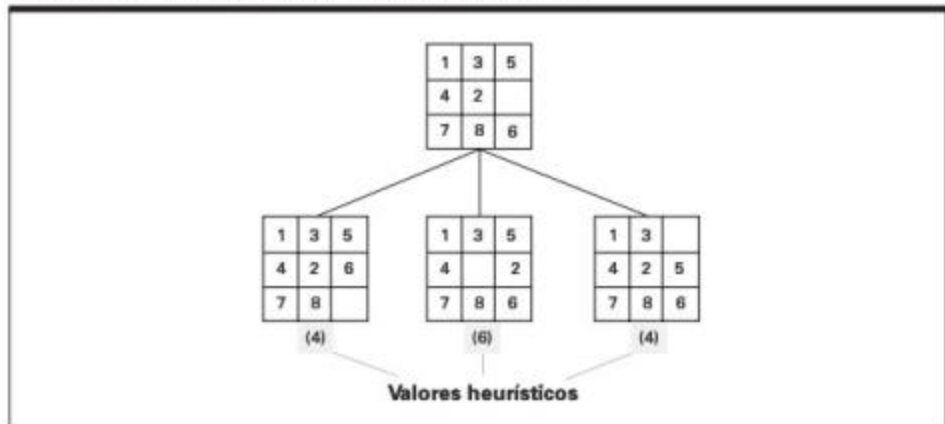
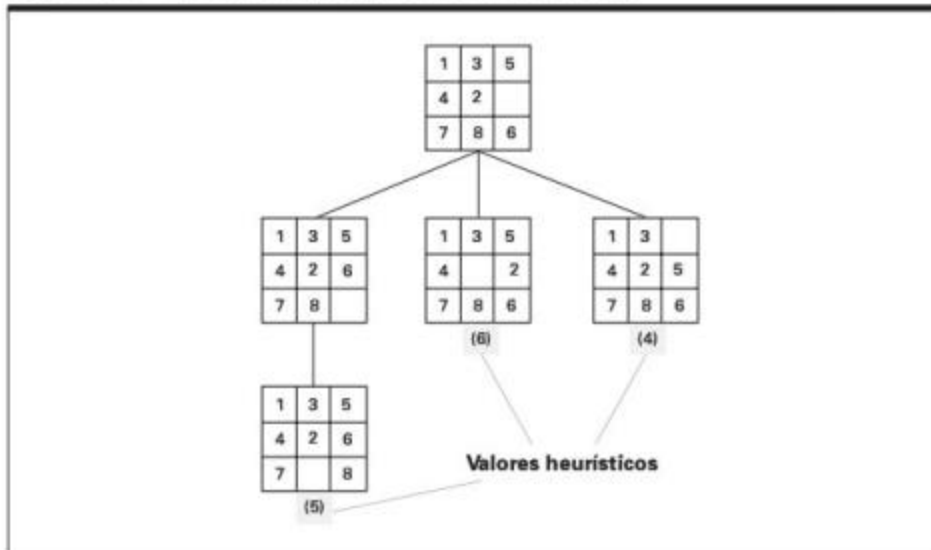
Figura 11.11 Comienzo de nuestra búsqueda heurística.

Figura 11.12 El árbol de búsqueda después de dos pasadas.

Después de alcanzar el estado objetivo, la sentencia `while` termina y el algoritmo continúa recorriendo el árbol desde el nodo objetivo hasta la raíz, insertando en una pila las producciones que se va encontrando. La pila resultante tendrá el aspecto que hemos mostrado anteriormente en la Figura 11.8.

Finalmente, el algoritmo ejecuta esas producciones a medida que las va sacando de la pila. Llegados a este punto, observaríamos cómo la máquina solucionadora de puzzles hace descender su dedo actuador y comienza a mover las piezas.

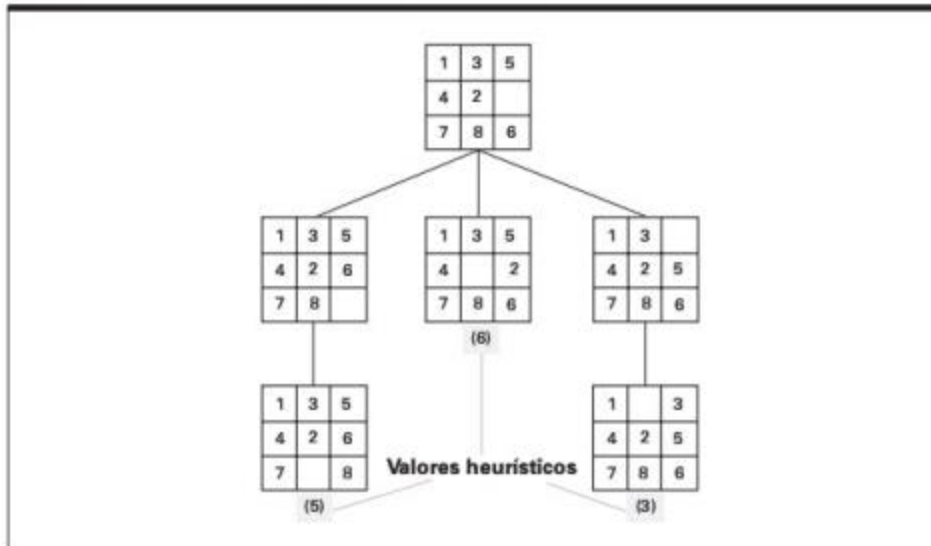
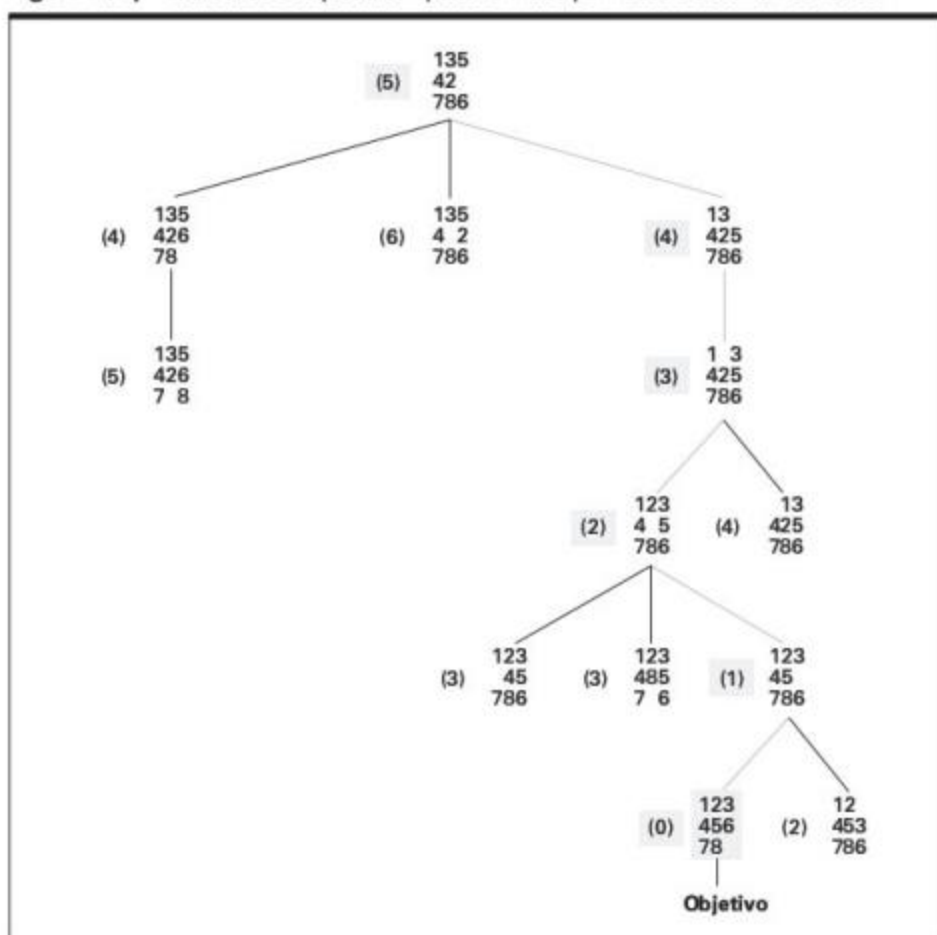
Figura 11.13 El árbol de búsqueda después de tres pasadas.

Figura 11.14 El árbol de búsqueda completo formado por nuestro sistema heurístico.

Conviene hacer un último comentario en relación con la búsqueda heurística. El algoritmo que hemos propuesto en esta sección, que a menudo se denomina algoritmo de búsqueda por elección del mejor (*best-fit search*), no garantiza poder encontrar la mejor solución en todas las aplicaciones. Por ejemplo, a la hora de buscar una ruta hasta una ciudad utilizando un Sistema de posicionamiento global (GPS, *Global Positioning System*) en un automóvil, lo que queríamos es encontrar la ruta más corta y no cualquier ruta. El **algoritmo A*** (léase algoritmo A asterisco) es una versión modificada de nuestro algoritmo de búsqueda por elección del mejor que permite encontrar una solución óptima. La principal diferencia entre los dos algoritmos es que, además de un valor heurístico, el algoritmo A* tiene en consideración el "coste acumulado" en el que se incurre para alcanzar cada nodo hoja, antes de seleccionar el siguiente nodo que hay que expandir (en el caso del GPS de un automóvil, este coste es la distancia recorrida, que el GPS obtiene de su base de datos interna). Por tanto, el algoritmo A* basa sus decisiones en estimaciones del coste requerido para completar cada ruta potencial, en lugar de basarlas tan solo en estimaciones de los restantes costes.

Cuestiones y ejercicios

1. ¿Qué importancia tienen los sistemas de producción en el campo de la inteligencia artificial?
2. Dibuje una parte del grafo de estados del puzzle de ocho piezas que rodee al nodo representado por el siguiente estado:

4	1	3
	2	6
7	5	8

3. Utilizando la solución de búsqueda en anchura, dibuje el árbol de búsqueda que un sistema de control construiría a la hora de resolver el puzzle de ocho piezas correspondiente al siguiente estado inicial:

1	2	3
4	8	5
7	6	

4. Utilice lápiz, papel y la solución de búsqueda en anchura para tratar de construir el árbol de búsqueda que se generará al resolver el puzzle de ocho piezas a partir del siguiente estado inicial. (No es necesario que termine el problema.) ¿Con qué problemas se ha encontrado?

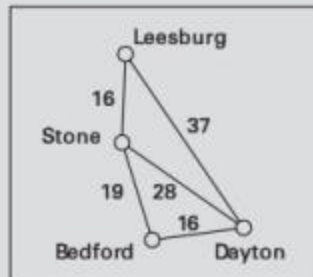
4	3	
2	1	8
7	6	5

5. ¿Qué analogía podría establecerse entre nuestro sistema heurístico para la resolución del puzzle de ocho piezas y un montañero que tratara de alcanzar el pico de una montaña considerando únicamente el terreno local y siguiendo siempre en la dirección con la pendiente más pronunciada?
6. Utilizando el heurístico presentado en esta sección, aplique el algoritmo de búsqueda por elección del mejor de la Figura 11.10 al problema de resolver el siguiente puzzle de ocho piezas:

1	2	3
4		8
7	6	5

7. Refine nuestro método de cálculo del valor heurístico para el estado del puzzle de ocho piezas, de modo que el algoritmo de búsqueda de la Figura 11.10 no tome la decisión incorrecta, como hacía en el ejemplo de esta sección. ¿Puede encontrar un ejemplo en el que su heurístico siga haciendo que la búsqueda recorra un camino equivocado?

8. Utilice el árbol de búsqueda generado mediante el algoritmo de búsqueda por elección del mejor (Figura 11.10) para calcular la ruta entre las ciudades de Leesburg y Bedford. Cada nodo del árbol de búsqueda será una ciudad en el mapa. Comience con un nodo para Leesburg. A la hora de expandir un nodo, añada solamente las ciudades que estén directamente conectadas con la ciudad que esté expandiendo. Anote en cada nodo la distancia en línea recta hasta Bedford y utilice ese valor como valor heurístico. ¿Cuál es la solución encontrada por el algoritmo de búsqueda por elección del mejor? ¿Se corresponde esa solución encontrada con la ruta más corta?



Distancia en línea recta a Bedford desde

Dayton	16
Leesburg	34
Stone	19

9. El algoritmo A* modifica el algoritmo de búsqueda por elección del mejor de dos formas significativas. En primer lugar, anota el coste real necesario para alcanzar cada estado. En el caso de una ruta en un mapa, el coste real es la distancia recorrida. En segundo lugar, a la hora de seleccionar el nodo que hay que expandir, elige el nodo que tenga un valor mínimo de la suma entre el coste real y el valor heurístico. Dibuje el árbol de búsqueda de la Cuestión 8 que resultaría al efectuar estas dos modificaciones. Anote en cada nodo la distancia recorrida hasta la ciudad, el valor heurístico para alcanzar el objetivo y su suma. ¿Cuál es la solución encontrada? ¿Se corresponde la solución encontrada con la ruta más corta?

11.4 Áreas adicionales de investigación

En esta sección, vamos a explorar las cuestiones relacionadas con el manejo del conocimiento, el aprendizaje y la manera de abordar problemas muy complejos, cuestiones todas ellas que continúan planteando desafíos a los investigadores en el campo de la inteligencia artificial. Estas actividades implican capacidades que parecen sencillas para la mente humana, pero que son aparentemente muy complicadas para las máquinas. Por ahora, buena parte del progreso obtenido a la hora de desarrollar agentes "inteligentes" se ha conseguido, básicamente, evitando la confrontación directa con estos problemas, por ejemplo aplicando atajos ingeniosos o limitando el ámbito de aplicación de un problema.

Representación y manipulación del conocimiento

En las explicaciones acerca de la percepción vimos que la comprensión de imágenes requiere una cantidad significativa de conocimientos acerca de los elementos de la imagen y que el significado de una frase puede depender de su

contexto. Estos son ejemplos del papel que desempeña el almacén de conocimientos, que a menudo se denomina **conocimientos del mundo real**, mantenido por la mente humana. De alguna forma, las personas almacenamos cantidades masivas de información y sabemos utilizar esa información con una notable eficiencia. Proporcionar a las máquinas esta capacidad es uno de los desafíos principales en el campo de la inteligencia artificial.

El objetivo subyacente es encontrar formas de representar y almacenar el conocimiento. Esto se ve complicado por el hecho de que, como ya hemos visto, el conocimiento puede presentarse tanto de forma declarativa como procedimental. Por ello, representar el conocimiento no consiste simplemente en la representación de hechos, sino que abarca un espectro mucho más amplio. Es por tanto cuestionable la tesis de que pueda llegar a encontrarse un único esquema para representar todas las formas de conocimiento.

Sin embargo, el problema no consiste simplemente en almacenar y representar conocimiento. También se debe poder acceder fácilmente al conocimiento almacenado y conseguir esta accesibilidad es un auténtico desafío. Las redes semánticas, tal como las hemos presentado en la Sección 11.2, suelen utilizarse como medio de representación y almacenamiento del conocimiento, pero extraer información de ellas puede ser problemático. Por ejemplo, la importancia de la frase "María golpeó a Juan" depende de las edades relativas de María y Juan (¿son esas edades 2 y 30 años, o viceversa?). Esta información estaría almacenada en la red semántica completa sugerida en la Figura 11.3, pero extraer dicha información durante el análisis contextual podría requerir una tarea significativa de búsqueda a través de la red.

Otro problema más que también está relacionado con el acceso al conocimiento es el de identificar aquellos conocimientos que se relacionan implícitamente, en lugar de explícitamente, con la tarea que tengamos entre manos. En lugar de responder a la cuestión "¿Ganó Arturo la carrera?" con un seco "No", lo que queremos es disponer de un sistema que pueda responder diciendo "No, se puso enfermo y no pudo competir". En la siguiente sección exploraremos el concepto de memoria asociativa, que es una de las áreas de investigación que está intentando resolver este problema de la información relacionada. Sin embargo, la tarea no consiste simplemente en extraer información relacionada; necesitamos sistemas que puedan distinguir entre información relacionada e información relevante. Por ejemplo, una respuesta como "No, nació en enero y su hermana se llama Elisa" no se consideraría una respuesta adecuada para la pregunta anterior, aún cuando la información comunicada esté relacionada en cierta manera.

Otra técnica para desarrollar sistemas mejores de extracción del conocimiento consiste en insertar diversos tipos de razonamiento dentro del proceso de extracción, lo que da como resultado una técnica que se denomina **meta-razonamiento**, lo que quiere decir razonamiento acerca del razonamiento. Un ejemplo, originalmente utilizado en el contexto de las búsquedas de bases de datos, consiste en aplicar la **suposición del mundo cerrado**, que es la suposición de que un enunciado es falso a menos que pueda derivarse explícitamente de la información disponible. Por ejemplo, es esa suposición del mundo cerrado la que permite a una base de datos concluir que Juan García no está suscrito a ninguna revista concreta, aún cuando la base de datos no contenga ninguna información en absoluto acerca de Juan. El proceso consiste en observar que

Juan García no se encuentra en la lista de subscriptores y luego aplicar la suposición del mundo cerrado para concluir que Juan García no es un subscriptor.

Si se analiza superficialmente, la suposición del mundo cerrado parece trivial, pero tiene consecuencias que ilustran cómo algunas técnicas de meta-razonamiento aparentemente inocentes pueden tener efectos sutiles indeseados. Por ejemplo, suponga que el único conocimiento que tenemos es el enunciado

Mickey es un ratón OR Donald es un pato.

A partir de este enunciado aislado no podemos concluir que Mickey sea realmente un ratón. Por tanto, la suposición del mundo cerrado nos fuerza a concluir que el enunciado

Mickey es un ratón.

es falso. De manera similar, la suposición del mundo cerrado nos fuerza a concluir que el enunciado

Donald es un pato.

es falso. Por tanto, la suposición del mundo cerrado nos ha conducido a la conclusión contradictoria de que, aunque al menos uno de los enunciados debe ser cierto, ambos son falsos. Entender las consecuencias de estas técnicas de meta-razonamiento de apariencia inocente es un objetivo de las actuales investigaciones tanto en el campo de la inteligencia artificial como en el de las bases de datos, y resalta algunas de las complejidades inherentes al desarrollo de sistemas inteligentes.

Finalmente, existe el problema conocido con el nombre de **problema del marco**, de mantener actualizado el conocimiento almacenado dentro de un entorno cambiante. Si un agente inteligente va a utilizar sus conocimientos con el fin de determinar su comportamiento, entonces esos conocimientos deben estar actualizados. Pero la cantidad de conocimiento requerido para permitir un comportamiento inteligente puede ser enorme y mantener ese conocimiento en un entorno cambiante puede ser una tarea tremendamente compleja. Un factor que viene a complicar aún más el panorama es que los cambios dentro de un entorno suelen alterar indirectamente otros elementos de información y tener en cuenta esas consecuencias indirectas es difícil. Por ejemplo, si un jarrón se cae y se rompe, nuestro conocimiento de la situación ya no contendrá el hecho de que hay agua en el jarrón, aún cuando el vertido del agua solo está relacionado indirectamente con la ruptura del jarrón. Por tanto, resolver el problema del marco no solo requiere la capacidad de almacenar y extraer cantidades masivas de información de una manera eficiente, sino que también exige que el sistema de almacenamiento reaccione apropiadamente a las consecuencias indirectas.

Aprendizaje

Además de representar y manipular el conocimiento, nos gustaría proporcionar a los agentes inteligentes la capacidad de adquirir nuevos conocimientos. Siempre podemos "enseñar" a un agente basado en computadora escribiendo e instalando un nuevo programa o almacenando nuevas informaciones en su conjunto de datos almacenados, pero lo que nos gustaría es que los agentes

inteligentes fueran capaces de aprender por su cuenta. Queremos disponer de agentes que se adapten a entornos cambiantes y que realicen tareas para las que no podemos escribir fácilmente programas de antemano. Un robot diseñado para tareas domésticas se encontrará con nuevos muebles, nuevos electrodomésticos, nuevas mascotas e incluso nuevos propietarios. Un vehículo autónomo que se conduzca solo deberá adaptarse a las variaciones en las líneas que marcan el perfil de las carreteras. Los agentes que forman parte en un juego deben ser capaces de desarrollar y aplicar nuevas estrategias.

Una forma de clasificar las soluciones para el problema de aprendizaje en una computadora es según el nivel de intervención humana requerido. En el primer nivel se encontraría el aprendizaje por **imitación**, en el que una persona demuestra directamente los pasos que componen una tarea (quizá llevando a cabo una secuencia de operaciones en la computadora o guiando físicamente a un robot a través de una secuencia de movimientos) y la computadora se limita a anotar los pasos. Esta forma de aprendizaje se ha utilizado a lo largo de los años en programas de aplicación tales como las hojas de cálculo y los procesadores de texto, en los que pueden guardarse las secuencias de comandos más frecuentemente utilizadas y posteriormente reproducirlas mediante una única orden. Observe que el aprendizaje por imitación asigna muy poca responsabilidad al agente.

En el siguiente nivel se encuentra el aprendizaje mediante **entrenamiento supervisado**. En el entrenamiento supervisado, una persona identifica la respuesta correcta para una serie de ejemplos y luego el agente generaliza a partir de esos ejemplos, con el fin de desarrollar un algoritmo que se pueda aplicar a nuevos casos. La serie de ejemplos se denomina **conjunto de entrenamiento**. Entre las aplicaciones típicas del entrenamiento supervisado se incluyen el aprendizaje para reconocer la escritura manuscrita o la voz de una persona, el aprendizaje para distinguir entre correo electrónico basura y deseado y el aprendizaje para identificar una enfermedad a partir de un conjunto de síntomas.

Un tercer nivel sería el aprendizaje por **refuerzo**. En este tipo de aprendizaje, al agente se le proporciona una regla general para que juzgue por sí mismo si ha tenido éxito o ha fallado a la hora de realizar una tarea, durante un proceso de prueba y error. El aprendizaje por refuerzo es adecuado para aprender a jugar al ajedrez o las damas, donde el éxito o el fracaso son fáciles de definir. A diferencia del entrenamiento supervisado, el aprendizaje por refuerzo permite al agente actuar autónomamente a medida que aprende a mejorar su comportamiento a lo largo del tiempo.

El aprendizaje continúa siendo un campo de investigación lleno de desafíos, ya que no se ha encontrado ningún principio general de carácter universal que cubra todas las posibles actividades de aprendizaje. Sin embargo, existen numerosos ejemplos de los progresos realizados. Uno de ellos es ALVINN (*Autonomous Land Vehicle in a Neural Net*, Vehículo terrestre autónomo en una red neuronal), un sistema desarrollado en la Universidad Carnegie Mellon para aprender a conducir un camión con una computadora de abordo utilizando una videocámara como entrada. La técnica empleada era el entrenamiento supervisado. ALVINN recopilaba datos de un conductor humano y utilizaba esos datos para ajustar sus propias decisiones de conducción. A medida que aprendía, predecía qué acciones realizar, comprobaba su predic-

El conocimiento en la programación lógica

Una preocupación importante a la hora de representar y almacenar conocimiento es que hay que llevar a cabo esa tarea de una forma que sea compatible con el sistema que luego vaya a acceder a ese conocimiento. Es por esta razón por lo que la programación lógica (véase la Sección 6.7) suele resultar ventajosa. En ese tipo de sistemas, el conocimiento está representado por enunciados “lógicos” como

Dumbo es un elefante.

y

X es un elefante implica que X es gris.

Dichos enunciados pueden representarse mediante sistemas de notación que están muy adaptados a la aplicación de reglas de inferencia. A su vez, las secuencias de razonamiento deductivo, como la que vimos en la Figura 11.5, pueden implementarse de manera muy sencilla. Por tanto, en la programación lógica, la representación y el almacenamiento de conocimiento están bien integrados con el proceso de extracción y aplicación del conocimiento. Podríamos decir que los sistemas de programación lógica proporcionan una interfaz “adaptada” entre el conocimiento almacenado y su aplicación.

ción con los datos obtenidos del conductor humano y luego modificaba sus parámetros para aproximarse más a las decisiones de conducción tomadas por esa persona. ALVINN tuvo tanto éxito que podía conducir el camión a cien kilómetros por hora, lo que condujo a investigaciones adicionales que han terminado produciendo sistemas de control que han sido capaces de conducir en una autopista llena de tráfico a las velocidades apropiadas.

Por último, es necesario realizar algunos comentarios acerca de un fenómeno estrechamente relacionado con el aprendizaje: nos referimos al descubrimiento. La distinción entre estos dos conceptos es que el aprendizaje está “basado en objetivos”, mientras que el descubrimiento no lo está. El término *descubrimiento* tiene una connotación de algo inesperado que no está presente en el concepto de aprendizaje. Podemos disponernos a aprender un idioma extranjero o a conducir un vehículo, pero podemos descubrir que esas tareas son más difíciles de lo esperado. Un explorador puede descubrir un gran lago, aún cuando el objetivo fuera simplemente aprender qué es lo que había en esa región.

Desarrollar agentes con la capacidad de realizar descubrimiento de manera eficiente requiere que el agente sea capaz de identificar “cadenas de pensamientos” potencialmente fructíferas. Aquí, la habilidad del descubrimiento depende enormemente de la capacidad de razonar y del uso de heurísticos. Además, muchas aplicaciones potenciales de la capacidad de descubrimiento requieren que un agente sea capaz de distinguir resultados significativos de otros que sean insignificantes. Un agente de minería de datos, por ejemplo, no debe informar de toda relación trivial que encuentre.

Como ejemplos de éxito a la hora de desarrollar sistemas de descubrimiento basados en computadora podemos citar a Bacon, denominado así en honor del filósofo Sir Francis Bacon, que ha descubierto (o quizá deberíamos decir “redescubierto”) la ley de Ohm de la electricidad, la tercera ley de Kepler

del movimiento de los planetas y el principio de conservación de la cantidad de movimiento. Quizá más persuasivo sea el sistema AUTOCLASS que, utilizando datos espectrales infrarrojos ha descubierto nuevas clases de estrellas que antes eran desconocidas en el campo de la astronomía, lo que constituye un verdadero descubrimiento científico efectuado por una computadora.

Algoritmos genéticos

El algoritmo A* (presentado en la sección anterior) permite encontrar la solución óptima a muchos problemas de búsqueda; sin embargo, existen muchos problemas que son demasiado complejos para poderlos resolver mediante esas técnicas de búsqueda (la ejecución sobrepasa la memoria disponible o no puede completarse dentro de un periodo de tiempo razonable). Para este tipo de problemas, en ocasiones puede descubrirse una solución mediante un proceso evolutivo que implica múltiples generaciones de soluciones de prueba. Esta estrategia es la base de lo que se denomina **algoritmos genéticos**. En esencia, los algoritmos genéticos descubren una solución combinando un comportamiento aleatorio con una simulación de la teoría reproductiva y del proceso evolutivo de la selección natural.

Un algoritmo genético comienza generando un conjunto aleatorio de soluciones de prueba. Cada solución se selecciona al azar. (En el caso del puzzle de ocho piezas, una solución de prueba podría ser una secuencia aleatoria del movimiento de las piezas.) Cada solución de prueba se denomina **cromosoma** y cada componente del cromosoma se llama **gen** (en el caso del puzzle de ocho piezas un gen sería un único movimiento de una pieza).

Puesto que los cromosomas iniciales se han seleccionado aleatoriamente, es bastante poco probable que ninguno de ellos represente una solución al problema que intentamos resolver. Por tanto, el algoritmo genético pasa a generar un nuevo conjunto de cromosomas, en el que cada cromosoma es un descendiente (hijo) de dos cromosomas (padres) del conjunto anterior. Los padres se seleccionan aleatoriamente dentro del conjunto, proporcionándose una preferencia probabilística a aquellos cromosomas que parezcan proporcionar las mejores oportunidades de conducir a una solución, con lo que se simula el principio evolutivo de la supervivencia del más adaptado. (Determinar qué cromosomas son los mejores candidatos para reproducirse es quizá el paso más problemático dentro del proceso de un algoritmo genético.) Cada hijo se forma mediante la combinación aleatoria de los genes de los padres. Además, un hijo puede ocasionalmente sufrir una mutación de carácter aleatorio (por ejemplo, intercambiar dos movimientos). La esperanza es que al repetir este proceso una y otra vez vayan evolucionando soluciones de prueba cada vez mejores, hasta descubrir una que sea muy buena, si es que no es la mejor posible. Lamentablemente, no existe ninguna garantía de que el algoritmo genético termine por encontrar una solución, a pesar de lo cual las investigaciones han demostrado que los algoritmos genéticos pueden resultar efectivos a la hora de resolver un rango sorprendentemente amplio de problemas complejos.

Cuando se aplica a la tarea del desarrollo de programas, la solución mediante algoritmo genético se conoce con el nombre de **programación evolutiva**. En este caso, el objetivo es desarrollar programas permitiendo que estos evolucionen, en lugar de escribirlos explícitamente. Los investigadores han

aplicado técnicas de programación evolutiva al proceso de desarrollo de programas utilizando lenguajes de programación funcional. La solución adoptada consiste en comenzar con un conjunto de programas que contengan una amplia variedad de funciones. Las funciones de este conjunto inicial forman el "conjunto de genes" a partir de los cuales se construirán las generaciones futuras de programas. Después, se permite que el proceso evolutivo funcione durante muchas generaciones, esperando que al producir cada generación a partir de los programas que mejor se han comportado en la generación anterior, irá evolucionando hacia una solución para nuestro problema objetivo.

Cuestiones y ejercicios

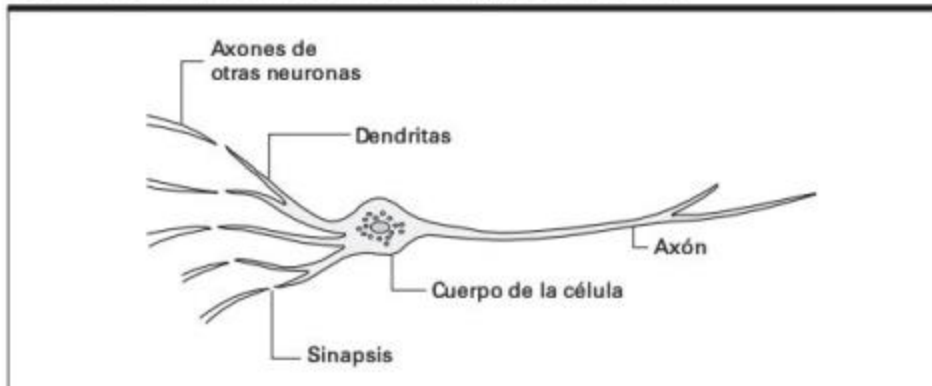
1. ¿Qué queremos decir con la frase *conocimiento del mundo real* y cuál es su importancia en el campo de la inteligencia artificial?
2. Una base de datos de subscriptores a revistas suele contener una lista de los subscriptores de cada revista pero no contiene una lista de personas que no están suscritas. ¿Cómo puede entonces una de esas bases de datos determinar que una persona no está suscrita a una determinada revista?
3. Resuma el denominado problema del marco.
4. Identifique tres formas de entrenar a una computadora. ¿Cuál de ellas no requiere una intervención humana directa?
5. ¿En qué difieren las técnicas evolutivas de las técnicas más tradicionales de resolución de problemas?

11.5 Redes neuronales artificiales

A pesar de todos los progresos realizados en el campo de la inteligencia artificial, muchos problemas continúan desafiando las habilidades de las computadoras que utilizan soluciones algorítmicas tradicionales. Las secuencias de instrucciones no parecen ser capaces de percibir y razonar a niveles comparables con los de la mente humana. Por esta razón, muchos investigadores están volviendo su atención hacia soluciones que tratan de aprovechar fenómenos observados en la naturaleza. Una de esas técnicas son los algoritmos genéticos presentados en la sección anterior y otra es la utilización de redes neuronales artificiales.

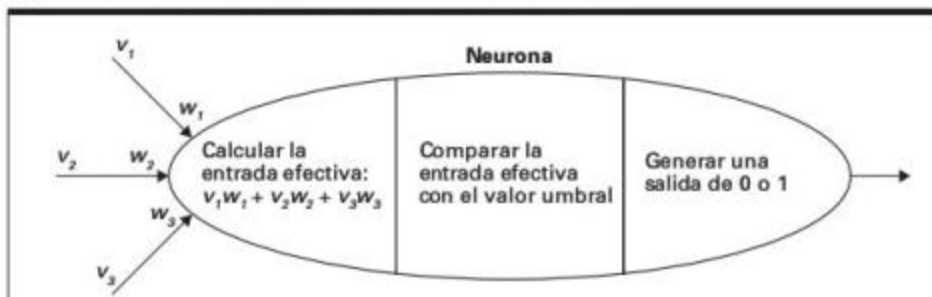
Propiedades básicas

Las redes neuronales artificiales proporcionan un modelo de procesamiento por computadora que simula las redes de neuronas de los sistemas biológicos de los seres vivos. Una neurona biológica es una única celda con una serie de tentáculos de entrada denominadas dendritas y un tentáculo de salida denominado axón (Figura 11.15). Las señales transmitidas a través del axón de una célula reflejan si la célula se encuentra en un estado inhibido o excitado. Este estado está determinado por la combinación de las señales recibidas a través de

Figura 11.15 Una neurona de un sistema biológico de un ser vivo.

las dendritas de la célula. Estas dendritas captan señales procedentes de los axones de otras células a través de pequeños huecos denominados sinapsis. Las investigaciones sugieren que la conductividad a través de una sinapsis está controlada por la composición química de la sinapsis. Es decir, el que la señal de entrada concreta tenga un efecto excitador o inhibidor sobre la neurona está determinado por la composición química de la sinapsis. Así, se cree que una red neuronal biológica aprende ajustando estas conexiones químicas entre neuronas.

Una neurona en una red neuronal artificial es una unidad software que imita este modelo básico de una neurona biológica. Genera una salida de 1 o 0, dependiendo de si su entrada efectiva excede un cierto valor, que se denomina valor **umbral** de la neurona. Esta entrada efectiva es una suma ponderada de las entradas reales, tal como se representa en la Figura 11.16. En esta figura, la neurona se presenta mediante un óvalo y las conexiones entre neuronas se representan mediante flechas. Los valores obtenidos de los axones de otras neuronas (designados mediante v_1 , v_2 , y v_3) se utilizan como entradas a la neurona mostrada. Además de estos valores, cada conexión tiene asociado un **peso** (designado mediante w_1 , w_2 y w_3). La neurona que recibe estos valores de entrada multiplica cada uno de ellos por el peso asociado con esa conexión y luego suma dichos productos para obtener la entrada efectiva ($v_1w_1 + v_2w_2 + v_3w_3$). Si esta suma excede del valor umbral de la neurona, esta genera una salida igual a 1 (simulando un estado excitado); en caso contrario, la neurona genera un 0 como salida (simulando un estado inhibido).

Figura 11.16 Las actividades en el interior de una neurona.

De acuerdo con lo que se muestra en la Figura 11.16, adoptaremos el convenio de representar las neuronas como círculos. Allí donde se conecte cada entrada con una neurona anotaremos el peso asociado con dicha entrada. Finalmente, escribiremos el valor umbral de la neurona dentro del propio círculo. Por ejemplo, la Figura 11.17 representa una neurona con un valor umbral de 1,5 y con pesos de -2, 3 y -1 asociados con cada una de sus conexiones de entrada. Por tanto, si la neurona recibe las entradas 1, 1 y 0, su entrada efectiva será $(1)(-2) + (1)(3) + (0)(-1) = 1$, y su salida será igual a 0. Pero si la neurona recibe las entradas 0, 1 y 1, su entrada efectiva será $(0)(-2) + (1)(3) + (1)(-1) = 2$, que excede del valor umbral, por lo que la salida de la neurona pasará a ser 1.

El hecho de que un peso pueda ser positivo o negativo significa que la entrada correspondiente puede tener un efecto inhibitorio o excitador sobre la neurona receptora. (Si el peso es negativo, entonces un 1 en esa posición de entrada reducirá la suma ponderada y tenderá por tanto a mantener la entrada efectiva por debajo del valor umbral. Por el contrario, un peso positivo hace que la entrada asociada tenga el efecto de incrementar la suma ponderada y aumentar, por tanto, la posibilidad de que dicha suma exceda del valor umbral.) Además, la magnitud real del peso controla el grado con el que la entrada correspondiente puede inhibir o excitar a la neurona receptora. En consecuencia, ajustando los valores de los pesos a través de una red neuronal artificial, podemos programar la red para que responda a diferentes entradas de una manera predeterminada.

Las redes neuronales artificiales suelen disponerse en una topología compuesta por varias capas. Las neuronas de entrada se encuentran en la primera capa y las neuronas de salida en la última. Pueden incluirse capas adicionales de neuronas (denominadas capas ocultas) entre las capas de entrada y de salida. Cada neurona de una capa está interconectada con todas las neuronas de la siguiente capa. Por ejemplo, la red simple mostrada en la Figura 11.18a está programada para generar una salida de 1 si sus dos entradas difieren y una salida igual a 0 en caso contrario. Sin embargo, si cambiamos los pesos y los sustituimos por los que se indican en la Figura 11.18b, obtenemos una red que responde con un 1 si ambas entradas son iguales a 1 y con un 0 en caso contrario.

Es preciso observar que la configuración de red de la Figura 11.18 es mucho más simple que la de una red biológica real. El cerebro humano contiene aproximadamente 10^{11} neuronas con unas 10^4 sinapsis por neurona. De hecho, las dendritas de una neurona biológica son tan numerosas que se parecen más a una malla fibrosa que a los tentáculos individuales representados en la Figura 11.15.

Figura 11.17 Representación de una neurona.

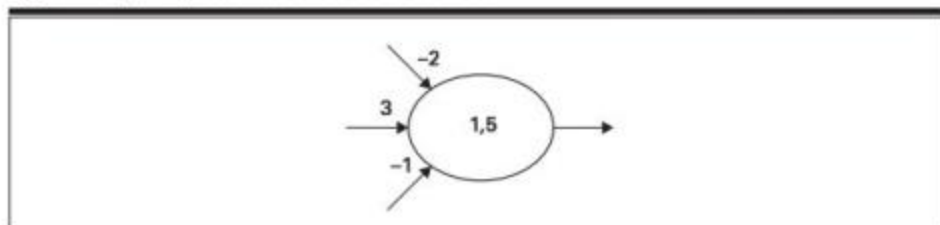
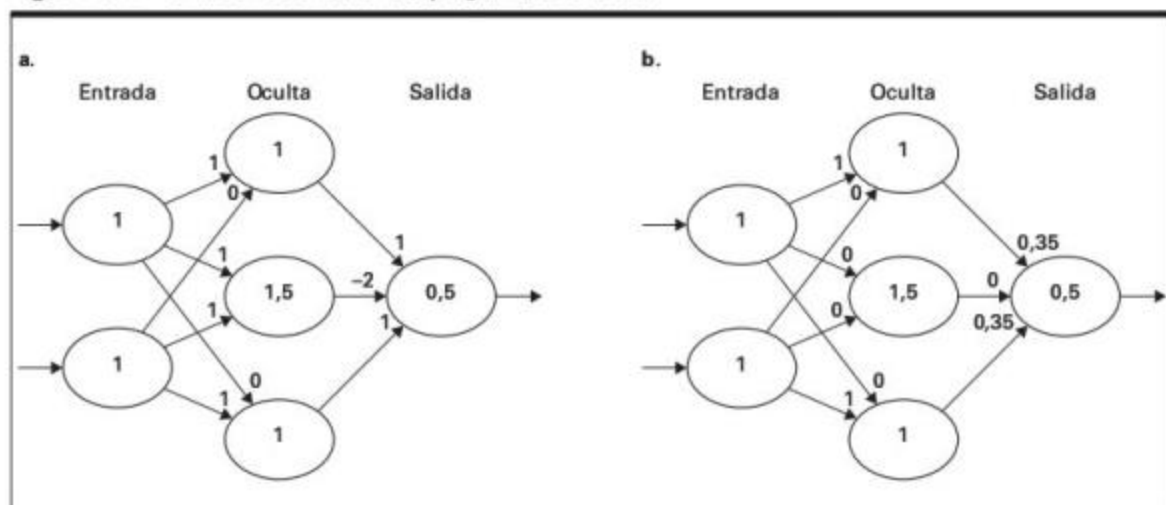


Figura 11.18 Una red neuronal con dos programas diferentes.

Entrenamiento con las redes neuronales artificiales

Una característica importante de las redes neuronales artificiales es que no se programan en el sentido tradicional, sino que en lugar de ello se las entrena. Es decir, el programador no determina los valores de los pesos necesarios para resolver un problema concreto y luego "inserta" esos valores en la red. En lugar de ello, una red neuronal artificial aprende cuáles son los valores apropiados de los pesos mediante un entrenamiento supervisado (Sección 11.4) que implica un proceso repetitivo en el que se aplican las entradas del conjunto de prueba a la red y luego los pesos se ajustan en incrementos pequeños, para que el comportamiento de la red se vaya aproximando al comportamiento deseado.

Es interesante observar que las técnicas de algoritmos genéricos se han aplicado a la tarea de entrenar redes neuronales artificiales. En particular, para entrenar una red neuronal, se puede generar un serie aleatoria de conjuntos de pesos para la red (cada conjunto actuará como un cromosoma del algoritmo genético). Después, en un proceso paso a paso, pueden asignarse a la red los pesos representados por cada cromosoma y probar la red con diversas entradas. Entonces puede asignarse a los cromosomas que generen el menor número de errores durante este proceso de prueba una mayor probabilidad de ser seleccionados como padres para la siguiente generación. En numerosos experimentos, esta técnica ha terminado por conducir a la obtención de un conjunto de pesos adecuado.

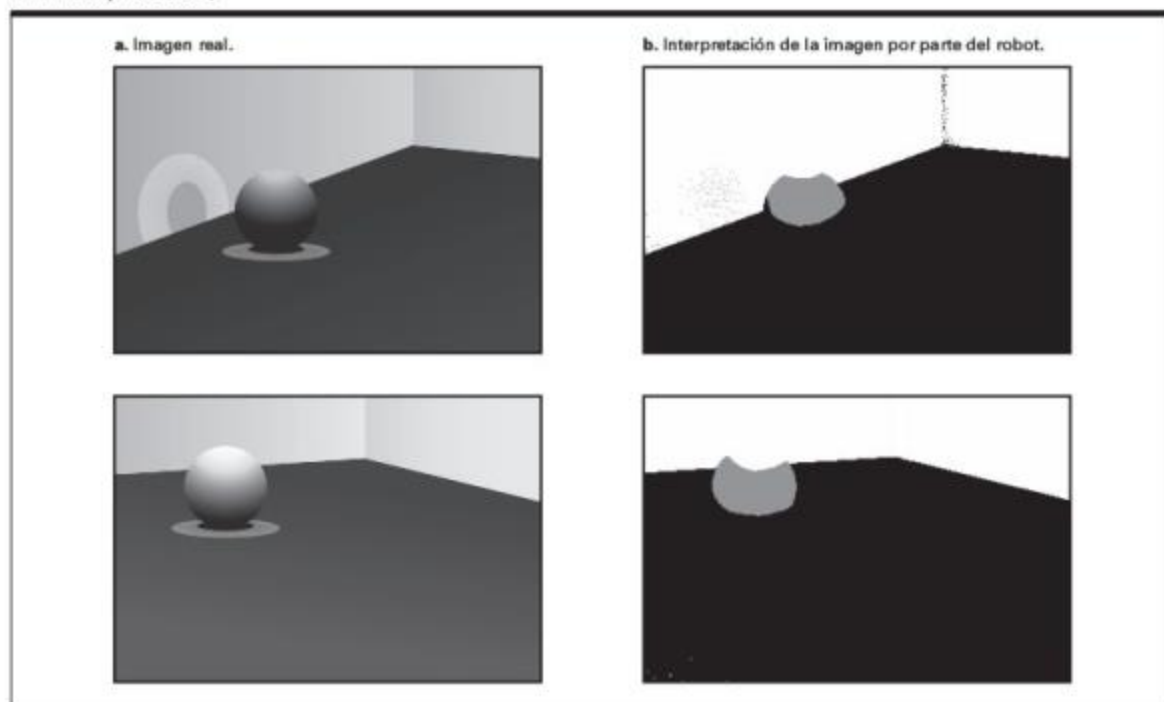
Consideremos un ejemplo en el que entrenar a una red neuronal artificial para resolver un problema ha tenido éxito y ha sido, quizá, más productivo que tratar de proporcionar una solución por medio de técnicas de programación tradicional. El problema es uno con el que un robot podría encontrarse al tratar de comprender su entorno gracias a la información recibida a través de una videocámara. Por ejemplo, suponga que el robot debe distinguir entre las paredes de una habitación, que son de color blanco, y el suelo, que es negro. A primera vista, parece que esto debería ser una tarea sencilla: bastaría con clasificar los píxeles blancos como parte de una pared y los píxeles negros como parte del

suelo. Sin embargo, a medida que el robot mira en diferentes direcciones o se mueve por la habitación, las distintas condiciones de iluminación pueden hacer que la pared parezca gris en algunos casos, mientras que en otras ocasiones es el suelo lo que puede parecer gris. Por tanto, el robot necesita aprender a distinguir las paredes y el suelo en una amplia variedad de condiciones de iluminación.

Para resolver esto, podríamos construir una red neuronal artificial cuyas entradas estuvieran compuestas por valores que indiquen las características de color de cada píxel individual de la imagen, así como por un valor que indique el brillo global de la imagen completa. Después, podríamos entrenar a la red proporcionándole numerosos ejemplos de píxeles que representen partes de las paredes y del suelo bajo distintas condiciones de iluminación.

Los resultados de entrenar a una red neuronal artificial con estas técnicas se ilustran en la Figura 11.19. La primera columna representa las imágenes originales, la siguiente muestra la interpretación realizada por el robot. Observe que, aunque las paredes en la imagen original de la parte superior son más bien oscuras, el robot ha identificado correctamente la mayoría de los píxeles asociados como píxeles blancos de una pared, sin por ello dejar de identificar correctamente el suelo presente en la imagen inferior (la bola presente en las imágenes era parte de un experimento más amplio). Observará también que el sistema de procesamiento de imágenes del robot no es perfecto. La red neuronal ha identificado incorrectamente algunos píxeles de la pared como píxeles del suelo (y algunos de los píxeles del suelo como píxeles de una pared). Estos son ejemplos de realidades que hay que tener en cuenta durante la aplicación

Figura 11.19 Resultados de utilizar una red neuronal para clasificar los píxeles de una imagen (inspirado en www.actapress.com)

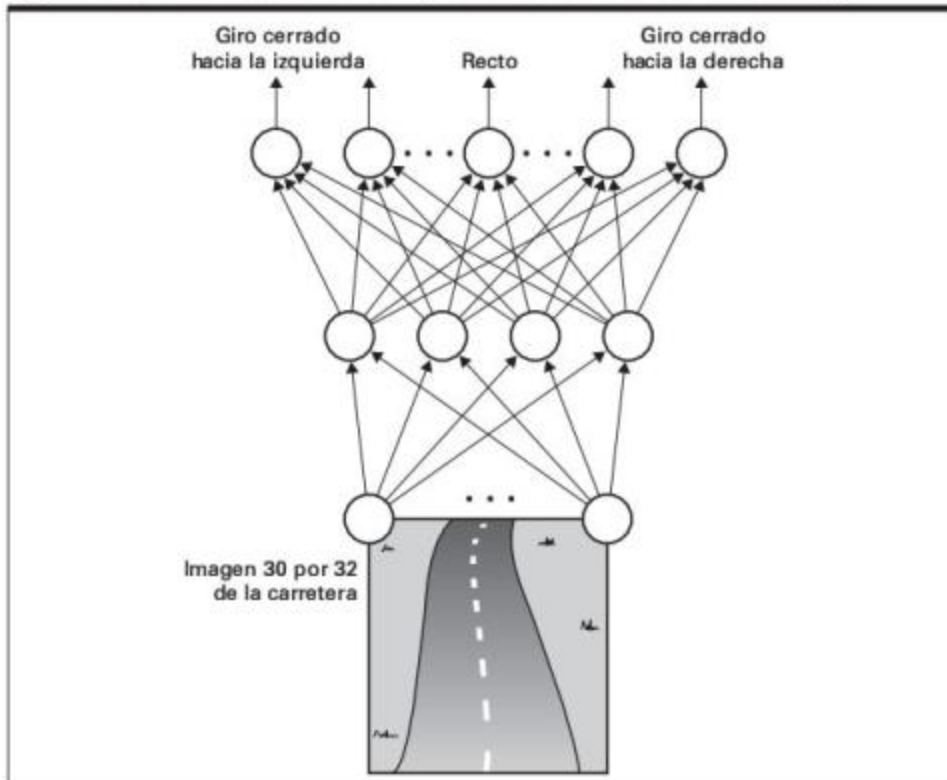


de una teoría. En este caso, los errores pueden corregirse programando al robot para que ignore los píxeles de suelo individuales que parezcan rodeados por una multitud de píxeles de pared (y viceversa).

Además de para problemas simples de aprendizaje (como el de clasificación de los píxeles), las redes neuronales artificiales se han estado utilizando para aprender comportamientos inteligentes sofisticados como atestigüa el proyecto ALVINN que hemos citado en la sección anterior. De hecho, ALVINN era una red neuronal artificial, cuya composición era tremendamente simple (Figura 11.20). Su entrada se obtenía a partir de una matriz de 30 por 32 sensores, cada uno de los cuales observaba una parte diferente de la imagen de vídeo de la carretera situada delante del vehículo e informaba de sus descubrimientos a cuatro neuronas situadas en una capa oculta (por tanto, cada una de estas cuatro neuronas tenía 960 entradas). La salida de cada una de estas cuatro neuronas estaba conectada a cada una de las 30 neuronas de salida, cuyas salidas indicaban la dirección en la que girar. La presencia de neuronas excitadas en uno de los extremos de la fila de 30 neuronas indicaba un giro cerrado hacia la izquierda, mientras que la presencia de neuronas excitadas en el otro extremo indicaba un giro cerrado hacia la derecha.

ALVINN era entrenado "observando" a un ser humano conducir, mientras que tomaba sus propias decisiones sobre hacia dónde girar, comparaba esas decisiones con las de la persona y realizaba pequeñas modificaciones en los pesos para hacer que sus decisiones se parecieran más a las que la persona

Figura 11.20 La estructura de ALVINN (Autonomous Land Vehicle in a Neural Net).



tomaba. Sin embargo, existe una cuestión colateral interesante. Aunque ALVINN aprendió a conducir mediante esta técnica simple, lo que no aprendió fue a recuperarse de los errores. Por tanto, los datos recopilados a partir de la observación del conductor humano se enriquecieron artificialmente con el fin de incluir también las situaciones en que hubiera que recuperarse de un error. Una solución para este entrenamiento de cara a la recuperación de errores que se adoptó inicialmente fue que el conductor humano efectuara virajes bruscos con el vehículo, para que ALVINN pudiera ver cómo recuperaba luego el control y aprendiera así a recuperar el control por su cuenta. Pero a menos que se desactivara a ALVINN mientras que el ser humano realizaba el giro brusco inicial, lo que ALVINN hacía era aprender también a realizar giros bruscos, además de a recuperar luego el control, lo que obviamente era una consecuencia indeseable.

Memoria asociativa

La mente humana tiene la increíble habilidad de poder extraer información que esté asociada con el tema que actualmente estemos considerando. Cuando percibimos ciertos olores, podemos fácilmente recordar cosas de nuestra infancia. El sonido de la voz de un amigo nos puede traer fácilmente a la mente la imagen de esa persona o quizá el recuerdo de algunos momentos agradables pasados en su compañía. Cierta música puede generar pensamientos relacionados con unas vacaciones concretas. Todos estos son ejemplos de **memoria asociativa**, la extracción de información que está asociada o relacionada con la información que en ese momento tenemos entre manos.

Construir máquinas con memoria asociativa ha sido un objetivo de investigación durante muchos años. Una aproximación consiste en aplicar técnicas de redes neuronales artificiales. Por ejemplo, considere una red compuesta por muchas neuronas que estén interconectadas para formar una especie de tela de araña sin entradas ni salidas. (En algunos diseños denominados redes de Hopfield, la salida de cada neurona se conecta como entrada a cada una de las otras neuronas; en otros casos, la salida de una neurona puede estar solo conectada a sus vecinas inmediatas.) En ese tipo de sistema, las neuronas excitadas tenderán a excitar a otras neuronas, mientras que las neuronas inhibidas tenderán a inhibir a otras. A su vez, el sistema completo puede encontrarse en un estado constante de cambio, o puede que termine por encontrar su camino hacia una configuración estable, en la que las neuronas excitadas permanecerán excitadas y las neuronas inhibidas permanecerán inhibidas. Si iniciamos la red en una configuración no estable que esté próxima a otra estable, cabría esperar que la red fuera derivando hacia esa configuración estable. En cierto sentido, cuando se le proporciona una parte de una configuración estable, la red puede ser capaz de completar esa configuración.

Ahora suponga que representamos un estado excitado mediante un 1 y un estado inhibido mediante un 0, de modo que la condición de toda la red en cualquier momento pueda representarse mediante una configuración de 0s y 1s. Entonces, si configuramos la red con un patrón de bits que esté próximo a un patrón estable cabría esperar que la red derivara hacia ese patrón estable. En otras palabras, la red podría encontrar el patrón estable de bits que esté próximo a ese patrón que se le ha proporcionado. De este modo, si algunos de los

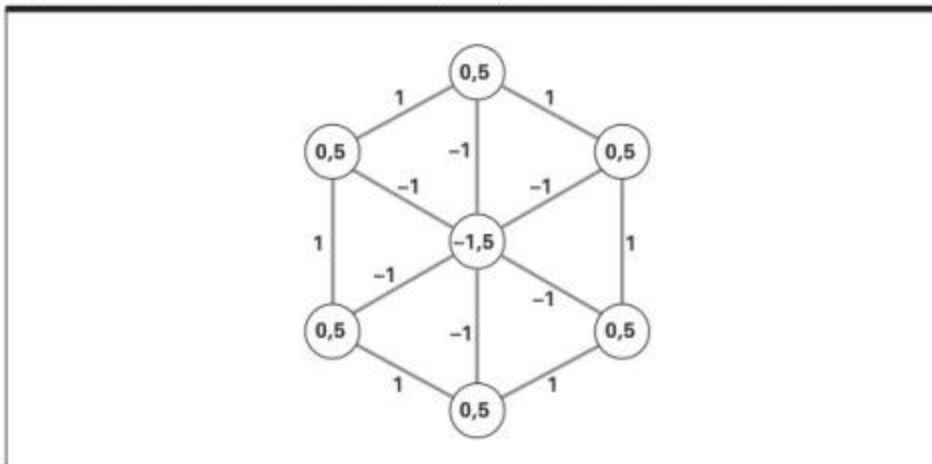
bits se emplean para codificar olores y otros se usan para codificar recuerdos de la infancia, entonces la inicialización de los bits correspondientes a los olores de acuerdo con una cierta configuración estable podría provocar que los bits restantes encontraran su camino hacia los recuerdos de infancia asociados.

Considere ahora la red neuronal artificial mostrada en la Figura 11.21. Siguiendo el convenio utilizado para dibujar redes neuronales artificiales, cada uno de los círculos de la figura representa una neurona cuyo valor umbral está anotado dentro del círculo. En lugar de flechas, las líneas que conectan los círculos representan conexiones bidireccionales entre las correspondientes neuronas. Es decir, si una línea conecta dos neuronas, eso indica que la salida de cada neurona está conectada como entrada de la otra. Por tanto, la salida de la neurona central está conectada como entrada a cada una de las neuronas situadas alrededor del perímetro, y la salida de cada una de las neuronas del perímetro está conectada como entrada a la neurona central además de estar conectada como entrada también a cada una de sus vecinas inmediatas del perímetro. Dos neuronas conectadas asociarán el mismo peso con la salida de la otra neurona. Este peso común se anota junto a la línea que conecta a ambas neuronas. Por tanto, la neurona situada en la parte superior del diagrama asocia un peso de -1 con la entrada que recibe de la neurona central y un peso de 1 con las entradas que recibe de sus dos vecinas del perímetro. De la misma forma, la neurona central asocia un peso de -1 con cada uno de los valores que recibe de cada una de las neuronas situadas alrededor del perímetro.

La red opera en pasos discretos en los que todas las neuronas responden a sus entradas de manera sincronizada. Para determinar la siguiente configuración de la red a partir de su configuración actual, calculamos las entradas efectivas de cada neurona de la red y luego permitimos que todas las neuronas respondan a sus entradas simultáneamente. El efecto es que toda la red sigue una secuencia coordinada en la que se calculan las entradas efectivas, se responde a las entradas, se calculan de nuevo la entradas efectivas, se responde otra vez a las entradas, etc.

Considere la secuencia de sucesos que tendría lugar si inicializáramos la red con las dos neuronas de la derecha inhibidas y las otras neuronas excitadas

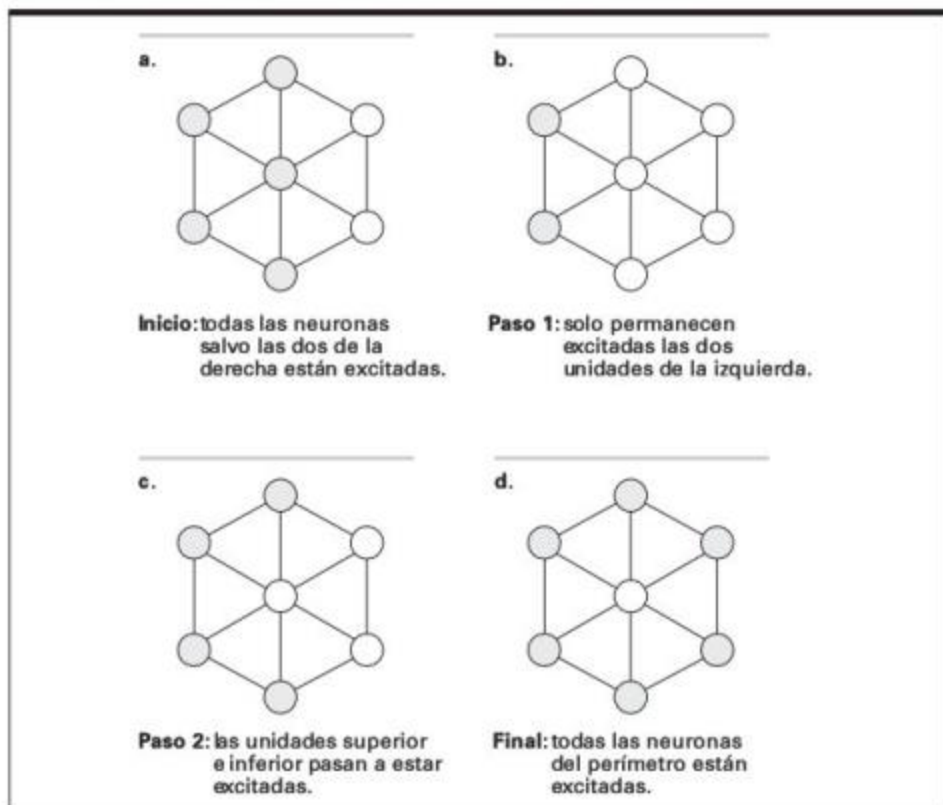
Figura 11.21 Una red neuronal artificial que implementa una memoria asociativa.



(Figura 11.22a). Las dos neuronas de la izquierda tendrán una entrada efectiva igual a 1, así que continuarán excitadas. Pero sus vecinas del perímetro tendrán una entrada efectiva igual a 0, por lo que quedarán inhibidas. De la misma forma, la neurona central tendrá una entrada efectiva de -4 , así que también se inhibirá. Por tanto, toda la red se desplazará a la configuración mostrada en la Figura 11.22b, en la que solo aparecen excitadas las dos neuronas de la izquierda. Puesto que la neurona central estaría ahora inhibida, la condición de excitadas de las neuronas de la izquierda hará que las neuronas superior e inferior vuelvan a excitarse. Mientras tanto, la neurona central seguirá inhibida, ya que tendrá una entrada efectiva igual a -2 . Por tanto, la red pasará a la configuración mostrada en la Figura 11.22c, de la cual después pasará a la configuración ilustrada en la Figura 11.22d. Pruebe a confirmar que si la red se inicializa con solo las cuatro neuronas superiores excitadas, se producirá un fenómeno de parpadeo. La neurona superior permanecerá excitada, mientras que su dos vecinas del perímetro y la neurona central irán alternando entre el estado excitado y el inhibido.

Por último, observe que la red tiene dos configuraciones estables: una en la que la neurona central está excitada y las demás están inhibidas y otra en la que la neurona central está inhibida y las restantes neuronas están excitadas. Si inicializamos la red con la neurona central excitada y no más de dos de las otras neuronas excitadas, la red irá derivando hacia la primera de esas dos con-

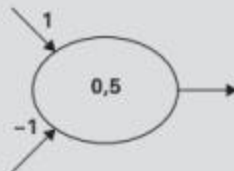
Figura 11.22 Los pasos que conducen a una configuración estable.



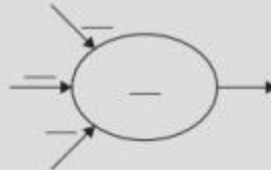
figuraciones estables. Si inicializamos la red con al menos cuatro neuronas adyacentes del perímetro en su estado excitado, la red derivará hacia la segunda configuración estable. Por tanto, podemos decir que la red asocia la primera de las configuraciones estables con aquellos patrones iniciales en los que su neurona central y menos de tres de sus neuronas perimetrales están excitadas, mientras que asocia la segunda configuración estable con aquellos patrones iniciales en los que hay cuatro o más de las neuronas perimetrales excitadas. En resumen, la red representa una memoria asociativa de carácter elemental.

Cuestiones y ejercicios

1. ¿Cuál será la salida de la siguiente neurona cuando sus dos entradas sean iguales a 1? ¿Y qué pasa con los patrones de entrada 0, 0; 0, 1; y 1, 0?



2. Ajuste los pesos y el valor umbral de la siguiente neurona para que su salida sea 1 si y solo si al menos dos de sus entradas son iguales a 1.



3. Identifique un problema que puede surgir a la hora de entrenar una red neuronal artificial.
4. ¿Hacia qué configuración estable derivará la red de la Figura 11.22 si se inicializa con todas sus neuronas inhibidas?

11.6 Robótica

La **robótica** es el estudio de los agentes físicos autónomos que se comportan de manera inteligente. Al igual que sucede con cualquier otro agente, los robots deben ser capaces de percibir, razonar y actuar dentro de su entorno. Las investigaciones en robótica abarcan, por tanto, todas las áreas de la inteligencia artificial, además de utilizar intensamente los resultados de la ingeniería mecánica y eléctrica.

Para interactuar con el mundo, los robots necesitan mecanismos para manipular los objetos y para desplazarse. En los primeros días de la robótica, el campo estaba estrechamente relacionado con el desarrollo de manipuladores, que muy a menudo eran brazos mecánicos con codos, muñecas y

manos o herramientas. La investigación no trataba solo de determinar cómo podían maniobrarse esos dispositivos, sino que también intentaba ver cómo podía mantenerse y aplicarse el conocimiento acerca de su posición y de su orientación. (Los seres humanos somos capaces de cerrar los ojos y seguir tocándonos la nariz con el dedo, porque nuestro cerebro mantiene un registro de dónde se encuentran nuestros dedos y nuestra nariz.) Con el tiempo, los brazos de robot se han hecho mucho más sofisticados, hasta el punto de que, con un cierto sentido del tacto basado en la realimentación dinámica, pueden manejar huevos o tazas de papel sin ningún problema.

Recientemente, el desarrollo de computadoras más rápidas y ligeras ha dado un impulso a las investigaciones en el campo de los robots móviles que pueden desplazarse. Conseguir esta movilidad ha conducido a una gran abundancia de diseños creativos. Los investigadores en el campo de la locomoción de los robots han desarrollado robots que nadan como los peces, que vuelan como las libélulas, que saltan como los saltamontes y que reptan como las serpientes.

Los robots con ruedas son muy populares ya que son relativamente fáciles de diseñar y de construir, pero están limitados en cuanto al tipo de terreno por el que se pueden desplazar. Sobreponerse a estas restricciones, utilizando combinaciones de ruedas o cadenas para subir escaleras o para moverse por terrenos rocosos es el objetivo de muchas de las investigaciones actuales. Por ejemplo, los exploradores marcianos de la NASA utilizan ruedas especialmente diseñadas para moverse por suelo rocoso.

Los robots con piernas ofrecen una mayor movilidad, pero son bastante más complejos. Por ejemplo, los robots con dos patas diseñados para caminar como los seres humanos, deben constantemente monitorizar y ajustar su posición para mantener el equilibrio, ya que de lo contrario se caen. Sin embargo, dichas dificultades pueden ser resueltas como ilustra el robot humanoide de dos piernas, denominado Asimo, desarrollado por Honda, que puede subir escaleras e incluso correr.

A pesar de los grandes avances en el campo de los manipuladores y de la locomoción, la mayoría de los robots todavía no son muy autónomos. Los bra-

Robots que han hecho historia

a. El robot "Boss" de Tartan Racing, ganador del Urban Challenge (desafío urbano), un concurso patrocinado por DARPA, en el que los vehículos tienen que conducir por sí mismos en un entorno urbano (© DARPA). **b.** Uno de los exploradores de la NASA, un robot geólogo que explora la superficie de Marte (Cortesía de NASA/JPL-Caltech).



zos robóticos industriales están, normalmente, rígidamente programados para cada tarea y trabajan sin sensores, suponiendo que los componentes se les entregarán en posiciones exactas. Otros robots móviles, como los exploradores marcianos de la NASA y los vehículos aéreos no tripulados de carácter militar dependen de los operadores humanos en lo que a inteligencia se refiere.

Sobreponerse a esta dependencia de los seres humanos es uno de los principales objetivos de las investigaciones actuales. Una de las cuestiones que se plantean trata con lo que un robot autónomo necesita conocer acerca de su entorno y con el grado de antelación con el que necesita planificar sus acciones. Un enfoque consiste en construir robots que mantengan registros detallados de sus entornos, que contengan un inventario de objetos junto con sus posiciones con el que desarrollar planes de acción precisos. Las investigaciones en esta dirección dependen en gran medida del progreso que se realice en el campo de la representación y almacenamiento del conocimiento, así como del desarrollo de técnicas mejoradas de razonamiento y de desarrollo de planes.

Un enfoque alternativo consiste en desarrollar robots reactivos que, en lugar de mantener registros complejos e invertir un gran esfuerzo en construir planes de acción detallados, simplemente aplican reglas sencillas de interacción con el mundo para guiar su comportamiento en cada momento. Los defensores de la robótica reactiva argumentan que a la hora de planificar un viaje largo en un vehículo, los seres humanos no hacen planes detallados de antemano en los que se contemplen todas las alternativas. En lugar de ello, simplemente seleccionan las carreteras principales, dejando para luego detalles tales como dónde comer, qué salidas tomar y cómo reaccionar en el caso de encontrarse con un desvío. De la misma forma, un robot reactivo que necesite desplazarse por un pasillo atestado o que necesite ir de un edificio a otro no tiene por qué desarrollar de antemano un plan enormemente detallado, sino que simplemente tiene que aplicar una serie de reglas sencillas para evitar cada obstáculo a medida que se lo encuentra. Este es el enfoque adoptado por el robot más vendido de la historia, el robot aspiradora iRobot Roomba, que se mueve por el suelo en modo reactivo sin preocuparse de recordar los detalles acerca de los muebles y otros obstáculos. Después de todo, lo más probable es que la mascota de la familia no se encuentre en el mismo lugar la siguiente vez.

Por supuesto, no es probable que ninguna solución resulte ser la óptima para todas las situaciones posibles. Los robots verdaderamente autónomos utilizarán, probablemente, múltiples niveles de razonamiento y de planificación, aplicando técnicas de alto nivel para establecer y conseguir los objetivos principales y sistemas reactivos de bajo nivel para lograr los subobjetivos secundarios. Un ejemplo de este tipo de razonamiento multinivel lo podemos encontrar en el concurso Robocup, un concurso internacional de equipos de fútbol robóticos, que sirve como foro para las investigaciones tendentes a desarrollar equipos de robots que puedan jugar mejor que los equipos profesionales formados por seres humanos en el año 2050. Aquí, el énfasis no está solo en construir robots móviles que puedan "patear" un balón, sino en diseñar un equipo de robots que puedan cooperar unos con otros con el fin de conseguir un objetivo común. Estos robots no solo tienen que desplazarse y razonar acerca de sus acciones, sino que también tienen que razonar acerca de las acciones de sus compañeros y de sus oponentes.

Otro ejemplo de investigación en el campo de la robótica es el subcampo conocido como robótica evolutiva, en el que se aplican las teorías de la evolución para el desarrollo de esquemas tanto de reglas reactivas de bajo nivel como razonamiento de alto nivel. En este subcampo nos encontramos con que la teoría de la supervivencia del más adaptado se utiliza para desarrollar dispositivos que a lo largo de múltiples generaciones adquieren sus propios medios de equilibrio o de movilidad. Buena parte de la investigación en esta área distingue entre el sistema de control interno de un robot (que es fundamentalmente software) y la estructura física de su cuerpo. Por ejemplo, el sistema de control de un robot renacuajo nadador fue transferido a un robot similar que tenía piernas. Entonces, se aplicaron técnicas evolutivas en el sistema de control para obtener un robot capaz de reptar. En otros casos se han aplicado técnicas evolutivas al cuerpo físico de un robot para descubrir las posiciones óptimas de los sensores para la realización de una tarea concreta. Otras investigaciones más complejas tratan de encontrar formas para hacer evolucionar los sistemas de control software de manera simultánea con las estructuras físicas del cuerpo.

Enumerar todo el conjunto de impresionantes resultados generados por las investigaciones en el campo de la robótica sería una tarea abrumadora. Los robots actuales están todavía lejos de esos potentes robots que se presentan en las novelas y películas de ficción, pero se han conseguido éxitos impresionantes en tareas específicas. Disponemos de robots que pueden conducir en condiciones de tráfico intenso, comportarse como mascotas y guiar misiles hasta su objetivo. Sin embargo, aunque nos alegremos de estos éxitos, es preciso observar que el cariño que sentimos por un perro artificial y el increíble poder de las armas inteligentes plantean cuestiones sociales y éticas que representan un auténtico desafío para nuestra sociedad. Nuestro futuro será tal como nosotros lo construyamos.

Cuestiones y ejercicios

1. ¿En qué sentido difiere la solución reactiva para fijar el comportamiento de un robot del comportamiento más tradicional "basado en planes"?
2. ¿Podría citar algunos de los temas actuales de investigación en el campo de la robótica?
3. ¿Podría citar dos niveles en los que se estén aplicando teorías evolutivas para el desarrollo de robots?

11.7 Consideración de las consecuencias

Sin ninguna duda, los avances que se están realizando en el campo de la inteligencia artificial ofrecen el potencial de beneficiar a la humanidad, y es fácil verse atrapado por el entusiasmo que generan esos potenciales beneficios. Sin embargo, también hay peligros potenciales acechando en el futuro, cuyas ramificaciones podrían ser tan devastadoras como beneficiosas son sus contrapartidas. La distinción está a menudo, simplemente, en el punto de vista de cada uno o quizá en la posición de cada uno dentro de la sociedad, las ventajas obte-

nidas por una persona pueden ser las pérdidas de otra. Es conveniente, por tanto, que dediquemos unos instantes a examinar estos avances de la tecnología desde algunas perspectivas alternativas.

Algunas personas ven los avances de la tecnología como un regalo para la humanidad, un medio de librar a las personas de las aburridas tareas mundanas y de abrir la puerta a estilos de vida más placenteros. Pero otras personas ven este mismo fenómeno como una maldición que quita empleos a los ciudadanos y canaliza la riqueza hacia aquellos que disponen del poder. De hecho, este era uno de los mensajes de Mahatma Gandhi en la India. Él insistía una y otra vez que para la India sería mejor sustituir las grandes fábricas de tejidos por ruecas instaladas en las casas de los trabajadores. De esta forma, decía, la producción centralizada en masa que solo da empleo a unos cuantos se vería sustituido por un sistema de distribución masivo que beneficiaría a las multitudes.

La historia está llena de revoluciones que hunden sus raíces en la desproporcionada distribución de la riqueza y los privilegios. Si se permite que los actuales avances tecnológicos aumenten esas diferencias, las consecuencias podrían ser catastróficas.

Pero las consecuencias de construir máquinas cada vez más inteligentes son más sutiles, más fundamentales, que las que se refieren a las luchas de poder entre distintos segmentos de la sociedad. Los problemas que se plantean afectan al auténtico núcleo de la imagen que la humanidad tiene de sí misma. En el siglo XIX, la sociedad se escandalizó por la teoría de la evolución de Charles Darwin y por el pensamiento de que los seres humanos podían haber evolucionado a partir de formas de vida menos complejas. ¿Cómo reaccionará entonces la sociedad si se la enfrenta con el fenómeno de la aparición de máquinas cuyas capacidades mentales desafien las de los propios seres humanos?

En el pasado, la tecnología se ha desarrollado lentamente, dejando así tiempo para que la imagen que tenemos de nosotros mismos quedara preservada reajustando nuestro concepto de la inteligencia. Nuestros antepasados de hace muchos siglos habrían interpretado los dispositivos mecánicos del siglo XIX como dotados de una inteligencia sobrenatural, pero hoy día no concedemos a esas máquinas ninguna inteligencia en absoluto. ¿Pero cómo reaccionará la humanidad si las máquinas comienzan verdaderamente a desafiar la inteligencia de los humanos o, más probablemente, si las capacidades de las máquinas comienzan a avanzar más rápidamente que nuestra capacidad de adaptación?

Una pista acerca de cuál sería la reacción potencial de la humanidad a la aparición de máquinas que desafien nuestro intelecto nos la ofrece el análisis de cuál fue la respuesta social a los test de inteligencia a mediados del siglo XX. Esos test se consideraron para tratar de identificar el nivel de inteligencia de los niños. Los niños en Estados Unidos se clasificaron según su rendimiento en este tipo de test y se les dirigió hacia los distintos programas educativos de acuerdo con los resultados. Esto hizo que se abrieran oportunidades educativas para aquellos niños que obtenían buenos resultados en esos test, mientras que los niños con resultados mediocres se canalizaban hacia programas de estudio compensatorios. En resumen, al proporcionarle una escala con la que medir la inteligencia, la sociedad tendió a no considerar las capacidades de aquellos que se vieron situados en el extremo inferior de la escala. ¿Cómo gestionaría entonces nuestra sociedad la situación si las capacidades "intelectuales" de las máquinas pasaran a ser comparables o incluso parecieran comparables, a las de los huma-

nos? ¿Volvería la espalda nuestra sociedad a aquellos cuyas capacidades fueran vistas como "inferiores" a las de las máquinas? En caso afirmativo, ¿cuáles serían las consecuencias para esos miembros de la sociedad? ¿Debería estar sujeta la dignidad de una persona a cómo se compare esa persona con una máquina?

Ya hemos comenzado a ver cómo la capacidad intelectual de los seres humanos se ve desafiada por las máquinas en una serie de campos específicos. Las máquinas son capaces ahora de derrotar a los jugadores expertos de ajedrez; existen sistemas expertos computerizados que son capaces de proporcionar consejos de tipo médico y existen programas simples de gestión de carteras de inversión que a menudo obtienen resultados mejores que los de los inversores profesionales. ¿Cómo afectan esos sistemas a la imagen que tienen de sí mismos los individuos afectados? ¿Cómo se verá afectada la autoestima de una persona al verse desplazada por las máquinas en un número cada vez mayor de áreas?

Muchos argumentan que la inteligencia que poseen las máquinas siempre será inherentemente distinta de la de los seres humanos, ya que los humanos somos entes biológicos y las máquinas no. Por tanto, argumentan que las máquinas no serán nunca capaces de reproducir el proceso de toma de decisiones de los seres humanos. Las máquinas podrían tomar las mismas decisiones que los humanos, pero esas decisiones no se realizarían basándose en las mismas cosas que los seres humanos. ¿Hasta qué grado existen entonces diferentes tipos de inteligencia y hasta qué grado sería ético que nuestra sociedad siguiera las directrices propuestas por una inteligencia no humana?

En su libro, *Computer Power and Human Reason*, Joseph Weizenbaum argumenta en contra de la aplicación indiscriminada de la inteligencia artificial diciendo lo siguiente:

Las computadoras pueden tomar decisiones judiciales. Las computadoras pueden emitir opiniones de carácter psiquiátrico. Pueden lanzar monedas al aire en formas mucho más sofisticadas que los seres humanos más pacientes, pero lo importante es que no *deberíamos* asignarles dichas tareas. Puede incluso que sean capaces de llegar a tomar decisiones "correctas" en algunos casos, pero siempre y necesariamente sobre bases que ningún ser humano debería estar dispuesto a aceptar.

Ha habido numerosos debates acerca de las "computadoras y la mente". Como conclusión me gustaría decir aquí que los problemas relevantes no son tecnológicos ni tampoco matemáticos; se trata de problemas éticos, problemas que no pueden solucionarse planteando preguntas que comiencen con "podemos". Los límites de la aplicabilidad de las computadoras solo pueden enunciarse, en definitiva, en términos de deberes. La conclusión más elemental de todo esto es que, puesto que ahora no disponemos de ninguna forma de hacer sabias a las computadoras, no deberíamos asignarles tareas que requieran sabiduría.

El lector podría argumentar que buena parte de esta sección bordea la ciencia ficción más que las Ciencias de la computación. Sin embargo, no hace tanto tiempo muchas personas descartaban la pregunta "¿Qué sucederá si las computadoras se hacen con el control de la sociedad?" con la misma aptitud de "eso no sucederá nunca". Pero en muchos aspectos, ese día ya ha llegado.

Si una base de datos computerizada informara erróneamente de que nuestro límite de crédito es muy bajo, de que poseemos antecedentes penales o de que nuestra cuenta bancaria no tiene saldo, ¿qué es lo que prevalecerá, la afirmación hecha por la computadora o nuestras demandas de inocencia? Si un sistema de navegación que funcione mal indica que una pista cubierta por la niebla se encuentra en el lugar incorrecto, ¿dónde aterrizará el avión? Si se emplea una máquina para predecir la reacción de la opinión pública a diversas decisiones políticas, ¿qué decisión tomará un político? ¿Cuántas veces se ha encontrado usted en esa situación en la que un administrativo le dice que no puede ayudarle porque "la computadora no funciona"? ¿Quién (o qué) está entonces a cargo de las cosas? ¿Acaso no hemos ya entregado nuestra sociedad a las máquinas?

Cuestiones y ejercicios

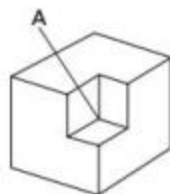
1. ¿Qué parte de la población actual sobreviviría si nos deshiciéramos de las máquinas desarrolladas a lo largo de los últimos cien años? ¿Y a lo largo de los últimos cincuenta años? ¿Y a lo largo de los últimos veinte años? ¿Dónde estarían localizados los supervivientes?
2. ¿Hasta qué grado está su vida controlada por las máquinas? ¿Quién controla las máquinas que afectan a su vida?
3. ¿Dónde obtiene la información en la que basa sus decisiones cotidianas? ¿Y sus decisiones más importantes? ¿Qué confianza tiene en la precisión de esa información? ¿Por qué?

Problemas de repaso

(Los problemas marcados con asterisco están asociados con las secciones opcionales.)

1. Como se ha ilustrado en la Sección 11.2, las personas pueden utilizar una pregunta con propósitos distintos del de realmente preguntar. Por ejemplo, la frase "¿Sabes que tienes una rueda pinchada?" se utilizaría más bien para informar a alguien que para preguntarle. Proporcione ejemplos de preguntas utilizadas para reafirmar, advertir y criticar.
2. Analice un cajero automático como si fuera un agente. ¿Cuáles son sus sensores? ¿Cuáles son sus actuadores? ¿Qué nivel de respuesta exhibe (refleja, basada en conocimientos, basada en objetivos)?
3. Identifique cada una de las siguientes respuestas como refleja, basada en conocimientos o basada en objetivos. Justifique sus respuestas.
 - a. Un partido de tenis.
 - b. Escalar una montaña.
 - c. Una partida de ajedrez.
4. ¿Cómo se analiza el lenguaje natural?
5. ¿Cómo se clasifican los diferentes métodos de aprendizaje?
- *6. ¿Cuál es la diferencia entre el conocimiento procedimental y el conocimiento declarativo? Explique su respuesta utilizando un ejemplo.

7. ¿Cuáles de las siguientes actividades esperarías que estuvieran orientadas al rendimiento y cuáles orientadas a la simulación?
- El diseño de un sistema de vehículos de transporte automatizado (empleado a menudo en aeropuertos para trasladarse de una terminal a otra).
 - El diseño de un modelo que predice la ruta seguida por un huracán.
 - El diseño de una base de datos de búsqueda en la Web utilizada para crear y mantener índices de documentos almacenados en la World Wide Web
 - El diseño de un modelo de la economía de una nación, con el fin de probar teorías.
 - El diseño de un programa para monitorizar los signos vitales de un paciente.
8. ¿Puede aplicarse el test de Turing a un sistema en el que la entrada y la salida están restringidas? ¿Puede aplicarse si el sistema solo toma datos binarios y proporciona resultados en formato binario?
9. Identifique un pequeño conjunto de propiedades geométricas que pueda utilizarse para distinguir entre los símbolos F, E, L y T.
- *10. ¿Qué es una red neuronal artificial?
11. Describa dos interpretaciones del siguiente dibujo lineal, basándose en si la "esquina" marcada por la letra "A" es convexa o cóncava:



12. Compare los papeles de las cláusulas preposicionales incluidas en las dos frases siguientes (que sólo difieren en una palabra). ¿Cómo podría programarse una máquina para poder hacer tales distinciones?

La pared fue construida por el viento.
La pared fue construida por el granjero.

13. ¿Cómo difieren los resultados de analizar sintácticamente las dos frases siguientes? ¿Cómo difieren los resultados del análisis semántico?

Una maravillosa puesta de sol fue contemplada por Andrea.

Andrea contempló una maravillosa puesta de sol.

14. ¿Cómo difieren los resultados de analizar sintácticamente las dos frases siguientes? ¿Cómo difieren los resultados del análisis semántico?

Si $X < 10$ then restar 1 de X else sumar 1 a X.

Si $X > 10$ then sumar 1 a X else restar 1 de X.

15. En el texto hemos explicado brevemente los problemas de comprender los lenguajes naturales, por oposición a los lenguajes de programación formales. Como ejemplo de las complejidades en el caso de los lenguajes naturales, identifique situaciones en las que la pregunta "¿Sabes qué hora es?" tenga diferentes significados.
16. Los cambios en el contexto de una frase pueden variar la importancia de la frase además de su significado. En el contexto de la Figura 11.3, ¿cómo cambiaría la importancia de la frase "María golpeó a Juan" si las fechas de nacimiento estuvieran ambas a finales de la década de 2000? ¿Y si una estuviera en la década de 1980 y otra a finales de la década de 2000?

17. Dibuje una red semántica que represente la información contenida en el siguiente párrafo:

Diana arrojó la pelota a Joaquín, que la lanzó hacia el centro del campo. El centrocampista trató de atraparla, pero en lugar de ello rebotó en la pared.

18. En ocasiones, la capacidad de responder a una pregunta depende tanto de conocer los límites del conocimiento, como de los propios hechos. Por ejemplo, suponga que las bases de datos A y B contienen, las dos, una lista completa de empleados adheridos al